

Introduction

FuriosaAI's Tensor Contraction Processor (TCP) is a massively parallel AI accelerator targeting inference workloads. Unlike high-level frameworks like PyTorch and XLA, which abstract away memory layouts and hardware scheduling, TCP exposes direct programmer control without requiring the byte-level reasoning of low-level kernel APIs.

TCP's Virtual Instruction Set Architecture (Virtual ISA, or vISA) is the programming interface that exposes this control. It lets programmers reason in tensors while directly managing memory allocation and Tensor Unit scheduling. This manual primarily helps authors guide an AI agent to write, verify, and optimize vISA kernels. It also provides the technical material that compiler developers need when generating vISA. [The `furiosa-opt-std` `rustdoc`](#) is the authoritative API source for published releases. This book explains how to make and review kernel design decisions. Both audiences assume basic Rust familiarity. See [the language manual](#) if needed.

Read This Book by Task

[Quick Start](#) introduces the base-template kernel pattern and the learning route through Mapping, Moving, Computing, End-to-End Cases, Scheduling, and Tools.

Warning

Alpha Test Build: Experimental Software

This software is an early, experimental, and incomplete build intended strictly for technical evaluation and internal testing.

Before using this software for any production work, critical tasks, or for important data, you must consult with Furiosa engineers.

Your feedback is vital to our development. Please provide it.

License

This documentation and the entire `furiosa-opt` repository are licensed under the [Apache License Version 2.0](#).

Quick Start

This index bootstraps a runnable base-template vector kernel. The five runnable patterns in [Kernel Design](#) add one hardware decision at a time. That single chapter owns the source-backed device and host-oracle walkthroughs for constant addition, elementwise multiplication, dot product, GEMV, and GEMM.

Validation meanings and caveats are defined only in [Kernel Validation](#).

HBM is high-bandwidth memory used for host transfers, and DM is on-chip data memory used by the Tensor Unit.

After these basic patterns, continue through Mapping, Moving, Computing, End-to-End Cases, Scheduling, and Tools. The subsystem chapters now own the matching advanced examples, while End-to-End Cases collects composed model tutorials.

Read the ordered child pages

Read [Setup and Tooling](#) for installation and the first project. Read [Tensor and Contraction](#) for tensor shapes and contraction math. Read [Kernel Design](#) for the five runnable pattern sections. Read [Kernel Validation](#) for compile, CPU, NPU, and schedule evidence.

Continue to Reference Chapters

The runnable patterns process tensors that fit in one hardware pass and within the 512 KB per-slice DM capacity. Use temporal partitioning when tiles run sequentially over time and spatial partitioning when tiles distribute across parallel hardware units. Read [Mapping Tensors](#), then [Moving Tensors](#), then [Computing Tensors](#). After those reference chapters, choose a matching advanced pattern in the relevant Mapping, Moving, or Computing chapter.

Choose the next pattern

The three reference chapters establish the mapping, movement, and compute contracts

before you choose an example. Use the simplest example route that matches the kernel's next constraint.

- **Fits one pass:** No escalation; the preceding basic patterns apply.
- **Layout permutation or packet handling:** Read [Case Study: Tensor Unit I/O](#) under Moving Tensors.
- **Composed model patterns:** Read the [Case Study: Transformer](#) under End-to-End Cases.

Setup and Tooling

Installation and First Program

This chapter owns the installation and first-project command path. Run it before reading the kernel patterns.

Host Requirements

The host must be `x86_64-unknown-linux-gnu` on Ubuntu 22.04 or newer, since the published binaries need `GLIBC_2.34` and 20.04 ships 2.31. On macOS or arm64, run everything inside a `linux/amd64` container.

```
sudo apt install build-essential libclang-dev    # every build
sudo apt install gcc-aarch64-linux-gnu          # only for the NPU build
(cargo furiosa-opt)
```

Install these before the toolchain below, which does not cover them: `rustc` links host binaries with the system `cc`, and `furiosa-opt-std`'s build script loads `libclang.so`.

Install and Run the Starter

The Furiosa optimizer is a `rustc` driver and is ABI-locked to the pinned nightly. Install the toolchain and host tools, create the standard starter, then run it on the CPU and compile its kernels:

```
rustup toolchain install nightly-2026-05-01
cargo +nightly-2026-05-01 install cargo-binstall
cargo +nightly-2026-05-01 binstall cargo-furiosa-opt
cargo install cargo-generate
cargo generate furiosa-ai/furiosa-opt base-template
cd base-template
cargo run --release --bin gemm
cargo furiosa-opt compile
```

`rust-toolchain.toml` pins the same channel in the generated project. Cargo activates it

automatically in that directory.

A plain `cargo` invocation builds and runs on the CPU. `cargo furiosa-opt` builds for the NPU and pre-compiles the kernels.

NPU Requirements

The NPU build also requires the Furiosa SDK and a physical NPU. It dispatches through the SDK kernel driver and PE runtime (`furiosa-driver-rngd` , `furiosa-smi` , and related components). See the [SDK documentation](#).

The CPU build does not require the SDK; it runs host-side with no NPU dependency.

The Generated Project

The command `path` scaffolds the `base-template` starter. The template is the standard executable example, and continuous integration (CI) generates the current source under an arbitrary name before testing it on the CPU. Read [Tensor and Contraction](#) and [Kernel Validation](#) before adapting the generated template. Use the Kernel Authoring Skill when an agent implements the change.

Template Layout

The generated `Cargo.toml` declares `[package.metadata.furiosa-opt]` , which marks the package for kernel compilation. `src/kernel/` contains each `#[device]` function and `src/kernel/mod.rs` re-exports those kernel modules. Direct `src/*.rs` files are host binaries that call `launch(kernel, ...)` and contain their host-oracle tests. Each host binary has an explicit `[[bin]]` entry with `path = "src/<name>.rs"` in `Cargo.toml` . Do not move host binaries into `src/bin/` , `examples/` , or `tests/` , because the compiler plugin scans cargo targets rooted at `src/` .

```

base-template/
├── Cargo.toml
├── rust-toolchain.toml
├── src/
│   ├── lib.rs
│   ├── kernel/
│   │   ├── mod.rs
│   │   └── <name>_kernel.rs
│   └── <name>.rs

```

Add a Kernel

1. Add `src/kernel/<name>_kernel.rs` with a `#[device(...)] pub fn <name>_kernel(...)` function.
2. Re-export it from `src/kernel/mod.rs`.
3. Add `src/<name>.rs` with the host program, host oracle, and a call to `launch(<name>_kernel, ...)`.
4. Register the host program as a `[[bin]]` target in `Cargo.toml`.
5. Run the kernel with the [Kernel Optimizer command reference](#).

Development Tools

The Furiosa IR Optimizer complements the [Furiosa SDK compiler](#). It gives programmers fine-grained control when they write vISA by hand or generate it from another compiler.

Kernel Optimizer

The [Kernel Optimizer tool reference](#) owns CLI syntax and compilation behavior.

Language Server

`furiosa-rust-analyzer-proxy` provides standard Rust IDE features with readable Furiosa mapping expressions. It renders verbose mapping types such as `Stride<Symbol<A>, 8> as m![A / 8]`. See the [Language Server tool reference](#) for installation and configuration.

Schedule Viewer

The Schedule Viewer visualizes the execution timeline to help identify performance bottlenecks. Use `furiosa-opt` to export a schedule JSON file, then open it with `furiosa-schedule-viewer`.

For installation and usage, see the [Schedule Viewer tool reference](#).

Tensor and Contraction

A tensor maps each named index in its shape to a value. A shape is an unordered set of named axes, so $\{ N = 4, C = 3 \}$ and $\{ C = 3, N = 4 \}$ identify the same tensor. An ordered representation of that shape behaves like a familiar multidimensional array. A tensor index supplies one value for every axis, so $\{ N = 4, C = 3 \}$ includes indices such as $\{ N: 0, C: 0 \}$ and $\{ N: 0, C: 1 \}$.

Tensor	Dimension	Example	Named shape
Scalar	0D	5.2	$\{ \}$
Vector	1D	[1, 2, 3]	$\{ I = 3 \}$
Matrix	2D	a 2×4 grid	$\{ I = 2, J = 4 \}$
Image batch	4D	four RGB images	$\{ N = 4, C = 3, H = 256, W = 512 \}$

Tensor contraction multiplies two inputs elementwise and reduces every shared axis that is absent from the output. Every contraction consists of broadcast, multiply, and reduce steps.

Operation	Einsum	Broadcast	Multiply	Reduce
Dot product	$(I, I \rightarrow 1)$	None.	$(x_i y_i)$	$(\sum_i x_i y_i)$
GEMV	$(I, J \rightarrow I)$	(x) across (I).	$(A_{ij} x_j)$	$(y_i = \sum_j A_{ij} x_j)$
GEMM	$(IK, KJ \rightarrow IJ)$	(A) across (J) and (B) across (I).	$(A_{ik} B_{kj})$	$(C_{ij} = \sum_k A_{ik} B_{kj})$

This page owns the introductory math only. Mapping, movement, and engine contracts remain in their respective reference chapters.

Kernel Design

First Vector Kernel

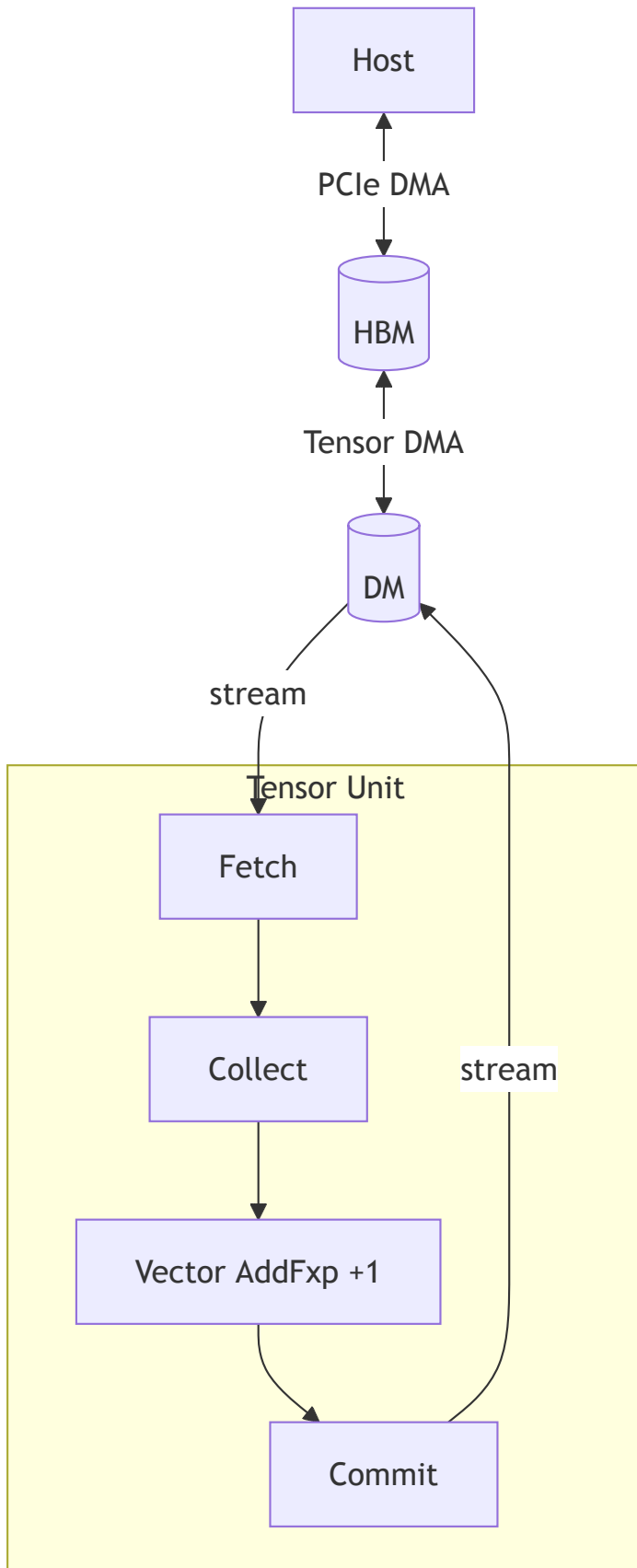
Constant Addition

Goal

Compute $\text{out}[i] = \text{in}[i] + 1$ for every element of a vector. This pattern covers residual and bias addition.

Data Movement

Move the vector from HBM to DM, stream each DM slice through Fetch, Collect, the Vector Engine, and Commit, and move the result from DM back to HBM.



Device Source

The kernel uses one chip, one of two clusters, and all 256 slices in that cluster. Each slice

receives eight `i32` values. `TagMode::Zero` executes the Vector Engine on every cycle. `to_dm` distributes the vector across slices, and the `begin` → `fetch` → `collect` → `vector_init` → `vector_intra_slice_tag` → `vector_fxp` → `vector_final` → `commit` chain processes each slice in one pass.

Device source: `src/kernel/constant_add_kernel.rs`.

```
use furiosa_opt_std::prelude::*;

axes![A = 2048];

pub type Chip = m![1];
pub type Cluster = m![1 # 2];
pub type Slice = m![A / 8 # 256];

#[device(chip = 1)]
pub fn constant_add_kernel(ctx: &mut Context, input: &HbmTensor<i32, Chip, m![A]>) -> HbmTensor<i32, Chip, m![A]> {
    // HBM → DM: split 2048 elements across 256 slices (8 elements per slice)
    let dm = input.to_dm::<Cluster, Slice, m![A % 8]>(&mut ctx.tdma);

    let result = ctx
        .main
        .begin(dm.view())
        // Fetch: stream 8-element packets from DM into the pipeline
        .fetch::<m![1], m![A % 8]>()
        // Collect: normalize the stream into 32-byte flits (8 × i32)
        .collect::<m![1], m![A % 8]>()
        // Vector Engine: enter pipeline and arm unconditionally
        .vector_init()
        .vector_intra_slice_tag(TagMode::Zero)
        // Add the scalar constant 1 to every element
        .vector_fxp(FxpBinaryOp::AddFxp, 1)
        // Exit VE and commit: trim the packet to the commit width, then
write
        // results back to DM
        .vector_final()
        .commit_trim::<m![A % 8]>()
        .commit::<m![A % 8]>();

    // DM → HBM
    result.to_hbm(&mut ctx.tdma)
}
```

Host Program and Oracle

The host program creates input, uploads it to HBM, launches the device function, and compares the result with the host equation in its test.

Base-template host program and oracle: `src/constant_add.rs`.

```
use furiosa_opt_std::prelude::*;
use {{ crate_name }}::kernel::constant_add_kernel::{A, constant_add_kernel};
use rand::SeedableRng;
use rand::rngs::SmallRng;

#[tokio::main]
async fn main() {
    let mut ctx = Context::acquire();
    let mut rng = SmallRng::seed_from_u64(42);
    let input = HostTensor::<i32, m![A]>::rand(&mut rng);
    let in_hbm = input.to_hbm(&mut ctx.pdma).await;
    let _out_hbm = launch(constant_add_kernel, (&mut ctx, &in_hbm)).await;
    println!("Constant Add: kernel ran");
}

#[cfg(test)]
mod tests {
    use super::*;

    #[tokio::test]
    async fn matches_reference() {
        let mut ctx = Context::acquire();

        let mut rng = SmallRng::seed_from_u64(42);
        let input = HostTensor::<i32, m![A]>::rand(&mut rng);
        let in_hbm = input.to_hbm(&mut ctx.pdma).await;

        // Reference: out[i] = in[i] + 1.
        let in_buf: Vec<i32> = input.into_vec();
        let expected: Vec<i32> = in_buf.iter().map(|&x|
x.wrapping_add(1)).collect();

        let out_hbm = launch(constant_add_kernel, (&mut ctx, &in_hbm)).await;

        let actual: Vec<i32> = out_hbm.to_host::<m![A]>(&mut
ctx.pdma).await.into_vec();
        for (i, (&e, &a)) in expected.iter().zip(&actual).enumerate() {
            assert_eq!(e, a, "constant_add mismatch at i={i}: expected {e},
actual {a}");
        }
    }
}
```

Pattern-Specific Mapping and Memory

TCP has four nested hardware levels.

Level	Count (RNGD)	Role
Chip	System-dependent	Top-level unit that holds HBM.
Cluster	Two per chip	Group of 256 slices.
Slice	256 per cluster	One Tensor Unit.
Lane	Eight per slice	One row of the Contraction Engine MAC array.

A tensor type encodes its element type and the distribution of each logical axis across this hierarchy. For example, `DmTensor<bf16, m![1], m![1 # 2], m![A / 8 # 256], m![A % 8]>` represents a `bf16` vector with axis `A` split across 256 slices with eight elements per slice. Each element of `A` has one well-defined position in one slice.

Mapping operators describe where an axis goes.

- `/` splits an axis by stride.
- `%` gives the in-unit count.
- `#` pads a mapping to the hardware unit count.
- `=` reduces how much of an existing term a view covers, which is how a tile selects one piece of a tensor.

`%` and `=` both narrow an axis, so it is worth stating what separates them. `%` introduces a new inner term: in `m![B / 2048, B % 2048]` the first term counts the 2048-element blocks of `B` and the second counts the positions inside one block. `=` leaves the terms alone and shrinks the extent the view reads, so `m![B / 2048 = 1 # 2]` selects one of the two blocks, where `# 2` keeps the stride of the full axis rather than of the selection. A tiled kernel uses them together, as in `m![B / 2048 = 1 # 2, B % 2048]`, which picks one block and covers every position within it. See [Tiling](#) for the general rule.

Tensor Unit tensors also use `Time` for pipeline iterations and `Packet` for elements within an iteration. The Tensor Unit pipeline is `Fetch → Switch → Collect → Contraction → Vector → Cast → Transpose → Commit`. The Switch Engine connects slices while most stages operate independently within a slice.

Type	Location	Capacity (RNGD)	Role

HbmTensor	On-package	48 GB and 1.5 TB/s	Long-term weight and activation storage.
DmTensor	On-chip SRAM	256 MB total and 512 KB per slice	Primary working memory.
TrfTensor	On-chip SRAM	8 KB per lane and eight lanes per slice	Contraction Engine register file.
VrfTensor	On-chip SRAM	8 KB per slice	Vector Engine operand register file.

These choices specialize the constant-add pattern. See [Mapping Tensors](#) for the complete mapping model, [Moving Tensors](#) for memory movement, and [Computing Tensors](#) for the pipeline APIs.

Stationary Vector Operand

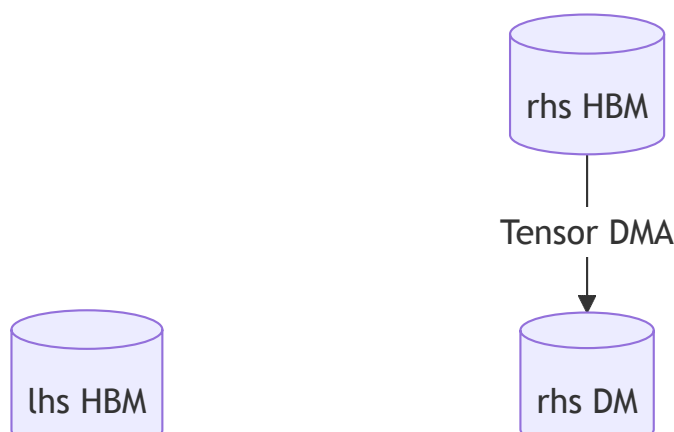
Elementwise Multiplication

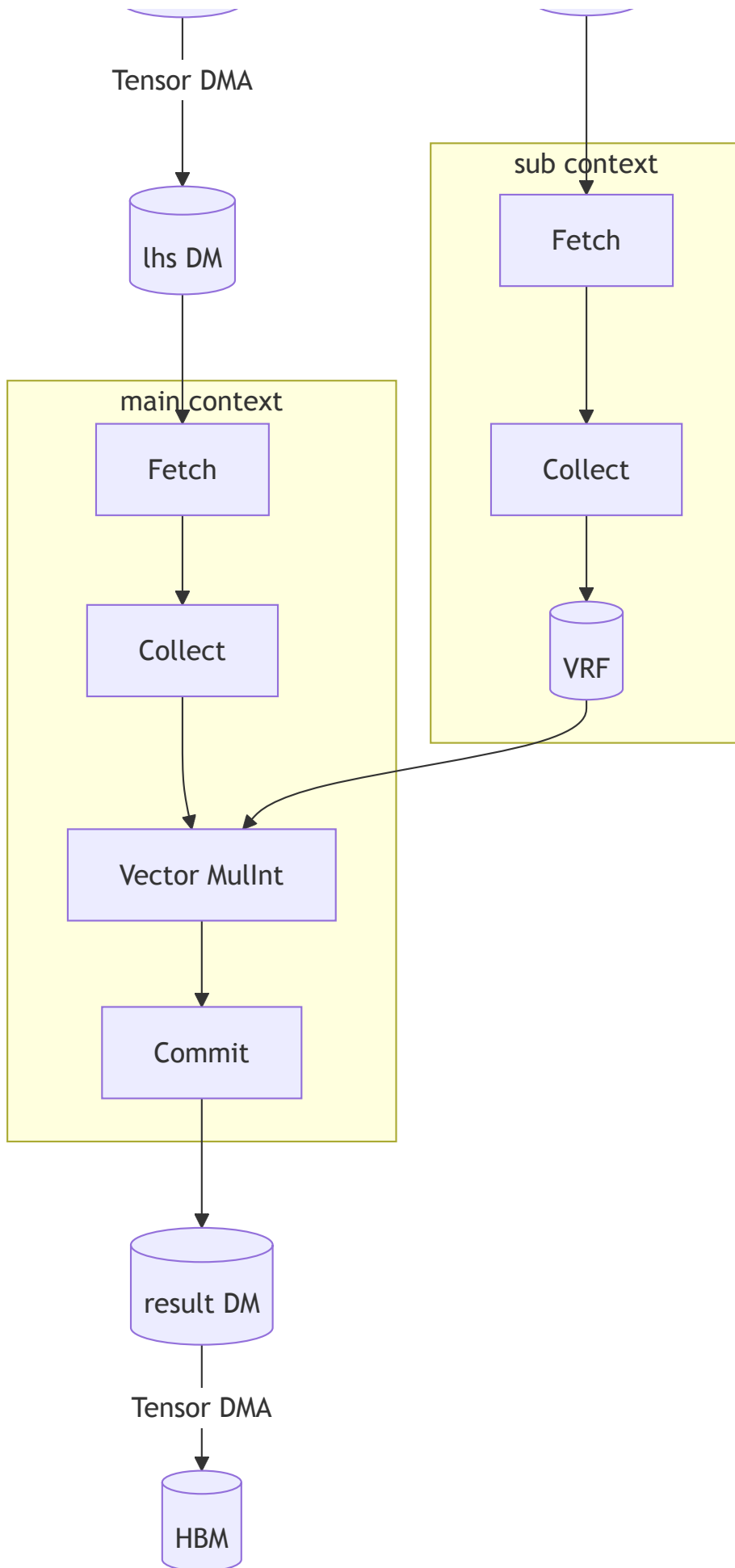
Goal

Compute $\text{out}[i] = \text{lhs}[i] \times \text{rhs}[i]$. This pattern covers gate scaling, GLU, and attention scaling.

Mapping and Data Movement

The `sub` context preloads `rhs` into the VRF while the `main` context streams `lhs` and reads the VRF on every Vector Engine cycle. The two DM regions must not overlap.





Every device kernel has `ctx.main` and `ctx.sub` execution contexts on separate hardware resources. `main` runs the primary computation while `sub` commonly prefetches operands. `main` waits when it needs data that `sub` has not produced, and both contexts share the flat on-chip SRAM.

Omit DM addresses for automatic placement. Use `_at` APIs only when an algorithm needs explicit non-overlapping addresses. The `Tu` const generic identifies whether a tensor flows through `{ Tu::Main }` or `{ Tu::Sub }`. The `sub` context loads `rhs_dm` through `Fetch`, `Collect`, and `.to_vrf()`, while `main` streams `lhs_dm` through `MulInt` with that stationary VRF operand. The two contexts run concurrently when their resource dependencies allow it.

Device Source

Device source: `src/kernel/elementwise_mul_kernel.rs`.

```

use furiosa_opt_std::prelude::*;

axes![A = 2048];

pub type Chip = m![1];
pub type Cluster = m![1 # 2];
pub type Slice = m![A / 8 # 256];

#[device(chip = 1)]
pub fn elementwise_mul_kernel(
    ctx: &mut Context,
    lhs: &HbmTensor<i32, Chip, m![A]>,
    rhs: &HbmTensor<i32, Chip, m![A]>,
) -> HbmTensor<i32, Chip, m![A]> {
    // Move both operands from HBM to DM (DM placement is assigned
    automatically).
    let lhs_dm = lhs.to_dm::<Cluster, Slice, m![A % 8]>(&mut ctx.tdma);
    let rhs_dm = rhs.to_dm::<Cluster, Slice, m![A % 8]>(&mut ctx.tdma);

    // Sub context: load rhs into VRF (runs concurrently with the main
    context below).
    // VRF holds a per-slice operand that the Vector Engine reads every
    cycle.
    let rhs_vrf: VrfTensor<i32, Chip, Cluster, Slice, m![A % 8]> = ctx
        .sub
        .begin(rhs_dm.view())
        .fetch::<m![1], m![A % 8]>()
        .collect::<m![A % 8 / 8], m![A % 8 % 8]>()
        .to_vrf();

    // Main context: multiply every lhs element by its rhs counterpart from
    VRF
    let result = ctx
        .main
        .begin(lhs_dm.view())
        .fetch::<m![1], m![A % 8]>()
        .collect::<m![1], m![A % 8]>()
        .vector_init()
        .vector_intra_slice_tag(TagMode::Zero)
        // Each slice multiplies its 8 lhs elements by the matching 8 rhs
        elements in VRF
        .vector_fxp(FxpBinaryOp::MulInt, &rhs_vrf)
        .vector_final()
        .commit_trim::<m![A % 8]>()
        .commit::<m![A % 8]>();

    result.to_hbm(&mut ctx.tdma)
}

```

Host Program and Oracle

Base-template host program and oracle: `src/elementwise_mul.rs`.

```
use furiosa_opt_std::prelude::*;
use {{ crate_name }}::kernel::elementwise_mul_kernel::{A,
elementwise_mul_kernel};
use rand::SeedableRng;
use rand::rngs::SmallRng;

#[tokio::main]
async fn main() {
    let mut ctx = Context::acquire();
    let mut rng = SmallRng::seed_from_u64(42);
    let lhs = HostTensor::<i32, m![A]>::rand(&mut rng);
    let rhs = HostTensor::<i32, m![A]>::rand(&mut rng);
    let lhs_hbm = lhs.to_hbm(&mut ctx.pdma).await;
    let rhs_hbm = rhs.to_hbm(&mut ctx.pdma).await;
    let _out_hbm = launch(elementwise_mul_kernel, (&mut ctx, &lhs_hbm,
&rhs_hbm)).await;
    println!("Elementwise Mul: kernel ran");
}

#[cfg(test)]
mod tests {
    use super::*;

    #[tokio::test]
    async fn matches_reference() {
        let mut ctx = Context::acquire();

        let mut rng = SmallRng::seed_from_u64(42);
        let lhs = HostTensor::<i32, m![A]>::rand(&mut rng);
        let rhs = HostTensor::<i32, m![A]>::rand(&mut rng);

        let lhs_hbm = lhs.to_hbm(&mut ctx.pdma).await;
        let rhs_hbm = rhs.to_hbm(&mut ctx.pdma).await;

        // Reference: out[i] = lhs[i] * rhs[i].
        let lhs_buf: Vec<i32> = lhs.into_vec();
        let rhs_buf: Vec<i32> = rhs.into_vec();
        let expected: Vec<i32> = lhs_buf.iter().zip(&rhs_buf).map(|(&a, &b)|
a.wrapping_mul(b)).collect();

        let out_hbm = launch(elementwise_mul_kernel, (&mut ctx, &lhs_hbm,
&rhs_hbm)).await;

        let actual: Vec<i32> = out_hbm.to_host::<m![A]>(&mut
ctx.pdma).await.into_vec();
```

```

        for (i, (&e, &a)) in expected.iter().zip(&actual).enumerate() {
            assert_eq!(e, a, "elementwise_mul mismatch at i={i}: expected
{e}, actual {a}");
        }
    }
}

```

Contraction Patterns

Dot Product

Goal

Compute $\sum_i x_i y_i$. This pattern covers attention scores and similarity.

Mapping and Data Movement

One operand flows through the pipeline while the `sub` context stores the other operand in the TRF. The `sub` context loads that operand through Fetch, Collect, and `.to_trf()`.

`.contract_outer()` pairs 32-byte flits into 64-byte packets and reads the stationary TRF operand. The Stream Adapter performs the pairing and the TRF Sequencer reads the stationary operand. Both operands feed the per-lane multiplier. `.contract_packet()` reduces products spatially through the hardware reduction tree. `.contract_time::<m! [1]>()` accumulates across time and produces one scalar per slice. `.contract_lane()` folds the lanes, which is trivial when `Lane = m! [1]`. `.cast()` converts the `f32` accumulator to `bf16`.

Contraction performs multiply-and-accumulate over many terms and needs a widened type. Contraction widens `i4` and `i8` to `i32`, and it widens `fp8` and `bf16` to `f32`. The annotations on `.align()` and `.contract()` select that widened type.

Device Source

Device source: `src/kernel/dot_product_kernel.rs`.

```

use furiosa_opt_std::prelude::*;

axes![A = 2048];

pub type Chip = m![1];
pub type Cluster = m![1 # 2];
pub type Slice = m![1 # 256]; // 1 active slice; m![A / 8 # 256] would
distribute across all 256
pub type Time = m![1]; // No temporal iteration
pub type Lane = m![1]; // No lane parallelism

#[device(chip = 1)]
pub fn dot_product_kernel(
    ctx: &mut Context,
    lhs: &HbmTensor<bf16, Chip, m![A]>,
    rhs: &HbmTensor<bf16, Chip, m![A]>,
) -> HbmTensor<bf16, Chip, m![1]> {
    // HBM → DM
    let lhs: DmTensor<bf16, Chip, Cluster, Slice, m![A]> = lhs.to_dm(&mut
ctx.tdma);
    let rhs: DmTensor<bf16, Chip, Cluster, Slice, m![A]> = rhs.to_dm(&mut
ctx.tdma);

    // Sub context: load rhs into TRF
    let rhs: TrfTensor<bf16, Chip, Cluster, Slice, Lane, m![A]> = ctx
.sub
    .begin(rhs.view())
    .fetch::<Time, m![A]>()
    .collect::<m![{ Time }, A / 16], m![A % 16]>()
    .to_trf();

    // Main context: stream lhs through the Contraction Engine, reduce along
A
    let result: DmTensor<bf16, Chip, Cluster, Slice, m![1 # 8]> = ctx
    .main
    .begin(lhs.view())
    .fetch::<Time, m![A]>()
    .collect::<m![A / 16], m![A % 16]>()
    // Pair consecutive 32-byte flits into 64-byte packets, halving time
steps (A/16 → A/32)
    .contract_outer::<m![A / 32], m![A % 32], _, _, _>(&rhs)
    .contract_packet::<m![1]>()
    .contract_time::<m![1]>()
    .contract_lane::<m![1], m![1 # 8]>(LaneMode::Interleaved)
    .cast::<bf16, m![1 # 16]>() // cast f32 accumulator output back to
bf16

    .commit_trim::<m![1 # 8]>()
    .commit();

    // DM → HBM

```

```

    result.to_hbm(&mut ctx.tdma)
}

```

Host Program and Oracle

Base-template host program and oracle: `src/dot_product.rs`.

```

use furiosa_opt_std::prelude::*;
use {{ crate_name }}::kernel::dot_product_kernel::{A, dot_product_kernel};
use rand::SeedableRng;
use rand::rngs::SmallRng;

#[tokio::main]
async fn main() {
    let mut ctx = Context::acquire();
    let mut rng = SmallRng::seed_from_u64(42);
    let lhs = HostTensor::<bf16, m![A]>::rand(&mut rng);
    let rhs = HostTensor::<bf16, m![A]>::rand(&mut rng);
    let lhs_hbm = lhs.to_hbm(&mut ctx.pdma).await;
    let rhs_hbm = rhs.to_hbm(&mut ctx.pdma).await;
    let _out_hbm = launch(dot_product_kernel, (&mut ctx, &lhs_hbm,
&rhs_hbm)).await;
    println!("Dot Product: kernel ran");
}

#[cfg(test)]
mod tests {
    use super::*;

    #[tokio::test]
    async fn matches_reference() {
        let mut ctx = Context::acquire();

        let mut rng = SmallRng::seed_from_u64(42);
        let lhs = HostTensor::<bf16, m![A]>::rand(&mut rng);
        let rhs = HostTensor::<bf16, m![A]>::rand(&mut rng);

        let lhs_hbm = lhs.to_hbm(&mut ctx.pdma).await;
        let rhs_hbm = rhs.to_hbm(&mut ctx.pdma).await;

        // Reference: sum_i lhs[i] * rhs[i] in f32, then round to bf16.
        let lhs_buf: Vec<bf16> = lhs.into_vec();
        let rhs_buf: Vec<bf16> = rhs.into_vec();
        let expected_f32: f32 = lhs_buf
            .iter()
            .zip(&rhs_buf)
            .map(|(&a, &b)| f32::from(a) * f32::from(b))
            .sum();
    }
}

```

```

    let expected = bf16::from_f32(expected_f32);

    let out_hbm = launch(dot_product_kernel, (&mut ctx, &lhs_hbm,
&rhs_hbm)).await;

    let actual_buf: Vec<bf16> = out_hbm.to_host:::<m![1]>(&mut
ctx.pdma).await.into_vec();
    if let Some(&actual) = actual_buf.first() {
        let diff = (f32::from(actual) - f32::from(expected)).abs();
        let tol = (0.02 * f32::from(expected).abs()).max(0.5);
        assert!(
            diff <= tol,
            "dot_product mismatch: expected {expected:?}, actual
{actual:?}, diff {diff} > tol {tol}"
        );
    }
}

```

GEMV

Goal

Compute $y_i = \sum_j A_{ij}x_j$. This pattern covers LLM decode.

Mapping and Data Movement

Map output dimension **I** across slices so each slice computes one row. Broadcast the full vector **x** so every slice can contract it with its row. Unlike dot product, the rows are independent but every row needs the same complete vector operand.

Device Source

Device source: `src/kernel/gemv_kernel.rs`.

```

use furiosa_opt_std::prelude::*;

axes![I = 256, J = 2048];

pub type Chip = m![1];
pub type Cluster = m![1 # 2];
pub type Slice = m![I]; // Distribute output dimension across slices

```

```

pub type Time = m![J / 32]; // Temporal iterations for reduction dimension
pub type Packet = m![J % 32]; // Packet size for reduction dimension
pub type Lane = m![1];

#[device(chip = 1)]
pub fn gemv_kernel(
    ctx: &mut Context,
    matrix: &HbmTensor<bf16, Chip, m![I, J]>,
    vector: &HbmTensor<bf16, Chip, m![J]>,
) -> HbmTensor<bf16, Chip, m![I]> {
    // Move data from HBM to DM
    let matrix: DmTensor<bf16, Chip, Cluster, Slice, m![J]> =
matrix.to_dm(&mut ctx.tdma);
    let vector: DmTensor<bf16, Chip, Cluster, Slice, m![J]> =
vector.to_dm(&mut ctx.tdma);

    // Load vector into TRF
    let vector_trf: TrfTensor<bf16, Chip, Cluster, Slice, Lane, m![J]> = ctx
        .sub
        .begin(vector.view())
        .fetch::(&vector_trf)
        .contract_packet::

```

Host Program and Oracle

Base-template host program and oracle: `src/gemv.rs`.


```

use furiosa_opt_std::prelude::*;
use {{ crate_name }}::kernel::gemv_kernel::{I, J, gemv_kernel};
use rand::SeedableRng;
use rand::rngs::SmallRng;

#[tokio::main]
async fn main() {
    let mut ctx = Context::acquire();
    let mut rng = SmallRng::seed_from_u64(42);
    let matrix = HostTensor::<bf16, m![I, J]>::rand(&mut rng);
    let vector = HostTensor::<bf16, m![J]>::rand(&mut rng);
    let matrix_hbm = matrix.to_hbm(&mut ctx.pdma).await;
    let vector_hbm = vector.to_hbm(&mut ctx.pdma).await;
    let _out_hbm = launch(gemv_kernel, (&mut ctx, &matrix_hbm,
&vector_hbm)).await;
    println!("GEMV: kernel ran");
}

#[cfg(test)]
mod tests {
    use super::*;

    #[tokio::test]
    async fn matches_reference() {
        let mut ctx = Context::acquire();

        let mut rng = SmallRng::seed_from_u64(42);
        let matrix = HostTensor::<bf16, m![I, J]>::rand(&mut rng);
        let vector = HostTensor::<bf16, m![J]>::rand(&mut rng);

        let matrix_hbm = matrix.to_hbm(&mut ctx.pdma).await;
        let vector_hbm = vector.to_hbm(&mut ctx.pdma).await;

        // Reference: y[i] = sum_j matrix[i, j] * vector[j] in f32, rounded
to bf16.
        let mat_buf: Vec<bf16> = matrix.into_vec();
        let vec_buf: Vec<bf16> = vector.into_vec();
        let expected: Vec<bf16> = mat_buf
            .chunks(J::SIZE)
            .map(|row| {
                let acc: f32 = row
                    .iter()
                    .zip(&vec_buf)
                    .map(|(&a, &b)| f32::from(a) * f32::from(b))
                    .sum();
                bf16::from_f32(acc)
            })
            .collect();

        let out_hbm = launch(gemv_kernel, (&mut ctx, &matrix_hbm,

```

```

&vector_hbm)).await;

    let actual: Vec<bf16> = out_hbm.to_host::<m![I]>(&mut
ctx.pdma).await.into_vec();
    for (i, (&e, &a)) in expected.iter().zip(&actual).enumerate() {
        let diff = (f32::from(a) - f32::from(e)).abs();
        let tol = (0.02 * f32::from(e).abs()).max(0.5);
        assert!(
            diff <= tol,
            "gemv mismatch at i={i}: expected {e:?}, actual {a:?}, diff
{diff} > tol {tol}"
        );
    }
}

```

GEMM

Goal

Compute $C_{ij} = \sum_k A_{ik} B_{kj}$. A broadcasts across J , and B broadcasts across I .

Mapping and Data Movement

Map both output dimensions with `type Slice = m![I / 32, J / 32]` so each slice computes a 16×16 output tile. The Switch Engine routes each B tile to its matching slice, so each slice receives only the J portion that belongs to its output tile.

`.contract_packet::<m![1]>()` reduces along K spatially. `.contract_time::<m![I]>()` accumulates over time while preserving I . `.contract_lane::<m![I], m![J # 8]> (LaneMode::Interleaved)` preserves I and J in the output packet.

Device Source

Device source: `src/kernel/gemm_kernel.rs`.

```

use furiosa_opt_std::prelude::*;

axes![I = 512, J = 512, K = 64];

pub type Chip = m![1];
pub type Cluster = m![1 # 2];

```

```

// Distribute output dimensions `I` and `J` across slices
pub type Slice = m![I / 32, J / 32]; // Each slice handles a 16 × 16 output
tile
pub type Lane = m![J % 8];

#[device(chip = 1)]
pub fn gemm_kernel(
    ctx: &mut Context,
    a: &HbmTensor<bf16, Chip, m![I, K]>,
    b: &HbmTensor<bf16, Chip, m![J, K]>,
) -> HbmTensor<bf16, Chip, m![I, J]> {
    // Move data from HBM to DM
    let a: DmTensor<bf16, Chip, Cluster, Slice, m![I % 32, K]> = a.to_dm(&mut
ctx.tdma);
    let b: DmTensor<bf16, Chip, Cluster, Slice, m![J % 32, K]> = b.to_dm(&mut
ctx.tdma);

    // Load matrix B into TRF
    // Switch Engine distributes B across 256 slices
    // Each slice gets the full `K` dimension but only its (16 × 16) output
tile
    // See: Switch Engine topologies for details on distribution
    let b_trf: TrfTensor<bf16, Chip, Cluster, Slice, Lane, m![J / 8 % 4, K]>
= ctx
        .sub
        .begin(b.view())
        .fetch::

```

```

    result.to_hbm(&mut ctx.tdma)
}

```

Host Program and Oracle

Base-template host program and oracle: `src/gemm.rs`.

```

use furiosa_opt_std::prelude::*;
use {{ crate_name }}::kernel::gemm_kernel::{I, J, K, gemm_kernel};
use rand::SeedableRng;
use rand::rngs::SmallRng;

#[tokio::main]
async fn main() {
    let mut ctx = Context::acquire();
    let mut rng = SmallRng::seed_from_u64(42);
    let a = HostTensor::<bf16, m![I, K]>::rand(&mut rng);
    let b = HostTensor::<bf16, m![J, K]>::rand(&mut rng);
    let a_hbm = a.to_hbm(&mut ctx.pdma).await;
    let b_hbm = b.to_hbm(&mut ctx.pdma).await;
    let _out_hbm = launch(gemm_kernel, (&mut ctx, &a_hbm, &b_hbm)).await;
    println!("GEMM: kernel ran");
}

#[cfg(test)]
mod tests {
    use super::*;

    #[tokio::test]
    async fn matches_reference() {
        let mut ctx = Context::acquire();

        let mut rng = SmallRng::seed_from_u64(42);
        let a = HostTensor::<bf16, m![I, K]>::rand(&mut rng);
        let b = HostTensor::<bf16, m![J, K]>::rand(&mut rng);

        let a_hbm = a.to_hbm(&mut ctx.pdma).await;
        let b_hbm = b.to_hbm(&mut ctx.pdma).await;

        // Reference: C[i, j] = sum_k A[i, k] * B[j, k] in f32, rounded to
bf16.
        let a_buf: Vec<bf16> = a.into_vec();
        let b_buf: Vec<bf16> = b.into_vec();
        let expected: Vec<bf16> = a_buf
            .chunks(K::SIZE)
            .flat_map(|a_row| {
                b_buf.chunks(K::SIZE).map(move |b_row| {
                    let acc: f32 = a_row

```

```

        .iter()
        .zip(b_row)
        .map(|(&a, &b)| f32::from(a) * f32::from(b))
        .sum();
        bf16::from_f32(acc)
    })
})
.collect();

let out_hbm = launch(gemm_kernel, (&mut ctx, &a_hbm, &b_hbm)).await;

let actual: Vec<bf16> = out_hbm.to_host::<m![I, J]>(&mut
ctx.pdma).await.into_vec();
for (idx, (&e, &av)) in expected.iter().zip(&actual).enumerate() {
    let diff = (f32::from(av) - f32::from(e)).abs();
    let tol = (0.05 * f32::from(e).abs()).max(1.0);
    assert!(diff <= tol, "gemm mismatch at idx={idx}: expected {e:?},
actual {av:?}");
}
}
}

```

Blocked GEMM extends GEMM with tiling for matrices that exceed on-chip DM capacity. It covers temporal partitioning over κ and spatial partitioning of I and J across chips. Flash Attention combines GEMM, Vector Engine softmax, and main/sub prefetch across a transformer attention head.

Kernel Validation

The validation sequence distinguishes structural validity, numerical correctness, hardware behavior, and static schedule data.

1. **Kernel compilation** (`cargo furiosa-opt compile`) translates and verifies every `# [device]` body, catching mapping and shape errors without executing anything.
2. **CPU runs** (plain cargo) compare host-side tensor results with a host oracle.
3. **NPU execution** runs the compiled executable device format (EDF) through the Furiosa SDK and physical hardware.
4. **Schedule data** inspects emitted schedule JSON and compares makespan with release, shapes, types, and mappings fixed.

Kernel compilation is static: it proves that every selected kernel translates and verifies, without providing numerical-correctness proof. Compare schedule makespan only when the release, shapes, data types, and mapping are fixed. A schedule is not a throughput measurement by itself. Use [Scheduling and Tuning](#) to compare fixed schedule candidates. Record the source revision, backend, shapes, mappings, oracle result, and schedule artifact for each comparison.

Mapping Tensors

A mapping is the source of truth for logical order and physical placement. Validate its cells and dimensions before selecting movement or compute operations. Every mapping expression maps a logical index to a physical location. Read the result categories defined in [Position Cell Summary](#) before selecting movement or compute operations.

Place and Reduce Dimensions

The table places axes before movement or compute operations are selected.

Dimension	Type	Defined in	Reduced in
Chip	Spatial	HBM, DM, Stream	DMA + Vector
Cluster	Spatial	DM, Stream	DMA + Vector
Slice	Spatial	DM, Stream	Vector
Lane	Spatial	TRF	Contraction
Time	Temporal	Stream	Contraction
Packet	Spatial	Stream	Contraction

Chip and Cluster reductions use DMA redistribution followed by Vector Engine reduction.

Mapping Order and Performance

A tensor has values but no intrinsic storage order. A mapping chooses that order, and hardware accesses are most efficient when adjacent values occupy contiguous buffer positions. The outermost mapping dimension is major and changes slowest. The innermost is minor and changes fastest. For a height-by-width tensor, $m![H, W]$ makes rows contiguous, while $m![W, H]$ makes columns contiguous. Each order favors locality along a different axis: an H-major mapping makes a scan along W contiguous, while a W-major mapping makes a scan along H contiguous. A tiled mapping such as $m![H / 2, W / 2, H \% 2, W \% 2]$ places small two-dimensional neighborhoods contiguously, so neither H nor W is globally the sole contiguous direction. This changes which access patterns map to nearby buffer positions, but does not inherently guarantee faster

hardware access. The additional mapping state needed to represent the split dimensions is its main extra cost.

For axes H (six rows) and W (eight columns), the same values can therefore have different physical access patterns:

$H \backslash W$	0	1	2	3	4	5	6	7
0	a	b	c	d	e	f	g	h
1	i	j	k	l	m	n	o	p
2	.	.	.	.	.	.	.	.
3	.	.	.	.	.	.	.	.
4	.	.	.	.	.	.	.	.
5	.	.	.	.	.	.	.	.

- **H-major, W-minor:** $m![H, W]$ — a scan along W is contiguous. A scan along H touches one value per cache line.

$H=0$								$H=1$				
a	b	c	d	e	f	g	h	i	j	k	l	m

- **W-major, H-minor:** $m![W, H]$ — a scan along H is contiguous. A scan along W touches one value per cache line.

W=0						W=1							
a	i	.	.	.	.	b	j	.	.	.	.	c	k

- **2×2 tiles:** $m![H / 2, W / 2, H \% 2, W \% 2]$ — H-major and W-major each sacrifice locality along one axis. Tiling places small neighborhoods contiguously along both axes, changing the locality distribution without inherently guaranteeing faster hardware access. The split dimensions require a non-trivial address formula and additional mapping state.

$t(0,0)$				$t(0,1)$				$t(0,2)$				...
a	b	i	j	c	d	k	l	e	f	m	n	...

The selected mapping also constrains later execution. Changing a representation's order after allocation generally requires copying or transposing its data, so allocation-time choices affect subsequent operations. Hardware geometry, alignment, and scheduling remain architectural constraints that compiler lowering must account for. The mapping

expresses the logical order rather than a raw address calculation. Device-specific alignment checks apply, and misaligned accesses can require read-modify-write cycles with substantial performance cost (historically observed at roughly 50× for affected DM accesses).

The mapping is the complete technical description of this choice. A physical representation is the concrete buffer, device placement, and stream decomposition produced from that mapping. The same values can move between different physical representations without changing the logical tensor.

Position Cell Summary

Every mapping expression maps a physical position to a `cell`. Identify the result category before interpreting any coordinates.

```
axes![A = 4];
type E = m![A];

assert_eq!(E::map(0), i![A: 0]);
assert_eq!(E::map(4), Cell::OutOfBounds);
```

`Cell::Index` carries live `Index` terms. `Cell::OutOfBounds` identifies a position outside the mapping. `Cell::Padding` is introduced with padding in [Mapping Expressions](#).

This section summarizes the result categories. After identifying the `cell`, use [Mapping Expressions](#) for constructors and API syntax.

Representation and Distribution

Physical Representations

Storage and stream tensors split one mapping across hardware dimensions. An HBM representation may distribute channels across chips. A DM representation may repartition those channels across slices. A stream representation may put the remaining dimensions in `Time` and `Packet`. [Spatial and Temporal Dimensions](#) follows one NCHW tensor through those representations with a concrete coordinate trace.

This representation trace closes the chapter: choose a mapping, derive each physical representation, then verify that the value at a traced `Index` reaches the expected stream cell. The formal tensor-value definition is in [Tensor Semantics](#). This chapter keeps the explanation operational.

For a minimal computation over a representation, see the [Vector Engine](#). Elementwise operations belong with that execution API, while this chapter focuses on mapping and representation.

Distribution Across Space and Time

Mapping dimensions are assigned explicitly to spatial hardware dimensions (`Chip` , `Cluster` , `Slice` , and `Packet`) or to the temporal `Time` loop. These names describe where a mapping is consumed, not a second representation vocabulary. See [Spatial and Temporal Dimensions](#) for the tensor declarations and constraints.

An assignment holds for one stage rather than for the whole pipeline, and four routes move an axis between `Time` and a spatial dimension.

Route	Moves	Available when	Cost
Switch Engine	an axis either way between <code>Slice</code> and <code>Time</code>	one of its configurations expresses the slice placement	<code>ring_size × Time::SIZE × flits_per_packet</code> cycles of ring traversal
DMA	an axis into <code>Chip</code> , <code>Cluster</code> or <code>Slice</code> , by writing the placement it is given	always	a transfer of the whole tensor
Inter-slice reducer	an axis out of <code>Time</code> into the <code>Slice</code> slot a reduced axis vacates	the stream is reduced over a <code>Slice</code> axis anyway	none beyond that reduction
Axis lifting, at fetch	an axis out of <code>Time</code> onto <code>Chip</code> , <code>Cluster</code> or <code>Slice</code>	the selected dimension holds a broadcast	the original read, plus a DMA, a context-local add, and an SFR store for <code>Slice</code>

The Switch Engine is the general route across the `slice / Time` boundary because it can deliver a value to a slice that does not already contain it. The Switch Engine operates only across slices, so moving an axis onto `chip` or `cluster` requires a DMA that writes the desired placement.

Axis lifting requires a broadcast on the selected dimension, meaning each chip, cluster, or slice already holds the source region. Different read offsets then move the axis out of `Time` without a data transfer.

Declarative Mapping Context

Declarative mappings state order in terms of logical axes instead of raw strides or offsets. The H×W examples above use `m! [H, W]` for H-major order, `m! [W, H]` for W-major order, and `m! [H / 2, W / 2, H % 2, W % 2]` for 2×2 tiles. The first two tile dimensions identify a tile and the last two identify its position within that tile.

Mapping expressions can be normalized to a standard representation, which makes equivalent constructor forms easier to inspect. The tested normalization properties and their limits are documented with [Equivalent Mappings](#). This chapter does not claim a universal proof or unique representation.

Mapping Expressions

A mapping expression is a Rust type that encodes a mapping. Read a position's `cell` first; then use the `Index` terms, padding kind, or out-of-bounds result to debug the access. This section defines the constructors and supported behavior that produce those results.

Axis Sizes

The `axes!` macro declares axis identifiers and their sizes. The following declaration applies throughout this section:

```
axes![A = 8, B = 512];
```

Position Results

A mapping expression like `m![H, W]` maps each physical position to a `cell`. See the result before reading its terms:

```
axes![A = 8];
type E = m![A];
assert_eq!(E::map(0), i![A: 0]);
```

`Cell::Index` is a structural live result: it preserves the mapping terms, while `Index::finalize()` validates their scalar coordinates and can return an error for a modulo overshoot. Padding and out-of-bounds positions do not contain live terms. `Cell::Index(Index::new())` is a valid live result with no axis terms; it is distinct from `Cell::Padding` and `Cell::OutOfBounds`, so an empty `Index` never stands for a non-live position. The `i!` macro combines assignments in the order written, which is the major-to-minor order for a composed mapping; because `Cell::combine` is non-commutative for padding, reorder assignments only when that order is intended.

M Constructor Rules

Every mapping expression implements the `M` trait, which provides the buffer size and position mapping:

```
// Inside `furiosa_opt_std::prelude`...
pub trait M: Debug + Clone {
    /// The computed size for the given shape.
    const SIZE: usize;

    /// Converts the mapping expression type into a value.
    fn to_value() -> Mapping;

    /// Converts a buffer index to a physical cell result.
    fn map(i: usize) -> Cell;
}

/// A live tensor index: a map from axis identifiers to coordinate values.
pub struct Index { /* ... */ }

/// Results for live, padded, and out-of-range positions.
pub enum Cell { Index(Index), Padding(PaddingKind), OutOfBounds }

/// Constructs classified cells.
/// `i![A: 2, B: 3]` creates a live `Cell::Index` with A = 2 and B = 3;
/// non-live contributions remain `Cell::Padding` or `Cell::OutOfBounds`.
macro_rules! i {
    /* ... */
}
```

Host Tensor Usage

The simplest concrete type built on the `M` trait is `HostTensor<D, E>`: a host memory buffer of element type `D` whose representation is fully determined by mapping `E`. `E` determines both the buffer size (`E::SIZE`) and the correspondence from buffer positions to cells (`E::map`). `HostTensor<bf16, m![A, B]>` contains 4,096 elements of `bf16` data. For a host tensor, each `Cell::Index(index)` returned by `E::map` identifies the structural `Index` stored at that physical position. Call `Index::finalize()` to validate its scalar coordinates before treating it as a tensor index; an overshoot can fail there. Convert `Cell::OutOfBounds` with `CellExt::finalize(PaddingKind::Top)` for reads or `CellExt::finalize(PaddingKind::Bottom)` for writes. The formal definition of *holds* appears in [Tensor Semantics](#).

Device tensors such as `HbmTensor` and `DmTensor` have more complex representations spanning multiple mapping expressions; see [Spatial and Temporal Dimensions](#) for details.

Constructors

Mapping expressions, including the mapping `E` in `HostTensor<D, E>`, are built by composing small constructors, each of which transforms or combines simpler mappings. These expressions use arithmetic-like operators (`/`, `%`, and `#` for padding) to concisely define the mapping between tensor and linear buffer indices.

Symbol

A symbol is a single uppercase letter whose size comes from the shape declaration. The mapping `m![A]` maps 8 buffer indices linearly to tensor indices along the axis:

```
type E = m![A]; // Symbol<Ident::A, 8>

fn test_symbol() {
    assert_eq!(E::map(0), i![A: 0]);
    assert_eq!(E::map(1), i![A: 1]);
    assert_eq!(E::map(2), i![A: 2]);
    for i in 0..E::SIZE {
        assert_eq!(E::map(i), i![A: i]);
    }
    assert_eq!(E::map(E::SIZE), Cell::OutOfBounds);
}
```

```

impl<S: AxisName> M for Symbol<S> {
    const SIZE: usize = S::SIZE;

    fn to_value() -> Mapping {
        Mapping::Symbol {
            symbol: S::NAME,
            size: S::SIZE,
        }
    }

    fn map(i: usize) -> Cell {
        if i >= S::SIZE {
            return Cell::OutOfBounds;
        }
        Cell::Index({
            let mut index = Index::new();
            index.add_term(
                Term {
                    inner: Atom::Symbol {
                        symbol: S::NAME,
                        size: S::SIZE,
                    },
                    stride: 1,
                    modulo: S::SIZE,
                },
                i,
            );
            index
        })
    }
}

```

Note

For every symbol A , the zeroth index $i![A: 0]$ is equivalent to the empty tensor index $i![]$.

Pair

The pair mapping $m![A, B]$ stores a 2D tensor with shape $\{A = 8, B = 512\}$ as a buffer of 4,096 elements. The mapping $\text{Pair}\langle L, R \rangle$ maps the Cartesian product of two spaces into a linear buffer where L is the major dimension and R is the minor dimension. The size is $L::SIZE * R::SIZE$, and the mapping uses floor division and modulo to decompose indices. If the major (L) result is padding, its kind wins before the

minor (R) result is considered; an out-of-bounds result propagates when no live combination is possible. `m![A, B, C, D]` expands to `Pair<A, Pair<B, Pair<C, D>>>` and is right-associative.

```
type E = m![A, B]; // Pair<m![A], m![B]>

fn test_pair() {
    // First 512 elements hold A=0, next 512 hold A=1
    assert_eq!(E::map(0), i![A: 0, B: 0]);
    assert_eq!(E::map(511), i![A: 0, B: 511]);
    assert_eq!(E::map(512), i![A: 1, B: 0]);
    assert_eq!(E::map(519), i![A: 1, B: 7]); // 519 == 512 * 1 + 7
    for i in 0..E::SIZE {
        assert_eq!(E::map(i), i![A: i / <m![B]>::SIZE, B: i % <m!
[B]>::SIZE]);
    }
    assert_eq!(E::map(E::SIZE), Cell::OutOfBounds);
}
```



```

impl<L, R> M for Pair<L, R>
where
    L: M,
    R: M,
{
    const SIZE: usize = L::SIZE * R::SIZE;

    fn to_value() -> Mapping {
        Mapping::Pair {
            left: RBox::new(L::to_value()),
            right: RBox::new(R::to_value()),
        }
    }

    fn map(i: usize) -> Cell {
        if i >= Self::SIZE {
            return Cell::OutOfBounds;
        }
        let l = L::map(i / R::SIZE);
        let r = R::map(i % R::SIZE);
        match l {
            Cell::OutOfBounds => Cell::OutOfBounds,
            Cell::Padding(kind) => Cell::Padding(kind),
            Cell::Index(mut left) => match r {
                Cell::Index(right) => {
                    left.add(right);
                    Cell::Index(left)
                }
                Cell::Padding(kind) => Cell::Padding(kind),
                Cell::OutOfBounds => Cell::OutOfBounds,
            },
        }
    }
}

```

Identity

The identity mapping `m![1]` creates a single-element buffer that maps buffer index `0` to the empty tensor index `i![]`. It serves as the identity element for `Pair: m![1, A]` and `m![A, 1]` are both equivalent to `m![A]`. More generally, a broadcast mapping returns the same live empty `Cell::Index(Index::new())` for every position below its size, and returns `Cell::OutOfBounds` beyond that size.

```

type E = m![1]; // Identity

fn test_identity() {
    assert_eq!(E::map(0), i![]);
    assert_eq!(E::map(1), Cell::OutOfBounds);
}

/// The identity mapping (size-1 broadcast), the unit written `m![1]`.
pub type Identity = Broadcast<1>;

```

Padding

Padding aligns data to hardware requirements by adding unused buffer space. For example, the DMA engine requires rows to start on 64-byte boundaries. With `axes![C = 13, D = 61]`, `m![C, D]` creates misaligned rows since 61 is not divisible by 64. `m![C, D # 64]` fixes this by aligning each row to 64-byte boundaries, using 3 extra elements per row.

```

axes![C = 13, D = 61];

type E = m![C, D # 64]; // Pair<m![C], Padding<m![D], 64>>

fn test_padding() {
    assert_eq!(E::map(0), i![C: 0, D: 0]);
    assert_eq!(E::map(60), i![C: 0, D: 60]);
    assert_eq!(E::map(61), Cell::Padding(PaddingKind::Top));
    assert_eq!(E::map(62), Cell::Padding(PaddingKind::Top));
    assert_eq!(E::map(63), Cell::Padding(PaddingKind::Top));
    assert_eq!(E::map(64), i![C: 1, D: 0]);
}

```

```

impl<L, const SIZE: usize, const KIND: PaddingKind> M for Padding<L, SIZE,
KIND>
where
    L: M,
{
    const SIZE: usize = SIZE;

    fn to_value() -> Mapping {
        Mapping::Padding {
            inner: RBox::new(L::to_value()),
            padding: SIZE,
            kind: KIND,
        }
    }

    fn map(i: usize) -> Cell {
        if i >= SIZE {
            Cell::OutOfBounds
        } else if i >= L::SIZE {
            Cell::Padding(KIND)
        } else {
            L::map(i)
        }
    }
}

```

The padded positions' content is part of the type, not just their count. Three kinds are tracked.

- $m![A \# m]$ (or $m![A \# \{*\} m]$) is top padding to size m . These positions are accessible but hold arbitrary values. Raw DM tensors carry this. $\#$ is the shorthand; $\{*\}$ spells the kind out explicitly.
- $m![A \#\{0\} m]$ is zero-filled padding to size m . These positions are accessible and known to hold zero. The mapping records that value property; the expression itself does not perform the fill.
- $m![A \#\{!\} m]$ is bottom padding to size m . These positions are inaccessible and reads/writes are undefined behavior. This models addresses the compiler must avoid.

$\#$ defaults to top kind. The Rust type level mirrors this via a const generic of `PaddingKind ON Padding<L, SIZE, KIND>`. `Padding<L, N>` is `KIND = PaddingKind::Top`, `Padding<L, N, { PaddingKind::Zero }>` is the zero-filled variant, and `Padding<L, N, { PaddingKind::Bottom }>` is inaccessible.

`Cell::Padding(kind)` reports a position inside the padded extent but outside the inner

mapping. The `kind` is `Top`, `Zero`, or `Bottom`; positions beyond the padded extent remain `Cell::OutOfBounds`. The complete `cell` result definition appears in [Position Results](#).

Resize

`Resize` constrains a mapping to a smaller logical size by truncating indices beyond the new size, discarding elements outside that range. Unlike padding, which expands the buffer, `Resize` shrinks the logical view. The mapping `m![D = 2]` takes only the first 2 elements of axis `D`, producing indices `D = 0` and `D = 1`.

```
axes![C = 2, D = 3];
type E = m![C, D = 2]; // Pair<m![C], Resize<m![D], 2>>

fn test_resize() {
    assert_eq!(E::map(0), i![C: 0, D: 0]);
    assert_eq!(E::map(1), i![C: 0, D: 1]);
    assert_eq!(E::map(2), i![C: 1, D: 0]);
    assert_eq!(E::map(3), i![C: 1, D: 1]);
    assert_eq!(E::map(4), Cell::OutOfBounds);
}

impl<L, const SIZE: usize> M for Resize<L, SIZE>
where
    L: M,
{
    const SIZE: usize = SIZE;

    fn to_value() -> Mapping {
        Mapping::Resize {
            inner: RBox::new(L::to_value()),
            resize: SIZE,
        }
    }

    fn map(i: usize) -> Cell {
        if i < SIZE { L::map(i) } else { Cell::OutOfBounds }
    }
}
```

Tiling

Tiling is implemented through indexed views: `tile` validates a mapping split, then creates a metadata view without copying data. The generic view API is

`TensorView::tile<I, E2, LEN>(start)`. `I` is the mapping used to locate the tile, `E2` is the requested view mapping, `LEN` is the number of `I` cells in the tile, and `start` is the logical starting coordinate along `I`.

```
let tensor = HbmTensor::<bf16, m![1], m![A, B]>::new();
let view = tensor.view(); // HbmTensorView::<'_, bf16, m![1], m![A, B]>
let tile01 = view.tile::<m![B], 2, m![A, B = 2 # 512]>(0); // HbmTensorView::
<'_, bf16, m![1], m![A, B = 2 # 512]>
let tile23 = view.tile::<m![B], 2, m![A, B = 2 # 512]>(2); // HbmTensorView::
<'_, bf16, m![1], m![A, B = 2 # 512]>
```

The HBM example uses `I = m![B]`, `LEN = 2`, and `E2 = m![A, B = 2 # 512]`. The `B = 2 # 512` mapping gives the view two live `B` positions inside a 512-cell physical extent; without that footprint, the split does not validate against the source mapping. `start = 0` and `start = 2` are logical `B` coordinates, so these views cover ranges `0..2` and `2..4`. They are starts, not a separate tile-number API.

Read views fill cells outside the tile with `PaddingKind::Top`; mutable views require `PaddingKind::Bottom` there so writes cannot escape the tile. HBM and DM expose tier-specific tile wrappers, while TRF, VRF, and DPE use the generic view; `HostTensor` has no view/tile API.

Stride and Modulo

Stride (`/`) and modulo (`%`) decompose a single dimension into two: the outer (block index) and the inner (position within block). Consider the 512-element axis `B` divided into 8 blocks of 64 elements each. The mapping `m![B / 64, B % 64]` creates an 8×64 grid where the first dimension selects which block and the second dimension selects the position within that block:

```

type D1 = m![B / 64]; // stride with size 8
type D2 = m![B % 64]; // modulo with size 64

type E = m![B / 64, B % 64]; // equivalent to `m![B]`

fn test_stride_modulo() {
    assert_eq!(E::map(130), i![B / 64: 2, B % 64: 2]); // block 2, position
2: B = 64*2 + 2 = 130
    assert_eq!(E::map(130), <m![B]>::map(130)); // same result
as flat m![B]

    for i in 0..8 {
        assert_eq!(D1::map(i), i![B / 64: i]);
    }
    assert_eq!(D1::map(8), Cell::OutOfBounds);

    for j in 0..64 {
        assert_eq!(D2::map(j), i![B % 64: j]);
    }
    assert_eq!(D2::map(64), Cell::OutOfBounds);

    for i in 0..8 {
        for j in 0..64 {
            assert_eq!(
                E::map(64 * i + j),
                <m![B]>::map(64 * i + j),
            );
        }
    }
    assert_eq!(E::map(512), Cell::OutOfBounds);
}

impl<L, const SIZE: usize> M for Stride<L, SIZE>
where
    L: M,
{
    const SIZE: usize = {
        assert!(L::SIZE % SIZE == 0, "Stride size must divide the original
size");
        L::SIZE / SIZE
    };

    fn to_value() -> Mapping {
        Mapping::Stride {
            inner: RBox::new(L::to_value()),
            stride: SIZE,
        }
    }
}

```

```

fn map(i: usize) -> Cell {
    if i < Self::SIZE {
        L::map(i * SIZE)
    } else {
        Cell::OutOfBounds
    }
}

impl<L, const SIZE: usize> M for Modulo<L, SIZE>
where
    L: M,
{
    const SIZE: usize = {
        assert!(L::SIZE % SIZE == 0, "Modulo size must divide the original
size");
        SIZE
    };

    fn to_value() -> Mapping {
        Mapping::Modulo {
            inner: RBox::new(L::to_value()),
            modulo: SIZE,
        }
    }

    fn map(i: usize) -> Cell {
        if i < Self::SIZE {
            L::map(i % L::SIZE)
        } else {
            Cell::OutOfBounds
        }
    }
}

```

Stride and modulo mappings can be visualized in tabular form. Consider the mapping $m!$ $[B / 4, B \% 4]$ with $B::SIZE = 16$. The following table shows how buffer indices are arranged: each row corresponds to a specific index of $B / 4$ (the stride axis), and each column corresponds to an index of $B \% 4$ (the modulo axis):

	$i![B \% 4: 0]$	$i![B \% 4: 1]$	$i![B \% 4: 2]$	$i![B \% 4: 3]$
$i![B / 4: 0]$	$i![B: 0]$	$i![B: 1]$	$i![B: 2]$	$i![B: 3]$
$i![B / 4: 1]$	$i![B: 4]$	$i![B: 5]$	$i![B: 6]$	$i![B: 7]$

<code>i![B / 4: 2]</code>	<code>i![B: 8]</code>	<code>i![B: 9]</code>	<code>i![B: 10]</code>	<code>i![B: 11]</code>
<code>i![B / 4: 3]</code>	<code>i![B: 12]</code>	<code>i![B: 13]</code>	<code>i![B: 14]</code>	<code>i![B: 15]</code>

Modulo differs from `resize` in how it handles buffer size:

- `Resize` shrinks the buffer by truncating indices beyond the new size.
- `Modulo` preserves the original buffer size while partitioning it into equal-sized blocks.

These operations can be nested for complex decompositions. The following example splits `B` into three dimensions where the buffer's bit arrangement differs from that of the tensor index.

```
// B's bits: 6 - 8, 0 - 4, 5
// Values: 0 - 7, 0 - 31, 0 - 1
type E = m![B / 64, B % 32, B / 32 % 2];

fn test_nested_stride() {
    assert_eq!(E::map(67), i![B: 97]); // 67 = 64*1 + 2*1 + 1 (i=1,j=1,k=1) →
    B = 64*1 + 1 + 32*1 = 97
    // Verify B=97 round-trips: 97/64=1, 97%32=1, (97/32)%2=1
    assert_eq!(97 / 64, 1);
    assert_eq!(97 % 32, 1);
    assert_eq!((97 / 32) % 2, 1);

    // buffer index: 64 * i + 2 * j + k (i = block, j = position within
    // block, k = sub-block)
    // tensor index B: 64 * i + j + 32 * k (rearranges bit positions)
    for i in 0..8 {
        for j in 0..32 {
            for k in 0..2 {
                assert_eq!(
                    E::map(64 * i + 2 * j + k),
                    i![B: 64 * i + j + 32 * k],
                );
            }
        }
    }
    assert_eq!(E::map(512), Cell::OutOfBounds);
}
```

This kind of bit rearrangement maps naturally to hardware representations where address bits are reordered for bank interleaving or cache efficiency. In binary, this rearranges bit positions: buffer `001_00001_1` becomes `B = 001_1_00001`. The buffer

groups bits as `[8:6]_[5:1]_[0]` while `B` groups them as `[8:6]_[5]_[4:0]`.

Tiling can operate on blocks rather than individual elements. The following example tiles by block using `m![B / 32]` and creates overlapping tiles:

```
let tensor = HbmTensor::<bf16, m![1], m![A, B]>::new();
for i in 0..15 {
    let tile = tensor.view().tile::<m![B / 32], 2, m![A, B / 32 = 2 # 16, B % 32]>(i);
}
```

With `B = 512`, the dimension `B / 32` has 16 blocks numbered 0-15. Each tile takes 2 consecutive blocks starting at index `i`. Tile 0 covers blocks `{0, 1}`, tile 1 covers blocks `{1, 2}`, and so on through tile 14 covering blocks `{14, 15}`. These tiles overlap because consecutive tiles share one block.

The tile mapping `B / 32 = 2` resizes the block dimension to 2 since each tile contains exactly 2 blocks. When tiling with a single block, `B / 32 = 1` simplifies to the identity `m![1]` since the dimension has only one value.

Escape

For complex mappings, define type aliases and reference them using `{ ... }`. With separate mappings `L = m![A]` and `R = m![B]`, combining them as `m![{ L }, { R }]` produces the same result as `m![A, B]`:

```
type L = m![A];
type R = m![B];
type E = m![{ L }, { R }]; // equivalent to `m![A, B]`

fn test_escape() {
    for i in 0..E::SIZE {
        assert_eq!(E::map(i), <m![A, B]>::map(i));
    }
}
```

This escape syntax breaks down complex mappings into named, reusable components.

Equivalent Mappings

Different constructor combinations can produce the same position behavior. This behavioral relation is distinct from structural equality of Rust mapping values. Mappings E_1 and E_2 are *equivalent* when:

- $E_1::\text{SIZE} == E_2::\text{SIZE}$, and
- For every i , $E_1::\text{map}(i) == E_2::\text{map}(i)$.

The equivalence relation is reflexive, symmetric, and transitive. The identities below describe useful behavior; they do not assert that normalization gives a unique structural representative for every equivalent input.

Normalization has a narrower tested behavior: applying it twice is idempotent; generated mapping corpora preserve their tested position and padding behavior; and normalized mappings round-trip through the mapping representation. These tests do not prove equivalence, guarantee a unique representation, or establish universal semantic preservation. The following identities capture common equivalences:

- **Identity of pairs:** for every E , E is equivalent both to $m![\{E\}, 1]$ and $m![1, \{E\}]$.
- **Stride-modulo decomposition:** for every E whose size $E::\text{SIZE}$ is divisible by n , E and $m![\{E\} / n, \{E\} \% n]$ are equivalent.
- **Pair projection:** for every A and B , $m![\{A\}, \{B\}] / B::\text{SIZE}$ is equivalent to $m![A]$ and $m![\{A\}, \{B\}] \% B::\text{SIZE}$ is equivalent to $m![B]$.
- **Associativity of pairs:** for every E_1, E_2, E_3 , $m![\{E_1\}, \{E_2\}, \{E_3\}]$, $m![\{E_1\}, \{E_2\}], \{E_3\}]$, and $m![\{E_1\}, [\{E_2\}, \{E_3\}]]$ are equivalent.
- **Idempotent operations:** for every E , E is equivalent to $m![\{E\} / 1]$, to $m![\{E\} \# E::\text{SIZE}]$, and to $m![\{E\} = E::\text{SIZE}]$.
- **Modulo by 1:** for every E , $m![E \% 1]$ is equivalent to the identity mapping $m![1]$.

Spatial and Temporal Dimensions

`HostTensor<D, E>` uses a single mapping to fully capture its physical representation. Device tensors split their mapping across multiple dedicated dimensions:

- **Spatial dimensions:** `Chip`, `Cluster`, and `Slice` distribute data across the hardware hierarchy. In stream tensors, `Packet` additionally sizes parallel delivery within each temporal iteration.
- **Temporal dimension:** `Time` sequences the delivery iterations in stream tensors.

Spatial Dimensions

Each spatial level in the hardware hierarchy gets its own type parameter in the tensor type, enabling spatial parallelism. All units at each level are assumed to share the same mapping.

The notation below names physical dimensions but is not a public type or constructor. The storage modules document the public tensor APIs and views.

```
// Assumed throughout this page.
axes![A = 8, B = 512];

// HBM tensors
struct HbmTensor<D: Scalar, Chip: M, Element: M> {
    /* ... */
}

// SRAM tensors
// DM (Data Memory), TRF (Tensor Register File), and VRF (Vector Register File)
struct DmTensor<D: Scalar, Chip: M, Cluster: M, Slice: M, Element: M> {
    /* ... */
}
struct TrfTensor<D: Scalar, Chip: M, Cluster: M, Slice: M, Lane: M, Element: M> {
    /* ... */
}
struct VrfTensor<D: Scalar, Chip: M, Cluster: M, Slice: M, Element: M> {
    /* ... */
}
```

HBM tensors distribute data across chips for spatial parallelism: each chip processes its own portion of the data simultaneously. For example, `HbmTensor<bf16, m![A], m![B]>` distributes $8 \times 512 = 4096$ elements across 8 chips with 512 elements per chip. The i -th chip's j -th element stores tensor index `i![A: i, B: j]`.

SRAM tensor types add `cluster` and `slice` dimensions for finer-grained parallelism. `TrfTensor` additionally has a `Lane` dimension that distributes TRF data across the 8 lanes per slice. See [Contraction Engine](#) for details.

HBM tensors carry concrete addresses. DM, TRF, and VRF tensors may receive addresses from the backend. Host tensors remain host-side values. Mapping parameters determine physical representation; storage owns addresses separately.

Constraints

- **Chip, Cluster, and Slice size:** they must exactly match the hardware counts:

Unit	Count	Constraint	Padding Example
Chip	System-dependent	<code>Chip::SIZE == NUM_CHIPS</code>	<code>m![1 # NUM_CHIPS]</code>
Cluster	2 / Chip	<code>Cluster::SIZE == 2</code>	<code>m![1 # 2]</code>
Slice	256 / Cluster	<code>Slice::SIZE == 256</code>	<code>m![X / N # 256]</code>

Any dimension can be padded with `#` when the kernel uses fewer units than the hardware provides. For example, type `Cluster = m![1 # 2]` uses 1 active cluster and 1 padding-only cluster, satisfying the hardware's 2-cluster-per-chip requirement.

Note

The runtime operates at chip granularity (`#[device(chip = N)]`), so partial chip or cluster usage is not yet supported. This may be relaxed in future releases.

- **Element size:** `Element::SIZE * size_of::<D>()` must not exceed the per-unit SRAM capacity, which varies by tensor type:

Type	Unit	Constraint
DmTensor	512KB / Slice	<code>Element::SIZE * size_of::<D>() <= 512KB</code>
TrfTensor	8KB / Lane	<code>Lane::SIZE <= 8, Element::SIZE * size_of::<D>() <= 8KB</code>
VrfTensor	8KB / Slice	<code>Element::SIZE * size_of::<D>() <= 8KB</code>

- **Element alignment:** Device storage APIs impose alignment constraints on addresses; consult the tier-specific constructor and backend checks rather than assuming one address rule for every tensor.

Temporal Dimension

`TuTensor` represents tensor data flowing through the [Tensor Unit](#) as a stream. It retains the same `Chip`, `Cluster`, and `Slice` dimensions as the SRAM types, and adds `Time` and `Packet` for streaming. `Time` is the temporal dimension: it sequences the delivery iterations. Unlike the spatial dimensions, `Time` has no hardware-imposed size limit and grows with the amount of data to process. `Packet` is an additional spatial dimension that determines how many elements each slice receives per temporal iteration.

```

axes![N = 4, C = 64, H = 32, W = 32];

/// Pipeline stage.
/// `Vector` is intentionally absent: the Vector Engine uses a separate
typestate
/// (`VectorBranchTensor` and friends) that tracks branch, ALU, and other
Vector-specific state.
/// `Commit` is intentionally absent: once the Commit Engine writes results
back to DM,
/// the data is at rest and the type becomes `DmTensor`, not `TuTensor`.
enum Position {
    Begin,          // After the start of the pipeline
    Fetch,          // After the Fetch Engine
    Switch,          // After the Switch Engine
    Collect,         // After the Collect Engine
    Contraction,     // After the Contraction Engine
    Reduce,          // After the Reduce Engine
    Cast,            // After the Cast Engine
    Transpose,       // After the Transpose Engine
}

struct TuTensor<
    'l,                // Lifetime tied to the Tensor Unit context
    const P: Position,
    D: Scalar,
    Chip: M,
    Cluster: M,
    Slice: M,
    Time: M,
    Packet: M,
> {
    /* ... */
}

type T<'l> = TuTensor<
    'l,
    { Position::Fetch }, // Fetch Engine's output
    bf16,
    m![1],               // Chip: single chip
    m![1],               // Cluster: single cluster
    m![C / 2],           // Slice: distribute 64 channels across 32 slices
    m![N, H, W],         // Time: iterate over batch (N) and spatial (H, W)
dimensions
    m![C % 2],           // Packet: 2 channels per cycle
>;

```

Type `T` streams a tensor with an aggregate shape of

$\{N = 4, C = 64, H = 32, W = 32\}$ across 32 slices (`Slice::SIZE = m![C / 2]::SIZE`)

= 32). The `Time` dimension (`m![N, H, W]`) has size $4 * 32 * 32 = 4096$, which means there are 4,096 temporal iterations. For each temporal iteration, the `Packet` dimension `m![C % 2]` delivers 2 channels to each slice. Since 32 slices operate in parallel, each temporal iteration processes $32 * 2 = 64$ channels total.

NCHW Representation Trace

The NCHW example makes the representation changes traceable from an HBM representation that distributes batches across four chips:

```
type Hbm = HbmTensor<bf16, m![N], m![C, H, W]>;
```

Move it to DM with one active chip and one element slice per pair of channels:

```
type Dm = DmTensor<bf16, m![1], m![1], m![C / 2], m![N, H, W, C % 2]>;
```

The stream type `τ` above uses the same channel split, but places `N, H, W` in `Time` and the two channels of each slice in `Packet`.

Trace Index (`N=1, C=17, H=3, W=5`):

- HBM selects chip 1; its element offset is $17 * 32 * 32 + 3 * 32 + 5 = 17,509$.
- DM selects slice $17 / 2 = 8$; the element mapping selects `N=1, H=3, W=5` and packet position $17 \% 2 = 1$.
- The stream selects `Time = 1 * 32 * 32 + 3 * 32 + 5 = 1,125` and `Packet = 1` on slice 8.

The value changes physical coordinates at each stage, but its `Index` remains (1, 17, 3, 5). This traces the mapping-to-representation path. [Tensor Semantics](#) defines what it means for a tensor to hold values.

Tensor Semantics

Tensors reside in HBM, on-chip DM, or the pipeline stream, and operations transform them. This chapter defines their mathematical meaning: what it means for a tensor variable to *hold* a mathematical tensor, and what it means for an operation to *specify* a mathematical function. These definitions enable tensor-level reasoning about vISA programs: a function is correct when its output holds the right mathematical tensor, regardless of which mapping or memory tier is used.

Tensor Holding Semantics

A tensor variable *holds* mathematical tensor T when each element stores the value of T at the tensor index formed by summing the partial indices produced by each dimension's mapping.

`HostTensor<D, E>` is the simplest case: a single mapping `E` fully determines the correspondence between buffer positions and tensor indices. `HostTensor<bf16, m![A, B]>` with `A = 8` and `B = 512`, for instance, stores 4,096 `bf16` elements in A-major, B-minor order. It holds tensor T when:

- for every buffer index `i` where `E::map(i) = Cell::Index(ti)`,
- the `i`-th element stores the value of T at `ti`.

`HbmTensor<D, Chip, Element>` extends this by splitting the single mapping into two: `Chip` maps chip indices to partial tensor indices, and `Element` maps per-chip element indices to the remaining partial indices, with each covering a disjoint subset of axes so their sum recovers the full tensor index. It holds T when:

- for every chip index `i` and element index `j` where `Chip::map(i) = Cell::Index(ti)` and `Element::map(j) = Cell::Index(tj)`,
- the `i`-th chip's `j`-th element stores T at the index `ti + tj`.

All other tensor types apply the same rule to more dimensions: each element stores T at the sum of the partial indices returned by all its mapping parameters.

Linear Combination Semantics

Linear combination expressions $\$(e1:n1, \dots, ed:nd)$ combine multiple dimensions with specified strides. Their size is $size_S(\$(e1:n1, \dots, ed:nd)) = 1 + \sum_k ((size_S(ek) - 1) * nk)$. The mapping $S, \$(e1:n1, \dots, ed:nd) \vdash si \sim ti$ is valid if there exist $si1...sid, ti1...tid$ such that for every $k, S, ek \vdash sik \sim tik, si = \sum_k (sik * nk)$, and $ti = \sum_k (tik * nk)$.

Linear combinations can encode outer sum: $e1 * e2$ is equivalent to $\$(e1 : size_S(e2), e2 : 1)$. Outer sum is preferred when axis reordering matters because changing $e1 * e2$ to $e2 * e1$ does not require manual stride updates.

Sliding operations access overlapping data blocks. Consider a buffer of 9 elements representing a tensor with shape $\{N = 5, F = 3\}$, where each row is a 3-element slice that slides one element at a time. The tensor element at N, F maps to buffer index $N + 2F$:

	$F = 0$	$F = 1$	$F = 2$
$N = 0$	0	2	4
$N = 1$	1	3	5
$N = 2$	2	4	6
$N = 3$	3	5	7
$N = 4$	4	6	8

In this sliding pattern, a single space index can map to multiple tensor indices. For example, space index 4 maps to $\{4_N\}$, $\{2_N, 1_F\}$, and $\{2_F\}$ simultaneously, illustrating the non-one-to-one nature of $(S, e).maps(si, ti)$. The linear combination uses stride 1 for N and stride 2 for F , yielding $1 + (5-1)*1 + (3-1)*2 = 9$.

Function Specification

Specifying a function means declaring what its output holds in terms of its inputs. For example, the function `elementwise_add` specifies the mathematical operation $f(T_1, T_2) = T_1 + T_2$ in that:

- For every tensor T_1 and T_2 ,
- if `lhs` holds T_1 and `rhs` holds T_2 ,
- then the return value holds $T_1 + T_2$.

```
axes![A = 8, B = 512];

// This signature specifies the mathematical contract; the engine pipeline
// implementation is shown in the Computing Tensors chapter.
fn elementwise_add(
    lhs: &HbmTensor<bf16, m![A], m![B]>,
    rhs: &HbmTensor<bf16, m![A], m![B]>,
) -> HbmTensor<bf16, m![A], m![B]> {
    // The implementation is intentionally omitted; this example states only
    // the function's tensor-level contract.
}
```

A *mathematical tensor move* specifies $f(T) = T$: the output holds the same mathematical tensor as the input, regardless of representation. `.to_dm()` is a mathematical tensor move. The `.to_dm()` method, for instance, specifies $f(T) = T$ in that:

- if `hbm` holds T ,
- the return value holds T .

```
axes![A = 8, B = 512];

fn hbm_to_dm(
    ctx: &mut Context,
    hbm: &HbmTensor<bf16, m![A], m![B]>,
) -> DmTensor<bf16, m![A], m![1], m![B / 2], m![B % 2]> {
    hbm.to_dm(&mut ctx.tdma)
}
```

Moving Tensors

Furiosa-opt kernels are Rust functions compiled for the Furiosa NPU. This chapter explains how a kernel transfers tensor data between host memory, HBM, DM, and the Tensor Unit. The [Quick Start kernel design](#) introduces these tiers in a complete kernel. Choose the source and destination in the movement table before reading the worked transfer. The [Sequencer](#) describes the common loop model. The engine pages document each concrete boundary.

Choose a movement route

The movement table selects operations from the current value to its destination. It gives the operation and context for each public boundary.

Current value	Destination	Operation and context
HostTensor	HbmTensor	HostTensor::to_hbm with Context::pdma (async); see DMA Engine
HbmTensor	HostTensor	HbmTensor::to_host with Context::pdma (async); see DMA Engine
HbmTensor	DmTensor	HbmTensor::to_dm with Context::tdma ; see DMA Engine
DmTensor	HbmTensor	DmTensor::to_hbm with Context::tdma ; see DMA Engine
HbmTensor	HbmTensor	HbmTensor::to_hbm with the DMA context required by the call; see DMA Engine
DmTensor	DmTensor	DmTensor::to_dm with Context::tdma , or a fetch/commit round trip through the Tensor Unit; see DMA Engine and Commit Engine
DmTensorView	Tensor Unit stream	TuContext::begin , then fetch
Tensor Unit stream	DmTensor	commit , or commit_view for an existing mutable view

Copyable end-to-end transfer

This page defines movement boundaries, context ownership, and lifetime rules. The worked transfer widens and reorders an HBM tensor before returning it to HBM. The device function stages HBM into DM, fetches a stream, casts and collects it, commits to DM, and transfers the result back to HBM. The host test awaits only host-to-HBM and HBM-to-host I/O and checks the reordered result.

Read and change the worked transfer in this order:

Step	Declared choice
Input	<code>A = 4096 , B = 8 , input element type i8 , and input HBM mapping m![1] , m![A, B] .</code>
HBM→DM	<code>The DM Cluster is m![1 # 2] , Slice is m![A / 16] , and Element is m![A / 8 % 2, A % 8, B] .</code>
Fetch	<code>OutTime is m![A / 8 % 2] and OutPacket is m![A % 8, B] .</code>
Compute and Collect	<code>The stream casts i8 to i32 , then Collect uses m![A / 8 % 2, A % 8] for Time and m![B] for Packet .</code>
Commit→DM→HBM	<code>commit_trim::<m![B]>() writes the DM tensor in m![A, B] ; to_hbm then returns HbmTensor<i32, m![1] , m![B, A]> through DMA relayout.</code>

Change only these declared axes, mappings, or element types while preserving the alignment assertions in the source. The host test awaits `to_hbm` and `to_host` through `pdma` , then asserts the expected `B` -major result.

```

#[expect(clippy::type_complexity)]

use furiosa_opt_std::prelude::*;

axes![A = 4096, B = 8];

type Chip = m![1];
type Cluster = m![1 # 2];

#[device(chip = 1)]
pub fn fetch_commit_simple(
    ctx: &mut Context,
    input: &HbmTensor<i8, m![1], m![A, B]>,
) -> HbmTensor<i32, m![1], m![B, A]> {
    // Element's innermost axis must match the source's innermost (B, stride
    1) so that the
    // DMA tail contains the full B axis (8 × i8 = 8 bytes), satisfying
    min_align = 8.
    let input_dm = input.to_dm::<Cluster, m![A / 16], m![A / 8 % 2, A % 8,
    B]>(&mut ctx.tdma);

    let fetch_and_commit_tensor: DmTensor<i32, Chip, Cluster, m![A / 16], m!
    [A / 8 % 2, A % 8, B]> = ctx
        .main
        .begin(input_dm.view())
        .fetch::<m![A / 8 % 2], m![A % 8, B]>()
        .fetch_cast::<i32>()
        .collect::<m![A / 8 % 2, A % 8], m![B]>()
        .commit_trim::<m![B]>()
        .commit();

    fetch_and_commit_tensor.to_hbm(&mut ctx.tdma)
}

```

```

use furiosa_opt_examples::fetch_commit::fetch_commit_simple;
use furiosa_opt_std::prelude::*;

#[tokio::test]
async fn test_fetch_commit_simple_host() {
    use furiosa_opt_examples::fetch_commit::{A, B};

    let mut ctx = Context::acquire();

    // Create input tensor with shape (A=4096)(B=8).
    let input = HostTensor::<i8, m![A, B]>::from_vec((0..32768).map(|x| x as i8).collect::<Vec<_>>())
        .to_hbm::<m![1], m![A, B]>(&mut ctx.pdma)
        .await;

    // Call the device function.
    let output = launch(fetch_commit_simple, (&mut *ctx, &input)).await;

    let mut expected = vec![0i32; 4096 * 8];
    let mut idx = 0;
    for b in 0..8 {
        for a in 0..4096 {
            expected[idx] = ((a * 8 + b) as i8) as i32;
            idx += 1;
        }
    }

    assert_eq!(
        output.to_host::<m![B, A]>(&mut ctx.pdma).await.into_inner(),
        Tensor::<_, m![B, A], CurrentBackend>::from_vec(expected)
    );
}

```

The included source is the complete example for the ordering and lifetime rules described below. This chapter keeps that transfer as a readable HBM↔DM and lifetime tutorial; the linked Tensor Unit I/O pattern owns the detailed packet and performance comparisons.

The type-level `Chip`, `Cluster`, `Slice`, `Time`, and `Element` mappings define each value's storage and stream behavior. A DMA call creates a destination with the mapping supplied by its type parameters. It does not mutate the source. Fetch and commit are the hand-off points into and out of the Tensor Unit pipeline.

For a first pipeline, stage an HBM input into DM, call `begin(...).fetch(...)`, run the compute stages, then commit the stream to DM and transfer the result back to HBM. The [Case Study: Tensor Unit I/O](#) shows this order with concrete mappings and source code.

Keep transfers ordered

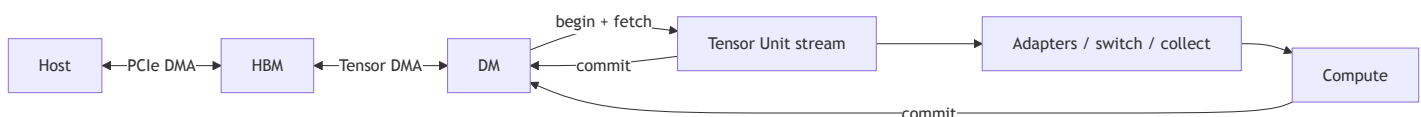
`Context::acquire()` provides one process-wide context containing independent `main` and `sub` Tensor Unit contexts plus `tdma` and `pdma` DMA contexts. Pass the matching mutable context to each operation. Rust borrows enforce that a Tensor Unit stream keeps its source view alive. `begin` borrows a `DmTensorView`, and the stream lifetime cannot outlive that view. `Fetch` consumes the `BeginTensor` and produces a stream. Adapters such as `fetch_cast` and `collect` consume and return the next stage. `Commit` consumes the final stream, or consumes it while writing to a supplied `DmTensorViewMut`.

Host transfers `to_hbm` and `to_host` are async I/O and must be awaited. Kernel-side `to_dm`, `to_hbm`, `fetch`, and `commit` operations enqueue device commands on their selected contexts. Keep source and destination handles alive until the enclosing kernel or backend submission has completed. Use a mutable view only when the destination region is intentionally overwritten. Do not read a destination before its producing transfer or `commit` has completed, and do not overlap writes to aliased views without an ordering edge.

Engine boundaries

- **Fetch** reads DM with one per-slice sequencer and emits a packet stream for the Tensor Unit. Its output mapping chooses the stream `Time` and `Packet` axes. **Axis lifting** can also move an axis from `Time` to `Chip`, `Cluster`, or `Slice` when the selected dimension contains a broadcast.
- **Commit** writes a Tensor Unit stream to DM. Its `Element` mapping chooses the destination layout and can transpose stream axes during the write.
- **DMA** pairs read and write sequencers for memory-to-memory movement. The supported public paths are HBM to HBM, HBM to DM, DM to HBM, and DM to DM. Indirect gather/scatter APIs have separate index-unit contracts: unscaled gather uses a `DmTensor` index with raw row positions, while unscaled scatter is currently unimplemented. SPM has no public tensor type.

The Tensor Unit pipeline is therefore:



Collect can place a stream operand in the TRF or VRF register files for Contraction or Vector work. Those compute boundaries are covered in [Computing Tensors](#).

Tune movement

Every movement is a mathematical tensor move: the logical values are preserved while the destination mapping may change. The compiler derives sequencer strides from both mappings. Respect the engine constraints before tuning performance: `DM cluster` must be 1 or 2, `DM slice` must be 64, 128, or 256, and DMA packet tails must satisfy the element-size alignment checked by the API. Fetch and commit also validate their mapping-specific sequencer rules. Invalid mappings fail during compilation or command verification.

Prefer contiguous packet layouts and mappings that distribute traffic across clusters, slices, DM banks, and HBM channels. Small or strided packets increase the number of sequencer accesses. Conflicting DM-bank patterns can starve DMA. See [Memory Performance](#) for the bank-starvation rule, HBM interleaving, and packet-size trade-offs.

For a complete end-to-end movement pattern, see [Case Study: Tensor Unit I/O](#). For indirect HBM row movement, see [DMA gather and scatter](#).

See also

The [furiosa-opt-std rustdoc](#) documents the release API. Use the engine pages above for operation-specific contracts and [Scheduling and Tuning](#) when a valid movement plan needs measured overlap or hazard diagnosis.

Sequencer

A sequencer reads a memory buffer as a packet stream and writes a packet stream back to memory. Mappings and packet sizes are the author-facing controls.

Sequencer mechanics explain the loops, strides, access sizes, and lifetimes shared by Fetch, Commit, and DMA. The pseudo-type examples introduce the model; Access Size and Architecture explain the concrete engine behavior. The Fetch and Commit Engines each use one sequencer to address DM. The DMA Engine chains a read sequencer and a write sequencer to move data among DM and HBM without intermediate buffers.

Interface

`MemTensor` and `StreamTensor` are teaching-only pseudo types that isolate the buffer-to-stream pattern, so this section can explain sequencer mechanics without introducing every engine type. The real engine APIs use different types (`DmTensor`, `HbmTensor`, `TuTensor`, ...), but every concrete pair maps onto the same `MemTensor` $\rightarrow$ `StreamTensor` shape illustrated below.

`MemTensor` holds data in some memory mapping `Buf`. DM and HBM tensors play this role in the public API.

```
/// A generic buffer-backed tensor.
/// Anything that holds data in a memory `Buf` and can be streamed in or out.
#[derive(Debug)]
pub struct MemTensor<D: Scalar, Buf: M> {
    inner: Tensor<D, Buf>,
}
```

`StreamTensor` is a tensor in flight. The lifetime `'l` ties the stream to its source buffer so a stream cannot outlive its data. `Time` is the temporal mapping (iteration over time) and `Packet` is the spatial mapping (contents of a single packet).

```

/// A streaming view of a tensor in flight.
/// `Packet` is the per-cycle shape and `Time` is the multi-cycle shape.
#[derive(Debug)]
pub struct StreamTensor<'l, D: Scalar, Time: M, Packet: M> {
    inner: Tensor<D, Pair<Time, Packet>>,
    _marker: PhantomData<&'l ()>,
}

```

`read` converts a `MemTensor` into a `StreamTensor` and `write` reverses it, both preserving values. Each engine's full API adds spatial dimensions (`Chip`, `Cluster`, `Slice`) on top, covered in the engine-specific pages.

```

impl<D: Scalar, Buf: M> MemTensor<D, Buf> {
    /// Reads a stream from this buffer with the supplied `Time` and `Packet`
    mapping.
    /// `(Time, Packet)` may be a broadcast of `Buf` (matches Fetch / Switch
    / DMA-read behavior).
    pub fn read<'l, Time: M, Packet: M>(&'l self) -> StreamTensor<'l, D,
    Time, Packet> {
        StreamTensor {
            inner: self.inner.transpose(true),
            _marker: PhantomData,
        }
    }

    /// Writes a stream back into this buffer.
    /// Broadcast is rejected: each `Buf` slot must have exactly one source
    position in `(Time, Packet)` (matches Commit behavior).
    pub fn write<'l, Time: M, Packet: M>(&mut self, stream: StreamTensor<'l,
    D, Time, Packet>) {
        self.inner = stream.inner.transpose(false);
    }
}

```

For any `MemTensor`, many valid `Time` and `Packet` combinations exist, each producing a different `StreamTensor`. Among valid choices, larger `Packet` sizes improve bandwidth utilization, and [Memory Performance](#) covers the trade-offs in detail.

Examples

The following examples show common read and write patterns using the core API above. [Architecture](#) below explains how the compiler derives each pattern's hardware configuration.

```

axes![A = 8, B = 512, N = 4, C = 3, H = 8, W = 8, T = 4, P = 4];

/// Strided access: read 8×512 tensor as 128 packets of 32 elements.
/// Time = m![A, B / 32] produces 8 * 16 = 128 time steps.
/// Packet = m![B % 32] delivers 32 consecutive elements per packet.
fn strided_read<'l>(
    buf: &'l MemTensor<bf16, m![A, B]>,
) -> StreamTensor<'l, bf16, m![A, B / 32], m![B % 32]> {
    buf.read() // Automatic type inference
}

/// Strided write: write 128 packets of 32 elements back to 8×512 tensor.
fn strided_write(
    buf: &mut MemTensor<bf16, m![A, B]>,
    stream: StreamTensor<bf16, m![A, B / 32], m![B % 32]>,
) {
    buf.write(stream)
}

/// Axis reordering read: change traversal from [N, C, H, W] to [W, H, C, N].
/// Time = m![W, H, C, N] iterates in reversed axis order.
/// Packet = m![1] delivers single-element packets.
fn axis_reordering_read<'l>(
    buf: &'l MemTensor<bf16, m![N, C, H, W]>,
) -> StreamTensor<'l, bf16, m![W, H, C, N], m![1]> {
    buf.read()
}

/// Axis reordering write: write [W, H, C, N] stream back to [N, C, H, W]
buffer.
fn axis_reordering_write(
    buf: &mut MemTensor<bf16, m![N, C, H, W]>,
    stream: StreamTensor<bf16, m![W, H, C, N], m![1]>,
) {
    buf.write(stream)
}

/// Tiling read: break axes into sub-blocks for cache efficiency.
/// Time = m![A % 2, B % 4, A / 2, B / 4] tiles A into 2 × 4, B into 4 × 128
blocks.
/// Packet = m![C # 32] pads C to 32 elements per packet.
fn tiling_read<'l>(
    buf: &'l MemTensor<i8, m![A, B, C # 8]>,
) -> StreamTensor<'l, i8, m![A % 2, B % 4, A / 2, B / 4], m![C # 32]> {
    buf.read()
}

/// Tiling write: write tiled stream back to buffer.
fn tiling_write(
    buf: &mut MemTensor<i8, m![A, B, C # 8]>,

```

```

    stream: StreamTensor<i8, m![A % 2, B % 4, A / 2, B / 4], m![C # 32]>,
) {
    buf.write(stream)
}

/// Broadcasting read: replicate elements absent from `Buf`.
/// Time = m![T, A] broadcasts T temporally (same data repeated T times).
/// Packet = m![P] broadcasts P spatially (same element fills packet).
fn broadcasting_read<'l>(<
    buf: &'l MemTensor<i8, m![A]>,
) -> StreamTensor<'l, i8, m![T, A], m![P]> {
    buf.read()
}

/// Broadcasting write: write broadcast stream back to buffer.
/// This is rejected as each `Buf` slot must have exactly one source position
in `(Time, Packet)`
/// This code will panic when run
fn broadcasting_write(
    buf: &mut MemTensor<i8, m![A]>,
    stream: StreamTensor<i8, m![T, A], m![P]>,
) {
    buf.write(stream)
}

```

Architecture

Each sequencer call is compiled from its input and output tensor mappings into a nested-loop configuration that the sequencer hardware executes. Each configuration takes the form `[size_0 : stride_0, size_1 : stride_1, ...] : packet_size`, where subscript 0 is the outermost loop:

```

struct Config {
    /// Each entry defines a nested loop level.
    entries: Vec<Entry>,
    /// Number of elements per packet.
    packet_size: usize,
}

struct Entry {
    /// Number of iterations for this loop level.
    size: usize,
    /// Memory address distance (in elements) to skip after each iteration.
    stride: isize,
}

```

Each entry encodes one dimension of tensor traversal. The `size` field determines how many times this loop iterates, while the `stride` field determines the memory offset between consecutive iterations.

Access Size

`max_access_size = gcd(Packet::SIZE, contiguous_run)` is the number of elements per hardware access.

Here, `contiguous_run` is the element count of the innermost physically contiguous run of `Config` entries. A larger `max_access_size` means fewer accesses per packet. The following shows how `max_access_size` is computed from a `Config`:

```

impl Config {
    fn contiguous_run(&self) -> usize {
        // Walk pairs from innermost outward; stop at the first non-
        contiguous pair.
        // Two adjacent entries (n_outer : s_outer) and (n_inner : s_inner)
        // are contiguous when s_outer == n_inner * s_inner.
        let mut contiguous_run = self.entries.last().map_or(1, |e| e.size);
        for w in self.entries.windows(2).rev() {
            if w[0].stride == w[1].size as isize * w[1].stride {
                contiguous_run *= w[0].size;
            } else {
                break;
            }
        }
        contiguous_run
    }

    fn max_access_size(&self) -> usize {
        gcd(self.packet_size, self.contiguous_run())
    }
}

```

In most cases the packet layout is fully contiguous in DM and `max_access_size == Packet::SIZE`. See [Non-Contiguous Packets](#) for a case where `max_access_size < Packet::SIZE`.

How It Works

The `Config` for $m![N, C, H, W] \rightarrow m![W, H, C, N]$ has one entry per axis in the stream, each with a stride equal to that axis's span in the source buffer. Since `Packet = m![1]`, `Packet::SIZE = max_access_size = 1` and the sequencer issues one DM access per loop iteration.

```
axes![N = 4, C = 3, H = 8, W = 8];
```

```
fn read_nchw_whcn(buf: &MemTensor<bf16, m![N, C, H, W]>) ->
    StreamTensor<bf16, m![W, H, C, N], m![1]> {
    // Compiler-generated configuration: [8 : 1, 8 : 8, 3 : 64, 4 : 192] : 1
    let config = Config {
        entries: vec![
            Entry { size: 8, stride: 1 },    // W
            Entry { size: 8, stride: 8 },    // H
            Entry { size: 3, stride: 64 },   // C
            Entry { size: 4, stride: 192 },  // N
        ],
        packet_size: 1,
    };

    // The hardware executes the configuration as nested loops:
    for w in 0..8 {
        for h in 0..8 {
            for c in 0..3 {
                for n in 0..4 {
                    // Read each address
                    let addr = 1 * w + 8 * h + 64 * c + 192 * n;
                    // yield buf[addr];
                }
            }
        }
    }

    buf.read()
}

fn write_whcn_nchw(buf: &mut MemTensor<bf16, m![N, C, H, W]>,
    stream: StreamTensor<bf16, m![W, H, C, N], m![1]> {
    // The compiler generates an identical config for writing
    // The hardware executes the configuration as nested loops:
    for w in 0..8 {
        for h in 0..8 {
            for c in 0..3 {
                for n in 0..4 {
                    // Write to each address
                    let addr = 1 * w + 8 * h + 64 * c + 192 * n;
                    // buf[addr] = stream.next();
                }
            }
        }
    }
}
```

Configurations

The following patterns cover most configurations a kernel writer is likely to encounter.

Transposing Axes

Axes may be transposed so that the stream visits them in a different order than the buffer, and the compiler computes the strides needed to traverse memory in that order.

```
axes![A = 8, B = 8, C = 8];

fn read_rearranging<'l>(<
  buf: &'l MemTensor<i8, m![A, B, C # 32]>, // Buf
) -> StreamTensor<'l, i8, m![B, A], m![C # 16]> { // Time, Packet
  buf.read()
}
```

The compiler generates configuration entries by processing the combined mapping `m!` `[B, A, C # 16]` term by term, transforming `Buf` along the way. For each term, the entry size equals the term size, and the stride equals the volume that term occupies within the current `Buf`. After processing a term, `Buf` is updated to reflect that the axis has been consumed:

Term	Entry	Stride Source	Buf After
B	8 : 32	<code>m![C # 32]::SIZE</code>	<code>m![A, 1 # 8, C # 32]</code>
A	8 : 256	<code>m![1 # 8, C # 32]::SIZE</code>	<code>m![1 # 64, C # 32]</code>
C # 16	16 : 1	contiguous (Packet dimension)	<code>1 # 2048</code>

`Packet::SIZE = max_access_size = 16`. The innermost entry `16 : 1` is contiguous, so the hardware transfers the full packet in one access.

Splitting Axes

Tiling breaks a logical axis into sub-blocks for cache efficiency or to match tensor unit buffer sizes, and the compiler achieves this by splitting the axis into multiple entries.


```
axes![A = 8, B = 8, C = 4];
```

```
fn read_splitting<'l>(  
    buf: &'l MemTensor<i8, m![A, B, C # 8]>, // Buf  
) -> StreamTensor<'l, i8, m![A % 2, B % 4, A / 2, B / 4], m![C # 32]> { //  
    Time, Packet  
        buf.read()  
}
```

Expressions like $A \% 2$ and $A / 2$ split axis A into separate entries. The compiler processes $m![A \% 2, B \% 4, A / 2, B / 4, C \# 32]$ term by term:

Term	Entry	Stride Source	Buf After
$A \% 2$	2 : 64	$m![B, C \# 8]::\text{SIZE}$	$m![A / 2, 1 \# 2, B, C \# 8]$
$B \% 4$	4 : 8	$m![C \# 8]::\text{SIZE}$	$m![A / 2, 1 \# 2, B / 4, 1 \# 4, C \# 8]$
$A / 2$	4 : 128	$m![1 \# 2, B / 4, 1 \# 4, C \# 8]::\text{SIZE}$	$m![1 \# 8, B / 4, 1 \# 4, C \# 8]$
$B / 4$	2 : 32	$m![1 \# 4, C \# 8]::\text{SIZE}$	$m![1 \# 64, C \# 8]$
$C \# 32$	32 : 1	contiguous (Packet dimension)	1 # 512

`Packet::SIZE = max_access_size = 32.`

Slicing Axes

Slicing reads only a partial range of indices from the memory layout, a condition that arises when an indexed view selects a subset of the original tensor.

```
axes![A = 16, B = 8, C = 8];
```

```
fn read_slicing<'l>(  
    buf: &'l MemTensor<i8, m![A, B, C]>, // Buf  
) -> StreamTensor<'l, i8, m![A / 4, A % 4 = 3, B / 4, B % 4 = 2], m![C]> {  
    // Time, Packet  
        buf.read()  
}
```

The $= 3$ notation limits $A \% 4$ to only 3 iterations instead of 4, restricting the hardware to a sub-region of the tensor. The compiler processes $m![A / 4, A \% 4 = 3, B / 4, B \% 4 = 2, C]$ term by term:

Term	Entry	Stride Source	Buf After
$A / 4$	4 : 256	$m![A \% 4, B, C]::SIZE$	$m![1 \# 4, A \% 4, B, C]$
$A \% 4 = 3$	3 : 64	$m![B, C]::SIZE$ (sliced to 3)	$m![1 \# 16, B, C]$
$B / 4$	2 : 32	$m![B \% 4, C]::SIZE$	$m![1 \# 32, B \% 4, C]$
$B \% 4 = 2$	2 : 8	$m![C]::SIZE$ (sliced to 2)	$m![1 \# 128, C]$
C	8 : 1	contiguous (Packet dimension)	1 # 1024

$Packet::SIZE = max_access_size = 8.$

Broadcasting Axes

Broadcasting replicates an element across multiple packets or time steps when the stream visits axes that **Buf** does not carry. An axis or partial-axis fragment such as $N / 512$ can appear in **Time** or **Packet** but not **Buf**. The sequencer then emits a broadcast entry $: 0$ and revisits the same address.

```
axes![A = 16, T = 4, P = 4];
```

```
fn read_broadcasting<'l>(<br>  buf: &'l MemTensor<i8, m![A]>, // Buf<br>)-> StreamTensor<'l, i8, m![T, A], m![P]> { // Time, Packet<br>  buf.read()<br>}
```

The compiler processes $m![T, A, P]$ term by term:

Term	Entry	Stride Source	Buf After
T	4 : 0	not in Buf (broadcast)	$m![A]$
A	16 : 1	A in $m![A]$	1 # 16

P	4 : 0	not in Buf (broadcast)	1 # 16
---	-------	------------------------	--------

`Packet::SIZE = max_access_size = 4`. `P` is broadcast, so the same element is replicated across the packet (spatial broadcast). `T` is broadcast, so the same data is repeated across time steps (temporal broadcast).

The same rule applies when `Time` or `Packet` references a fragment of an axis that `Buf` does not carry. For example, a buffer of `m![N % 512]` read as `StreamTensor<m![N / 512], m![N % 512]>` broadcasts on the `N / 512` time entry: the buffer's 512 elements are reused across each of the `N / 512` outer iterations.

Merging Entries

The hardware supports at most 8 entries per configuration, so when a transformation produces more, the compiler merges adjacent entries to satisfy that limit. Adjacent entries $(n1 : s1)$ and $(n2 : s2)$ merge into $(n1 * n2 : s2)$ when physically contiguous: `s1 == n2 * s2`.

```
axes![N = 8, C = 8, H = 8, W = 32];
```

```
fn read_merging<'l>(<br>  buf: &'l MemTensor<i8, m![N, C, H, W]>, // Buf<br>) -> StreamTensor<'l, i8, m![W / 16, H % 2, H / 2, C / 2, C % 2, N / 2, N %<br>2, W / 8 % 2], m![W % 8]> { // Time, Packet<br>  buf.read()<br>}
```

The compiler processes `m![W / 16, H % 2, H / 2, C / 2, C % 2, N / 2, N % 2, W / 8 % 2, W % 8]` term by term, producing 9 initial entries:

Term	Entry	Stride Source
<code>W / 16</code>	<code>2 : 16</code>	<code>m![W % 16]::SIZE</code>
<code>H % 2</code>	<code>2 : 32</code>	<code>m![W]::SIZE</code>
<code>H / 2</code>	<code>4 : 64</code>	<code>m![H % 2, W]::SIZE</code>
<code>C / 2</code>	<code>4 : 512</code>	<code>m![C % 2, H, W]::SIZE</code>
<code>C % 2</code>	<code>2 : 256</code>	<code>m![H, W]::SIZE</code>
<code>N / 2</code>	<code>4 : 4096</code>	<code>m![N % 2, C, H, W]::SIZE</code>
<code>N % 2</code>	<code>2 : 2048</code>	<code>m![C, H, W]::SIZE</code>

$W / 8 \% 2$	$2 : 8$	$m![W \% 8]::SIZE$
$W \% 8$	$8 : 1$	contiguous (packet dimension)

Since 9 entries exceed the hardware limit of 8, the compiler merges contiguous pairs where $s_1 == n_2 * s_2$. The entries for $H \% 2 \rightarrow (2 : 32)$ and $H / 2 \rightarrow (4 : 64)$ are not merged because they are not physically contiguous ($s_1 \neq n_2 \times s_2 \iff 32 \neq 4 \times 64$). The final configuration has 6 entries. The last merge crosses the time/packet boundary: $W/8\%2$ ($2:8$) and $W\%8$ ($8:1$) merge into $W\%16$ ($16:1$).

Term	Entry	Merged Entries
$W / 16$	$2 : 16$	
$H \% 2$	$2 : 32$	
$H / 2$	$4 : 64$	
C	$8 : 256$	$C / 2 (4 : 512),$ $C \% 2 (2 : 256)$
N	$8 : 2048$	$N / 2 (4 : 4096),$ $N \% 2 (2 : 2048)$
$W \% 16$	$16 : 1$	$W / 8 \% 2 (2 : 8),$ $W \% 8 (8 : 1)$

$Packet::SIZE = \text{max_access_size} = 8.$

Non-Contiguous Packets

When the DM layout has stride discontinuities within the packet span, $\text{max_access_size} < Packet::SIZE$ and the hardware issues one access per contiguous sub-block rather than one per packet. The example below writes a packet of 32 elements ($m![A, B]$) to a buffer where each B row is padded to 16 slots in DM. A's stride is 16 rather than 8, so the packet span is not contiguous and the hardware issues 4 accesses instead of 1:

```

axes![A = 4, B = 8];

fn write_padded(
    buf: &mut MemTensor<i8, m![A, B # 16]>,
    stream: StreamTensor<i8, m![1], m![A, B]>,
) {
    // Compiler-generated configuration: [
    //   A -> 4 : 16,    (16 != 8 * 1, NOT contiguous – padding gap after each
B row)
    //   B -> 8 : 1,    (packet sub-block, contiguous)
    // ] : 32
    buf.write(stream)
}

```

```
Packet::SIZE = 32, contiguous_run = 8, max_access_size = 8.
```

Non-contiguous strides also arise when `Packet` contains non-adjacent axes from the source layout. The example below reads the same `m![N, C, H, W]` buffer with two different `Packet` choices. Placing only the innermost axis `W` in `Packet` gives `max_access_size = Packet::SIZE = 8`, one access per packet. Placing `m![N, H, W]` in `Packet` skips `C`, so `N`'s stride in source (96) does not equal `H×W` (32): `contiguous_run = 32`, `max_access_size = 32`, and the hardware issues 4 accesses per packet instead of 1.

```

axes![N = 4, C = 3, H = 4, W = 8];

// Compiler-generated configuration: [
//   N -> 4 : 96,    (96 == 3 × 32, contiguous)
//   C -> 3 : 32,    (32 == 4 × 8,  contiguous)
//   H -> 4 : 8,     (8  == 8 × 1,  contiguous)
//   W -> 8 : 1,     (packet dimension)
// ] : 8
// contiguous_run = 8 (W); ×4 (H): 8==8×1 ✓; ×3 (C): 32==4×8 ✓; ×4 (N):
// 96==3×32 ✓; all axes contiguous
// max_access_size = gcd(packet_size, contiguous_run) = packet_size = 8
fn read_contiguous<'l>(<
  buf: &'l MemTensor<i8, m![N, C, H, W]>,
) -> StreamTensor<'l, i8, m![N, C, H], m![W]> {
  buf.read()
}

// Compiler-generated configuration: [
//   C -> 3 : 32,    (time dimension)
//   N -> 4 : 96,    (96 != 4 × 8 = 32, NOT contiguous – C axis interspersed)
//   H -> 4 : 8,     (8  == 8 × 1,  contiguous)
//   W -> 8 : 1,     (packet dimension)
// ] : 128
// contiguous_run = 8 (W); ×4 (H): 8==8×1 ✓ → 32; ×4 (N): 96!=4×8 × stop → 32
// max_access_size = gcd(128, 32) = 32; hardware issues 128/32 = 4 accesses
// per packet
fn read_non_contiguous<'l>(<
  buf: &'l MemTensor<i8, m![N, C, H, W]>,
) -> StreamTensor<'l, i8, m![C], m![N, H, W]> {
  buf.read()
}

```

Constraints

In RNGD, exceeding any of the following hardware limits causes a compilation error:

- **Entry limit:** Maximum 8 entries, so the compiler merges adjacent entries where possible (see [Merging Entries](#) in Configurations).
- **Iteration limit:** `size <= 65,536` per entry.
- **Packet size:** Must be 1, 2, 4, 8, 16, or 32 bytes.
- **Packet fetch:** The innermost entry `n : s` must satisfy one of:
 - Contiguous access (adjacent elements): `(s == 0 || s == 1) && n % packet_size == 0`

- Discrete access (single-element packets): `packet_size == 1`

If merging fails or limits are exceeded, redesign the tensor mapping or split the operation across multiple sequencer calls.

Compatible Axis Decompositions

Each axis named in both `Buf` and the stream must use the same decomposition. The compiler walks the stream term by term and consumes axes from `Buf` (see [Architecture](#)). When `Buf` splits an axis one way and the stream splits it another with no common refinement, no traversal order works and the configuration is rejected, even when both sides have the same total element count.

```
axes![A = 15];
```

```
fn read_incompatible<'l>(  
    buf: &'l MemTensor<i8, m![A % 5, A / 5]>, // Buf  
) -> StreamTensor<'l, i8, m![1], m![A % 3, A / 3]> { // Time, Packet  
    buf.read() // Compilation error: incompatible decomposition  
}
```

`Buf` decomposes `A` as 5×3 while the stream decomposes it as 3×5 . Since $\gcd(5, 3) = 1$, neither decomposition refines the other: the compiler cannot consume `A % 3` from a `Buf` that has already committed to a 5-block split.

Fetch Engine

The Fetch Engine reads a DM tensor and produces a packet stream for the Tensor Unit, with `OutTime` and `OutPacket` choosing the stream consumed by later stages.

Choose Fetch when data in DM must enter the Tensor Unit stream with a specific temporal and packet order.

Interface

`BeginTensor` represents a tensor resident in DM, at the entry of the Tensor Unit pipeline. Its `Time` is `m![1]` (no temporal iteration before the pipeline starts) and `Packet` is the element layout in DM.

`BeginTensor::fetch()` runs the sequencer and produces a `FetchTensor` packet stream. The stream can feed the [Fetch Adapter](#), [Switch Engine](#), or [Collect Engine](#). The `assert_eq!` calls enforce hardware constraints on `Cluster::SIZE`, `Slice::SIZE`, and packet alignment (see [Constraints](#)).

```
impl<'l, const T: Tu, P: CanApplyFetch, D: Scalar, Chip: M, Cluster: M,
    Slice: M, Time: M, Packet: M, B: Backend>
    TuTensor<'l, T, P, D, Chip, Cluster, Slice, Time, Packet, B>
{
    /// Runs the Fetch Sequencer.
    #[primitive(TuTensor::fetch)]
    pub fn fetch<OutTime: M, OutPacket: M>(self) -> FetchTensor<'l, T, D,
        Chip, Cluster, Slice, OutTime, OutPacket, B> {
        verify_fetch::<Cluster, Slice, Time, Packet, OutTime, OutPacket>();
        FetchTensor::new(self.ctx, self.inner.transpose(true))
    }
}
```

As introduced in [Mapping Tensors](#), the `Chip`, `Cluster`, `Slice`, `Time`, `Packet` mapping distributes data across space and time. `.fetch()` preserves the `Chip`, `Cluster`, and `slice` dimensions unchanged from the input, because each slice independently reads its own DM partition. Later the [Switch Engine](#) changes the `slice` mapping by moving data across slices. [Axis lifting](#) changes `chip`, `cluster`, or `slice` without transferring data, using different DM read offsets across the selected dimension.

`fetch()` takes `OutTime` and `OutPacket` type parameters that configure the Fetch Sequencer. `OutTime` sets the number of time steps in the output stream, and `OutPacket` sets the element layout within each packet. For performance implications of `OutPacket` choices, see [Optimizations](#).

The following example fetches an `i8` matrix from DM as an `i8` packet stream. The output `FetchTensor` streams 512 time steps, each a 32-element `i8` packet (32 bytes). Here `OutTime = m![A]` and `OutPacket = m![B]`.

```
axes![CH = 4, CL = 2, S = 256, A = 512, B = 32];

fn fetch_matrix_example<'l, const T: Tu>(
    input: BeginTensor<'l, T, i8, m![CH], m![CL], m![S], m![1], m![A, B]>,
) -> FetchTensor<'l, T, i8, m![CH], m![CL], m![S], m![A], m![B]> {
    input.fetch::<m![A], m![B]>()
}
```

`Chip`, `Cluster`, and `Slice` are the hardware spatial parallelism dimensions. A Fetch Sequencer runs independently in every slice, each operating on its own local DM partition. In this example, `CH = 4`, `CL = 2`, and `S = 256` describe a 4-chip system with two clusters per chip and 256 slices per cluster. Each slice runs the same sequencer over its own $A \times B$ sub-tensor.

Axis Lifting

`fetch_chip_lift`, `fetch_cluster_lift`, and `fetch_slice_lift` move axes from `Time` to `Chip`, `Cluster`, or `Slice`. A plain `fetch` starts every chip, cluster, and slice at the same DM offset and traverses the axes in `Time`. A lift gives each chip, cluster, or slice a different read *base*, so the lifted axes are read in parallel and removed from `Time`.

A lift requires the source region to be replicated across the selected dimension because a read base changes only the starting offset. The DM placement represents this replication as a broadcast, which the lift replaces with the lifted axes.

For `Slice`, use [InterTranspose](#) when data must move between slices or from `Slice` to `Time`. For `Chip` and `Cluster`, a [DMA](#) is the alternative when the source region is not replicated.

Stage	Dimension	Where the base comes from
-------	-----------	---------------------------

<code>fetch_chip_lift</code>	Chip	the read command encodes a static base, so the lift adds no setup operation
<code>fetch_cluster_lift</code>	Cluster	the read command encodes a static base, so the lift adds no setup operation
<code>fetch_slice_lift</code>	Slice	the compiler materializes one runtime base per slice with a DMA, an add in the fetch's execution context, and an SFR store

Slice bases depend on the source tensor's runtime address and therefore cannot be encoded as static command fields. The compiler copies constant relative offsets from DRAM to SRAM, adds the source address in the fetch's execution context, and stores the results in the per-slice base SFRs. The fetch waits for this setup, which uses the same SFR-store mechanism as the Switch Engine's [custom bitmap](#).

For a sub-context fetch, the setup store overwrites SFRs that the fetch itself needs. The compiler restores those parameters after writing the bases; both contexts support every lift variant.

```

    /// Gives each chip its own fetch base, lifting a DM axis onto `Chip`.
    #[primitive(TuTensor::fetch_chip_lift)]
    pub fn fetch_chip_lift<OutChip: M, OutTime: M>(
        self,
    ) -> FetchChipLiftTensor<'l, T, D, OutChip, Cluster, Slice, OutTime,
Packet, B>
    where
        P: CanApplyFetchChipLift,
    {

    /// Gives each cluster its own fetch base, lifting a DM axis onto
    `Cluster`.
    #[primitive(TuTensor::fetch_cluster_lift)]
    pub fn fetch_cluster_lift<OutCluster: M, OutTime: M>(
        self,
    ) -> FetchClusterLiftTensor<'l, T, D, Chip, OutCluster, Slice, OutTime,
Packet, B>
    where
        P: CanApplyFetchClusterLift,
    {

    /// Gives each slice its own fetch base, lifting a DM axis onto `Slice`.
    #[primitive(TuTensor::fetch_slice_lift)]
    pub fn fetch_slice_lift<OutSlice: M, OutTime: M>(
        self,
    ) -> FetchSliceLiftTensor<'l, T, D, Chip, Cluster, OutSlice, OutTime,
Packet, B>
    where
        P: CanApplyFetchSliceLift,
    {

```

Example: Lifting Q onto Slice

```

axes![A = 128, Q = 4, V = 16];

fn fetch_halves_per_slice<'l, const T: Tu>(
    input: BeginTensor<'l, T, bf16, m![1], m![1 # 2], m![A, 2], m![1], m![Q,
V]>,
) -> FetchSliceLiftTensor<'l, T, bf16, m![1], m![1 # 2], m![A, Q / 2], m![Q %
2], m![V]> {
    input
        .fetch::<m![Q], m![V]>()
        .fetch_slice_lift::<m![A, Q / 2], m![Q % 2]>()
}

```

With $A = 128$, the slice placement $m![A, 2]$ describes 128 pairs of slices; both slices in

each pair hold the same $m! [Q, V]$ data. A plain `fetch` reads all four Q values on both slices. The lift replaces the broadcast `2` with $Q / 2$, leaving $Q \% 2$ in `Time`: the first slice reads $Q = 0, 1$ from base 0, and the second reads $Q = 2, 3$ from base 32 elements (64 bytes of `bf16`).

	placement	base	reads	time steps
fetch, either slice	$m! [A, 2]$	0	$Q = 0, 1, 2, 3$	4
lift, first slice	$m! [A, Q / 2]$	0	$Q = 0, 1$	2
lift, second slice	$m! [A, Q / 2]$	32 elements	$Q = 2, 3$	2

The base for any chip, cluster, or slice is the sum of each lifted-axis value multiplied by its original DM stride. Here the second slice has $Q / 2 = 1$, so its base is $1 \times (Q \% 2) \times V = 32$ elements.

Requirements

A lift must satisfy all of the following:

- The input and output sizes of the selected dimension must match.
- Only broadcasts may change. The verifier compares the mappings in size-2 groups; a changed broadcast `2` must become an axis component of size 2. To replace a broadcast with padding, reshape the DM placement and leave that padding unread. Several groups may change in one lift, but an existing axis or a broadcast of 3 may not.
- Lifted axes must be removed from `OutTime`. `OutPacket` remains unchanged.
- Lift methods must be called in `chip`, `cluster`, `slice` order, at most once per dimension.
- Every calculated base must be a multiple of 8 bytes.
- If an unsafe `reshape` introduces the broadcast, the data must already be replicated. `reshape` checks element order, not replication; violating this requirement can produce different hardware and CPU results.

```

axes![A = 128, H = 2, V = 16, Rep = 2, G = 2];

// Valid: DMA replicates `[H, V]` across `Rep`; reshape exposes `Rep` as a
broadcast.
fn named_broadcast(
    ctx: &mut Context,
    input: &HbmTensor<bf16, m![1], m![A, H, V]>,
) -> DmTensor<bf16, m![1], m![1 # 2], m![A, H], m![V]> {
    let written: DmTensor<bf16, m![1], m![1 # 2], m![A, Rep], m![H, V]> =
        input.to_dm::<m![1 # 2], m![A, Rep], m![H, V]>(&mut ctx.tdma);
    let dm: DmTensor<bf16, m![1], m![1 # 2], m![A, 2], m![H, V]> = unsafe {
        written.reshape() };

    ctx.main
        .begin(dm.view())
        .fetch::<m![H], m![V]>()
        .fetch_slice_lift::<m![A, H], m![1]>()
        .collect::<m![1], m![V]>()
        .commit_trim::<m![V]>()
        .commit()
}

// Invalid for lifting: `G` stores different values, but reshape names it as
a broadcast.
fn without_broadcast(
    ctx: &mut Context,
    input: &HbmTensor<bf16, m![1], m![A, G, H, V]>,
) -> DmTensor<bf16, m![1], m![1 # 2], m![A, H], m![V]> {
    let written: DmTensor<bf16, m![1], m![1 # 2], m![A, G], m![H, V]> =
        input.to_dm::<m![1 # 2], m![A, G], m![H, V]>(&mut ctx.tdma);
    let dm: DmTensor<bf16, m![1], m![1 # 2], m![A, 2], m![H, V]> = unsafe {
        written.reshape() };

    ctx.main
        .begin(dm.view())
        .fetch::<m![H], m![V]>()
        .fetch_slice_lift::<m![A, H], m![1]>()
        .collect::<m![1], m![V]>()
        .commit_trim::<m![V]>()
        .commit()
}

```

Constraints

- **Hardware dimensions:** `Chip::SIZE`, `Cluster::SIZE`, and `Slice::SIZE` must

match the hardware configuration (see [Sequencer](#)).

Multi-Read Packet

Preparing a packet may require multiple hardware reads because packet axes may not be contiguous in DM, and the hardware reads at most 32 bytes at once. In the [main-context](#), `read_size` is the largest divisor of `max_access_size` for which `D[read_size]` is 1, 2, 4, 8, 16, or 32 bytes. See [Sequencer Architecture](#) for `max_access_size`. In the [sub-context](#), `read_size` is fixed at 8 bytes. The compiler derives `read_size` from the input element type and any [Fetch Adapter](#) cast. Users do not set it directly. Multi-read occurs whenever `Packet::SIZE > read_size`. For example, a 24-byte packet in the main-context forces `read_size = 8` and 3 reads per packet. The total cycle count is `Time::SIZE * (Packet::SIZE / read_size)`.

The following examples fetch the same `i4` tensor of shape `m![N, C, H, W]` (with `N=4`, `C=3`, `H=4`, `W=16`) using four different `OutPacket` choices.

```

axes![N = 4, C = 3, H = 4, W = 16];

/// Sequencer config: [N = 4 : 192, C = 3 : 64, H = 4 : 16, W = 16 : 1].
/// max_access_size = 16; read_size = 16 (8 bytes); reads per packet = 1;
cycles = 48
fn fetch_batch_1<'l, const T: Tu>(
    input: BeginTensor<'l, T, i4, m![1], m![1 # 2], m![1 # 256], m![1], m![N,
C, H, W]>,
) -> FetchTensor<'l, T, i4, m![1], m![1 # 2], m![1 # 256], m![N, C, H], m!
[W]> {
    input.fetch()
}

/// Sequencer config: [N = 4 : 192, C = 3 : 64, H / 2 = 2 : 32, H % 2 = 2 :
16, W = 16 : 1].
/// max_access_size = 32; read_size = 32 (16 bytes); reads per packet = 1;
cycles = 24
fn fetch_batch_2<'l, const T: Tu>(
    input: BeginTensor<'l, T, i4, m![1], m![1 # 2], m![1 # 256], m![1], m![N,
C, H, W]>,
) -> FetchTensor<'l, T, i4, m![1], m![1 # 2], m![1 # 256], m![N, C, H / 2],
m![H % 2, W]> {
    input.fetch()
}

/// Sequencer config: [N = 4 : 192, C = 3 : 64, H = 4 : 16, W = 16 : 1].
/// max_access_size = 64; read_size = 64 (32 bytes); reads per packet = 1;
cycles = 12
fn fetch_batch_3<'l, const T: Tu>(
    input: BeginTensor<'l, T, i4, m![1], m![1 # 2], m![1 # 256], m![1], m![N,
C, H, W]>,
) -> FetchTensor<'l, T, i4, m![1], m![1 # 2], m![1 # 256], m![N, C], m![H,
W]> {
    input.fetch()
}

/// Sequencer config: [N = 4 : 192, C = 3 : 64, H = 4 : 16, W = 16 : 1].
/// max_access_size = 192; read_size = 64 (32 bytes); reads per packet = 3;
cycles = 12
fn fetch_batch_4<'l, const T: Tu>(
    input: BeginTensor<'l, T, i4, m![1], m![1 # 2], m![1 # 256], m![1], m![N,
C, H, W]>,
) -> FetchTensor<'l, T, i4, m![1], m![1 # 2], m![1 # 256], m![N], m![C, H,
W]> {
    input.fetch()
}

```

Interleaving

Interleaving combines two tensors with identical mappings into a single sequencer operation, reducing overhead when both tensors are needed for the same computation. An explicit `Time` axis encodes alternation between the two tensors.

In the following example, the main-context creates an interleaved tensor using `begin_interleaved()`. The first temporal iteration fetches from `lhs`, the second from `rhs`, the third from `lhs` again, and so on. At most two tensors can be interleaved in a single fetch operation.

```
axes![A = 16, B = 32, I = 2];

/// Interleaves two input tensors into a single packet stream.
/// Useful for operations like 'input1 + input2' in the Vector Engine.
/// The interleaved BeginTensor is created via Tu.begin_interleaved().
/// The `I = 2` axis in Time encodes alternation between the two tensors.
fn fetch_interleaved<'l>(
    ctx: &'l mut Context,
    lhs: &'l DmTensor<i8, m![1], m![1 # 2], m![1 # 256], m![A, B]>,
    rhs: &'l DmTensor<i8, m![1], m![1 # 2], m![1 # 256], m![A, B]>,
) -> FetchTensor<'l, { Tu::Main }, i8, m![1], m![1 # 2], m![1 # 256], m![A,
I], m![B]> {
    ctx.main.begin_interleaved::

```

Optimizations

Three factors determine Fetch Sequencer throughput.

- **Input bandwidth:** `read_size` is limited by axis contiguity in DM and packet size. Non-adjacent axes reduce `max_access_size` and therefore `read_size` (see [Non-Contiguous Packets](#)). A packet smaller than the contiguous run also limits `read_size`. Padding to a larger power-of-two raises it (see [Packet padding](#)).

Repeated access to the same bank can starve lower-priority [Commit Engine](#) and [DMA Engine](#) operations. The limit is 64 consecutive accesses; exceeding it can cause a NoC timeout.

See [Memory Performance](#) for details.

- **Output bandwidth:** the downstream [Collect Engine](#) converts Fetch's packets to 32-byte *flits*, so packet sizes that don't align to 32 bytes waste bandwidth. A 20-byte packet fills one flit with 12 bytes of zero-padding, wasting $12 / 32 = 37.5\%$. A 40-byte packet spans two flits (64 bytes total) and zero-pads the final 24 bytes of the second flit, wasting $24 / 64 = 37.5\%$.
- **Spatial parallelism:** Distributing fetches across slices maximizes throughput.

Example: Packet padding

Padding `OutPacket` to a larger power-of-two element count can increase `read_size`. The three examples below fetch the same 30-byte tensor in 15, 3, and 1 cycles by growing the packet from 2 to 16 to 32 bytes:

```
axes![A = 3, B = 5, C = 2];
```

```
/// Smallest packet: only C dimension padded to 8bytes. Takes 15 cycles.
```

```
fn fetch_packet_C<'l, const T: Tu>(
    input: BeginTensor<'l, T, f8e4m3, m![1], m![1 # 2], m![1 # 256], m![1],
    m![A, B, C]>,
) -> FetchTensor<'l, T, f8e4m3, m![1], m![1 # 2], m![1 # 256], m![A, B], m![C
# 8]> {
    input.fetch()
}
```

```
/// Medium packet: B and C dimensions padded to 16 bytes. Takes 3 cycles.
```

```
fn fetch_packet_BC<'l, const T: Tu>(
    input: BeginTensor<'l, T, f8e4m3, m![1], m![1 # 2], m![1 # 256], m![1],
    m![A, B, C]>,
) -> FetchTensor<'l, T, f8e4m3, m![1], m![1 # 2], m![1 # 256], m![A], m![[B,
C] # 16]> {
    input.fetch()
}
```

```
/// Largest packet: all dimensions padded to 32 bytes. Takes 1 cycle.
```

```
fn fetch_packet_ABC<'l, const T: Tu>(
    input: BeginTensor<'l, T, f8e4m3, m![1], m![1 # 2], m![1 # 256], m![1],
    m![A, B, C]>,
) -> FetchTensor<'l, T, f8e4m3, m![1], m![1 # 2], m![1 # 256], m![1], m![[A,
B, C] # 32]> {
    input.fetch()
}
```

In these examples, padding reads beyond the actual data, but this is safe because

padding values do not affect computation. Different padding strategies produce different `FetchTensor` mappings, which may affect downstream components.

The same $A = 3, B = 5, C = 2$ `f8e4m3` tensor can use these packet mappings:

Function	Fetch Time	Fetch Packet	Fetch work	Collect padding
<code>fetch_packet_c</code>	A, B	C # 8	15 packets of 8 bytes	24 bytes for each packet.
<code>fetch_packet_bc</code>	A	[B, C] # 16	3 packets of 16 bytes	16 bytes for each packet.
<code>fetch_packet_abc</code>	1	[A, B, C] # 32	1 packet of 32 bytes	None.

The 8-byte and 16-byte candidates require Collect to pad each packet to a 32-byte flit. The 32-byte candidate reaches Collect as one full flit, but its extra physical capacity can increase DM usage. The schedule comparison decides whether fewer fetches outweigh the padding and downstream layout cost for the fixed workload.

Commit Engine

The Commit Engine writes a Tensor Unit stream packet to DM, the inverse of the [Fetch Engine](#), after adapters have selected the final element type and layout. It is the Commit Sequencer: a [mathematical tensor move](#) that runs independently in every slice, each writing to its own local DM partition.

Choose Commit when a computed Tensor Unit stream must become a DM tensor with a deliberate `Element` layout.

Interface

A `TuTensor` carries `Chip`, `Cluster`, `Slice`, `Time`, and `Packet` dimensions at the end of the Tensor Unit pipeline. Its `Time` reflects the temporal unrolling of the computation, and `Packet` is the element layout in the output stream.

`.commit()` writes the stream to a `DmTensor` in DM.

```
impl<'l, const T: Tu, P: CanApplyCommit, D: Scalar, Chip: M, Cluster: M,
Slice: M, Time: M, Packet: M, B: Backend>
    TuTensor<'l, T, P, D, Chip, Cluster, Slice, Time, Packet, B>
{
    /// Commits to data memory.
    #[primitive(TuTensor::commit)]
    pub fn commit<Element: M>(self) -> DmTensor<D, Chip, Cluster, Slice,
Element, B> {
        verify_commit::<D, Time, Packet, Element>();
        DmTensor::from_parts(self.inner.transpose(false), None)
    }

    /// Commits to a mutable tensor view in data memory.
    #[primitive(TuTensor::commit_view)]
    pub fn commit_view<Element: M>(self, mut dst: DmTensorViewMut<'l, D,
Chip, Cluster, Slice, Element, B>) {
        verify_commit::<D, Time, Packet, Element>();
        dst.inner.transpose(self.inner.view(), false);
    }
}
```

`.commit()` preserves the `chip`, `cluster`, and `slice` dimensions unchanged, because each slice independently writes to its own DM partition. The output `Element` mapping

replaces `Time` and `Packet`, defining how the stream is laid out in DM. `Element` configures both the Commit Sequencer and the [Commit Adapter](#), and can reorder `Time` axes relative to the input stream, performing a transpose during the commit. For performance implications of the `Element` mapping, see [Optimizations](#).

The following example commits a cast accumulation result to DM as `bf16`. The output `DmTensor` stores 16 time steps $\times$ 8 `bf16` elements across 256 slices. Here `D = bf16` and `Element = m![M, N # 16]`.

```
axes![P = 256, M = 16, N = 8];
```

```
fn cast_commit<'l, const T: Tu>(
    input: ContractTensor<'l, T, f32, m![1], m![1 # 2], m![P], m![M], m![N]>,
) -> DmTensor<bf16, m![1], m![1 # 2], m![P], m![M, N # 16]> {
    // Cast f32 to bf16 (Cast Engine), then commit to DM (Commit Engine).
    // Input: M = 16 time steps, N = 8 f32 elements per packet (32 bytes).
    // After cast: N = 8 bf16 elements padded to 16 (32 bytes).
    // After trim: N # 16 trimmed into N = 8.
    // The sequencer writes across P = 256 slices.
    input.cast::

```

Constraints

- **Hardware dimensions:** `Chip::SIZE`, `Cluster::SIZE`, and `Slice::SIZE` must match the hardware configuration (see [Sequencer](#)).
- **Address alignment:** All sequencer strides must be multiples of 8 bytes.
- **Write unit alignment:** `D[valid_size]` must be 8, 16, 24, or 32 bytes (see the [Commit Adapter's Trimming](#) stage).

Multi-Write Packet

Writing a packet may require multiple hardware writes because packet axes may not be contiguous in DM. The compiler derives `write_size = gcd(valid_size, access_size)`. The [Commit Adapter](#) supplies `valid_size`; the [Sequencer](#) supplies `access_size`. In the [sub-context](#), `D[write_size]` is fixed at 8 bytes. The total cycle count is `Time::SIZE * (valid_size / write_size)`. The division is always exact: in the main-context,

`valid_size == write_size`, so each packet commits in a single cycle. In the sub-context, `write_size` is fixed at 8 bytes and `valid_size` is one of 8, 16, 24, or 32 bytes (from the trimming constraint), so `valid_size / write_size` is always 1, 2, 3, or 4.

```
axes![M = 4, K = 2, W = 8, N = 16, L = 32];
```

```
// Compiler-generated configuration: [
//   M -> 4 : 64,   (64 == 2 * 32,   contiguous)
//   K -> 2 : 32,   (32  == 32 * 1,   contiguous)
//   M -> 32 : 1    (packet dimension, contiguous)
// ] : 8
// access_size = 64; valid_size = 8; write_size = gcd(64, 8) = 8; writes per
// packet = 1
fn no_transpose<'l, const T: Tu>(
    input: CastTensor<'l, T, i8, m![1], m![1 # 2], m![1 # 256], m![M, K], m!
[L]>,
) -> DmTensor<i8, m![1], m![1 # 2], m![1 # 256], m![M, K, L]> {
    input.commit_trim::<m![L]>().commit()
}

// Compiler-generated configuration: [
//   M -> 4 : 8,    (8   != 2 * 32, NOT contiguous)
//   K -> 2 : 32,   (32 != 8 * 1,   NOT contiguous)
//   W -> 8 : 1     (packet dimension, contiguous)
// ] : 32
// access_size = 8; valid_size = 8; write_size = gcd(8, 8) = 8; writes per
// packet = 1
fn transpose<'l, const T: Tu>(
    input: ContractTensor<'l, T, f32, m![1], m![1 # 2], m![1 # 256], m![M,
K], m![W]>,
) -> DmTensor<f32, m![1], m![1 # 2], m![1 # 256], m![K, M, W]> {
    input.commit_trim::<m![W]>().commit()
}

// Compiler-generated configuration: [
//   M -> 4 : 8,    (8   != 2 * 32, NOT contiguous)
//   K -> 2 : 32,   (32 != 8 * 1,   NOT contiguous)
//   N -> 8 : 1     (trimmed packet dimension, contiguous)
// ] : 16
// access_size = 8; valid_size = 8 (trimmed from 16); write_size = gcd(8, 8)
= 8; writes per packet = 1
fn transpose_with_trimming<'l, const T: Tu>(
    input: CastTensor<'l, T, i8, m![1], m![1 # 2], m![1 # 256], m![M, K], m!
[N # 32]>,
) -> DmTensor<i8, m![1], m![1 # 2], m![1 # 256], m![K, M, N]> {
    input.commit_trim::<m![N]>().commit()
}
```

Optimizations

Three factors determine Commit Sequencer throughput.

- **Sequential Addresses:** Writing to sequential DM addresses within each slice enables parallel bank access (128 B/cycle per DMN, 256 B/cycle with DMN interleaving). Patterns that hit the same bank 64+ times consecutively trigger [DM Bank Starvation](#).
- **Spatial parallelism:** Distributing writes across all active slices maximizes throughput.
- **Aligned writes** (invariant): Partial bank writes never occur, because both the write address and the write unit are always 8-byte aligned. Sequencer strides are multiples of 8 bytes (see [Constraints](#)), and the [Commit Adapter's Trimming](#) stage holds `D[valid_size]` to a multiple of 8 bytes.

Case Study: Tensor Unit I/O

This end-to-end Tensor Unit I/O pattern uses Fetch and Commit to stage a tensor through DM while preserving every logical index. It is a composed movement pattern, distinct from the individual Fetch Engine and Commit Engine reference pages.

The canonical public case is `fetch_commit_simple`. It accepts an `i8` HBM tensor with mapping `[A, B]`, widens values to `i32` through the Tensor Unit, and returns an HBM tensor with mapping `[B, A]`. The final `to_hbm` DMA call selects the output HBM mapping, so the test proves the complete HBM-to-DM, Fetch, Collect, Commit, and DM-to-HBM pipeline. It intentionally does not claim a streaming permutation: the public example's `[A, B]` to `[B, A]` relay occurs at the final DMA boundary. The device source is included below.

```

#[expect(clippy::type_complexity)]

use furiosa_opt_std::prelude::*;

axes![A = 4096, B = 8];

type Chip = m![1];
type Cluster = m![1 # 2];

#[device(chip = 1)]
pub fn fetch_commit_simple(
    ctx: &mut Context,
    input: &HbmTensor<i8, m![1], m![A, B]>,
) -> HbmTensor<i32, m![1], m![B, A]> {
    // Element's innermost axis must match the source's innermost (B, stride
    1) so that the
    // DMA tail contains the full B axis (8 × i8 = 8 bytes), satisfying
    min_align = 8.
    let input_dm = input.to_dm::<Cluster, m![A / 16], m![A / 8 % 2, A % 8,
    B]>(&mut ctx.tdma);

    let fetch_and_commit_tensor: DmTensor<i32, Chip, Cluster, m![A / 16], m!
[A / 8 % 2, A % 8, B]> = ctx
        .main
        .begin(input_dm.view())
        .fetch::<m![A / 8 % 2], m![A % 8, B]>()
        .fetch_cast::<i32>()
        .collect::<m![A / 8 % 2, A % 8], m![B]>()
        .commit_trim::<m![B]>()
        .commit();

    fetch_and_commit_tensor.to_hbm(&mut ctx.tdma)
}

```

The host oracle and test are included in the examples test target.


```

use furiosa_opt_examples::fetch_commit::fetch_commit_simple;
use furiosa_opt_std::prelude::*;

#[tokio::test]
async fn test_fetch_commit_simple_host() {
    use furiosa_opt_examples::fetch_commit::{A, B};

    let mut ctx = Context::acquire();

    // Create input tensor with shape (A=4096)(B=8).
    let input = HostTensor::<i8, m![A, B]>::from_vec((0..32768).map(|x| x as i8).collect::<Vec<_>>())
        .to_hbm::<m![1], m![A, B]>(&mut ctx.pdma)
        .await;

    // Call the device function.
    let output = launch(fetch_commit_simple, (&mut *ctx, &input)).await;

    let mut expected = vec![0i32; 4096 * 8];
    let mut idx = 0;
    for b in 0..8 {
        for a in 0..4096 {
            expected[idx] = ((a * 8 + b) as i8) as i32;
            idx += 1;
        }
    }

    assert_eq!(
        output.to_host::<m![B, A]>(&mut ctx.pdma).await.into_inner(),
        Tensor::<_, m![B, A], CurrentBackend>::from_vec(expected)
    );
}

```

Run `cargo furiosa-opt test --test fetch_commit_tests` to execute the canonical case. The host oracle checks every `output[b, a]` value against the widened `input[a, b]` value. The streaming path is input DM, Fetch, Cast, Collect, Commit, and output DM with the original `[A, B]` DM layout. The [Fetch Engine](#), [Collect Engine](#), and [Commit Engine](#) document the APIs used here.

DMA Engine

The DMA Engine moves tensors directly between memory tiers without engaging the Tensor Unit pipeline, so use it for residency or layout changes that require no compute stage.

Choose DMA for HBM↔HBM, HBM↔DM, or DM↔DM movement when no Fetch, compute, or Commit stage is needed. The Interface examples show how source tensors, destination mappings, and `tdma` or `pdma` contexts combine. Each transfer pairs two coordinated stages:

- **Read Sequencer**: Reads from the source tier.
- **Write Sequencer**: Writes to the destination tier, possibly with a layout transformation.

A DMA transfer is a [mathematical tensor move](#): the output holds the same mathematical tensor as the input even when the layouts differ. Tensor DMA spans cross-DMN, cross-cluster, and cross-chip transfers, with chip IDs globally agreed across the system.

See [Optimizations](#) for transfer throughput considerations.

Interface

A DMA transfer takes a tensor in one memory tier and produces a tensor in another (or the same) tier. The kernel writer calls `.to_dm()`, `.to_hbm()`, or related methods on the source tensor, passing in a `DmaContext`:

- `Context::tdma`: Tensor DMA context for on-chip transfers (HBM ↔ HBM, HBM ↔ DM, DM ↔ DM).
- `Context::pdma`: PCIe DMA context for host ↔ HBM transfers (see [PCIe DMA](#)).

```
impl<D: Scalar, Chip: M, Element: M, B: Backend> HbmTensor<D, Chip, Element, B> {
    /// Converts to data memory tensor.
    #[primitive(HbmTensor::to_dm)]
    pub fn to_dm<Cluster: M, Slice: M, Element2: M>(
        &self,
        _dma: &mut DmaContext<{ Dma::Tensor }>,
    ) -> DmTensor<D, Chip, Cluster, Slice, Element2, B> {
        assert_dma_layout::<
```

```

        D,
        m![{ Chip }, { Element }],
        Element,
        m![{ Chip }, { Cluster }, { Slice }, { Element2 }],
        Element2,
    >(DMA_SRAM_WRITE_WIDTH);
    DmTensor::from_parts(self.inner.transpose(true), None)
}

    /// Reshapes the tensor to a different mapping at the same HBM address,
    consuming `self`.
    /// The HBM analogue of [`DmTensor::reshape`]; both delegate to
    [`Tensor::reshape`].
    ///
    /// # Safety
    ///
    /// The per-level sizes (`Chip::SIZE == Chip2::SIZE`, `Element`) are
    asserted at compile time below
    /// (see [`constraints::assert_hbm_reshape_dimension_preserved`]); the
    genuine precondition is
    /// [`Tensor::reshape`]'s: the old and new mappings must lay the elements
    out in the SAME physical
    /// (wire) order, so the relabel moves no data. Axis regrouping
    (merge/split) preserves wire order
    /// and is valid; a permutation is not (use a transpose). Equal sizes do
    not guarantee this.
    /// Consuming `self` is the safety contract made explicit: the old-shaped
    handle cannot survive to
    /// alias the same HBM bytes under a conflicting mapping.
    #[primitive(HbmTensor::reshape)]
    pub unsafe fn reshape<Chip2: M, Element2: M>(self) -> HbmTensor<D, Chip2,
    Element2, B> {
        constraints::assert_hbm_reshape_dimension_preserved::<Chip, Chip2,
    Element, Element2>();
        let reshaped = unsafe { self.inner.reshape::<m![{ Chip2 }, { Element2
    }]>() };
        HbmTensor {
            inner: reshaped,
            address: self.address,
            owner: self.owner,
        }
    }
}

```

The compiler derives the read and write sequencer configurations from the source and destination tensor types. The kernel writer specifies the destination type's `cluster`, `slice`, and `Element` (for DM tensors) or `Element` (for HBM tensors), which encode the layout transformation.

The example below transposes a tensor from $[A, B, C]$ to $[C, A, B]$ using two HBM-to-HBM transfers:

```
axes![A = 8, B = 16, C = 32];

fn transpose_simple(
    ctx: &mut Context,
    input: &HbmTensor<f32, m![1], m![A, B, C]>,
) -> HbmTensor<f32, m![1], m![C, A, B]> {
    // Step 1: [A, B, C] → [A, C, B]
    let intermediate: HbmTensor<f32, m![1], m![A, C, B]> = input.to_hbm(&mut
ctx.tdma);

    // Step 2: [A, C, B] → [C, A, B]
    intermediate.to_hbm(&mut ctx.tdma)
}
```

A transfer that crosses tiers also takes a layout transformation through the destination type's mapping. For an HBM-to-DM transfer, the destination DM tensor adds `cluster` and `slice` axes that distribute and broadcast the tensor across hardware partitions.

```
axes![A = 2048];

fn hbm_to_dm(
    ctx: &mut Context,
    input: &HbmTensor<i8, m![1], m![A]>,
) -> DmTensor<i8, m![1], m![1 # 2], m![A / 8], m![A % 8]> {
    input.to_dm::<m![1 # 2], m![A / 8], m![A % 8]>(&mut ctx.tdma)
}
```

Here the 2,048-element vector is distributed as 256 elements per slice (`slice = m![A / 8]`) with 8 elements per slice (`Element = m![A % 8]`), spread across 2 clusters.

Architecture

Each RNGD chip holds 8 DMA Engines, one per pair of DMNs, running up to 8 independent transfers in parallel. A single DMA Engine runs paired read and write sequencers, and a tensor move spreads across multiple engines through an aggregate. The subsections below describe its static structure, sequencer representation, dynamic behavior, compiler derivation, and aggregate operations.

Note

The Tensor Unit (via [Fetch](#) and [Commit](#) Engines) is often more efficient than DMA for SRAM-to-SRAM transfers, since DMA may underutilize SRAM slice bandwidth. HBM bandwidth, however, is typically the bottleneck in practice, making this gap less critical for HBM ↔ DM transfers.

Static Structure

`Chip`, `Cluster`, and `Slice` are the hardware spatial parallelism dimensions. The 8 DMA Engines per chip transfer between different memory components in parallel (e.g., engine #0 handles HBM ↔ DM while engine #1 handles DM ↔ DM).

Each DMA Engine runs paired read and write sequencers in lockstep. The read sequencer traverses source addresses, the write sequencer traverses destination addresses. Both share the same loop count but use different strides and base addresses, since the layout transformation reorders how the same logical elements appear in source vs. destination memory. The compiler represents the pair compactly as a single sequencer with paired strides per loop entry, exploiting the matched read and write counts.

The compiler distributes a single tensor move across the available DMA Engines, partitioning the work along chip, cluster, and slice dimensions and assigning each partition to a DMA Engine. Any DMA Engine handles any transfer. By default the compiler picks the source DM's local DMA Engine, since local DMN access is faster than cross-DMN access. The kernel writer can also specify an engine explicitly.

A homogeneous aggregate uses one descriptor template parameterized across all participating DMA Engines. A heterogeneous aggregate uses a `HashMap<DmnIndex, DmaDescriptor>` that pairs each DMN with its specific descriptor. DM tensor specifications must include chip, cluster, and slice in the mapping expression to identify the exact memory location.

Dynamic Behavior

Each loop in the `DmaSequencer` advances a counter in row-major order, deriving the read and write addresses from the paired strides. For the sequencer below with `base = (0, 2563)` :

```
[
  A -> 256 : (65,536, 256),
  B -> 256 : (256, 65,536),
  C -> 256 : (1, 1),
] : 256
```

iteration i	counters	read addr	write addr
0	(0, 0, 0)	0	write_base
1	(0, 0, 1)	1	1 + write_base
...	...	...	...
255	(0, 0, 255)	255	255 + write_base
256	(0, 1, 0)	256	65,536 + write_base
$i = a \cdot 256^2 + b \cdot 256 + c$	(a, b, c)	i	$256 \cdot a + 256^2 \cdot b + c + \text{write_base}$

With `stride0 = 256`, the hardware reads and writes 256 bytes per iteration, so iteration 0 processes all values for $(A, B, C) = (0, 0, 0..255)$ as a single packet. The transfer completes in approximately 500 cycles of startup latency plus 256×256 cycles of data transfer.

Compiler Derivation

Given source and destination tensor mappings (`In`, `Out`) and a stream shape (`Stream`), the compiler derives the read and write sequencers:

- **Read sequencer:** Projects `In` onto the stream shape, producing per-loop strides into the source tier.
- **Write sequencer:** Projects `Out` onto the stream shape, producing per-loop strides into the destination tier.
- **Unified sequencer:** Merges the two so each entry pairs the read and write strides.
- **Packet size:** Infers `stride0` from the consecutive read/write volume. When both read and write access 256 consecutive bytes, the optimal `stride0` is 256.

For $m![A, B, C] \rightarrow m![B, A, C]$ with $A=B=C=256$, the stream is $m![A, B, C]$. The compiler maps each linear index to A, B, and C using the relation below. It then derives the paired strides.

The relation is $m![A, B, C]::\text{map}(i) = i![A: i / 65,536, B: (i \% 65,536) / 256, C: i \% 256]$ (see [Mapping Expressions](#) for the notation):

```
read_sequencer = [
  A -> 256 : 65,536,
  B -> 256 : 256,
  C -> 256 : 1,
] : 256, HBM @ 0
write_sequencer = [
  A -> 256 : 256,
  B -> 256 : 65,536,
  C -> 256 : 1,
] : 256, HBM @ 2563
```

These combine into a single `DmaSequencer` with paired strides per entry. Each side of the unified `DmaSequencer` (read or write) can be displayed as a single-stride sequencer for that direction, omitting the paired-stride bracket for clarity.

Aggregate Operations

The aggregate takes one of two forms based on whether tensor shapes divide evenly across DMNs.

When shapes divide evenly, all participating engines run a *homogeneous* aggregate. Every DMA Engine uses the same parametric stream environment (`Stream = { chip, cluster, slice, time, packet }`), differing only by base address.

When shapes do not divide evenly, the compiler falls back to a *heterogeneous* aggregate. Each DMN gets its own stream environment via `StreamFn(chip, cluster, slice)`, and boundary DMNs split their work across multiple DMA commands to avoid writing past the valid region. The input and output mapping environments (`In` and `out`) remain structurally identical to the homogeneous case, so the overall logical tensor move is well-defined.

Two correctness invariants apply across both forms:

- **Same media types:** All participating DMA Engines must use the same source and

destination media.

- **Single unified mapping:** One input and one output tensor mapping govern the overall transfer.

Each command incurs its own startup latency, so prefer tensor shapes that divide evenly across DMNs to keep the aggregate homogeneous.

Constraints

The DMA Engine enforces hardware-level alignment and packet-size rules. Violations cause correctness errors or hardware exceptions, not just performance degradation.

- **Address alignment:**

Tier	Read	Write
HBM	1 byte	1 byte
DM (SRAM)	1 byte	8 bytes

HBM ↔ DM transfers additionally require 8-byte alignment for the read address, write address, and packet size, regardless of the table above. The asymmetric DM rule reflects asymmetric SRAM hardware. Read ports use byte-select logic to extract arbitrary byte ranges, but write ports operate on full 8-byte bank-width units. Misaligned DM writes therefore trigger a Read-Modify-Write operation that triples the write time and blocks other operations on the affected bank.

The compiler enforces these constraints as hardware invariants.

- **Packet size:** The maximum packet size is 4,096 bytes, set by the AXI protocol constraint that transactions cannot exceed 256 beats × 16-byte data width.

Optimizations

Three factors determine DMA throughput: memory bandwidth, channel and DMN interleaving, and startup latency with packet splitting.

Memory Bandwidth

Each tier has a peak bandwidth that bounds achievable throughput, and the actual rate is limited by the slowest component on the streaming path.

Tier	Peak bandwidth
HBM	1.5 TB/s per chip (32 channels × 48 GB/s per channel at 0.75 GHz)
DM	256 B/cycle per cluster (with DMN interleaving, 128 B/cycle per DMN)
PCIe	30 B/cycle for both reads and writes (see PCIe DMA)

Each DMA Engine moves up to 256 B/cycle on its own. HBM bandwidth is shared across all engines transferring HBM data, so an aggregate saturating HBM is bounded by the 1.5 TB/s HBM peak rather than by any per-engine sum.

Same-cluster DM-to-DM transfers serialize their reads and writes, since both phases contend for the same DM bank access. Cross-tier transfers like HBM ↔ DM pipeline the read and write phases.

Note

The Tensor Unit (via [Fetch](#) and [Commit](#) Engines) is often more efficient than DMA for SRAM-to-SRAM transfers, since DMA may underutilize SRAM slice bandwidth. HBM bandwidth, however, is typically the bottleneck in practice, making this gap less critical for HBM ↔ DM transfers.

Channel and DMN Interleaving

Sustaining peak bandwidth requires interleaving access patterns across the underlying memory partitions.

HBM channel selection uses address bits 9 to 28, and address bit 8 is the stack bit. Access patterns must toggle all of these bits to spread requests across all 32 channels. Missing the stack bit (address bit 8) alone halves effective bandwidth by routing all requests to only 16 of the 32 channels. Repeated access to the same HBM bank can toggle row-address bits 21 and above. Each conflict costs about 40 cycles and can reduce bandwidth by an order of magnitude. FR-FCFS memory scheduling recovers some throughput, but the fundamental cost remains severe.

DM bandwidth requires alternating between both DMNs (each 128 B/cycle), so a single-

DMN access pattern halves DM bandwidth.

Startup Latency and Packet Splitting

Each DMA command incurs approximately 500 cycles of fixed startup latency before data transfer begins. Combining multiple transfers into fewer commands amortizes this cost, while heterogeneous aggregates split into per-DMN commands and pay the latency on each command.

Within a single command, the hardware splits each packet into 256-byte units, so an n -byte packet becomes $\text{ceil}(n / 256)$ AXI requests. A 4,095-byte packet therefore costs 16 requests, while a 4,099-byte packet (a prime length, awkwardly placed past the 4,096-byte limit) requires splitting into multiple commands. The innermost-loop stride (`stride0`) determines packet alignment: when `stride0` is 256-byte aligned, the cycle count is $\text{ceil}(\text{stride0} / 256)$. When `stride0` is not 256-byte aligned, HBM writes additionally pay a Read-Modify-Write penalty for the partial 256-byte block. HBM reads incur only the `ceil` overhead, and DM operations are largely unaffected by this kind of misalignment.

DMA has the lowest DM bank-access priority, so 64+ consecutive same-bank accesses from the Fetch or Commit Engine can starve it and trigger a NoC timeout. See [DM Bank Starvation](#) for details.

Detailed Examples

The examples below give concrete sequencer configurations and cycle estimates for representative transfer patterns. Examples 1 to 3 cover well-tuned single-engine cases for each tier pair. Examples 4 and 5 contrast pathological access patterns that lose 10x or more. Example 6 illustrates heterogeneous segmentation when shapes do not divide evenly across DMNs.

The sequencer configurations use two address-stride symbols:

- `slice_stride`: The virtual address span of one in-slice DM partition (4 MB).
- `DMN_stride`: The address span between two DMNs within the same cluster.

Example 1: HBM ↔ HBM Layout Transformation

Arguments:

- `axes! [A = 8, B = 8, C = 256]`
- `dtype = i8`
- Source: HBM at offset 0, mapping `m! [A, B, C]`
- Destination: HBM at offset 16,384, mapping `m! [B, A, C]`
- Stream: time `m! [A, B]`, packet `m! [C]`

Generated sequencers:

```
read = [
  A -> 8    : 2,048,
  B -> 8    : 256,
  C -> 256  : 1,
] : 256, HBM @ 0

write = [
  A -> 8    : 256,
  B -> 8    : 2,048,
  C -> 256  : 1,
] : 256, HBM @ 16,384
```

The non-innermost strides (256 and 2,048) toggle HBM address bits 8 (stack) and 11 (channel), spreading every request across distinct HBM channels for parallel execution. A single 256-byte transfer takes 4 cycles per channel at 0.75 GHz, but parallel channel distribution sustains near-peak bandwidth. Total time: approximately 64 read requests + 64 write requests at 1 GHz, plus 500 cycles startup $\approx$ 628 cycles.

When 4 DMA Engines share HBM ↔ HBM traffic, each gets approximately 0.1875 TB/s out of the 0.75 TB/s read bandwidth. Even with `stride0 = 256`, no single engine completes one request per cycle under that share.

Example 2: HBM → DM with Full Bandwidth

This cross-tier transfer pipelines reads and writes by interleaving across HBM channels and both DMNs.

Arguments:

- `axes! [A = 256, B = 256, C = 256]`

- `dtype = i8`
- Source: HBM at chip 0, mapping `m![B, A, C]`
- Destination: DM at chip 0, cluster 0, slice 0. Slice mapping `m![A / 4]`, element mapping `m![A % 4, B, C]`
- Stream: time `m![B, A % 4, A / 4 % 32, A / 128]`, packet `m![C]`

Generated sequencers:

```
read = [
  B      -> 256 : 65,536,
  A%4    -> 4   : 256,
  A/4%32 -> 32  : 1,024,
  A/128  -> 2   : 32,768,
  C      -> 256 : 1,
] : 256, HBM @ 0

write = [
  B      -> 256 : 256,
  A%4    -> 4   : 65,536,
  A/4%32 -> 32  : slice_stride,
  A/128  -> 2   : DMN_stride,
  C      -> 256 : 1,
] : 256, DM @ 0
```

On the HBM side, the 32,768-stride on `A/128=2` interleaves access across channels, and the hardware command queue keeps all 65,536 requests ($256 \times 4 \times 32 \times 2$) flowing. On the DM side, `slice_stride` and `DMN_stride` interleave consecutive 256-byte writes across the two DMNs, keeping both at one request per cycle. Reads and writes pipeline across tiers, so total time is approximately $\max(65,536 \text{ read cycles}, 65,536 \text{ write cycles}) + 500 \text{ startup} \approx 66,036 \text{ cycles}$.

Example 3: DM → DM Within One Cluster

Same-cluster DM-to-DM transfers serialize reads and writes, since both contend for the same DM bank access.

Arguments:

- `axes![A = 256, B = 256, C = 256]`
- `dtype = i8`
- Source: DM at chip 0, cluster 0, slice 0, element offset `0`. Slice mapping `m![A / 4]`, element mapping `m![A % 4, B, C]`

- Destination: DM at chip 0, cluster 0, slice 0, element offset $4 \cdot 256 \cdot 256$. Slice mapping $m![A / 4]$, element mapping $m![B, A \% 4, C]$
- Stream: time $m![B, A \% 4, A / 4 \% 32, A / 128]$, packet $m![C]$

Generated sequencers:

```
read = [
  B      -> 256 : 1,
  A%4     -> 4   : 65,536,
  A/4%32  -> 32  : slice_stride,
  A/128   -> 2   : DMN_stride,
  C       -> 256 : 1,
] : 256, DM @ 0

write = [
  B      -> 256 : 1,024,
  A%4     -> 4   : 256,
  A/4%32  -> 32  : slice_stride,
  A/128   -> 2   : DMN_stride,
  C       -> 256 : 1,
] : 256, DM @ (4*256*256)
```

DMN and slice interleaving give each phase the full 256 B/cycle, but the two phases serialize. Total time: approximately 131,072 cycles (65,536 reads + 65,536 writes) + 500 startup.

Note

Choose c to be a multiple of 256 when possible. For $c = 256n + r$ with $0 < r < 256$, the cycle count grows by a factor of $n+1$ because each access splits into more requests, even though the total data volume changes only slightly.

Example 4: HBM Bank-Conflict Pathology

DM interleaving is healthy here, but a pathological HBM access pattern costs roughly 10x the well-tuned cycle count.

Arguments:

- 1 chip (8 DMNs)
- axes! $[A = 64, B = 2,048, C = 1,024]$
- dtype = i8

- Source: HBM, with cluster mapping $m![B / 1024]$, slice mapping $m![B / 256 \% 4, A]$, element mapping $m![B \% 256, C]$
- Destination: DM, with slice mapping $m![A / 4]$, element mapping $m![B, A \% 4, C]$
- Stream: cluster $m![B / 1024]$, slice $m![B / 256 \% 4]$, time $m![B \% 256, C / 256, A \% 32, A / 32]$, packet $m![C \% 256]$

Generated sequencers per $(cluster_i, dmn_j)$:

```
read = [
    B%256 -> 256 : 1,024,
    C/256 -> 4 : 256,
    A%32 -> 32 : 221,
    A/32 -> 2 : 226,
    C%256 -> 256 : 1,
] : 256, HBM @ (i·220 + j·218)

write = [
    B%256 -> 256 : 1,024,
    C/256 -> 4 : 256,
    A%32 -> 32 : slice_stride,
    A/32 -> 2 : DMN_stride,
    C%256 -> 256 : 1,
] : 256, DM @ (cluster_i, dmn_j, 0)
```

The strides on $A\%32$ and $A/32$ toggle HBM address bits 21 and 26, which select the row within an HBM bank. Consecutive accesses within each channel therefore close one row and open the next on nearly every request, paying approximately 40 cycles per access. Channel interleaving via $C / 256 = 4$ (stride 256) does spread requests across all 32 channels, but cannot hide the row-conflict cost within each channel.

Performance breakdown:

- HBM reads: 524,288 total requests across 32 channels = 16,384 per channel $\times$ ~40 cycles $\approx$ 655,360 cycles.
- DM writes: 65,536 requests per DMN at one per cycle, hidden under the read latency.

Total time: approximately 655,360 cycles + 500 startup $\approx$ 655,860 cycles. FR-FCFS scheduling recovers some throughput, but the order-of-magnitude penalty remains.

Example 5: Missing Stack Bit Pathology

The access pattern fails to interleave across HBM's stack dimension (address bit 8), routing all traffic to half the channels and halving effective bandwidth.

Arguments:

- 1 chip (8 DMNs)
- axes! [A = 8, B = 64, C = 8, D = 512]
- dtype = i8
- Source: HBM, mapping $m![A, B, C, D]$
- Destination: DM, with cluster mapping $m![A / 4]$, slice mapping $m![A \% 4, B]$, element mapping $m![C, D \% 256]$
- Stream: cluster $m![A / 4]$, slice $m![A \% 4]$, time $m![C, B \% 32, B / 32]$, packet $m![D \% 256]$

Generated sequencers per (cluster_i, dmn_j):

```
read = [
    C      -> 8    : 512,
    B%32   -> 32   : 4,096,
    B/32   -> 2    : 131,072,
    D%256  -> 256  : 1,
] : 256, HBM @ (i·220 + j·218)

write = [
    C      -> 8    : 256,
    B%32   -> 32   : slice_stride,
    B/32   -> 2    : DMN_stride,
    D%256  -> 256  : 1,
] : 256, DM @ (cluster_i, dmn_j, 0)
```

The `c` stride of 512 never toggles HBM address bit 8 (the stack bit), so the eight DMNs concentrate on 16 of the 32 HBM channels.

Performance breakdown:

- HBM reads (bottleneck): 4,096 total requests across 16 channels = 256 per channel × ~5.3 cycles per request at 1 GHz ≈ 1,357 cycles.
- DM writes: 512 requests per DMN, pipelined under the reads.

Total time: approximately 1,357 cycles + 500 startup ≈ 1,857 cycles. Restoring stack-bit interleaving across all 32 channels would halve the HBM cycle count.

Example 6: Heterogeneous DMN Segmentation

When tensor shapes do not divide evenly across DMNs, the compiler segments the boundary DMN's work into multiple commands, each paying its own startup latency.

Arguments:

- 4 chips
- `axes! [A = 15, B = 32, C = 256, D = 8]`
- `dtype = i8`
- Source: DM, with (writing $A' = A + 1\#$) chip mapping `m! [D / 2]`, cluster mapping `m! [D % 2]`, slice mapping `m! [A' / 4, A' / 2 % 2, B]`, element mapping `m! [A' % 2, C]`
- Destination: HBM, with chip mapping `m! [D / 2]`, element mapping `m! [D % 2, B, A, C]`
- Stream (per-DMN, expressed as `StreamFn(chip_i, cluster_j, slice_k)`):

```
StreamFn(chip_i, cluster_j, slice_k) = let A' = A + 1# in
{ chip: m! [(D / 2) @ i = 1], cluster: m! [(D % 2) @ j = 1],
  slice: m! [(A' / 4) @ k = 1],
  time: (k == 0,1,2): m! [A' % 2, B, A' / 2 % 2, C]
        (k == 3, exec #0): m! [A' % 2, B, A' / 2 = 1, C]
        (k == 3, exec #1): m! [A' = 1, B, A' / 2 % 2 @ 1, C],
  packet: m! [C] }
```

The dimension $A = 15$ does not divide across 4 DMNs ($15 = 3 \cdot 4 + 3$), so DMNs 0 to 2 each handle 4 elements while DMN 3 handles only 3. A single descriptor on DMN 3 would write a fourth element past the valid region, so the compiler segments DMN 3's work into two commands that together cover exactly 3 elements.

Performance breakdown:

- DMNs 0 to 2 (one command each): ~ 256 cycles + 500 startup ≈ 756 cycles.
- DMN 3 (two commands): ~ 192 data cycles + 1,000 startup (500 each) $\approx 1,192$ cycles.

Total time: approximately 1,192 cycles, gated by DMN 3. Choose tensor shapes that divide evenly across DMNs to avoid this segmentation cost.

Redistribution Operations

Cluster swap exchanges clusters 0 and 1. Chip shuffle redistributes a tensor according to a per-chip source pattern. These methods follow the `to_dm / to_hbm` convention.

`dm_cluster_swap` and `dm_chip_shuffle` are methods on `DmTensorView`;

`hbm_cluster_shuffle` and `hbm_chip_shuffle` are methods on `HbmTensor`. The chip-shuffle pattern specifies, for each destination chip, which source chip provides its data. A cluster swap exchanges clusters 0 and 1.

```
axes![A = 256, B = 4096];
```

```
fn cluster_swap(
    ctx: &mut Context,
    input: &DmTensor<i32, m![A / 4 % 4], m![A / 2 % 2], m![B % 16, B / 16 %
16], m![B / 256, A % 2, A / 16]>,
) -> DmTensor<i32, m![A / 4 % 4], m![A / 2 % 2], m![B % 16, B / 16 % 16], m!
[B / 256, A % 2, A / 16]> {
    input.view().dm_cluster_swap(&mut ctx.tdma)
}
```

Inter-chip shuffles use the system-wide global chip IDs. `hbm_chip_shuffle` is generic over the DMA context (`tdma` or `pdma`) because the cross-chip operation is $\text{HBM} \leftrightarrow \text{HBM}$, and $\text{HBM} \leftrightarrow \text{HBM}$ is the one DMA pair that both Tensor DMA and PCIe DMA support. The other shuffle methods, and DMA pairs like $\text{HBM} \leftrightarrow \text{DM}$ and $\text{DM} \leftrightarrow \text{DM}$ in general, are not context-generic.

Scatter and Gather

Scatter and gather move tensor elements at addresses computed from an index tensor rather than at fixed strides.

`DmTensor::dma_scatter` writes DM values to HBM rows chosen by an index tensor.

`HbmTensor::dma_gather_scaled` and `HbmTensor::dma_gather_unscaled` read HBM rows into DM at rows chosen by an index tensor. The two gather variants differ only in where the index lives and how its values are read, described below.

```

axes![K = 512, D = 128, C = 612, G = 512, CL = 2];

fn scatter_minimal(
    ctx: &mut Context,
    data: &HbmTensor<bf16, m![1], m![K, D]>,
    index: &HbmTensor<i32, m![1], m![K]>,
    output: &mut HbmTensor<bf16, m![1], m![C, D]>,
) {
    let data_dm: DmTensor<bf16, m![1], m![1 # 2], m![K / 2], m![K % 2, D]> =
        data.to_dm(&mut ctx.tdma);

    data_dm.dma_scatter::<m![K], _, _>(index, output);
}

fn gather_minimal(
    table: &HbmTensor<bf16, m![1], m![K, D]>,
    index: &HbmTensor<i32, m![1], m![G]>,
    // The gather axis itself partitions into Slice x Element (`G / 2 = 256`,
    // a valid slice count)
    // with `D` folded into the Element side alongside the `G % 2` remainder;
    `C = 612` (used above
    // for the scatter cache) has no divisor landing on a valid 64 | 128 |
    256 slice count, so the
    // gather count here is the separate, slice-friendly `G` instead.
) -> DmTensor<bf16, m![1], m![1 # 2], m![G / 2], m![G % 2, D]> {
    table.dma_gather_scaled(index)
}

fn gather_unscaled(
    ctx: &mut Context,
    table: &HbmTensor<bf16, m![1], m![K, D]>,
    // Raw row positions per cluster. The kernel stages the index on-chip
    with `to_dm`;
    // a real per-cluster (`CL`) partition avoids broadcast padding.
    index: &HbmTensor<i32, m![1], m![CL, G]>,
) -> DmTensor<bf16, m![1], m![CL], m![G / 2], m![G % 2, D]> {
    let index_dm: DmTensor<i32, m![1], m![CL], m![G / 2], m![G % 2]> =
        index.to_dm(&mut ctx.tdma);
    table.dma_gather_unscaled(&index_dm)
}

```

The scaled variants (`dma_gather_scaled` , `dma_scatter`) take the index from an `HbmTensor` in DRAM and read its values as byte offsets along the gather/scatter axis. To address row `r` , pass `r` times one row's byte size (its element count times the element's byte size, for example $128 * 2 = 256$ for a 128-wide `bf16` row). `dma_gather_unscaled` instead takes an on-chip `DmTensor` index, staged from DRAM with `to_dm` , and reads its values as raw row positions, for indices computed on-chip such as paged-attention block

tables.

PCIe DMA

PCIe DMA (`Context::pdma`) moves tensors between host system memory and device HBM. It is a separate physical engine from the on-chip Tensor DMA. PCIe DMA handles only host $\leftrightarrow$ HBM, while Tensor DMA handles all on-chip transfers.

The kernel writer calls `.to_hbm()` on a `HostTensor` (host $\rightarrow$ device) or `.to_host()` on an `HbmTensor` (device $\rightarrow$ host). Both are async operations.

```
axes![A = 8, B = 512];
```

```
async fn upload_and_download(ctx: &mut Context) {
    let mut rng = SmallRng::seed_from_u64(0);
    let host: HostTensor<i8, m![A, B]> = HostTensor::rand(&mut rng);

    // Host  $\rightarrow$  HBM (allocator-assigned address)
    let hbm: HbmTensor<i8, m![A], m![B]> = host.to_hbm(&mut ctx.pdma).await;

    // HBM  $\rightarrow$  host (back to system memory)
    let _round_tripped: HostTensor<i8, m![A, B]> = hbm.to_host(&mut
ctx.pdma).await;
}
```

`HostTensor` carries only an `Element` mapping (host memory has no chip/cluster/slice partitioning), while the destination `HbmTensor` adds the `chip` axis to distribute across chips. The destination element layout in HBM may differ from the host layout, since `to_hbm` accepts new `chip` and `Element` type parameters.

PCIe DMA bandwidth is 30 B/cycle, an order of magnitude slower than on-chip Tensor DMA (256 B/cycle). Algorithms should minimize host $\leftrightarrow$ device traffic, uploading data once and reusing it across many on-chip operations.

Memory Performance

Memory performance often limits kernel throughput. Compare fixed schedules before changing compute mappings. The Fetch, Commit, and DMA Engines each expose API choices (such as `Packet` size and access ordering) that map directly to performance outcomes. This section explains the hardware constraints behind those choices and how they affect measured throughput.

Each memory type has a peak bandwidth per chip:

Memory	Peak Bandwidth
DM	2 TB/s per chip
HBM	1.5 TB/s per chip

Reaching these peaks requires specific access patterns. The following table lists rules whose violation degrades throughput, and the sections below explain each memory type and factor in detail:

Memory	Issue	Rule	Penalty
DM	Bank starvation	< 64 consecutive same-bank accesses	NoC timeout → hardware reset
DM	DMN interleaving	Alternate across 2 DMNs per cluster	50% bandwidth loss
DM	Slice interleaving	Spread across 32 slices per DMN	Command queue contention
HBM	Alignment	256-byte aligned access	Unaligned read: 2× penalty. Unaligned write: ~50× penalty (RMW)
HBM	Bank conflicts	Avoid row switches within same bank	30–40× degradation
HBM	Channel interleaving	Spread across 32 channels	Reduced parallelism

Use the tables first to choose alignment, interleaving, and packet sizes, then use the detailed sections to diagnose bank starvation or HBM conflicts. The rule table identifies the limiting resource for a candidate mapping, and the DM, SPM, or HBM section explains it.

Data Memory (DM)

Data Memory (DM) holds 256MB per chip, organized hierarchically into clusters, Data Memory Networks (DMNs), slices, and banks. The following table summarizes the geometry:

Unit	Count
Clusters	2 / Chip
Data Memory Networks (DMNs)	8 / Cluster
Slices	32 / DMN
Banks	16 / Slice
Rows	4096 / Bank
Bytes	8 / Row

Clusters can exchange data through the Switch Engine. See [the dedicated section](#) for details. The subsections below explain how this structure determines bandwidth and the bank access constraint.

Bank Structure in a Slice

Each slice provides 512KB of SRAM with a dedicated address space. The memory is organized into 16 parallel banks, each with an 8-byte data width, enabling a total data access rate of 128 B/cycle. Access to any individual bank is serialized, but the address space distributes 128 consecutive bytes across all 16 banks (8 bytes per bank) for parallel access. The following bit mapping defines this distribution:

Bit #	Component
0–2	Byte
3–6	Bank
7–18	Row

Consecutive addresses map to different banks, enabling parallel access during sequential scans.

DMN and Slice Interleaving

Each DMN provides only 128 B/cycle bandwidth (its 32 slices share data paths). Since the standard 256-byte transfer unit requires two cycles per DMN, pipeline accesses across both DMNs to maintain continuous throughput:

cycle	DMN #0	DMN #1
0	read #0 (1/2)	(idle)
1	read #0 (2/2)	read #1 (1/2)
2	read #2 (1/2)	read #1 (2/2)
3	read #2 (2/2)	read #3 (1/2)
...	...	...
2n-1	read #2n-2 (2/2)	read #2n-1 (1/2)
2n	(idle)	read #2n-1 (2/2)

Note

While command queues theoretically allow some burst access without interleaving, always interleave across DMNs when generating DMA streams, as this is the most natural approach.

Slices are shared by the DMA, Fetch, and Commit Engines, so spreading requests across the 32 slices within each DMN reduces contention. Data Memory Routers connect those slices in a ring topology within each DMN: slice0_in → slice31_out, slice32_in → slice63_out. Each Data Memory Slice has a 2-entry command queue for pending DMA requests. Distributing requests across M slices reduces required throughput per slice to 1/M, even when priority delays individual slices. DMN interleaving every n cycles achieves saturated 256 B/cycle.

Bank Starvation

Bank starvation occurs when the DMA Engine is indefinitely blocked waiting for a DM

bank held by higher-priority engines. The 64-access rule prevents this. Violating this rule causes a Network-on-Chip (NoC) timeout and a full cluster reset, losing all computation state.

Each DM bank is a shared resource. When high-priority engines continuously access it, lower-priority requesters are indefinitely blocked, a form of priority inversion. The DM controller prioritizes requests in this order:

- Main-context Fetch Engine
- Main-context Commit Engine
- Sub-context Fetch Engine
- Sub-context Commit Engine
- DMA Engine

DMA has the lowest priority among all memory engines because computation engines must get first access to data during normal operation. High-priority engines can continuously use the same bank. A queued DMA request then waits while those engines retain the bank. Tensor DMA communicates with DRAM and DMN through a NoC hub where each port (DMA, DRAM, DMN) must acknowledge requests within 4,096 cycles. After 4,096 cycles without a response, the NoC protocol declares the transaction dead and enters an exception state as a safety mechanism to detect deadlocks and indefinitely hung transactions. When the timeout triggers, the hardware lacks a graceful recovery mechanism. The only recovery is a full cluster domain reset, losing all computation state and requiring complete reinitialization.

The limit is 64 consecutive accesses. Fetch and Commit must not retain the same bank for 64 or more operations while DMA is active. Why 64? The constraint is $(\text{TDMA_IO_BYTE} / \text{DMN_IO_BYTE}) * \text{Max_Consecutive_Access} * \text{DMN_SIZE} < 4096$ (with $\text{TDMA_IO_BYTE} = 256$, $\text{DMN_IO_BYTE} = 128$, $\text{DMN_SIZE} = 32$), which yields $\text{Max_Consecutive_Access} < 64$. This ensures DMA requests complete before the NoC timeout even in the worst case.

For example, suppose the DMA Engine issues a request to bank 0 (along with 15 other banks), but the main-context's Fetch Engine continuously requests bank 0. The DMA request stalls, and if this exceeds 4,096 cycles, a NoC timeout forces a hardware reset.

See [Scheduling: Resource and bank contention](#) for context occupancy, the 64-access rule, and main/sub bank contention. See [Schedule Viewer](#) for a scheduling visualization utility that shows which operations run in parallel and verifies actual context assignments.

High-Bandwidth Memory (HBM)

HBM holds 48GB per chip and delivers 1.5 TB/s aggregate bandwidth, but reaching that peak requires 256-byte-aligned access and channel interleaving across all 32 channels. Misaligned writes and bank conflicts can degrade throughput by 30–50×. The following table summarizes the HBM geometry:

Unit	Count
Stacks	2 / Chip
Channels	16 / Stack
Slices	3 / Channel
Bank Groups	4 / Slice
Banks	4 / Bank Group
Rows	16K / Bank
Bytes	2K / Row

Peak Bandwidth

Saturating a single DMA Engine (256GB/s capacity) requires interleaving accesses across multiple channels. Peak HBM bandwidth reaches 1.5TB/s per chip through parallel operation of stacks and channels. The channel controller transfers 64B/cycle at 0.75GHz,¹ yielding 48GB/s per channel (0.75GHz x 64B/cycle) or 1.5TB/s per chip (48GB/s x 32 channels). The fundamental transfer unit is 256 bytes, requiring 4 clock cycles per channel.

Peak bandwidth is sensitive to access patterns. Misalignment, bank conflicts, and resource sharing can each severely degrade throughput. Each channel controller has a 64-entry command queue that interleaves accesses to minimize penalties, but pathological cases can still cause severe degradation. The following sections describe causes of performance degradation and how to avoid them.

Address Space in a Chip

The HBM address space uses a non-linear bit mapping optimized for parallel sequential access. This design maximizes parallelism and minimizes overhead:

Bit #	Main Component	Additional Components
0-7	Byte	
8	Stack	
9-12	Channel	
13	Bank Group	Channel
14-16	Byte	Channel
17-18	Bank	Channel
19	Bank Group	Channel
20	Slice	Channel
21-33	Row	Channel (21-28)
34	Slice	Row
35	Row	

The bit assignment for each component corresponds to the physical memory geometry. For instance, the byte component occupies 11 bits (bits 0-7, 14-16) to represent 2K (2^{11}) bytes per row. Three exceptions exist:

- **Slice representation:** Two bits (20 and 34) represent slice, even though there are only three slices.
- **Contiguous address space:** Bit 34 is influenced by the row component to ensure bits 34 and 35 are never both 1, guaranteeing a contiguous 48GB address space.
- **Channel XOR mapping:** The channel component equals the XOR of bits 9-12 and 13-28 (e.g., the channel's first bit equals the XOR of bits 9, 13, 21, and 25).

This bit ordering ensures that sequential accesses are spread across stacks, channels, bank groups, and banks simultaneously, keeping multiple memory resources busy in parallel.

Misaligned Access

Misaligned access degrades HBM performance substantially. Reads crossing a 256-byte boundary require two transfers (2x penalty), and unaligned writes require a Read-Modify-Write (RMW) operation (roughly 50x penalty). The 256-byte minimum access unit is defined by bits 0-7 (the eight LSBs), so data that crosses this boundary incurs these penalties.

- **Unaligned Read:** Read requests crossing a 256-byte boundary require two NoC

transfers, effectively halving bandwidth.

- **Unaligned or Partial Write:** An unaligned write arises because DMA packets are internally segmented into 256-byte transactions. When a packet's size is not 256-byte aligned (e.g., a 2,800-byte packet splits into ten 256-byte requests plus one 240-byte request), the final "leftover" transaction requires an RMW operation. RMW reads the entire 256-byte unit, updates the requested bytes, then writes the entire unit back. RMW can slow writes by roughly 50× compared to aligned writes.

Bank Conflict

HBM banks hold one open row at a time. Switching to a different row within the same bank requires closing the current row and opening the new one. This adds 40-50 ns (60-75 cycles at 1.5 GHz) of latency, which is 30-40x slower than accessing an already-open row. This penalty occurs whenever consecutive accesses target different rows within the same bank. All rows start closed, so the first access to any row always pays the open-row cost.

Channel interleaving mitigates bank conflicts. Interleaving accesses across all 32 channels distributes load and reduces conflicts. Bits 8-12 (the next five LSBs) represent independent stacks and channels. Placing these at low addresses prevents interference between adjacent accesses, which is vital for parallelizing contiguous operations. Non-contiguous operations often benefit from natural channel interleaving because the channel component spans bits 9-28. However, the stack component corresponds only to bit 8, so the programmer must explicitly ensure accesses alternate between the two stacks to achieve full stack interleaving.

The controller hides row-switch latency through command interleaving. Within each channel, the controller automatically interleaves commands across banks, enabling useful transfers while other banks perform row switches. The controller manages bank states using its command queue. It employs FR-FCFS (First Ready-First Come First Served) scheduling, prioritizing commands targeting already-open rows.

Despite this sophisticated scheduling, access patterns that continuously switch rows within the same bank still degrade performance significantly. Compilers and programmers should estimate row-switch costs when generating code.

Column-to-Column Delay

t_{CCD} (Column-to-Column Delay) is the minimum time between consecutive read or write commands on the same channel, which determines the maximum command issue rate. In most access patterns, bank conflicts or channel interleaving dominate before t_{CCD} becomes the bottleneck. Vendor specifications set t_{CCD} values based on analog constraints for accessing DRAM stack layers and shared resources.

The t_{CCD} value depends on which memory resources consecutive commands target:

Command Relation	t_{CCD} (cycles @ 1.5GHz)	Relative Performance	Reason for Penalty
Same Slice, Different Bank Group	2	1	Ideal interleaving of bank groups
Different Slice	3	2/3	Data path switching
Same Slice, Same Bank Group	4	1/2	Shared I/O buffer among four banks

Interleaving bank groups within one slice gives $t_{CCD} = 2$ cycles at 1.5GHz. It allows a new 64B command each 0.75GHz cycle and reaches full channel speed. Any t_{CCD} greater than 2 reduces the command rate and channel utilization.

Compared to bank conflicts, t_{CCD} degradation is less severe because the worst-case patterns either coincide with bank conflicts (making t_{CCD} the secondary effect) or are masked by channel interleaving:

- Different Slice ($t_{CCD} = 3$): Slice ID corresponds to bit 20, and bit 21 corresponds to the row. Interleaving across slices therefore likely causes bank conflicts simultaneously.
- Same Slice, Same Bank Group ($t_{CCD} = 4$): This pattern interleaves bits 8–35 except bits 13, 19, 20, and 34. Bits 29–35 relate to bank conflicts. Bits 8–28 relate to channel interleaving.

-
1. Although the channel controller operates at a frequency of 0.75GHz, it performs eight bursts per cycle, leading to an effective frequency of $0.75 \times 8 = 6\text{GHz}$. ↩

Computing Tensors

Computing Tensors explains how a valid Tensor Unit stream reaches the Vector or Contraction Engine. A stationary operand stays in a register file while the other operand streams through the engine. The Tensor Register File (TRF) stores contraction operands, and the Vector Register File (VRF) stores vector operands. This chapter explains the engine behavior that places operands and preserves stream dimensions.

Selecting a Compute Route

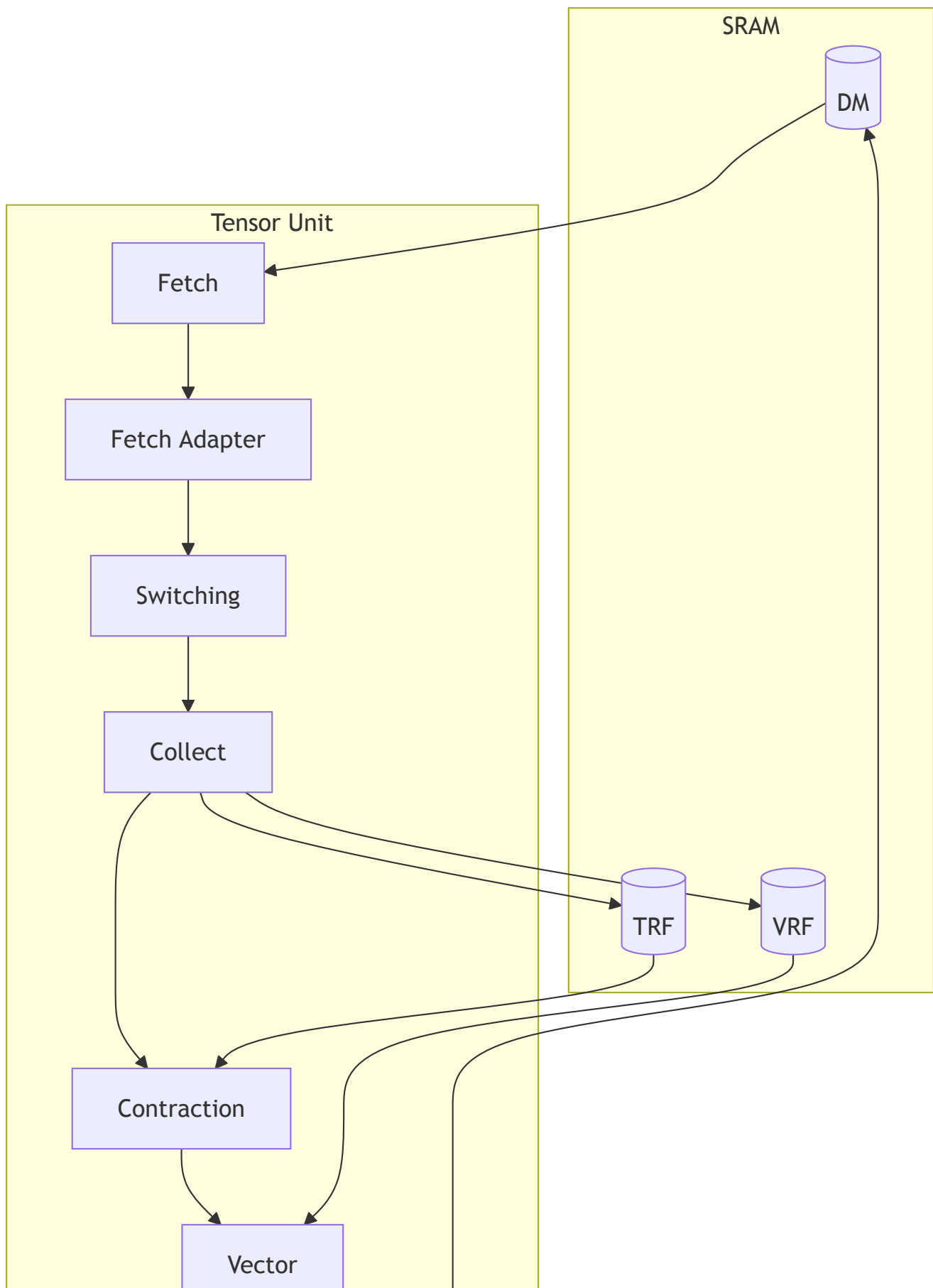
Kernel need	Select	Behavior to read next
Elementwise operation or reduction	Vector Engine	Vector stream and reduction contracts
Contraction such as matmul or convolution	Tensor Register File (TRF) plus Contraction Engine	TRF layout and contraction mapping
Cross-slice redistribution or reduction	Switch Engine or Inter-Slice Reducer	Slice movement and reducer contracts
Layout or precision change	Fetch/Commit Adapter, Cast Engine, or Transpose Engine	Adapter stage and output mapping contracts

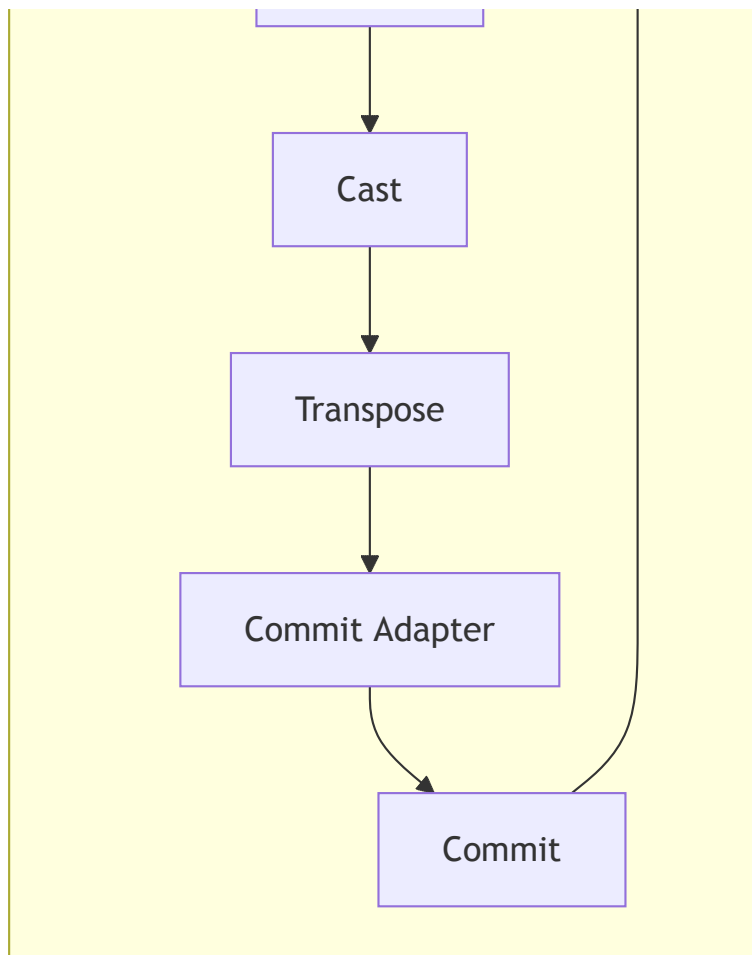
Tensor Unit

The Tensor Unit is the on-chip compute pipeline. It reads tensor data from DM, transforms it through ten engines, and writes results back to DM.

Each tensor flows through the pipeline as a stream of packets, one packet per cycle. The engines consume and produce these streams, reshaping the per-cycle layout and the iteration order along the way. The Collect Engine normalizes incoming packets to 32-byte *flits*. Every downstream engine operates on these flits. The pipeline includes Contraction,

Vector, Cast, Transpose, Commit Adapter, and Commit. See the linked engine pages for [Contraction](#), [Vector](#), [Cast](#), [Transpose](#), [Commit Adapter](#), and [Commit](#).





Engine	Function	Key Constraint
Fetch	Load data from DM into the pipeline	Packet must be 8-byte aligned. <code>slice</code> is preserved unless <code>axis lifting</code> replaces a broadcast with an axis from <code>Time</code>
Fetch Adapter	Per-element transforms after fetch (table lookup, cast)	Optional. Identity if skipped
Switching	Move data across slices	Ring network, <code>slice</code> can change
Collect	Normalize packets to 32-byte flits	Output = exactly one flit
Contraction	Einsum: matmul, convolution, attention	One operand resident in TRF. The other streams
Vector	Elementwise, binary, reduce operations	Only i32/f32 input

Cast	Precision lowering with batching	Output = exactly one flit
Transpose	Reorder elements within a flit	Within-flit only
Commit Adapter	Per-element transforms before commit (cast, ReLU, trim)	Optional. Chained before <code>.commit()</code>
Commit	Write results back to DM	Flit-aligned writes

Each tensor stream inside the Tensor Unit carries five dimensions, `[Chip, Cluster, Slice, Time, Packet]`, that split into two groups. `Chip`, `Cluster`, and `Slice` are spatial dimensions: each slice runs its own pipeline instance, with slices grouped by cluster and clusters grouped by chip. `Time` and `Packet` describe the per-slice stream (see [Spatial and Temporal Dimensions](#) for the definitions). The engines above reshape `Time` / `Packet` along the pipeline. Most engines preserve the spatial dimensions. [Switch](#) moves data across slices. Fetch's [axis lifting](#) assigns different DM read offsets along `Chip`, `Cluster`, or `Slice`, moving an axis out of `Time`. The [Vector inter-slice reducer](#) combines the 256 slices in a cluster.

The Contraction and Vector Engines each take one operand from the pipeline stream and the other operand from a dedicated per-slice register file. TRF (Tensor Register File) feeds the Contraction Engine, and VRF (Vector Register File) feeds the Vector Engine. The Collect Engine writes into TRF via `.to_trf()` and into VRF via `.to_vrf()`. For an end-to-end example using both files, see [Quick Start](#).

Fetch reads from DM and Commit writes back to DM. Their detailed sequencer behavior is documented in [Moving Tensors](#) rather than here.

Execution Context

The [scheduler](#) treats each *execution context* as an independent stream of operations. The hardware exposes three:

- **Main** drives the Tensor Unit pipeline for the kernel's primary computation.
- **Sub** drives a subset of the same pipeline, typically prefetching operands into TRF /

VRF while main computes.

- **DMA** drives the DMA Engine alone, external to the Tensor Unit.

The main context can drive every Tensor Unit engine. The sub context drops the Contraction Engine and a handful of other features. Everything else carries over from main.

Context ordering, overlap, resource conflicts, and memory-rule scheduling are defined in [Schedule](#).

Fetch Adapter

The Fetch Adapter applies element-wise transformations (type casting, masking, table lookup, zero-point subtraction) to the packet stream emitted by the [Fetch Engine](#), before the [Switch Engine](#) routes it across slices. The Fetch Engine itself does not run any of these transforms; they live here under Computing Tensors and are applied as separate stages between the Fetch Engine and the Switch Engine. The kernel writer composes the per-stage methods directly on a `FetchTensor`, and each call advances to the next stage.

The adapter has three usable stages, each optional and invoked by calling its method on the stream in hardware pipeline order. A `FetchTensor` may flow directly into the [Switch Engine](#) or the [Collect Engine](#) with no adapter call at all.

- [Table Lookup](#) replaces values via a hardware lookup table.
- [Type Casting](#) converts the element type.
- [Zero-Point Subtraction](#) subtracts a quantization zero point, widening an integer stream to the [Contraction Engine](#)'s staging type (`i4` to `i5`, `i8` to `i9`).

Table Lookup

Table lookup provides hardware-accelerated lookup tables during the fetch stage. Each value is treated as an index into a pre-configured table, and the corresponding table entry is output instead. This is useful for operations that cannot be efficiently implemented with standard arithmetic, such as non-linear activation functions like Sigmoid and GeLU, or quantization schemes that use custom encoding tables. This enables:

- **Non-linear activations:** Implements Sigmoid, GeLU, and other functions through pre-computed lookup tables.
- **Custom type casting:** Translates specialized encodings like `MXFP4` / `NVFP4` to standard formats using conversion tables.

Sigmoid and GeLU can also be expressed directly in the [Vector Engine](#), so table lookup is one option among several for these activations rather than the only path.

```
// Table lookup is a **main-context-only** hardware feature: only the main
Fetch Unit's
// `mode_indirect_table` can point at a resident lookup table, so this impl
```

```

is fixed to
// `{ Tu::Main }`. The sub-context Fetch Unit (used by the StoTrf weight-
staging path) has no
// table-lookup register, so decoding a packed weight must happen in a main-
context fetch that
// commits the decoded stream to DM, after which a plain convert-only StoTrf
stages it into the TRF.
impl<
  'l,
  P: CanApplyFetchTableLookup,
  D: MaterializableScalar,
  Chip: M,
  Cluster: M,
  Slice: M,
  Time: M,
  Packet: M,
  B: Backend,
> TuTensor<'l, { Tu::Main }, P, D, Chip, Cluster, Slice, Time, Packet, B>
{
  /// Runs the Fetch Adapter's table-lookup stage (main context only).
  ///
  /// Each input value indexes a decode table selected by the `TableLookup`
impl,
  /// not by a runtime argument: `f4e2m1` decodes to either 8-bit float
through the
  /// paired 4b->8b table (the NVFP4 / MXFP4 weight path), and either 8-bit
float
  /// decodes to `bf16` through the non-paired table. Chain a
  /// [`fetch_cast`](Self::fetch_cast) to widen a paired decode to `f32`;
the
  /// per-block scale is applied downstream in the Vector or Contraction
Engine.
  /// See the book chapter `computing-tensors/fetch-adapter.md` ("Table
Lookup").
  ///
  /// Only available on the main context: the lookup table lives in a main
Fetch Unit register the
  /// sub context lacks. To feed a decoded weight into a contraction,
decode here and commit the
  /// `f8e4m3` stream to DM, then stage that DM tensor into the TRF with a
plain convert-only StoTrf.
  #[primitive(TuTensor::fetch_table_lookup)]
  pub fn fetch_table_lookup<OutD: Scalar>(
    self,
  ) -> FetchTypeLookupTensor<'l, { Tu::Main }, OutD, Chip, Cluster, Slice,
Time, Packet, B>
  where
    D: TableLookup<OutD>,
  {
    FetchTypeLookupTensor::new(self.ctx, self.inner.map(|v| v.lookup()))

```

```

    }
}

```

The decode table is selected by the input scalar type (via the `TableLookup` trait), not by a runtime argument, exactly as `fetch_cast` selects its conversion by the input type. The key width picks the table and the requested output type picks the entry encoding:

Input	Output
<code>f4e2m1</code>	<code>f8e4m3</code> , <code>f8e5m2</code>
<code>f8e4m3</code>	<code>bf16</code>
<code>f8e5m2</code>	<code>bf16</code>

Widen a decoded 8-bit float to `f32` with a following `fetch_cast` , and apply any NVFP4 / MXFP4 per-block scale downstream in the [Vector Engine](#) or the [Contraction Engine](#) (it is intentionally not folded into the table, which stays a static 16-entry, block-independent decode).

The table for the `f4e2m1` decode is a compile-time constant baked into the fetch-sequencer configuration, so the `f4e2m1` weight stream is the only data input to the stage; there is no per-invocation table staging. The 16-entry e2m1 decode is block-independent, which is why the per-block scale stays out of it: folding the scale in would need a distinct table per block and defeat the shared static table.

The hardware sequencer walks byte-aligned keys (1 or 2 bytes), never a raw 4-bit nibble, and a 4-bit key must use a *paired* table. The `f4e2m1` decode therefore runs as a 256-entry byte-indexed table: each byte carries two nibbles and one lookup yields the pair of decoded `f8e4m3` values, decoding at two elements per key. The `f4e2m1` scalar is modelled accordingly as a byte holding two nibbles, with `BITS = 4` so the fetch mapping accounts elements rather than bytes.

```
axes![A = 8];
```

```
/// Decodes an e2m1 (NVFP4 / MXFP4) weight stream to f8e4m3 via the hardware
table,
```

```
/// then widens to f32 with a following fetch_cast.
```

```
fn fetch_decode_e2m1<'l, const T: Tu>(
    input: BeginTensor<'l, T, f4e2m1, m![1], m![1], m![1], m![1], m![A]>,
) -> FetchCastTensor<'l, T, f32, m![1], m![1], m![1], m![1], m![A]> {
    input
        .fetch::

```

Type Casting

`fetch_cast:: converts the element type from D to OutD, preserving the Time and Packet mapping. Type casting adds 1 to 2 cycles of latency. fetch_cast performs only the type conversion; the integer widenings that hold a zero-point offset (i4 to i5, i8 to i9) are a separate stage, Zero-Point Subtraction, so fetch_cast never produces an i5 / i9.`

```

impl<
  'l,
  const T: Tu,
  P: CanApplyFetchCast,
  D: MaterializableScalar,
  Chip: M,
  Cluster: M,
  Slice: M,
  Time: M,
  Packet: M,
  B: Backend,
> TuTensor<'l, T, P, D, Chip, Cluster, Slice, Time, Packet, B>
{
  /// Runs the Fetch Adapter's type-casting stage.
  ///
  /// Converts the stream's element type from `D` to `OutD`. The mapping
  /// shape is preserved.
  #[primitive(TuTensor::fetch_cast)]
  pub fn fetch_cast<OutD: Scalar>(self) -> FetchCastTensor<'l, T, OutD,
Chip, Cluster, Slice, Time, Packet, B>
  where
    D: FetchCast<OutD>,
    {
      FetchCastTensor::new(self.ctx, self.inner.map(|v| v.cast()))
    }
}

```

RNGD supports the conversions below and no others: every widening lands in the `i32` / `f32` compute width, and `f32` to `bf16` is the only narrowing.

Input	Output
<code>i4</code>	<code>i32</code>
<code>i8</code>	<code>i32</code>
<code>i16</code>	<code>i32</code>
<code>f8e4m3</code>	<code>f32</code>
<code>f8e5m2</code>	<code>f32</code>
<code>bf16</code>	<code>f32</code>
<code>f32</code>	<code>bf16</code>

In particular the Fetch Adapter has no cast from an 8-bit float to `bf16`, and none from `f32` to an 8-bit float. To land an `f8e4m3` stream in `bf16`, decode it through the non-paired [Table Lookup](#) table instead; to reach an 8-bit float, narrow with the [Cast Engine](#) later in the pipeline.

The example below fetches an 8-element `i8` stream and casts it to `i32`. The `Time` and `Packet` mapping is unchanged across the call.

```
axes![A = 8];
```

```
/// Fetches with type casting: converts i8 storage to i32 for computation.
/// Input:   i8 [0, 1, 2, 3, 4, 5, 6, 7]
/// Output: i32 [0, 1, 2, 3, 4, 5, 6, 7]
fn fetch_with_type_cast<'l, const T: Tu>(
    input: BeginTensor<'l, T, i8, m![1], m![1], m![1], m![1], m![A]>,
) -> FetchCastTensor<'l, T, i32, m![1], m![1], m![1], m![1], m![A]> {
    input.fetch::

```

Type casting adds an additional limit on `read_size`. The cast output per fetch must fit in a single 32-byte flit (see [Collect Engine](#)).

- Valid:
 - `i4 -> i32, read_size = 8` (4 bytes) : produces $8 \times 4 = 32$ B
 - `i8 -> i32, read_size = 8` (8 bytes) : produces $8 \times 4 = 32$ B
- Invalid:
 - `i4 -> i32, read_size = 16` (8 bytes) : produces $16 \times 4 = 64$ B
 - `i8 -> i32, read_size = 16` (16 bytes) : produces $16 \times 4 = 64$ B

Zero-Point Subtraction

`fetch_zero_point_sub:: subtracts the quantization zero_point from each element and widens the stream to the Contraction Engine's staging type: i4 to i5, i8 to i9. It is the only stage that produces an i5 / i9.`

```

impl<
  'l,
  const T: Tu,
  P: CanApplyFetchZeroPointSub,
  D: MaterializableScalar,
  Chip: M,
  Cluster: M,
  Slice: M,
  Time: M,
  Packet: M,
  B: Backend,
> TuTensor<'l, T, P, D, Chip, Cluster, Slice, Time, Packet, B>
{
  /// Runs the Fetch Adapter's zero-point-subtraction stage.
  ///
  /// Subtracts `zero_point` and widens the stream from `D` to its
contraction-engine staging
  /// type `OutD` (`i4 -> i5`, `i8 -> i9`), the only way to produce an
i5/i9
  /// stream. The result may only feed `contract_outer`; it is not
  /// [`MaterializableScalar`], so committing or re-routing it is a compile
  /// error. The mapping shape is preserved.
  ///
  /// Panics if `zero_point` is outside the source type's range
  /// ([`FetchZeroPointSub::ZERO_POINT_RANGE`]); a zero point in range
keeps
  /// every widened residual within `OutD`, so this one check (independent
of
  /// the stream data) is enough.
  #[primitive(TuTensor::fetch_zero_point_sub)]
  pub fn fetch_zero_point_sub<OutD: Scalar>(
    self,
    zero_point: i32,
  ) -> FetchZeroPointSubTensor<'l, T, OutD, Chip, Cluster, Slice, Time,
Packet, B>
  where
    D: FetchZeroPointSub<OutD>,
  {
    let zero_point_range = <D as
FetchZeroPointSub<OutD>>::ZERO_POINT_RANGE;
    assert!(
      zero_point_range.contains(&zero_point),
      "zero_point {zero_point} is outside the source type's quantized
range {zero_point_range:?}",
    );
    FetchZeroPointSubTensor::new(self.ctx, self.inner.map(|v|
v.zero_point_sub(zero_point)))
  }
}

```

Extra-Bit Rationale

Subtracting the zero point turns an unsigned-around- `zero_point` quantized value into a signed residual whose range no longer fits the input width. For a symmetric-signed input the residual is a difference of two same-width values:

- `i4` residual: $[-8, 7] - [-8, 7] = [-15, 15]$, which needs `i5`'s $[-16, 15]$.
- `i8` residual: $[-128, 127] - [-128, 127] = [-255, 255]$, which needs `i9`'s $[-256, 255]$.

The subtraction therefore produces one more bit than it consumes. The conversion checks this at runtime: a residual outside the `i5 / i9` range (an out-of-range `zero_point` or input) is rejected rather than silently wrapped.

Contraction-Only Staging

An `i5 / i9` stream may flow through the [Switch Engine](#) and [Collect Engine](#), but from there its **only** legal consumer is `contract_outer`, which pairs it with a weight as [Operand types](#) lays out. It cannot be committed to memory, stored to a register file (`to_trf / to_vrf`), transposed, or fed to any other engine.

This restriction is enforced at compile time, not by a runtime check: passing an `i5 / i9` stream to any consumer other than `contract_outer` is a compile error.

Switch Engine

While every other Tensor Unit engine runs per-slice on its own DM partition, the Switch Engine moves data across slices through a 256-slice ring network: broadcasting one slice's value to a group, swapping values between slices, or permuting which slice holds which value.

Interface

`FetchTensor::switch()` produces a `SwitchTensor`, preserving `Chip`, `Cluster`, `Packet`, and the underlying data values. Only the `slice` and `Time` mappings change to reflect the selected configuration.

```
impl<'l, const T: Tu, P: CanApplySwitch, D: Scalar, Chip: M, Cluster: M,
Slice: M, Time: M, Packet: M, B: Backend>
    TuTensor<'l, T, P, D, Chip, Cluster, Slice, Time, Packet, B>
{
    /// Applies switching network routing only. The packet passes through
    /// unchanged, no padding, no reshaping. Use `collect` afterwards to
    /// normalize the packet to flit-sized chunks.
    #[primitive(TuTensor::switch)]
    pub fn switch<OutSlice: M, OutTime: M>(
        self,
        config: SwitchConfig,
    ) -> SwitchTensor<'l, T, D, Chip, Cluster, OutSlice, OutTime, Packet, B>
{
    verify_switch::<Slice, Time, OutSlice, OutTime>(&config);
    SwitchTensor::new(self.ctx, self.inner.transpose(true))
}
}
```

The kernel writer picks `OutSlice`, `OutTime`, and a `SwitchConfig` argument that selects the configuration and its parameters. `SwitchConfig` is one of the predefined variants (`Broadcast01`, `Broadcast1`, `Transpose`, `InterTranspose`, `TransposedBroadcast1`) for common patterns, or a `CustomBroadcast` for more general patterns. The numeric suffix in each variant name lists the slice sub-dimensions that move out of `slice`. For example, `Broadcast01` broadcasts both `slice0` and `slice1`, while `Broadcast1` broadcasts only `slice1`.

The compiler verifies that `OutSlice` and `OutTime` match the configuration's required

dimension structure (each per-config section below shows the required structure with input/output diagrams), and compilation fails when they do not. Every configuration also requires `InSlice::SIZE == OutSlice::SIZE`: the switch preserves the total slice count.

Regular Configurations

Each regular configuration partitions the 256 slices on a chip into parallel **sub-rings** of `ring_size` slices each. `ring_size` is derived from the configuration's parameters (typically `slice1 × slice0`) and determines both the partitioning granularity and the cycle cost. See [Architecture](#) below for the ring topology and per-router decision logic, and [Performance](#) for the cycle-factor breakdown.

Regular configurations cover six common patterns. Arbitrary `slice` sub-dimension permutations not expressible by one of these patterns require [CustomBroadcast](#) instead.

Configurations that introduce broadcast axes (`X` or `Y` in `Broadcast01`, `Broadcast1`, `TransposedBroadcast1`) require those axes to be new: they must not already appear in input `slice` or input `Time`.

Configuration	Use case	<code>ring_size</code>
Forwarding	pass each slice's data through unchanged (no inter-slice exchange)	1
Broadcast01	broadcast both inner <code>slice</code> sub-dimensions (<code>slice1</code> and <code>slice0</code>) to every slice in a sub-ring	<code>slice1 × slice0</code>
Broadcast1	broadcast <code>slice1</code> while keeping <code>slice0</code> in <code>slice</code>	<code>slice1 × slice0</code>
Transpose	swap <code>slice1</code> and <code>slice0</code> within <code>slice</code>	<code>slice1 × slice0</code>
InterTranspose	swap a <code>slice</code> sub-dimension (<code>slice1</code>) with a <code>Time</code> sub-dimension (<code>time1</code>)	<code>slice1 × slice0</code>
TransposedBroadcast1	broadcast <code>slice0</code> to <code>Time</code> while shifting <code>slice1</code> to innermost <code>slice</code> (equivalent to <code>Transpose</code> then <code>Broadcast1</code>)	<code>slice1 × slice0</code>

Forwarding

Forwarding leaves the `slice` and `Time` mappings unchanged: every router outputs its own slice's input directly, with no cross-slice movement.

`SwitchConfig` has no `Forwarding` variant. When no inter-slice exchange is needed, skip `.switch()` and call `.collect()` directly on the `FetchTensor`.

```
axes![A = 256, B = 64, C = 32];
```

```
fn forwarding<'l, const T: Tu>(
    input: FetchTensor<'l, T, f32, m![1], m![1 # 2], m![A], m![B], m![C]>,
) -> CollectTensor<'l, T, f32, m![1], m![1 # 2], m![A], m![B, C / 8], m![C %
8]> {
    input.collect()
}
```

Broadcast01

Broadcast01 broadcasts each slice's data to every slice in a sub-ring, moving both inner `slice` sub-dimensions (`slice0` and `slice1`) into `Time`.

The input dimension structure (outermost to innermost, left to right):

Slice			Time	
slice2	slice1	slice0	time1	time0

After switching, `slice1` and `slice0` move from `slice` into `Time` to broadcast across the sub-ring. Two new broadcast dimensions, labeled `x` and `y`, fill their vacated `slice` positions:

Slice			Time			
slice2	x	y	time1	slice1	time0	slice0

The sub-ring spans `ring_size = slice1 × slice0` slices, one for each (`slice1`, `slice0`) combination at fixed `slice2`. Every slice sends its packet around the sub-ring,

and every slice receives all `ring_size` packets, so each output slice ends up holding the full broadcast group's data, indexed along the new `x` and `y` axes in the output `slice`.

Example

```
axes![A = 256, B = 64, C = 63, D = 2, X = 2, Y = 2];

fn broadcast01<'l, const T: Tu>(
    input: FetchTensor<'l, T, f32, m![1], m![D], m![A], m![B], m![C # 64]>,
) -> SwitchTensor<'l, T, f32, m![1], m![D], m![A / 4, X, Y], m![B / 4, A / 2
% 2, B % 4, A % 2], m![C # 64]> {
    input.switch::<m![A / 4, X, Y], m![B / 4, A / 2 % 2, B % 4, A % 2]>(<
        SwitchConfig::Broadcast01 {
            slice1: 2,
            slice0: 2,
            time0: 4
        }
    )
}
```

With `slice1 = 2` (size of broadcast `x`), `slice0 = 2` (size of broadcast `y`), and `time0 = 4`, the compiler derives `slice2 = 64`, `time1 = 16`, and `ring_size = 4` (64 sub-rings span 256 slices).

Sub-dimensions resolve to `slice2 = A / 4`, `slice1 = A / 2 % 2`, `slice0 = A % 2`, `time1 = B / 4`, `time0 = B % 4`, giving `OutSlice = m![A / 4, X, Y]` and `OutTime = m![B / 4, A / 2 % 2, B % 4, A % 2]`.

Cycle estimate: $\text{ring_size} \times \text{Time::SIZE} \times \text{flits_per_packet} = 4 \times 64 \times 8 = 2048$, where $\text{Time::SIZE} = 64$ and $\text{flits_per_packet} = \text{sizeof}(f32) \times \text{Packet::SIZE} / 32 = 4 \times 64 / 32 = 8$.

Broadcast1

The input dimension structure (outermost to innermost, left to right):

Slice			Time
slice2	slice1	slice0	time0

After switching, `slice1` moves from `slice` into `Time` to broadcast across the sub-ring while `slice0` stays in `slice`. A new broadcast dimension labeled `x` fills `slice1`'s vacated `slice` position:

Slice			Time	
<code>slice2</code>	<code>x</code>	<code>slice0</code>	<code>time0</code>	<code>slice1</code>

The sub-ring spans `ring_size = slice1 × slice0` slices, the same physical extent as `Broadcast01`'s. But broadcasting happens only along `slice1`: each output slice receives the `slice1` packets from sources at the same `slice0` position, sequenced along the innermost `Time`. The new `x` axis in the output `slice` (sized `slice1`) replicates this collected data, while `slice0` itself is preserved at its original position.

Example

```
axes![A = 256, B = 64, C = 63, X = 4];

fn broadcast1<'l, const T: Tu>(
    input: FetchTensor<'l, T, i8, m![1], m![1 # 2], m![A], m![B], m![C #
64]>,
) -> SwitchTensor<'l, T, i8, m![1], m![1 # 2], m![A / 32, X, A % 8], m![B, A
/ 8 % 4], m![C # 64]> {
    input.switch::<m![A / 32, X, A % 8], m![B, A / 8 % 4]>(<
        SwitchConfig::Broadcast1 {
            slice1: 4,
            slice0: 8,
        }
    )
}
```

With `slice1 = 4` (size of broadcast `x`) and `slice0 = 8`, the compiler derives `slice2 = 8` and `ring_size = 32` (8 sub-rings span 256 slices).

Sub-dimensions resolve to `slice2 = A / 32`, `slice1 = A / 8 % 4`, `slice0 = A % 8`, `time0 = B`, giving `OutSlice = m![A / 32, X, A % 8]` and `OutTime = m![B, A / 8 % 4]`.

Cycle estimate: `ring_size × Time::SIZE × flits_per_packet = 32 × 64 × 2 = 4096`, where `Time::SIZE = 64` and `flits_per_packet = sizeof(i8) × Packet::SIZE / 32 =`

$$1 \times 64 / 32 = 2.$$

Transpose

Transpose swaps `slice1` and `slice0` within the innermost part of `slice`.

The input and output `slice` orderings:



Each sub-ring spans `slice0 × slice1` slices and circulates data so every slice in the sub-ring ends up holding the value previously held by its swap partner.

Transpose requires input `Time` and output `Time` to match (after normalization).

Example

```
axes![A = 256, B = 64, C = 63];
```

```
fn transpose<'l, const T: Tu>(  
    input: FetchTensor<'l, T, i8, m![1], m![1 # 2], m![A], m![B], m![C #  
64]>,  
    ) -> SwitchTensor<'l, T, i8, m![1], m![1 # 2], m![A / 64, A % 2, A / 2 % 32],  
    m![B], m![C # 64]> {  
    input.switch::<m![A / 64, A % 2, A / 2 % 32], m![B]>  
    (SwitchConfig::Transpose {  
        slice1: 32,  
        slice0: 2,  
    })  
}
```

With `slice1 = 32` and `slice0 = 2`, the compiler derives `slice2 = 4` and `ring_size = 64` (4 sub-rings span 256 slices).

Sub-dimensions resolve to `slice2 = A / 64`, `slice1 = A / 2 % 32`, `slice0 = A % 2`, `time0 = B`, giving `OutSlice = m![A / 64, A % 2, A / 2 % 32]` and `OutTime = m![B]` (slice0 and slice1 swap; Time is unchanged).

Cycle estimate: $\text{ring_size} \times \text{Time::SIZE} \times \text{flits_per_packet} = 64 \times 64 \times 2 = 8192$,
 where $\text{Time::SIZE} = 64$ and $\text{flits_per_packet} = \text{sizeof}(i8) \times \text{Packet::SIZE} / 32 = 1 \times 64 / 32 = 2$.

InterTranspose

`InterTranspose` swaps a dimension between `slice` and `Time`: `slice1` moves into `Time` and `time1` moves into `slice` (regular `Transpose` stays within `slice`).

Note

Axis lifting can replace `InterTranspose` when every destination slice already holds a copy of the source region. It assigns a different read offset to each slice and performs no sub-ring traversal. Use `InterTranspose` when data must move between slices or from `slice` to `Time`.

The input dimension structure (outermost to innermost, left to right):

Slice			Time		
slice2	slice1	slice0	time2	time1	time0

After switching, `slice1` and `time1` swap positions across the `slice / Time` boundary:

Slice			Time		
slice2	time1	slice0	time2	time0	slice1

Each sub-ring spans `slice1 × slice0` slices and circulates data over `time1` time steps so each slice's value previously indexed by `slice1` ends up indexed by `time1`, and vice versa.

`InterTranspose` enforces three sizing constraints:

- `InSlice` spans all 256 slices: `slice2 × slice1 × slice0 == 256`.
- The swapped dimensions match in size: `time1.SIZE == slice1`.

- `InTime::SIZE` is divisible by `slice1 × time0` so the `time2` decomposition is integral.

Example

```
axes![A = 256, B = 8, C = 32];

fn inter_transpose<'l, const T: Tu>(
    input: FetchTensor<'l, T, i8, m![1], m![1 # 2], m![A], m![B], m![C #
32]>,
) -> SwitchTensor<'l, T, i8, m![1], m![1 # 2], m![A / 32, B / 2 % 2, A % 16],
m![B / 4, B % 2, A / 16 % 2], m![C # 32]> {
    input.switch::<m![A / 32, B / 2 % 2, A % 16], m![B / 4, B % 2, A / 16 %
2]>(<
        SwitchConfig::InterTranspose {
            slice1: 2,
            slice0: 16,
            time0: 2,
        })
}
```

With `slice1 = 2`, `slice0 = 16`, and `time0 = 2`, the compiler derives `slice2 = 8`, `time2 = 2`, and `ring_size = 32` (8 sub-rings span 256 slices).

Sub-dimensions resolve to `slice2 = A / 32`, `slice1 = A / 16 % 2`, `slice0 = A % 16`, `time2 = B / 4`, `time1 = B / 2 % 2`, `time0 = B % 2`, giving `OutSlice = m![A / 32, B / 2 % 2, A % 16]` and `OutTime = m![B / 4, B % 2, A / 16 % 2]` (`slice1` and `time1` swap between `slice` and `time`).

Cycle estimate: `ring_size × Time::SIZE × flits_per_packet = 32 × 8 × 1 = 256`, where `Time::SIZE = 8` and `flits_per_packet = sizeof(i8) × Packet::SIZE / 32 = 1 × 32 / 32 = 1`.

TransposedBroadcast1

`TransposedBroadcast1` broadcasts `slice0` to the innermost `time` position and shifts `slice1` to the innermost `slice` position, equivalent to applying `Transpose` followed by `Broadcast1`. An input tensor structured as:

Slice			Time
slice2	slice1	slice0	time0

becomes the output below, where `slice0` moves to the innermost `Time` position to broadcast across the sub-ring, `slice1` shifts to the innermost `slice` position, and a broadcast dimension fills `slice1`'s vacated middle slot:

Slice			Time	
slice2	Y	slice1	time0	slice0

Each sub-ring spans `slice0` $\times$ `slice1` slices and circulates data so every slice ends up with its swap partner's value, broadcast across the `slice0` positions at the innermost `Time` sub-dimension.

Example

```
axes![A = 256, B = 16, C = 32, Y = 8];

fn transposed_broadcast1<'l, const T: Tu>(
    input: FetchTensor<'l, T, i8, m![1], m![1 # 2], m![A], m![B], m![C #
32]>,
) -> SwitchTensor<'l, T, i8, m![1], m![1 # 2], m![A / 64, Y, A / 8 % 8], m!
[B, A % 8], m![C # 32]> {
    input.switch::<m![A / 64, Y, A / 8 % 8], m![B, A % 8]>(
        SwitchConfig::TransposedBroadcast1 {
            slice1: 8,
            slice0: 8,
        }
    )
}
```

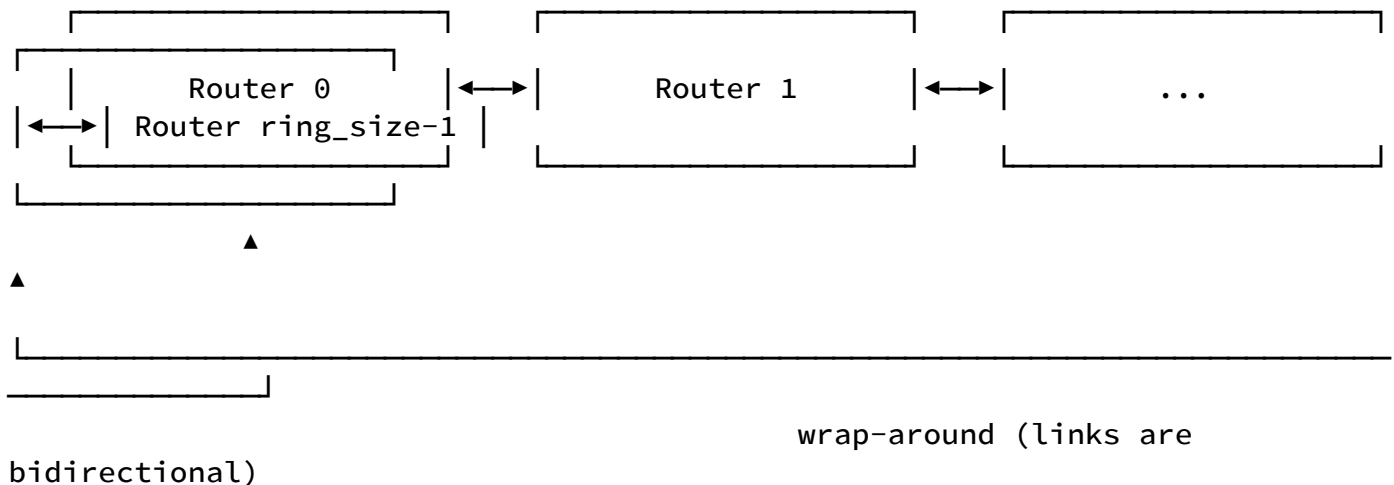
With `slice1 = 8` and `slice0 = 8` (size of broadcast `Y`), the compiler derives `slice2 = 4` and `ring_size = 64` (4 sub-rings span 256 slices).

Sub-dimensions resolve to `slice2 = A / 64`, `slice1 = A / 8 % 8`, `slice0 = A % 8`, `time0 = B`, giving `OutSlice = m![A / 64, Y, A / 8 % 8]` and `OutTime = m![B, A % 8]`.

Cycle estimate: $\text{ring_size} \times \text{Time}::\text{SIZE} \times \text{flits_per_packet} = 64 \times 16 \times 1 = 1024$, where $\text{Time}::\text{SIZE} = 16$ and $\text{flits_per_packet} = \text{sizeof}(i8) \times \text{Packet}::\text{SIZE} / 32 = 1 \times 32 / 32 = 1$.

Architecture

To execute the configurations introduced above, the Switch Engine arranges all 256 slices on a chip into a single physical ring (one router per slice), partitioned into $256 / \text{ring_size}$ parallel sub-rings of ring_size slices each (one such sub-ring is shown below). For regular configurations the compiler derives ring_size from the configuration's parameters (slice1 , slice0 , time0); for `CustomBroadcast` the kernel writer sets ring_size directly.



Performance

A switch operation takes roughly $\text{ring_size} \times \text{Time}::\text{SIZE} \times \text{flits_per_packet}$ cycles. All sub-rings advance in parallel, so this per-ring cycle count is also the chip-wide latency. The three factors are:

- ring_size : cycles for one flit to traverse one sub-ring. A larger ring_size reaches more slices per ring at a higher per-ring cost, while a smaller ring_size partitions the cluster into more parallel rings at a lower per-ring cost.
- $\text{Time}::\text{SIZE}$: number of time steps in the input tensor. The sub-ring traversal repeats once per time step.

- `flits_per_packet` : flits per packet, equal to size of `D[Packet::SIZE]` divided by 32 bytes. The traversal also repeats once per flit of the packet.

Custom Configurations

Custom configurations handle movement patterns no regular configuration expresses, such as arbitrary dimension permutations or partial dimension extractions. This flexibility comes with [configuration overhead](#) and the [constraints](#) listed at the end of the section.

The `SwitchConfig::CustomBroadcast` variant carries a single field:

```
/// Routes data across slices using a custom snoop bitmap.
/// The bitmap is computed by the compiler from the input shape and
/// topology parameters.
CustomBroadcast {
    /// Ring group size for the custom routing.
    ring_size: usize,
},
```

Where regular configurations supply built-in generators for the snoop bitmap, `CustomBroadcast` lets the compiler synthesize the bitmap directly from the kernel writer's input/output mappings together with `ring_size`.

Supported Transformation Patterns

Custom bitmaps cover two patterns regular configurations cannot:

- **Free transpose with broadcast:** arbitrary permutation and broadcast of partitioning dimensions, beyond the fixed forms in `Transpose` or `TransposedBroadcast1`.
- **Partial dimension extraction:** only a subset of a dimension's values moves to `Time` during broadcasting, whereas regular configurations like `Broadcast01` always move the whole dimension.

The examples below illustrate these patterns.

Configuration Overhead

Writing a custom snoop bitmap streams configuration data into the Switch Engine's Special Function Registers (SFRs), and this SFR write occupies both the DMA Engine and the sub-context for the duration. While the bitmap is loading, the DMA and sub contexts cannot run any other operation, so the cost manifests as reduced scheduling parallelism rather than a fixed-cycle stall.

Example 1: Arbitrary Permutation

This example reverses the four innermost slice sub-dimensions ($A / 4$, $A \% 4$, $B / 4$, $B \% 4$) into $[3, 2, 1, 0]$, a pattern no regular configuration expresses.

```
axes![A = 16, B = 16, C = 8, D = 8, E = 8];

fn arbitrary_permutation<'l, const T: Tu>(
    input: FetchTensor<'l, T, f32, m![1], m![1 # 2], m![A, B], m![C], m![D,
E]>,
) -> SwitchTensor<'l, T, f32, m![1], m![1 # 2], m![B % 4, B / 4, A % 4, A /
4], m![C], m![D, E]> {
    input.switch::<m![B % 4, B / 4, A % 4, A / 4], m![C]>(<
        SwitchConfig::CustomBroadcast { ring_size: 256 }
    )
}
```

The output `slice = m![B % 4, B / 4, A % 4, A / 4]` permutes the input slice shape $[0, 1, 2, 3]$ into $[3, 2, 1, 0]$, which no regular configuration covers but a custom bitmap does.

Bitmap Index	$(B \% 4, B / 4, A \% 4, A / 4)$	(A, B)	Ring Group
0	(0, 0, 0, 0)	(0, 0)	0
1	(0, 0, 0, 1)	(4, 0)	64
2	(0, 0, 0, 2)	(8, 0)	128
3	(0, 0, 0, 3)	(12, 0)	192
4	(0, 0, 1, 0)	(0, 1)	1
5	(0, 0, 1, 1)	(4, 1)	65
...	...	...	...
255	(3, 3, 3, 3)	(15, 15)	255

Plugging into the [cycle formula](#), $\text{cycles} \approx \text{ring_size} \times \text{Time::SIZE} \times \text{flits_per_packet}$
 $= 256 \times 8 \times 8 = 16384$:

- `ring_size = 256`
- `Time::SIZE = C::SIZE = 8`
- `flits_per_packet = sizeof(f32) × Packet::SIZE / 32 = 4 × 64 / 32 = 8`
`( Packet = m![D, E] , D::SIZE × E::SIZE = 64 )`

The maximum `ring_size = 256` is necessary because the permutation creates dependencies across all slices with no repeating structure, so input and output slices can be arbitrarily far apart in the ring index, and any smaller sub-ring would fail to cover at least one such pair.

Example 2: Multi-dimension Broadcast

Unlike Example 1's pure permutation, this example moves two non-contiguous dimensions (`A % 2` and `B % 2`) from `slice` to `Time`, broadcasting at their original positions.

```
axes! [A = 16, B = 16, C = 8, D = 8, E = 8, X = 2, Y = 2];

fn multi_axis_broadcast<'l, const T: Tu>(
    input: FetchTensor<'l, T, f32, m![1], m![1 # 2], m![A, B], m![C], m![D,
E]>,
) -> SwitchTensor<'l, T, f32, m![1], m![1 # 2], m![A / 2, X, B / 2, Y], m![C,
A % 2, B % 2], m![D, E]> {
    input.switch::<m![A / 2, X, B / 2, Y], m![C, A % 2, B % 2]>(
        SwitchConfig::CustomBroadcast { ring_size: 32 }
    )
}
```

The output moves `A % 2` and `B % 2` from `slice` to `Time`, broadcasting at their original positions via the broadcast dimensions `X` and `Y`. `Broadcast01` supports a similar form but requires the broadcast dimensions (`slice0`, `slice1`) to be contiguous in the input slice, so it cannot express non-contiguous dimensions moving from `slice` to `Time`. A custom bitmap expresses it instead.

Bitmap Index	(A / 2, A % 2, B / 2, B % 2)	(A, B)	Ring Group
0	(0, 0, 0, 0), (0, 0, 0, 1), (0, 1, 0, 0), (0, 1, 1, 0)	(0, 0), (0, 1), (0, 2), (0, 3)	0, 1, 2, 3

	0, 1)	(1, 0), (1, 1)	16, 17
1	(0, 0, 0, 0), (0, 0, 0, 1), (0, 1, 0, 0), (0, 1, 0, 1)	(0, 0), (0, 1), (1, 0), (1, 1)	0, 1, 16, 17
2	(0, 0, 1, 0), (0, 0, 1, 1), (0, 1, 1, 0), (0, 1, 1, 1)	(0, 2), (0, 3), (1, 2), (1, 3)	2, 3, 18, 19
3	(0, 0, 1, 0), (0, 0, 1, 1), (0, 1, 1, 0), (0, 1, 1, 1)	(0, 2), (0, 3), (1, 2), (1, 3)	2, 3, 18, 19
...	...	...	...
255	(7, 0, 7, 0), (7, 0, 7, 1), (7, 1, 7, 0), (7, 1, 7, 1)	(14, 14), (14, 15), (15, 14), (15, 15)	238, 239, 254, 255

Plugging into the [cycle formula](#), $\text{cycles} \approx \text{ring_size} \times \text{Time}::\text{SIZE} \times \text{flits_per_packet}$
 $= 32 \times 8 \times 8 = 2048$:

- `ring_size = 32` (the outermost `A / 2` partition needs no inter-sub-ring exchange, so only the innermost 32 slices within each sub-ring communicate)
- `Time::SIZE = 8`
- `flits_per_packet = sizeof(f32) × Packet::SIZE / 32 = 4 × 64 / 32 = 8`
`( Packet = m![D, E], D::SIZE × E::SIZE = 64 )`

Example 3: Partial Axis Extraction (Slicing)

Unlike Examples 1 and 2, which include every value of the moved dimensions, here only the 3 valid values of `c # 4` move from `slice` to `Time`; its padding cell stays behind.

```
axes![A = 16, B = 4, C = 3, D = 8, E = 8, X = 4];

fn partial_axis_extraction<'l, const T: Tu>(
    input: FetchTensor<'l, T, f32, m![1], m![1 # 2], m![A, B, C # 4], m![D],
    m![E]>,
) -> SwitchTensor<'l, T, f32, m![1], m![1 # 2], m![A, B, X], m![D, C], m![E]>
{
    input.switch::<m![A, B, X], m![D, C]>(
        SwitchConfig::CustomBroadcast { ring_size: 4 }
    )
}
```

The output moves `C # 4` from `slice` to `Time`, placing a broadcast axis `x` at its vacated `slice` position, and the fourth value (`C # 4 = 3`), a pure padding cell, is dropped, so only the three valid values `c` are extracted. `Broadcast1` supports a similar form but always moves the entire dimension, so it cannot express a subset.

This partial extraction is allowed only because the discarded value is padding: `C # 4` holds `c = 3` valid elements in a slot padded to 4, so slicing it down to `c` (`C # 4 → c`) throws away nothing real. Slicing a valid range would instead discard live input, which the [Dropped values must be padding](#) constraint forbids. A custom bitmap expresses this padding-only extraction instead.

Bitmap Index	(A, B, C # 4) sources	Ring Group
0	(0, 0, 0), (0, 0, 1), (0, 0, 2)	0, 1, 2
1	(0, 0, 0), (0, 0, 1), (0, 0, 2)	0, 1, 2
2	(0, 0, 0), (0, 0, 1), (0, 0, 2)	0, 1, 2
3	(0, 0, 0), (0, 0, 1), (0, 0, 2)	0, 1, 2
4	(0, 1, 0), (0, 1, 1), (0, 1, 2)	4, 5, 6
...	...	...
255	(15, 3, 0), (15, 3, 1), (15, 3, 2)	252, 253, 254

Plugging into the [cycle formula](#), $\text{cycles} \approx \text{ring_size} \times \text{Time::SIZE} \times \text{flits_per_packet}$
 $= 4 \times 24 \times 1 = 96$:

- `ring_size = 4` (the outer `A, B` partition needs no inter-sub-ring exchange, so only the innermost 4 slices (one `C # 4` group) within each sub-ring communicate)
- `Time::SIZE = D::SIZE × C::SIZE = 8 × 3 = 24`
- `flits_per_packet = sizeof(f32) × Packet::SIZE / 32 = 4 × 8 / 32 = 1`
 (`Packet = m![E], E::SIZE = 8`)

The bitmap shows padding-only extraction directly: `bitmap[0] = {0, 1, 2}` means output slice 0 receives from the 3 valid input slices in its `c # 4` group while skipping index 3, the padding cell. Reading `{0, 1, 2, 3}` would pull that padding in as if it were real data.

Constraints

Custom configurations come with seven constraints that bound this flexibility.

Broadcast axes must be new

Same rule as the [Regular Configurations](#) intro: each broadcast axis introduced in the output `slice` must not appear in the input `slice` or input `Time`.

For instance, with `axes![A = 256, B = 64, C = 32]` and input `slice = m![A]`, `Time = m![B]`, `Packet = m![C # 32]`, an output `slice = m![A / 4, B / 32, A % 4]` violates this constraint because `B` already appears in the input `Time`.

Each broadcast axis used exactly once

Each broadcast axis must appear exactly once in the output `slice`. Repeating the same axis at two output positions has no defined meaning for the routing bitmap.

For instance, output `slice = m![A / 4, X, X]` (where `X` is a new axis used twice) violates this constraint.

Broadcast axes must not be padded

Broadcast axes in the output `slice` must not carry padding (no `Axis # N` form). Padding on a broadcast axis would leave routing destinations undefined for the padded positions.

For instance, output `slice = m![A / 4, X # 4, Y]` violates this constraint because `X` is a broadcast axis with padding.

Order preservation

Axes moving from `slice` to `Time` must preserve their relative order from the input slice

dimension, and the verifier in `SwitchConfig::CustomBroadcast` panics at kernel compile time when they do not. Each router has minimal buffering (one packet) and must immediately decide to output locally or forward, with no opportunity to buffer multiple packets and reorder them.

For instance, with `axes![A = 16, B = 16, C = 8, D = 8, E = 8]` and `dtype = i8`, mapping input `slice = m![A, B]`, `Time = m![C]`, `Packet = m![D, E]` to output `slice = m![A, B / 4, 4]`, `Time = m![C, B % 2, B / 2]`, `Packet = m![D, E]` violates this constraint. Here `B % 2` and `B / 2` appear in reversed order relative to their input slice arrangement. Output `Time = m![C, B / 2, B % 2]` would be valid, since `B / 2, B % 2` match the input order.

Innermost time position

Axes moving from `slice` to `Time` must occupy the innermost positions of the output `Time`. Data from other slices arrives last within each packet in the pipeline, so `slice`-to-`Time` sub-dimensions naturally land at the innermost time dimensions. Placing them elsewhere would demand buffering and reordering full time sequences, which the hardware cannot do.

For instance, with the same `axes!` and `dtype` as above, mapping input `slice = m![A, B]`, `Time = m![C]`, `Packet = m![D, E]` to output `slice = m![A / 2, 2, B / 2, 2]`, `Time = m![A % 2, C, B % 2]`, `Packet = m![D, E]` violates this constraint. Here `A % 2` and `B % 2` preserve their relative order correctly, but `C` sits between them.

Note

`Broadcast01` works around this constraint via the `time0` parameter, but custom configurations lack that mechanism and must follow the constraint strictly.

Dropped values must be padding

When fewer values of a dimension move from `slice` to `Time` than the dimension spans (partial extraction, as in [Example 3](#)), every value left behind must be padding. Dropping a valid value would silently discard live input, so the verifier rejects it at kernel compile time.

For instance, with `axes![A = 16, B = 4, C = 3, D = 8, E = 8, X = 4]` and input `slice = m![A, B, C # 4]`, extracting `C # 4 → C` is allowed because the dropped fourth

value is a padding cell. Slicing a fully valid axis, for instance $B = 4 \rightarrow B = 3$, violates this constraint, since the dropped value carries real data.

Ring size

The `ring_size` parameter must be a power of 2. The compiler also derives the expected `ring_size` from the input/output mappings (the outermost non-direct-cast boundary) and rejects any user-supplied value that does not match.

Collect Engine

All downstream engines ([Contraction Engine](#), [Vector Engine](#), [Cast Engine](#), [Transpose Engine](#), and [Commit Engine](#)) consume exactly 32-byte *flits*. The Collect Engine normalizes arbitrary-sized packets to one flit in two steps:

1. **Pad** the input packet up to the next 32-byte boundary. Skipped if the packet is already 32-byte aligned.
2. **Split** at the flit boundary: the inner 32 bytes become `Packet2`, and the outer flit count is absorbed into `Time2`. Skipped if the packet is already 32 bytes.

The resulting `CollectTensor` either flows down the pipeline to a downstream engine or is stored in the [Register Files](#).

Interface

`SwitchTensor` and `FetchTensor` both expose `.collect()` with the same semantics. The `FetchTensor` entry point bypasses the Switch Engine when no slice distribution is needed.

```
impl<'l, const T: Tu, P: CanApplyCollect, D: Scalar, Chip: M, Cluster: M,
Slice: M, Time: M, Packet: M, B: Backend>
    TuTensor<'l, T, P, D, Chip, Cluster, Slice, Time, Packet, B>
{
    /// Normalizes packet to exactly 32 bytes (one flit).
    ///
    /// Pads to flit-aligned boundary, then splits: inner 32 bytes become
    /// `Packet2`, outer flit portion is absorbed into `Time2`. For packets
    /// already ≤ 32 bytes, only padding is added.
    #[primitive(TuTensor::collect)]
    pub fn collect<Time2: M, Packet2: M>(self) -> CollectTensor<'l, T, D,
Chip, Cluster, Slice, Time2, Packet2, B> {
        verify_collect::<D, Time, Packet, Time2, Packet2>();
        CollectTensor::new(self.ctx, self.inner.transpose(false))
    }
}
```

Examples

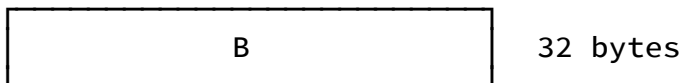
Single-Flit Packet

```
axes![A = 8, B = 32];
```

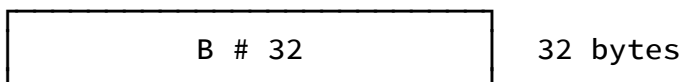
```
fn collect_identity<'l, const T: Tu>(
  input: SwitchTensor<'l, T, i8, m![1], m![1 # 2], m![1 # 256], m![A], m!
[B]>,
) -> CollectTensor<'l, T, i8, m![1], m![1 # 2], m![1 # 256], m![A], m![B #
32]> {
  // B=32 elements × 1 byte (i8) = 32 bytes = one flit.
  // Time and Packet pass through unchanged.
  input.collect()
}
```

When the input packet is already exactly 32 bytes, `collect` passes it through unchanged ($B = 32$ elements $\times$ 1 byte for $i8 = 32$ bytes).

Before: Time = m![A]
 Packet = m![B]



After: Time = m![A]
 Packet = m![B # 32]



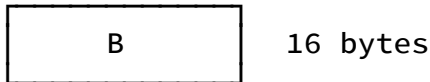
Sub-Flit Packet

```
axes![A = 8, B = 16];
```

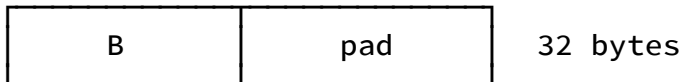
```
fn collect_padding<'l, const T: Tu>(
  input: SwitchTensor<'l, T, i8, m![1], m![1 # 2], m![1 # 256], m![A], m!
[B]>,
) -> CollectTensor<'l, T, i8, m![1], m![1 # 2], m![1 # 256], m![A], m![B #
32]> {
  // B=16 elements × 1 byte = 16 bytes < 32 bytes.
  // Padded to 32 bytes: Packet2 = m![B # 32].
  // Time unchanged since it fits in one flit.
  input.collect()
}
```

When the input packet is smaller than 32 bytes, `collect` pads to 32 bytes ($B = 16$ elements $\times 1$ byte for $bf16 = 16$ bytes).

Before: `Time = m![A]`
`Packet = m![B]`



After: `Time = m![A]`
`Packet = m![B # 32]`



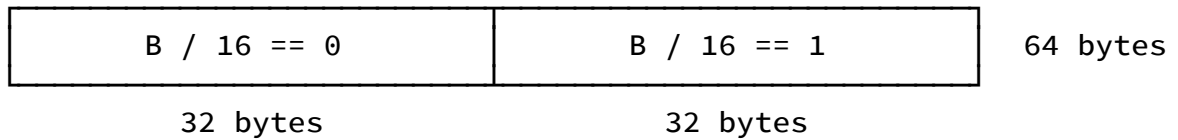
Multi-Flit Packet

`axes![A = 8, B = 32];`

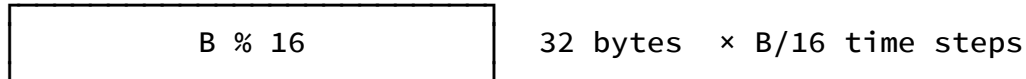
```
fn collect_multi_flit<'l, const T: Tu>(  
    input: SwitchTensor<'l, T, bf16, m![1], m![1 # 2], m![1 # 256], m![A], m!  
    [B]>,  
    ) -> CollectTensor<'l, T, bf16, m![1], m![1 # 2], m![1 # 256], m![A, B / 16],  
    m![B % 16]> {  
    // B=32 elements  $\times$  2 bytes (bf16) = 64 bytes = 2 flits.  
    // Inner 16 elements = 32 bytes  $\rightarrow$  Packet2 = m![B % 16].  
    // Outer 2 flits  $\rightarrow$  absorbed into Time2 = m![A, B / 16].  
    input.collect()  
}
```

When the input packet exceeds 32 bytes, `collect` splits into flits and absorbs the outer flit count into Time ($B = 32$ elements $\times$ 2 bytes for $bf16 = 64$ bytes, so $B / 16 = 2$ flits).

Before: `Time = m![A]`
`Packet = m![B]`



After: `Time = m![A, B / 16]`
`Packet = m![B % 16]`



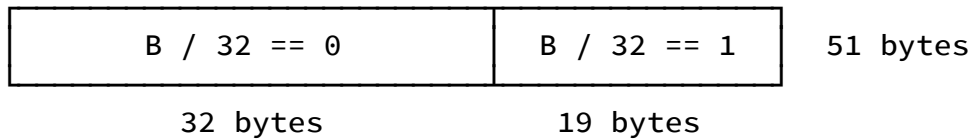
Multi-Flit Packet With Padding

`axes![A = 8, B = 56];`

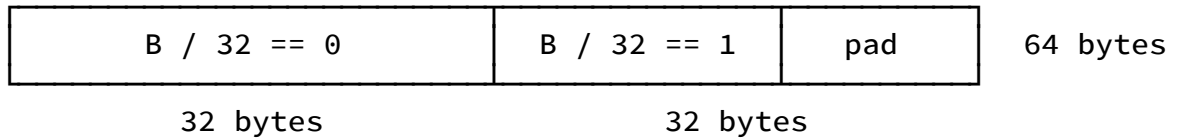
```
fn collect_multi_flit_padded<'l, const T: Tu>(
    input: SwitchTensor<'l, T, i8, m![1], m![1 # 2], m![1 # 256], m![A], m!
[B]>,
) -> CollectTensor<'l, T, i8, m![1], m![1 # 2], m![1 # 256], m![A, B # 64 /
32], m![B # 64 % 32]> {
    // B is not 32-byte aligned; first pad B to a multiple of 32 bytes.
    // B # 64=64 elements × 1 byte (i8) = 64 bytes = 2 flits.
    // Inner 32 elements = 32 bytes → Packet2 = m![B # 64 % 32].
    // Outer 2 flits → absorbed into Time2 = m![A, B # 64 / 32].
    input.collect()
}
```

When the input packet is not aligned to 32 bytes, it is first padded (`B = 51` elements × 1 byte for `i8 = 51` bytes, padded to 64). Then, `collect` splits into flits and absorbs the outer flit count (`B # 64 / 32 = 2`) into `Time`.

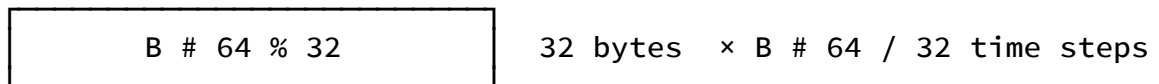
Before: `Time = m![A]`
`Packet = m![B]`



Padded: `Time = m![A]`
`Packet = m![B # 64]`



After: `Time = m![A, B # 64 / 32]`
`Packet = m![B # 64 % 32]`



Register File Loading

After normalization, store the `CollectTensor` into the Tensor Register File with `.to_trf()` or the Vector Register File with `.to_vrf()`. The [Register Files](#) chapter owns their APIs, regions, capacities, address modes, and examples.

Register Files

The [Collect Engine](#) streams into the [Contraction Engine](#) and the [Vector Engine](#). Both engines also take input from a register file (one per slice): the Tensor Register File (TRF) feeds the Contraction Engine, and the Vector Register File (VRF) feeds the Vector Engine. These register files must be populated before their consumer engine runs.

Tensor Register File

Interface

A `TrfTensor` is a tensor stored in the TRF:

```
/// Tensor stored in the tensor register file.
#[primitive(TrfTensor)]
#[derive(Debug)]
pub struct TrfTensor<D: Scalar, Chip: M, Cluster: M, Slice: M, Lane: M,
Element: M, B: Backend = CurrentBackend> {
    pub(crate) inner: Tensor<D, Pair<Chip, Pair<Cluster, Pair<Slice,
Pair<Lane, Element>>>>, B>,
    _marker: PhantomData<(D, Chip, Cluster, Slice, Lane, Element)>,
}
```

`Chip` / `Cluster` / `Slice` pass through from the source. `Lane` indexes the spatial parallelism (1, 2, 4, or 8 active lanes). `Element` holds the per-lane layout.

From Collect Engine

`.to_trf::<Lane, Element>()` on `CollectTensor` produces a `TrfTensor`. Where in the register file it lands is the compiler's to decide, so the call names no region:


```
impl<'l, const T: Tu, P: CanApplyToTrf, D: Scalar, Chip: M, Cluster: M,
Slice: M, Time: M, Packet: M, B: Backend>
    TuTensor<'l, T, P, D, Chip, Cluster, Slice, Time, Packet, B>
{
    /// Stores to the tensor register file.
    #[primitive(TuTensor::to_trf)]
    pub fn to_trf<Lane: M, Element: M>(self) -> TrfTensor<D, Chip, Cluster,
Slice, Lane, Element, B> {
        verify_to_trf::<D, Lane, Time, Packet, Element>();
        TrfTensor::new(self.inner.transpose(false))
    }
}
```

`.to_trf` reshapes the streaming `Time / Packet` into `Lane / Element`:

```
Lane    = Time / FlitsPerLane
Element = [Time % FlitsPerLane, Packet]
```

for some `FlitsPerLane` that the compiler derives from `Lane` and `Time`, so each lane is filled by `FlitsPerLane` consecutive flits.

The following simple case occupies the full TRF:

```
axes![B = 32];

fn load_trf<'l, const T: Tu>(
    input: CollectTensor<'l, T, i8, m![1], m![1 # 2], m![1 # 256], m![1], m!
[B]>,
) -> TrfTensor<i8, m![1], m![1 # 2], m![1 # 256], m![1], m![B]> {
    input.to_trf()
}
```

For example, in a matmul kernel `Lane` holds output channels and `Element` holds the contracted axis.

```

axes![V = 32, M = 32, N = 8, K = 32];

type Chip      = m![1];
type Cluster   = m![V / 16];
type Slice     = m![V % 16 # 256];
type Lane      = m![N];

/// Stores matmul weights into TRF for consumption by `bmatmul` in
/// [Contraction Engine: Example: Batched MatMul](./contraction-
engine/index.md#example-batched-matmul).
fn store_bmatmul_trf<'l, const T: Tu>(
    input: CollectTensor<'l, T, bf16, Chip, Cluster, Slice, m![N, K / 16], m!
[K % 16]>,
) -> TrfTensor<bf16, Chip, Cluster, Slice, Lane, m![K]> {
    input.to_trf()
}

```

From the Vector Engine

A store does not have to happen off the `collect`. A main-context stream can run the Vector Engine first and store what the pass computed:

```

axes![B = 64];

fn store_pass(ctx: &mut Context) -> VrfTensor<f32, m![1], m![1 # 2], m![1 #
256], m![B]> {
    let dm: DmTensor<f32, m![1], m![1 # 2], m![1 # 256], m![B]> =
DmTensor::new();
    ctx.main
        .begin(dm.view())
        .fetch::<m![1], m![B]>()
        .collect::<m![B / 8], m![B % 8]>()
        .vector_init()
        .vector_intra_slice_tag(TagMode::Zero)
        .vector_narrow_split::<m![B / 8, B % 8 / 4 % 2], m![B % 4]>()
        .vector_fp_unary(FpUnaryOp::Sqrt)
        .vector_widen_concat::<m![B / 8], m![B % 8]>()
        .vector_final()
        .to_vrf(&mut ctx.sub)
}

```

The two positions are different data paths, not two spellings of one. Off the `collect`, the Fetch Network hands the flits to the register file and no engine runs in between. Off a `vector_final`, the pass ends at the Vector Engine's own write port, so the value that lands in the register is the one the pass computed: `Element2` flattens the *post-pass* Time

/ `Packet` , which a reduce or a widen may have reshaped along the way.

That is the only way a computed value reaches the VRF, since the [DM-direct](#) store and the collect-position store both read Data Memory. Nothing after the Vector Engine can run in the same command, so a store cannot be combined with a cast, a transpose, or a commit.

Which context a store occupies

A store issued from the main context runs its write through the sub context, whichever of the two positions above it takes, so it occupies both and takes `&mut ctx.sub` as well.

`ctx.main` and `ctx.sub` are separate fields, so a kernel (which holds `&mut Context`) borrows both at once, as the example above does.

The borrow lasts the statement, which is the extent of the occupancy: the sub context is free again once the command retires. How long the *register* stays occupied is a different question with a different answer (until the operand's last read), and it is the compiler's allocation to make, which is why the returned `VrfTensor` carries no borrow.

A store issued from the sub context needs no argument, since it already holds that context.

From Data Memory

For completely contiguous input access (no gaps or reordering), TRF supports a *short command* (StoTRF), a compact hardware instruction that loads data from Data Memory directly into the TRF, bypassing the full Fetch → Switch → Collect → `to_trf()` pipeline. The shortcut trades arbitrary-layout support for lower setup overhead.

To Contraction Engine

Each read covers 8 lanes × (1 or 2) banks × 1 row × 320 bits per bank, producing 320 bits per lane per cycle for a narrow read (one bank) or 640 bits per lane per cycle for a wide read (both banks): all active lanes access one or both banks of the same row in parallel (one bank for narrow reads, both for wide reads). Per slice that totals 320 bytes/cycle (narrow) or 640 bytes/cycle (wide) across all 8 lanes. See [TRF Sequencer](#) for how the sequencer iterates these reads across rows and broadcasts.

Architecture

The TRF is a banked SRAM with the structure $8 \text{ lanes} \times 2 \text{ banks} \times 128 \text{ rows} \times 320 \text{ bits} = 80 \text{ KB}$ per slice. The 8 lanes operate in parallel, with 1, 2, 4, or 8 active per access.

How many elements pack into a single 320-bit row depends on the data type:

Type	Element size stored	Elements per row
<code>i4</code> $\rightarrow$ <code>i5</code> (hardware behaviour; not wired, so an <code>i4</code> weight stays <code>i4</code> today)	5 bits	64
<code>i8</code> $\rightarrow$ <code>i9</code>	9 bits (approx; rounds up to fit 320-bit row)	32
<code>i8</code> / <code>f8</code>	8 bits	32 (40 bytes per row)
<code>bf16</code>	16 bits	16 (32 bytes per row)

Only integers promote on store: `i4` $\rightarrow$ `i5` (5 bits) and `i8` $\rightarrow$ `i9` (9 bits) leave room for the fetch adapter's optional zero-point subtraction, which widens an integer weight by one bit. `i8` / `f8` and `bf16` stay at their native widths (8 and 16 bits respectively); the 320-bit row holds extra slack relative to a flat 8- or 16-bit packing so the same physical row width serves all types.

With fewer than 8 active lanes, each active lane sees more rows (as if the row count grew). Halving the active count doubles the rows per active lane (e.g., 4 active $\rightarrow$ 256 rows per bank, 1 active $\rightarrow$ 1024).

Double Buffering

The TRF enables double-buffering by splitting each bank into two halves: the [TRF Sequencer](#) loads from one half while a store fills the other, and the two can be flipped between iterations. Three address modes name the region, and the scheduler picks one at store time: `Full` uses all 128 rows per bank, `FirstHalf` uses rows 0–63, and `SecondHalf` uses rows 64–127. The half modes cap per-slice capacity at 40 KB.

See [Scheduling and Tuning: Double Buffering](#) for the tuning pattern that uses these

halves across the main and sub contexts.

Both halves share the same banks, so reads and writes contend at the bank level even though they target different rows. When both target the same bank in a cycle, the read takes priority because the contraction pipeline needs the data this cycle while the store can wait.

The TRF mitigates this contention with a read cache and bank alternation.

TRF reads have heavy reuse: the same data is typically broadcast across many cycles, so a direct-mapped read cache (8 lanes $\times$ 2 banks $\times$ 4 rows $\times$ 320 bits = 2.5 KB) sits in front of the banks and absorbs repeated reads. The cache also relieves contention with the concurrent store. On a hit the read skips the bank, so the store can use it that cycle. On a miss the cache refills from the bank, occupying it for that cycle.

For narrow reads (≤ 32 bytes), bank alternation adds a second mitigation. Reads use only one bank, so they can alternate at 32-byte granularity across the two banks. Reads and writes then end up on different banks on successive cycles, avoiding contention even on cache misses. Wide reads (64 bytes) occupy both banks every cycle, so each cache miss blocks the concurrent store; narrow reads preserve half-bandwidth alternation even when the cache misses.

Vector Register File

The VRF is written either from the [Collect Engine](#) or directly from Data Memory, and read by the [Vector Engine](#).

Interface

A `VrfTensor` is a tensor stored in the VRF:

```

/// Tensor stored in the vector register file (VRF).
#[primitive(VrfTensor)]
#[derive(Debug, Clone)]
pub struct VrfTensor<D: VeScalar, Chip: M, Cluster: M, Slice: M, Element: M,
B: Backend = CurrentBackend> {
    pub(crate) inner: Tensor<D, Pair<Chip, Pair<Cluster, Pair<Slice,
Element>>>, B>,
    _marker: PhantomData<(D, Chip, Cluster, Slice, Element)>,
}

```

`Chip` / `Cluster` / `Slice` pass through from the source. `Element` holds the per-(slice) layout.

Those three are what a Vector Engine op matches an operand against, so an operand carried under a different partition than the stream it feeds is a compile error rather than a read of another slice's data (see [Operands](#)). `.reshape::<Chip2, Cluster2, Slice2, Element2>()` restates one under another partition of the same slices, which is how an operand replicated under an anonymous broadcast (`m![256]`) feeds a stream partitioned by a named axis. It relabels the mapping and moves no data, so it is `unsafe` for the same reason `DmTensor::reshape` is: every physical position must already hold what the new mapping claims of it. Regrouping axes and naming a broadcast distribution axis are the two forms that satisfy that; the `# Safety` section on `VrfTensor::reshape` states both.

From Collect Engine

`.to_vrf::<Element2>()` on `CollectTensor` stores the flits into the VRF and produces a `VrfTensor`. Where in the register file they land is the compiler's to decide, so the call names no address:

```

impl<'l, P: CanApplyToVrf, D: VeScalar, Chip: M, Cluster: M, Slice: M, Time:
M, Packet: M, B: Backend>
  TuTensor<'l, { Tu::Sub }, P, D, Chip, Cluster, Slice, Time, Packet, B>
{
  /// Stores to the vector register file.
  #[primitive(TuTensor::to_vrf)]
  pub fn to_vrf<Element: M>(self) -> VrfTensor<D, Chip, Cluster, Slice,
Element, B> {
    verify_to_vrf::<D, Time, Packet, Element>();
    VrfTensor::new(self.inner.transpose(false))
  }
}

impl<'l, P: CanApplyToVrf, D: VeScalar, Chip: M, Cluster: M, Slice: M, Time:
M, Packet: M, B: Backend>
  TuTensor<'l, { Tu::Main }, P, D, Chip, Cluster, Slice, Time, Packet, B>
{
  /// Stores to the vector register file, occupying the sub context the
write runs through.
  ///
  /// The borrow lasts the statement. How long the *register* stays
occupied is the compiler's
  /// allocation to make, so the returned [VrfTensor] keeps no borrow.
  #[primitive(TuTensor::to_vrf_from_main)]
  pub fn to_vrf<Element: M>(
    self,
    _sub: &mut TuContext<{ Tu::Sub }>,
  ) -> VrfTensor<D, Chip, Cluster, Slice, Element, B> {
    verify_to_vrf::<D, Time, Packet, Element>();
    VrfTensor::new(self.inner.transpose(false))
  }
}

```

`.to_vrf` flattens the streaming `Time / Packet` into `Element2`:

```
Element2 = [Time, Packet]
```

The user picks `Element2`.

```

axes![B = 64];

fn store_vrf<'l>(
  input: CollectTensor<'l, { Tu::Sub }, i32, m![1], m![1 # 2], m![1 # 256],
m![B / 8], m![B % 8]>,
) -> VrfTensor<i32, m![1], m![1 # 2], m![1 # 256], m![B]> {
  input.to_vrf()
}

```

The store has one signature per context, and the example above takes the sub context's. A store issued from the main context takes `ctx.sub` as an argument, because the write registers live in the sub context's register map: see [Which context a store occupies](#).

From Data Memory

For completely contiguous input access (no gaps or reordering), VRF supports a *short command* (StoVRF), a compact hardware instruction that loads data from Data Memory directly into the VRF, bypassing the full Fetch → Switch → Collect → `to_vrf()` pipeline. The shortcut trades arbitrary-layout support for lower setup overhead.

Architecture

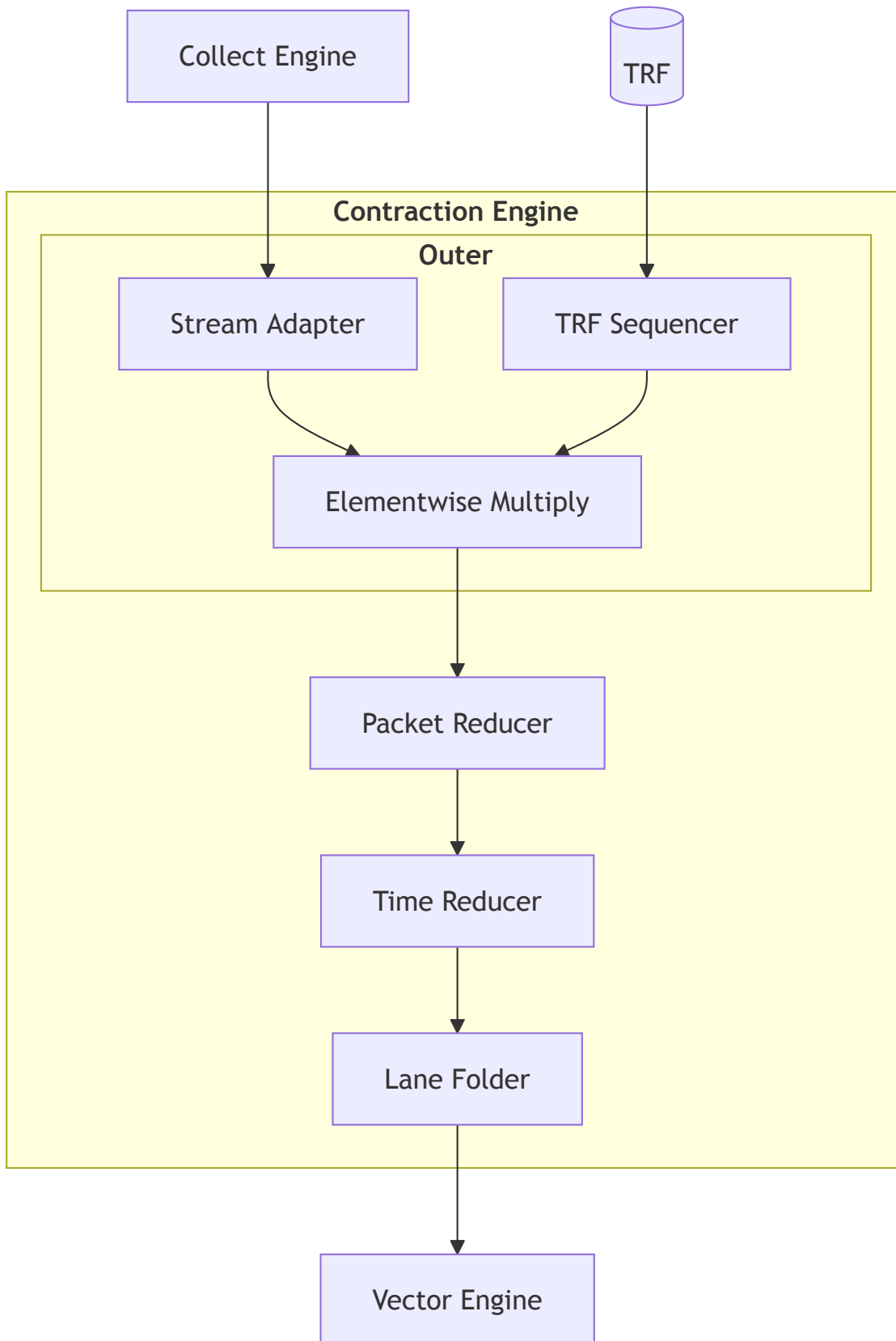
Contraction Engine

The Contraction Engine performs binary tensor contractions such as matmul and convolution. Recall from [Quick Start](#):

- A tensor contraction takes two input tensors and reduces along their shared (contracted) axes. Dot product, GEMV, and GEMM are the standard examples.
- A contraction decomposes into three steps: Broadcast, Multiply, Reduce.
- One operand streams from the [Collect Engine](#), and the other sits in the TRF (Tensor Register File).
- Contraction runs in the [main context](#). TRF preparation runs in the sub context via `.to_trf()`.

Architecture

Four pipeline stages factor the workload: one for Broadcast and Multiply, three for Reduce. Each stage handles its own non-overlapping dimension.



- **Outer** (*Broadcast and Multiply*): broadcasts the two operands to a matching shape [Chip, Cluster, Slice, Lane, Time, Packet] and multiplies them elementwise into a single product tensor. Chip / Cluster / Slice pass through. Lane indexes the spatial parallelism shared by the TRF and downstream reducers. Time and

`Packet` together represent [packet streams](#). Three sub-stages run in series: the Stream Adapter broadcasts the streaming operand, the TRF Sequencer broadcasts the TRF operand, and the Multiplier widens to the contraction output type (`i4 / i8 -> i32`, `f8 / bf16 -> f32`) and multiplies them elementwise.

- **Packet Reducer** (*Reduce within Packet*): reduces along contracted axes mapped to `Packet` via a parallel tree, one tree per lane.
- **Time Reducer** (*Reduce across Time*): accumulates per-cycle results in the shared accumulator.
- **Lane Folder** (*Fold Lane*): emits the buffer to the output stream, absorbing `Lane` into either `OutPacket` or `OutTime` depending on the mode. For reductions across slices or chips, the [Vector Engine](#) handles the reduction downstream.

The Outer stage caps `Lane` ≤ 8 and `Packet` ≤ 64 B (on RNGD); see [Packet Reducer](#) and [Time Reducer](#) for more details.

Example: Batched MatMul

[Quick Start](#) walks through dot product, GEMV, and GEMM. This section is the architecture reference for batched-matmul variants and keeps all runnable variant snippets together; Quick Start remains the end-to-end introductory path. Batched matmul extends GEMM with a leading batch axis `V`: $VMK, KN \rightarrow VMN$. For each of `V` independent ($M \times K$) inputs and a shared ($K \times N$) weight, the kernel produces the ($M \times N$) product.

The three variants below classify kernels by which axis sits in `Time`. The remaining axes are exploited as spatial parallelism. They share these axes:

```
axes![V = 32, M = 32, N = 8, K = 32]; // V batch, M×N output, K contraction
```

K in Time

`K` (the contraction axis) sits in `Time`. `M` splits across `Cluster` and `Slice`, and `V` splits as well: `v % 16` joins `Slice` while `v / 16 = 2` joins `Time` alongside `K` ($V \times M = 1024$ doesn't fit the 512 spatial cells per chip on RNGD, so `V`'s outer chunk must iterate).

`Packet` pads to `1 # 32` and the reduction proceeds sequentially across cycles instead of via the Packet Reducer's spatial tree, so only 1 of 32 multipliers does useful work per cycle (1/32 MAC utilization for bf16). The result is a degenerate kernel, shown only as an

educational baseline.

```

axes![V = 32, M = 32, N = 8, K = 32]; // V batch, M×N output, K contraction

type Chip      = m![1];                // single chip
type Cluster   = m![M / 16];           // outer M split across clusters (M /
16 = 2)
type Slice     = m![M % 16, V % 16];   // inner M × inner V = 16 × 16 = 256
slices per cluster
type Lane      = m![N];                // N (output channels) partitions the
8 hardware lanes

/// Batched matmul with K placed in Time.
fn bmatmul_k_in_time<'l, const T: Tu>(
    // Streaming operand: V outer + K in Time, with a one-element Packet m!
    [1].
    input: CollectTensor<'l, T, bf16, Chip, Cluster, Slice, m![V / 16, K], m!
    [1 # 16]>,
    // TRF operand: N in Lane, K in Element. Stored into TRF by a prior
    .to_trf() call.
    trf: &TrfTensor<bf16, Chip, Cluster, Slice, Lane, m![K]>,
    // Output: one (M × N) f32 matrix per (slice, V-outer) pair.
) -> ContractTensor<'l, T, f32, Chip, Cluster, Slice, m![V / 16], m![N]> {
    input
    // Outer: Lane = m![N] (inferred from trf), OutTime = m![V / 16, K],
    OutPacket = m![1 # 32].
    // input: 1 K-element broadcast across all N lanes.
    // trf: 1 K-element per lane, advancing one K-step per cycle.
    .contract_outer::<m![V / 16, K], m![1 # 16], _, _, _>(trf)
    // Packet Reducer: OutPacket = m![1]. Nothing to reduce.
    .contract_packet::<m![1]>()
    // Time Reducer: OutTime = m![V / 16]. K iterates over Time and
    accumulates; V outer survives.
    .contract_time::<m![V / 16]>()
    // Lane Folder: Lane folds into OutPacket. Interleaved mode emits 8
    lanes per cycle.
    .contract_lane::<m![V / 16], m![N]>(LaneMode::Interleaved)
}

```

To avoid this pathological case, keep `K` in `Packet` (parallel reduction via the `Packet Reducer`'s tree) and spread the surviving axes (`V`, `M`, `N`) across `Cluster`, `Slice`, and `Lane` to maximize spatial parallelism. The two strategies below apply this principle, each with one axis per class for simplicity; real kernels may split a single axis across multiple classes when sizes demand.

M in Time

V (batch) distributes across `cluster` and `slice` (one batch element per slice). M in `Time`, K in `Packet`. This strategy is applicable when (1) the slice count covers the batch, (2) N fits in `Lane`, and (3) K fits in a single `Packet`. When K is larger than `Packet`, split K across `Packet` (spatial) and `Time` (temporal). It maximizes MAC utilization across lanes.

```
axes![V = 32, M = 32, N = 8, K = 32]; // V batch, M×N output, K contraction

type Chip    = m![1]; // single chip
type Cluster = m![V / 16]; // outer V split across clusters (V /
16 = 2)
type Slice   = m![V % 16 # 256]; // inner V split across slices (V %
16 = 16 per cluster)
type Lane    = m![N]; // N (output channels) partitions the
8 hardware lanes (N = 8 fills the cap)

/// Batched matmul: V slices × (M × K) · (K × N) → V × M × N.
fn bmatmul_m_in_time<'l, const T: Tu>(
    // Streaming operand: M in Time, K in Packet.
    // Element type can be i4, i8, f8, or bf16; integers widen to i32 output,
    floats to f32.
    input: CollectTensor<'l, T, bf16, Chip, Cluster, Slice, m![M, K / 16], m!
[K % 16]>,
    // TRF operand: N in Lane (one output channel per lane), K in Element.
    // Stored into TRF by a prior .to_trf() call in the sub context.
    trf: &TrfTensor<bf16, Chip, Cluster, Slice, Lane, m![K]>,
    // Output: one (M × N) f32 matrix per slice.
) -> ContractTensor<'l, T, f32, Chip, Cluster, Slice, m![M], m![N]> {
    input
        // Outer: broadcast input and trf, multiply elementwise.
        // Lane = m![N] (inferred from trf), OutTime = m![M], OutPacket = m!
[K].

        // input: K elements broadcast across all N lanes.
        // trf: K elements per lane, broadcast across all M cycles.
        .contract_outer::<m![M], m![K], _, _, _>(trf)
        // Packet Reducer: OutPacket = m![1]. Sum K spatially via the
reduction tree.
        .contract_packet::<m![1]>()
        // Time Reducer: OutTime = m![M]. Nothing to reduce.
        .contract_time::<m![M]>()
        // Lane Folder: Lane folds into OutPacket. Interleaved mode emits 8
lanes per cycle.
        .contract_lane::<m![M], m![N]>(LaneMode::Interleaved)
}
```

V in Time

V (batch) in Time , K in Packet . M splits across cluster and slice . This strategy is applicable when (1) the slice count covers M ($M / 16$ in Cluster , $M \% 16$ in Slice), (2) N fits in Lane , and (3) K fits in a single Packet . Useful when batch is the dominant axis (e.g., batched inference).

```
axes![V = 32, M = 32, N = 8, K = 32]; // V batch, M×N output, K contraction

type Chip    = m![1]; // single chip
type Cluster = m![M / 16]; // outer M split across clusters (M /
16 = 2)
type Slice   = m![M % 16 # 256]; // inner M split across slices (M %
16 = 16 per cluster)
type Lane    = m![N]; // N (output channels) partitions the
8 hardware lanes

/// Batched matmul with V (batch) placed in Time.
fn bmatmul_v_in_time<'l, const T: Tu>(
    // Streaming operand: V in Time, K in Packet.
    input: CollectTensor<'l, T, bf16, Chip, Cluster, Slice, m![V, K / 16], m!
[K % 16]>,
    // TRF operand: N in Lane, K in Element. Stored into TRF by a prior
.to_trf() call.
    trf: &TrfTensor<bf16, Chip, Cluster, Slice, Lane, m![K]>,
    // Output: one (V × N) f32 matrix per slice.
) -> ContractTensor<'l, T, f32, Chip, Cluster, Slice, m![V], m![N]> {
    input
        // Outer: Lane = m![N] (inferred from trf), OutTime = m![V],
OutPacket = m![K].
        // input: K elements broadcast across all N lanes.
        // trf: K elements per lane, broadcast across all V cycles.
        .contract_outer::<m![V], m![K], _, _, _>(trf)
        // Packet Reducer: OutPacket = m![1]. Sum K spatially via the
reduction tree.
        .contract_packet::<m![1]>()
        // Time Reducer: OutTime = m![V]. Nothing to reduce.
        .contract_time::<m![V]>()
        // Lane Folder: Lane folds into OutPacket. Interleaved mode emits 8
lanes per cycle.
        .contract_lane::<m![V], m![N]>(LaneMode::Interleaved)
}
```

Outer

The Outer stage broadcasts the two operands into a matching shape and multiplies them elementwise.

“Outer” comes from the **outer product** of linear algebra. For vectors u (length n) and v (length m), $u v^T$ is the $n \times m$ matrix where $(u v^T)[i, j] = u[i] \times v[j]$. That matrix is produced by broadcasting u along the column axis (length m), broadcasting v along the row axis (length n), and multiplying elementwise. The Outer stage’s three sub-stages are the hardware embodiment of this exact semantics, run in series:

- The [Stream Adapter](#) handles (and broadcasts) the streaming operand from the [Collect Engine](#).
- The [TRF Sequencer](#) handles (and broadcasts) the TRF operand from TRF SRAM.
- The [Multiplier](#) widens the operand types ($i4 / i8$ to $i32$, $f8 / bf16$ to $f32$) and multiplies the two aligned operands elementwise.

The output is a single multiplied tensor in the joint mapping [Chip, Cluster, Slice, Lane, Time, Packet], ready for the [Packet Reducer](#) to reduce-add.

Interface

`.contract_outer(&trf)` on `CollectTensor` invokes the Outer stage.

```
impl<
    'l,
    const T: Tu,
    P: CanApplyContractOuter,
    D: Scalar + ContractionCast,
    Chip: M,
    Cluster: M,
    Slice: M,
    Time: M,
    Packet: M,
    B: Backend,
> TuTensor<'l, T, P, D, Chip, Cluster, Slice, Time, Packet, B>
{
    /// Runs the Outer stage: stashes the two un-broadcast operands (widened
    to the accumulator type)
    /// and the layouts [super::ContractTimeTensor::contract_lane`] needs to
    fuse them into a [`LazyContraction`]. No
```

```

    /// materializing alternative, no per-backend branch -- every backend
    fuses the same way.
    ///
    /// The impl block binds [`ContractionCast`], so this method does not
    exist on a `TuTensor` whose
    /// element type the Multiplier has no MAC for.
    #[primitive(TuTensor::contract_outer)]
    pub fn contract_outer<OutTime: M, OutPacket: M, Lane: M, TrfElement: M,
TrfD>(
        self,
        trf_tensor: &TrfTensor<TrfD, Chip, Cluster, Slice, Lane, TrfElement,
B>,
    ) -> ContractOuterTensor<'l, T, <D as ContractionCast>::Output, D, Chip,
Cluster, Slice, Lane, OutTime, OutPacket, B>
    where
        D: Cast<<D as ContractionCast>::Output>,
        // The weight (TRF) type must form a valid contraction-engine operand
pair with the
        // stream type `D`: same type, or a mixed integer precision within a
        // family (i4/i5 x i4/i5, i8/i9 x i8/i9). Both operands widen to the
        // stream's accumulator for the multiply.
        TrfD: Scalar + ContractionWeight<D> + Cast<<D as
ContractionCast>::Output>,
    {
        type Out<D> = <D as ContractionCast>::Output;

        // Skipping the broadcast transpose does not skip its validity
contract -- a malformed
        // contraction would otherwise slip past these asserts and panic far
downstream instead.
        stream_adapter::verify_stream_adapter::<D, Lane, Time, Packet,
OutTime, OutPacket>();
        trf_sequencer::verify_trf_sequencer::<TrfD, Lane, TrfElement,
OutTime, OutPacket>();

        // The operands keep their own compact layouts: lhs is `[Chip,
Cluster, Slice, Time, Packet]`
        // (`self.inner`), rhs is `[Chip, Cluster, Slice, Lane, TrfElement]`
(`trf_tensor`). A
        // bare-buffer backend reads its strides from these; `MathStorage`
ignores them (its axes live
        // in the storage).
        let lhs_map = <m![{ Chip }, { Cluster }, { Slice }, { Time }, {
Packet }]>::to_value();
        let rhs_map = <m![{ Chip }, { Cluster }, { Slice }, { Lane }, {
TrfElement }]>::to_value();
        let pre_reduce = <m![{ Chip }, { Cluster }, { Slice }, { Lane }, {
OutTime }, { OutPacket }]>::to_value();

        // Widen each operand to the `Out<D>` accumulator up front, at the

```



```

operand's own (compact) size,
    // not pre_reduce's -- the fold at `contract_lane` then runs entirely
in `Out<D>`. Parity-identical
    // to the old per-cell widen (same `Cast::cast` per element), it just
never allocates the
    // pre_reduce-shaped broadcast this stage used to build.
    //
let contraction = LazyContraction {
    lhs: self.inner.map(|v| -> Out<D> { v.cast() }).inner,
    rhs: trf_tensor.inner.map(|v| -> Out<D> { v.cast() }).inner,
    lhs_map,
    rhs_map,
    pre_reduce,
};
ContractOuterTensor::new(self.ctx, contraction)
}
}

```

After their respective adapters (Stream Adapter for the streaming path, TRF Sequencer for the TRF path), both paths feed `Lane / Time / Packet` of matching shape, and the Multiplier multiplies them elementwise at aligned positions.

The streaming operand's `Time / Packet` map to the output's `OutTime / OutPacket`: `OutPacket` absorbs the innermost size-1-or-2 factor of `Time` via [Packing](#). `OutTime` retains the remaining factors of `Time`, with broadcast factors added at its innermost positions via [Broadcast](#). Broadcast factors come from the TRF operand's `Lane / Element` (where the streaming operand replicates against the TRF mapping) and from any purely-output axes that appear in `OutTime / OutPacket` but in neither the input nor the TRF (e.g. einsum `AB, BC -> ABCD` where `D` is broadcast).

The `TrfTensor` has shape `[Chip, Cluster, Slice, Lane, Element]`, with `Chip / Cluster / Slice / Lane` spatially parallel: `Chip / Cluster / Slice` pass through to the output, and `Lane` partitions per-lane data across 1–8 hardware lanes. `Element` (the per-lane layout set by `.to_trf()`) is reshaped by the TRF Sequencer to fill `OutTime / OutPacket`.

Stream Adapter

The Stream Adapter transforms the streaming `Time / Packet` into the computation shape `(Lane / OutTime / OutPacket)` via two operations: [Packing](#) and [Broadcast](#). The compiler derives the three free variables `(PackSize, LaneBroadcast, TimeBroadcast)`

from the user-supplied `outTime / outPacket`, the TRF operand, and any purely-output broadcast axes, giving the mapping:

```
Lane      = LaneBroadcast
outTime   = [Time / PackSize, TimeBroadcast]
outPacket = [Time % PackSize, Packet] # (64 / D::SIZE)
```

Packing

The Collect Engine produces 32 B flits, and the Outer stage emits packets of `PackSize × 32 B` (32 or 64 on RNGD). Packing combines `PackSize ∈ {1, 2}` consecutive flits into one packet:

```
PackTime   = [Time / PackSize]
PackPacket = [Time % PackSize, Packet] # (PackSize × 32 / D::SIZE)
```

`PackSize` is set by matching `outPacket` against the input `Packet`: `PackSize = 2` if `outPacket` absorbs the innermost size-2 factor of `Time`, otherwise `PackSize = 1`. Equivalently, `PackSize = outPacket::SIZE × D::SIZE / 32`, so the user picks `outPacket` (32 B or 64 B) and Packing's collect-flit count follows.

Hardware always operates on 64 B packets internally; when `PackSize = 1`, the unused 32 B half holds zeros that do not propagate into the logical `outPacket` type. Downstream stages (Packet Reducer, Lane Folder) therefore see only the `PackSize × 32 B` payload, avoiding dummy cycles, see the [Lane Folder Sequential note](#).

Broadcast

After packing, the Stream Adapter broadcasts the data spatially via `LaneBroadcast` (the TRF's `Lane` mapping, $∈ \{1, 2, 4, 8\}$) and temporally via `TimeBroadcast`. `TimeBroadcast` covers factors of TRF `Element` not in the input `Time`, and also any purely-output axes in `outTime` that appear in neither the input nor the TRF: the same broadcast machinery replicates the packet across both. Each destination receives the same `outPacket`:

```
Lane      = LaneBroadcast
outTime   = [PackTime, TimeBroadcast]
outPacket = PackPacket
```

`TimeBroadcast` factors occupy the *innermost* positions of `OutTime`: the same `OutPacket` is re-sent across those factors before iterating any outer `PackTime` factor.

Examples

The example below exercises both operations: Packing absorbs the innermost size-2 factor `L` of `Time` into `Packet` (`PackSize = 2`), Lane Broadcast distributes the resulting packet to `N = 8` lanes, and Time Broadcast tiles the streaming data across a TRF-only `B = 5` axis.

```
axes![M = 32, N = 8, K = 16, L = 2, B = 5];
```

```
fn stream_adapter_example<'l, const T: Tu>(
    input: CollectTensor<'l, { T }, bf16, m![1], m![1 # 2], m![1 # 256], m!
[M, L], m![K]>,
    trf: &TrfTensor<bf16, m![1], m![1 # 2], m![1 # 256], m![N], m![B, L, K]>,
) -> ContractOuterTensor<'l, { T }, f32, bf16, m![1], m![1 # 2], m![1 # 256],
m![N], m![M, B], m![L, K]> {
    // Packing (PackSize = 2):
    //   L = 2 (innermost Time) absorbed into Packet.
    //   PackTime = [M = 32], PackPacket = [L = 2, K = 16] = 32 bf16 = 64B.
    // Lane Broadcast: same packet to all N = 8 lanes.
    // Time Broadcast: B = 5 (TRF-only) added at innermost OutTime.
    //   OutTime = [M, B = 5], OutPacket = [L = 2, K = 16].
    input.contract_outer::<m![M, B], m![L, K], _, _, _>(trf)
}
```

Constraints

- `OutPacket::SIZE * Storage::SIZE ∈ {32, 64}` bytes (on RNGD), where `Storage` is the pre-widen operand dtype (e.g. `bf16 = 2 B`, *not* the widened `f32` accumulator the result tensor carries). The size is 32 for `PackSize = 1` and 64 for `PackSize = 2`. The user picks this size and Packing's collect-flit count follows.
- `PackSize ∈ {1, 2}` (see [Packing](#)).
- `Lane::SIZE ∈ {1, 2, 4, 8}`.

Performance

`PackSize` sets MAC utilization. `PackSize = 2` fills the full 64 B and uses all MACs.

`PackSize = 1` fills only 32 B, so the zero-padded half always multiplies by zero and effective throughput halves.

`PackSize = 2` takes 2 cycles per Packet (two 32 B flits combine into one 64 B Packet), but this is not a pipeline bottleneck: upstream supplies one 32 B flit every cycle at the full fetch rate, so the Stream Adapter consumes flits as fast as they arrive and emits a Packet every 2 cycles to match downstream consumption.

Time Broadcasting amortizes fetches. Broadcast factors reuse the same streaming packet across cycles without re-fetching, which eliminates bandwidth cost for those factors.

Fetch bandwidth (up to 32 B/cycle per fetch) bounds the Stream Adapter overall. Interleave fetch patterns across slices to maximize utilization.

TRF Sequencer

The TRF Sequencer reads a `TrfTensor` and reshapes its `Element` into `OutTime / OutPacket` for the [Packet Reducer](#). See [Register Files](#) for the TRF storage layout (lanes, banks, rows, double-buffering, cache).

The mapping:

```
OutTime   = (sequencing over [Element / ReadSize] with broadcasts)
OutPacket = [PacketBroadcast, Element % ReadSize]
```

`OutPacket` is filled each cycle by one TRF read: one full `OutPacket` (640 bits per lane across both banks, or 320 bits when only one bank is read) is generated every cycle. See [To Contraction Engine](#) for the per-slice totals across 8 lanes (in bytes). The read pulls the innermost contiguous portion of `Element` and replicates it to fill the 64 B `OutPacket`. The compiler picks the largest `ReadSize` such that `Element % ReadSize == OutPacket % ReadSize` and `ReadSize * D::SIZE ≤ 64` bytes: a wider `ReadSize` spans both TRF banks per lane, a narrower one uses just one bank.

`OutTime` is filled across cycles by sequencing `Element / ReadSize` (plus optional broadcasts). The TRF Sequencer uses the same nested-loop configuration as all other [sequencers](#).

Examples

In this example, `ReadSize` covers all of `Element` in one 64 B read, so `Element / ReadSize` is trivial and the sequencer iterates only broadcasts:

```
axes![M = 32, N = 8, K = 32];

fn trf_sequencer_full_read<'l, const T: Tu>(
    input: CollectTensor<'l, T, bf16, m![1], m![1 # 2], m![1 # 256], m![M, K
/ 16], m![K % 16]>,
    trf: &TrfTensor<bf16, m![1], m![1 # 2], m![1 # 256], m![N], m![K]>,
) -> ContractOuterTensor<'l, T, f32, bf16, m![1], m![1 # 2], m![1 # 256], m!
[N], m![M], m![K]> {
    // Element          = K
    // ReadSize         = 32
    // PacketBroadcast   = 1
    // OutTime           = M      (sequencing over [K / 32] (= 1), broadcast M)
    // OutPacket         = K      (= [1, K % 32])
    input.contract_outer::<m![M], m![K], _, _, _>(trf)
}
```

In this example, `ReadSize` covers only part of `Element`, so `Element / ReadSize` is non-trivial and the sequencer iterates the outer `Element` factor alongside a broadcast:

```
axes![M = 32, N = 8, K = 16, L = 2, O = 2];

fn trf_sequencer_partial_read<'l, const T: Tu>(
    input: CollectTensor<'l, T, bf16, m![1], m![1 # 2], m![1 # 256], m![O, M,
L], m![K]>,
    trf: &TrfTensor<bf16, m![1], m![1 # 2], m![1 # 256], m![N], m![O, K]>,
) -> ContractOuterTensor<'l, T, f32, bf16, m![1], m![1 # 2], m![1 # 256], m!
[N], m![O, M], m![L, K]> {
    // Element          = [O, K]
    // ReadSize         = 16
    // PacketBroadcast   = L
    // OutTime           = [O, M]  (sequencing over [O, K] / 16 (= 0),
broadcast M)
    // OutPacket         = [L, K]  (= [L, [O, K] % 16])
    input.contract_outer::<m![O, M], m![L, K], _, _, _>(trf)
}
```

Constraints

- **Hardware dimensions:** `Chip::SIZE`, `Cluster::SIZE`, and `Slice::SIZE` must match the hardware configuration (see [Sequencer](#)).
- **Address alignment:** when `Element % ReadSize` covers all 64 B, the read spans

both TRF banks per lane, so the sequencer's base address and all strides must align to 64 B.

Architecture

Across cycles, the sequencer iterates the outer factors of `Element` (i.e. `Element / ReadSize`) using the same nested-loop configuration as all other [sequencers](#), so a single `TrfTensor` walks `Element / ReadSize` cycles before exhausting its content. `PacketBroadcast` factors replicate the same row within a single cycle, filling the 64 B `OutPacket` past the natural `ReadSize` without consuming additional TRF read bandwidth.

Performance

Throughput is one full `OutPacket` per lane per cycle: 640 bits per lane when both banks are read, 320 bits per lane when only one bank is read. See [Register Files: To Contraction Engine](#) for the per-slice byte totals. The TRF read cache and bank alternation (see [Register Files: Double Buffering](#)) keep the concurrent sub-context store unblocked across broadcast reuse and narrow reads.

Multiplier

The Multiplier consumes the two aligned operands from the Stream Adapter and TRF Sequencer, widens each input element to the contraction output type to keep the downstream accumulator from overflowing, and multiplies them elementwise. Its output, a single tensor in the joint mapping `[Chip, Cluster, Slice, Lane, Time, Packet]`, becomes the input to the [Packet Reducer](#). Each `Time` cycle, every `Lane` produces a full `packet` of products in parallel.

Operand types

These are the operand pairs the Multiplier accepts, with the type each pair widens to:

Stream (activation)	Weight (TRF)	Accumulator

i4 or i5	i4 or i5	i32
i8 or i9	i8 or i9	i32
f8e4m3	f8e4m3	f32
f8e5m2	f8e5m2	f32
bf16	bf16	f32

An integer operand may be the raw form (i4 / i8) or its zero-point-subtracted staging (i5 / i9 , produced by [Zero-Point Subtraction](#)), and the two operands need not match within a family. They may not cross families (no i4 against i8) or kinds (no integer against float), and a float pair must match exactly.

A contraction over an element type this table does not list runs in the [Vector Engine](#) instead: multiply elementwise, then reduce with [Intra-Slice Reduce](#), which computes in i32 / f32 .

Packet Reducer

The Packet Reducer reduce-adds the innermost contracted axes within a single Packet, with one reduction tree per lane.

Interface

`.contract_packet()` invokes the Packet Reducer. Each lane receives a 32 B or 64 B Packet of `i4`, `i8`, `f8`, or `bf16` elements, inherited from the [Outer](#) stage's `OutPacket`. Formally, it computes $\text{output}[i] = \sum_j \text{input}[i, j]$, where `i` ranges over the surviving (output) axes and `j` ranges over the contracted axes inside `Packet`.


```

impl<
    'l,
    const T: Tu,
    D: Scalar,
    Storage: ContractionCast<Output = D>,
    Chip: M,
    Cluster: M,
    Slice: M,
    Lane: M,
    Time: M,
    Packet: M,
    B: Backend,
> ContractOuterTensor<'l, T, D, Storage, Chip, Cluster, Slice, Lane, Time,
Packet, B>
{
    /// Spatial reduction within `Packet`: validates the reduce-add along the
    contracted axes inside
    /// `Packet` that the fused fold at `contract_lane` will perform. `D` is
    the widened accumulator the
    /// deferred carrier stays keyed on; the DPE input packet is still sized
    in `Storage` bytes.
    #[primitive(ContractOuterTensor::contract_packet)]
    pub fn contract_packet<OutPacket: M>(
        self,
    ) -> ContractPacketTensor<'l, T, D, Chip, Cluster, Slice, Lane, Time,
OutPacket, B> {
        verify_contract_packet(ContractPacketInput {
            in_packet: Packet::to_value(),
            out_packet: OutPacket::to_value(),
            element_bits: Storage::BITS,
        });
        // Carry the deferred operands forward unreduced: the fused
        contraction at `contract_lane`
        // performs this Packet reduction too. This stage only re-types the
        carrier to `OutPacket`.
        ContractPacketTensor::new(self.ctx, self.inner)
    }
}

```

The kernel below uses all 8 lanes in parallel: tree depth 5 sums over the 32 bf16 elements of B , producing one f32 per A position.

```

axes![A = 32, B = 32, C = 8];

fn matmul<'l, const T: Tu>(
    input: CollectTensor<'l, T, bf16, m![1], m![1 # 2], m![1 # 256], m![A, B
/ 16], m![B % 16]>,
    trf: &TrfTensor<bf16, m![1], m![1 # 2], m![1 # 256], m![C], m![B]>,
) -> ContractTensor<'l, T, f32, m![1], m![1 # 2], m![1 # 256], m![A], m![C]>
{
    //
    // Spatial reduction: tree depth 5 reduces 32 bf16 elements along B → f32
    // Output (Interleaved): Time = [A], Packet = [C]
    input.contract_outer::<m![A], m![B], _, _, _>(&trf)
        .contract_packet::<m![1]>()
        .contract_time::<m![A]>()
        .contract_lane::<m![A], m![C]>(LaneMode::Interleaved)
}

```

Architecture

$\text{ReducePacket} = \text{Packet} / 2^d$ for $0 \leq d \leq \log_2(\text{Packet}::\text{SIZE})$
 $\text{OutPacket} = \text{ReducePacket}$, outermost padding removed
 (size ≤ 32 ; when $\text{ReducePacket}::\text{SIZE} > 32$ only the innermost 32 survive)

The Packet Reducer first runs an independent **reduction tree** per lane on its input Packet. At depth 0, the tree leaves hold the input Packet's elements, and each subsequent depth sums pairs, halving the element count. The maximum tree depth is $\log_2(\text{Packet}::\text{SIZE})$, so 7 for i4 ($\text{Packet}::\text{SIZE} = 128$), 6 for i8 / f8 (64), 5 for bf16 (32). Given the user's OutPacket , the compiler derives the tree depth d , and the tree consumes the innermost 2^d elements to produce ReducePacket .

The Packet Reducer then trims ReducePacket to OutPacket , whose size is capped at 32 elements because the downstream Time Reducer's per-lane accumulator only has 32 columns. When $\text{ReducePacket}::\text{SIZE} > 32$, the outer dummy is trimmed and only the innermost 32 elements survive. For example, i4 arrives as a 128-element Packet, so $d \in \{0, 1\}$ produce a 128- or 64-element ReducePacket , both trimmed to $\text{OutPacket}::\text{SIZE} = 32$.

ReducePacket may also carry outermost padding (a padded packet axis), and the Packet Reducer keeps only the live columns, dropping that padding. For example, a ReducePacket of $[A \# 16]$ (A live, padded up to 16 columns) reduces to $\text{OutPacket} =$

`[A]` . So `OutPacket` may be declared with the padding present (`[A # 16]`) or already removed (`[A]`); both name the same result.

Performance

Latency depends on tree depth: 7 cycles for `i4` , 6 for `i8 / f8` , 5 for `bf16` . Wider element types reduce depth because fewer elements fit in each Packet. The adder tree is fully pipelined, so depth adds first-output latency but does not reduce steady-state throughput: one Packet enters and one reduced output emerges every cycle once the pipeline is filled.

When `Lane < 8` , the inactive lanes' reduction trees are idle, so per-cycle throughput drops proportionally to `Lane::SIZE / 8` .

Time Reducer

The Time Reducer accumulates the [Packet Reducer](#)'s [Lane, Packet] output across Time into OutTime, with *temporal accumulators*.

Interface

`.contract_time::<OutTime>()` invokes the Time Reducer. OutTime names the Time dimensions that survive (the rest are summed away).

```
impl<'l, const T: Tu, D: Scalar, Chip: M, Cluster: M, Slice: M, Lane: M,
Time: M, Packet: M, B: Backend>
    ContractPacketTensor<'l, T, D, Chip, Cluster, Slice, Lane, Time, Packet,
B>
{
    /// Accumulates per-cycle contractions over the `Time` dimension via the
    shared
    /// accumulator buffer, shrinking input `Time` to `OutTime`. The axes
    present in
    /// `Time` but absent from `OutTime` are reduce-added.
    #[primitive(ContractPacketTensor::contract_time)]
    pub fn contract_time<OutTime: M>(
        self,
    ) -> ContractTimeTensor<'l, T, D, Chip, Cluster, Slice, Lane, OutTime,
Packet, B> {
        verify_contract_time(ContractTimeInput {
            in_time: Time::to_value(),
            out_time: OutTime::to_value(),
        });
        // Carry the deferred operands forward unreduced: the fused
        contraction at `contract_lane`
        // performs this Time reduction too. This stage only re-types the
        carrier to `OutTime`.
        ContractTimeTensor::new(self.ctx, self.inner, Time::to_value())
    }
}
```

For example, the kernel below reduces a 2D tensor along B (surviving only A).

```

axes![A = 2048, B = 32];

/// Reduces along B; A survives.
fn reduce_b<'l, const T: Tu>(
    // Streaming operand: Slice = m![A / 8] (256 outer A chunks across
    slices).
    // Time = m![B / 16, A % 8]; Packet = m![B % 16].
    // B splits across Packet (B % 16) and Time (B / 16): each cycle produces
    a partial sum.
    input: CollectTensor<'l, T, bf16, m![1], m![1 # 2], m![A / 8], m![B / 16,
    A % 8], m![B % 16]>,
    // TRF operand: single-lane weight per slice.
    trf: &TrfTensor<bf16, m![1], m![1 # 2], m![A / 8], m![1], m![B]>,
    // Output: one f32 per (slice, A % 8) cell.
) -> ContractTensor<'l, T, f32, m![1], m![1 # 2], m![A / 8], m![A % 8], m![1
# 8]> {
    input
    // Outer: Lane = m![1], OutTime = m![B / 16, A % 8], OutPacket = m!
[B % 16].
    .contract_outer::<m![B / 16, A % 8], m![B % 16], _, _, _>(trf)
    // Packet Reducer: OutPacket = m![1]. Collapses B % 16 spatially.
    .contract_packet::<m![1]>()
    // Time Reducer: OutTime = m![A % 8]. Accumulator receives
    // Time::SIZE = (B / 16) × (A % 8) = 2 × 8 = 16 flits; B / 16 outer
    // chunks accumulate into 8 slots indexed by A % 8.
    .contract_time::<m![A % 8]>()
    // Lane Folder: Lane folds into OutPacket. Sequential mode (Lane =
m![1]).
    .contract_lane::<m![A % 8], m![1 # 8]>(LaneMode::Sequential)
}

```

Architecture

The Time Reducer receives the Packet Reducer's per-cycle `[Lane, Packet]` output. The hardware caps `Lane::SIZE ≤ 8` (spatially parallel lanes) and `Packet::SIZE ≤ 32` upstream.

Each cycle the Time Reducer folds the `[Lane, Packet]` spatial grid across `Time` into `OutTime`. `OutTime` must be a subset of `Time` with the relative order of surviving dimensions preserved (enforced by `verify_contract_time`). Dimensions in `Time` absent from `OutTime` are summed away, and the outermost such dimension iterates over flits.

Let `InnerTime` denote the inner non-reduce dimensions of `Time` (the dimensions inner

to the outermost reduce dimension that survive in `OutTime`). In `reduce_b` above, `Time = m![B / 4, A % 8]` and `OutTime = m![A % 8]`, so `B / 4` is the outermost reduce dimension (iterates over `Time::SIZE = 2 × 8 = 16` flits) and `InnerTime = m![A % 8]`.

Accumulation requires `InnerTime::SIZE` slots of `[Lane, Packet]`, one per `InnerTime` tuple value. Flits with the same tuple accumulate into the same slot. For `reduce_b`, 8 slots accumulate across the `B / 4 = 2` iterations, and after flit 15 the buffer contains the final reduced results and hands them off to the [Lane Folder](#):

```
Time = m![B / 4, A % 8]
        ~~~~~ ~~~~~
        outer R inner non-R (A % 8)
```

Flit sequence (`B / 4` has values 0,1; `A % 8` has values 0..7):

```
flit #0:  B/4=0, A%8=0  →
flit #8:  B/4=1, A%8=0  →  slot 0 (A%8=0)  accumulates B for A%8=0

flit #1:  B/4=0, A%8=1  →
flit #9:  B/4=1, A%8=1  →  slot 1 (A%8=1)  accumulates B for A%8=1
:

flit #7:  B/4=0, A%8=7  →
flit #15: B/4=1, A%8=7  →  slot 7 (A%8=7)  accumulates B for A%8=7
```

8 non-reduce positions → 8 slots used

Constraints

The mapping fits the buffer when `InnerTime::SIZE` does not exceed the slot capacity (the number of slots the buffer holds).

The slot capacity follows from the buffer's 1,024 cells and the downstream [Lane Folder's](#) `LaneMode`. Each slot is a `[Lane, Packet]` chunk whose shape the `LaneMode` decides, so the number of slots is 1,024 divided by the cells per chunk:

LaneMode	Chunk shape	Cells per chunk	Slot capacity
Interleaved	[Lane # 8,	8 ×	128 /

	Packet]	Packet::SIZE	Packet::SIZE
Sequential	[Lane, Packet # 32]	Lane::SIZE × 32	32 / Lane::SIZE

For `reduce_b` under the downstream `.contract_lane(LaneMode::Sequential)` with `Lane::SIZE = 1`, the slot capacity is 32 and `InnerTime::SIZE = 8` fits comfortably. If `InnerTime::SIZE` exceeded the slot capacity, you would restructure `Time` (e.g., split `B` further) or switch `LaneMode` to trade throughput for slot headroom.

Performance

Throughput is one packet per cycle on the input side. The effective output rate is $1 / N$ of the input after reducing N inputs into one output.

Latency for a `Time`-dimension reduction of size N is approximately N cycles.

Lane Folder

The Lane Folder is the Contraction Engine's final stage. It eliminates the `Lane` dimension by relocating its 8 values into either `OutPacket` (Interleaved) or `OutTime` (Sequential). No values are summed: the stage folds `Lane` into another axis rather than reducing it.

Interface

`.contract_lane(mode)` invokes the Lane Folder. The stage drains the upstream Time Reducer's buffer through an 8-element-wide output bus one cycle at a time, and `LaneMode` selects what each cycle's flit carries.


```

impl<'l, const T: Tu, D: ContractionAccumulator, Chip: M, Cluster: M, Slice:
M, Lane: M, Time: M, Packet: M, B: Backend>
    ContractTimeTensor<'l, T, D, Chip, Cluster, Slice, Lane, Time, Packet, B>
{
    /// Folds the `Lane` dimension into the output stream.
    /// `LaneMode::Interleaved` relocates `Lane` into `OutPacket`;
    /// `LaneMode::Sequential` relocates `Lane` into `OutTime`.
    #[primitive(ContractTimeTensor::contract_lane)]
    pub fn contract_lane<OutTime: M, OutPacket: M>(
        self,
        mode: LaneMode,
    ) -> ContractTensor<'l, T, D, Chip, Cluster, Slice, OutTime, OutPacket,
B> {
        verify_contract_lane(ContractLaneInput {
            in_lane: Lane::to_value(),
            in_time: Time::to_value(),
            in_packet: Packet::to_value(),
            out_time: OutTime::to_value(),
            out_packet: OutPacket::to_value(),
            pre_reduce_time: self.pre_reduce_time,
            mode: mode.into(),
        });
        // Earlier stages only retype the deferred operands. Perform their
fused contraction here,
        // then apply the lane-fold relayout.
        // `contraction_prewidened` validates `out` through mapping carve.
Symbol-wise comparison is
        // invalid because one contracted symbol may span spatial and reduced
slots.
        let contraction = self.inner;
        let out = <m![{ Chip }, { Cluster }, { Slice }, { Lane }, { Time }, {
Packet }]>::to_value();
        let reduced: Tensor<D, m![{ Chip }, { Cluster }, { Slice }, { Lane },
{ Time }, { Packet }], B> =
            Tensor::from_inner(B::contraction_prewidened(
                &contraction.lhs,
                &contraction.rhs,
                &contraction.lhs_map,
                &contraction.rhs_map,
                &contraction.pre_reduce,
                &out,
            ));
        ContractTensor::new(self.ctx, reduced.transpose(false))
    }
}

```

The minimal examples below take a `ContractTimeTensor` (the output of the upstream Time Reducer) and call only `.contract_lane(...)`, so each example shows the Lane

Folder in isolation. The input `Packet` carries the size that survived the [Packet Reducer](#), one of `{1, 2, 4, 8, 16, 32}` elements per lane.

Interleaved

The `Lane` dimension folds into `OutPacket`: each cycle reads one column position across all 8 lanes (one value per lane, 8 values per flit), with `Lane` materialized as the innermost `OutPacket`.

```
OutTime    = [Time, Packet]
OutPacket  = [Lane # 8]
```

```
axes![N = 8, M = 4, P = 16];
```

```
/// Lane folds into OutPacket.
fn lane_interleaved<'l, const T: Tu>(
    // Input from upstream Time Reducer: Lane = m![N], Time = m![M], Packet =
    m![P].
    input: ContractTimeTensor<'l, T, f32, m![1], m![1 # 2], m![1 # 256], m!
    [N], m![M], m![P]>,
    // Output: OutTime = m![M, P] = [Time, Packet], OutPacket = m![N] =
    [Lane].
) -> ContractTensor<'l, T, f32, m![1], m![1 # 2], m![1 # 256], m![M, P], m!
[N]> {
    input.contract_lane::<m![M, P], m![N]>(LaneMode::Interleaved)
}
```

Sequential

The `Lane` dimension folds into `OutTime`: each cycle reads 8 column positions from one lane's `Packet` (8 values per flit), with `Lane` iterating across successive cycles. Since each cycle is 8 elements wide, `Packet` is first padded up to a multiple of 8 (the bus width), then split into `PadPacket / 8` cycles per lane and `PadPacket % 8` elements per cycle.

```
PadPacket = Packet # align_up(Packet::SIZE, 8)    (pad Packet up to the next
multiple of 8)
OutTime    = [Time, Lane, PadPacket / 8]
OutPacket  = [PadPacket % 8]
```

For `Packet::SIZE < 32`, `[PadPacket / 8]::SIZE = ceil(Packet::SIZE / 8)` is the

number of cycles per packet (e.g., 1 cycle for `Packet::SIZE = 4`, 2 cycles for `Packet::SIZE = 16`).

```
axes![N = 8, M = 4, P = 16];

/// Lane folds into OutTime.
fn lane_sequential<'l, const T: Tu>(
    // Input from upstream Time Reducer: Lane = m![N], Time = m![M], Packet =
    m![P].
    input: ContractTimeTensor<'l, T, f32, m![1], m![1 # 2], m![1 # 256], m!
    [N], m![M], m![P]>,
    // Output: OutTime = m![M, N, P / 8] = [Time, Lane, Packet / 8],
    OutPacket = m![P % 8] = [Packet % 8].
) -> ContractTensor<'l, T, f32, m![1], m![1 # 2], m![1 # 256], m![M, N, P /
8], m![P % 8]> {
    input.contract_lane::<m![M, N, P / 8], m![P % 8]>(LaneMode::Sequential)
}
```

Constraints

The Lane Folder has no constraints of its own. The `LaneMode` selected here determines the slot capacity bound that the upstream Time Reducer enforces (see [Time Reducer Constraints](#)).

Performance

In Interleaved mode, throughput drops by `Lane::SIZE / 8` when `Lane < 8` (inactive lanes leave bus positions empty).

In Sequential mode, when `Packet::SIZE < 8` (e.g., `Packet::SIZE = 4` after the Packet Reducer collapses half of an `bf16` packet), each cycle carries exactly `Packet::SIZE` elements rather than the full 8-element bus width: the output is narrower per cycle, but there is no padding and no wasted bus slots.

Latency is negligible: the Lane Folder reshapes per-lane outputs and does not add cycles beyond the buffer's drain time.

Vector Engine

The Vector Engine performs elementwise computation and reduction. Examples include activations (GELU, SiLU), normalizations (softmax, layer norm), binary operations, and intra- and inter-slice reductions.

The engine accepts only 32-bit types, `i32` and `f32`. An upstream [Contraction Engine](#) widens types automatically (`bf16` products accumulate in `f32`, `i8` products in `i32`). When that engine is bypassed, the [Fetch Adapter](#) must widen the input via its type-cast adapter.

Interface

In a single Tensor Unit invocation, the Vector Engine portion is the method chain from `vector_init()` to `vector_final()`. The engine has two sub-pieces: the [Intra-Slice Chain](#) (elementwise / binary / per-slice reduce stages) and the [Inter-Slice Reducer](#) (reduces across the 256 slices in a cluster). Between `vector_init()` and `vector_final()`, the chain alone, the reducer alone, or both can run. When both run, the order is `IntraFirst` (chain then reducer) or `InterFirst` (reducer then chain).

The intra-slice chain is entered via `vector_intra_slice_tag()`, or `vector_intra_slice_unzip()` when the input carries a 2-way grouping axis to split into two parallel streams (see [Pair Mode](#)). Either entry point fires right after `vector_init()` or on the inter-slice reducer's output. The inter-slice reducer is entered via `vector_inter_slice_reduce()`, either right after `vector_init()` or from a [compatible intra-slice stage](#). For stage-by-stage API coverage, see [Intra-Slice Chain](#) and [Inter-Slice Reducer](#).

The signatures below cover the `vector_init()`-side entry methods only. The same method names (`vector_intra_slice_tag`, `vector_inter_slice_reduce`) also exist on chain and reducer tensors for the chain↔reducer transitions; those are documented in the child pages.

```
impl<'l, const T: Tu, P: CanApplyVectorInit, D: VeScalar, Chip: M, Cluster:
M, Slice: M, Time: M, Packet: M>
    TuTensor<'l, T, P, D, Chip, Cluster, Slice, Time, Packet>
{
    /// Initializes Vector Engine processing for this tensor.
```

```

    #[primitive(TuTensor::vector_init)]
    pub fn vector_init(self) -> VectorInitTensor<'l, T, D, Chip, Cluster,
Slice, Time, Packet> {
        VectorInitTensor::new(self.ctx, self.inner)
    }
}

impl<'l, const T: Tu, D: VeScalar, Chip: M, Cluster: M, Slice: M, Time: M,
Packet: M>
    VectorInitTensor<'l, T, D, Chip, Cluster, Slice, Time, Packet>
{
    /// Enters VE intra-slice pipeline (single stream).
    #[primitive(VectorInitTensor::vector_intra_slice_tag)]
    pub fn vector_intra_slice_tag(
        self,
        branch: TagMode<D>,
    ) -> VectorBranchTensor<'l, T, D, Chip, Cluster, Slice, Time, Packet,
Fresh, { VeOrder::IntraFirst }> {
        VectorBranchTensor::new(self.ctx, self.inner, branch)
    }

    /// Enters VE intra-slice pipeline (two-group / unzip).
    #[primitive(VectorInitTensor::vector_intra_slice_unzip)]
    pub fn vector_intra_slice_unzip<I: AxisName, TileTime: M, SplitTime: M>(
        self,
    ) -> VectorTensorPair<'l, T, D, stage::Tag, Chip, Cluster, Slice,
SplitTime, Packet> {
        verify_vector_intra_slice_unzip::<I, Time, Packet>();
        VectorTensorPair::new::<I, Time, TileTime>(self.ctx, self.inner)
    }
}

impl<'l, const T: Tu, Chip: M, Cluster: M, Slice: M, Time: M, Packet: M>
    VectorInitTensor<'l, T, i32, Chip, Cluster, Slice, Time, Packet>
{
    /// Performs inter-slice reduce for i32 as the first VE operation.
    #[primitive(VectorInitTensor::vector_inter_slice_reduce)]
    pub fn vector_inter_slice_reduce<OutSlice: M, OutTime: M>(
        self,
        op: InterSliceReduceOpI32,
    ) -> VectorInterSliceReduceTensor<'l, T, i32, Chip, Cluster, OutSlice,
OutTime, Packet, { VeOrder::InterFirst }>
    {
        let reduced = self.inner.reduce(op.reduce_fn(), op.identity(), true);
        create_inter_slice_reduce_tensor(self.ctx, reduced)
    }
}

```

Examples

A few representative examples follow to give a feel for what a Vector Engine call looks like, with the full API tour deferred to the child pages [Intra-Slice Chain](#) and [Inter-Slice Reducer](#).

ReLU Activation

This pass applies ReLU elementwise, computing $output[b, k, m, n] = \max(input[b, k, m, n], 0)$.

```
axes![B = 2, K = 256, M = 16, N = 16];

// ReLU activation after batched matrix multiplication.
// Chain-only pass, so the reducer is skipped and the path trivially resolves
// to IntraFirst.
// Both clusters (B = 2) and all 256 slices (K) carry real data, no padding.
fn relu<'l, const T: Tu>(
    input: ContractTensor<'l, T, f32, m![1], m![B], m![K], m![M, N / 8], m![N
% 8]>,
) -> VectorFinalTensor<'l, T, f32, m![1], m![B], m![K], m![M, N / 8], m![N %
8]> {
    input
        .vector_init()
        .vector_intra_slice_tag(TagMode::Zero)
        // max(x, 0), the ReLU itself
        .vector_clip(ClipBinaryOpF32::Max, 0.0f32)
        .vector_final()
}
```

ReLU Then Reduce

This pass applies ReLU per slice and then reduces across R , giving $output[a, b] = \sum_{r \in R} \max(input[a, b, r], 0)$.

```

axes![A = 512, B = 2, R = 4];

// Chain applies ReLU, then reducer sums across slices.
// IntraFirst shape (chain runs first, then reducer).
// Both clusters (B = 2), all 256 slices (A / 8 * R), and full Way8 packet (A
% 8) carry real data.
fn relu_then_reduce<'l, const T: Tu>(
    input: CollectTensor<'l, T, i32, m![1], m![B], m![A / 8, R], m![1], m![A
% 8]>,
) -> VectorFinalTensor<'l, T, i32, m![1], m![B], m![A / 8, 1 # 4], m![1], m!
[A % 8]> {
    input
        .vector_init()
        .vector_intra_slice_tag(TagMode::Zero)
        // max(x, 0), the ReLU
        .vector_clip(ClipBinaryOpI32::Max, 0)
        // sum across R slices
        .vector_inter_slice_reduce::<m![A / 8, 1 # 4], m![1]>
(InterSliceReduceOpI32::AddSat)
        .vector_final()
}

```

Reduce Then Bias

This pass reduces across R and then adds a constant bias, giving

$$output[a, b] = \left(\sum_{r \in R} input[a, b, r] \right) + 100.$$

```

axes![A = 512, B = 2, R = 4];

// Reducer sums across slices, then chain adds a bias to the reduced result.
// InterFirst shape (reducer runs first, then chain).
// Both clusters (B = 2), all 256 slices, and full Way8 packet carry real
data.
fn reduce_then_add<'l, const T: Tu>(
    input: CollectTensor<'l, T, i32, m![1], m![B], m![A / 8, R], m![1], m![A
% 8]>,
) -> VectorFinalTensor<'l, T, i32, m![1], m![B], m![A / 8, 1 # 4], m![1], m!
[A % 8]> {
    input
        .vector_init()
        // sum across R slices
        .vector_inter_slice_reduce::<m![A / 8, 1 # 4], m![1]>
(InterSliceReduceOpI32::AddSat)
        .vector_intra_slice_tag(TagMode::Zero)
        // add bias 100
        .vector_fxp(FxpBinaryOp::AddFxp, 100)
        .vector_final()
}

```

Intra- and Inter-Slice Reduction

This pass reduces R entirely by combining the intra-slice reducer (over R 's Time and Packet portions) and the inter-slice reducer (over R 's Slice portion). The einsum form is $BR \rightarrow B$, with saturating addition.


```

axes![B = 2, R = 8192];

// R splits across Slice (R / 32 = 256), Time (R % 32 / 4 = 8), and Packet (R
% 4, padded to 8 in Way8).
// Chain runs intra-slice reduce over R's Time and Packet portions, then the
reducer collapses the Slice portion.
// IntraFirst shape (chain runs first, then reducer).
fn full_sum<'l, const T: Tu>(
    input: CollectTensor<'l, T, i32, m![1], m![B], m![R / 32], m![R % 32 /
4], m![R % 4 # 8]>,
) -> VectorFinalTensor<'l, T, i32, m![1], m![B], m![1 # 256], m![1], m![1 #
8]> {
    input
        .vector_init()
        .vector_intra_slice_tag(TagMode::Zero)
        // Way8 → Way4 (back 4 packet positions were padding)
        .vector_narrow_trim::<m![R % 4]>()
        // sum over R's Time and Packet portions
        .vector_intra_slice_reduce::<R, m![1], m![1 # 4]>
(IntraSliceReduceOpI32::AddSat)
        // Way4 → Way8
        .vector_widen_pad::<m![1 # 8]>()
        // sum over R's Slice portion across all 256 slices
        .vector_inter_slice_reduce::<m![1 # 256], m![1]>
(InterSliceReduceOpI32::AddSat)
        .vector_final()
}

```

Pair Add

This pass unzips two interleaved groups along I and adds them pair-wise. The einsum form is $ABI \rightarrow AB$.

```

axes![A = 2048, B = 2, I = 2];

// Pair-mode entry via unzip, then a zip op fuses the two streams with an
add.
// Both clusters (B = 2), all 256 slices (A / 8), and full Way8 packet (A %
8) carry real data.
fn pair_add<'l, const T: Tu>(
    input: CollectTensor<'l, T, i32, m![1], m![B], m![A / 8], m![I], m![A %
8]>,
) -> VectorFinalTensor<'l, T, i32, m![1], m![B], m![A / 8], m![1], m![A % 8]>
{
    input
        .vector_init()
        // split into group 0 and group 1 along I
        .vector_intra_slice_unzip::<I, m![1 # 2], m![1]>()
        // group0 + group1
        .vector_clip_zip(ClipBinaryOpI32::AddFxp)
        .vector_final()
}

```

Intra-Slice Chain

The Intra-Slice Chain performs elementwise, binary, and intra-slice reduce operations on each slice's data independently. It handles post-contraction processing such as activation and normalization. For example, computing $\text{sigmoid}(XW + b)$ runs XW on the [Contraction Engine](#), and the addition plus sigmoid activation in the Intra-Slice Chain.

Interface

The example below applies a fixed-point bias, runs sigmoid on the float path (narrowing to 4-way and widening back), and finishes with a ReLU clip. It computes $\text{output}[a, b] = \max(\text{sigmoid}(\text{input}[a, b] + 100), 0)$.

```
axes![A = 512, B = 2];

fn staged_pipeline<'l, const T: Tu>(
    input: CollectTensor<'l, T, i32, m![1], m![B], m![A / 2], m![1], m![A % 2
# 8]>,
) -> VectorFinalTensor<'l, T, i32, m![1], m![B], m![A / 2], m![1], m![A % 2 #
8]> {
    input
    .vector_init()
    .vector_intra_slice_tag(TagMode::Zero)
    // input + 100
    .vector_fxp(FxpBinaryOp::AddFxp, 100)
    // i32 → f32 (fixed-point, int_width = 31)
    .vector_fxp_to_fp(31)
    // Way8 → Way4 for the float path
    .vector_narrow_trim::<m![A % 2 # 4]>()
    // sigmoid(input + 100)
    .vector_fp_unary(FpUnaryOp::Sigmoid)
    // Way4 → Way8
    .vector_widen_pad::<m![A % 2 # 8]>()
    // f32 → i32
    .vector_fp_to_fxp(31)
    // max(sigmoid(input + 100), 0)
    .vector_clip(ClipBinaryOpI32::Max, 0)
    .vector_final()
}
```

Pipeline

The chain starts (as shown above) with `vector_intra_slice_tag()` (right after `vector_init()` or on the inter-slice reducer's output), or with `vector_intra_slice_unzip()` directly after `vector_init()` for [Pair Mode](#) (see below).

```
#[primitive(VectorInitTensor::vector_intra_slice_tag)]
pub fn vector_intra_slice_tag(
    self,
    branch: TagMode<D>,
) -> VectorBranchTensor<'l, T, D, Chip, Cluster, Slice, Time, Packet,
Fresh, { VeOrder::IntraFirst }> {

#[primitive(VectorInitTensor::vector_intra_slice_unzip)]
pub fn vector_intra_slice_unzip<I: AxisName, TileTime: M, SplitTime: M>(
    self,
) -> VectorTensorPair<'l, T, D, stage::Tag, Chip, Cluster, Slice,
SplitTime, Packet> {
```

After entry, the chain steps through the pipeline stages below in a fixed order; software chains the relevant ones and skips the rest, as the example skips `Logic`, `FpDiv`, and `Filter`. Each row lists the stage's position in the chain (`#`), its name (`Stage`), the API method that triggers it (`Method`), the way it runs in (`Way`, either 8 or 4 elements per cycle), and whether it accepts an operand (`Operand`). The type system enforces every stage transition at compile time, so methods become callable only after the preceding chain reaches a compatible state. Per-stage detail is in [Stages](#) below.

#	Stage	Method	Way	Op
1	Entry	<code>vector_intra_slice_tag()</code>	Way8	–
2	Logic	<code>vector_logic()</code>	Way8	yes
3	Fxp	<code>vector_fxp()</code>	Way8	yes
4	FxpToFp	<code>vector_fxp_to_fp()</code>	Way8	–
5	Narrow	<code>vector_narrow_split()</code> / <code>vector_narrow_trim()</code>	Way8 → Way4	–
6	Float	<code>vector_fp_unary/binary/ternary()</code>	Way4	yes
7	IntraSliceReduce	<code>vector_intra_slice_reduce()</code>	Way4	–
8	FpDiv	<code>vector_fp_div()</code>	Way4	yes
		<code>vector_widen_concat()</code> /	Way4	

9	Widen	<code>vector_widen_pad()</code>	→ Way8	–
10	FpToFxp	<code>vector_fp_to_fxp()</code>	Way8	–
11	Clip	<code>vector_clip()</code>	Way8	yes
12	Filter	<code>vector_filter()</code>	Way8	–

`vector_reinterpret()` is deliberately absent from the table: it occupies no stage and can sit between any two of them (see [Reinterpret](#)).

Stages run either 8-way (8 elements per cycle) or 4-way (4 elements per cycle). The floating-point cluster runs 4-way to amortize its half-throughput ALUs against the rest of the chain. A chain that uses the float path therefore enters 8-way, calls `Narrow` (`vector_narrow_split` or `vector_narrow_trim`) before the float stages, and calls `Widen` (`vector_widen_concat` or `vector_widen_pad`) afterward to return to 8-way. The example does exactly this: `vector_narrow_trim` then `vector_fp_unary(Sigmoid)` then `vector_widen_pad`.

The chain exits via `vector_final()` (as in the example) or `vector_inter_slice_reduce()` (handing off to the [Inter-Slice Reducer](#)). Both exits require 8-way, so any active 4-way stage must pass through `Widen` first.

Operands

Binary and ternary ops have two slot kinds: a **stream** (the running tensor, i.e., the `self` of the method chain, fixed by the chain) and one or two **operands** (the extra inputs). Each operand comes from one of three sources.

Source	Example	Description
Constant	<code>100</code> , <code>2.5f32</code>	Scalar broadcast to all elements
VRF tensor	<code>&vrf_tensor</code>	Pre-loaded via <code>.to_vrf()</code> before entering the Vector Engine
Stash	<code>Stash</code>	Snapshot of an earlier chain step, described below

Ternary ops (`FmaF`) take a pair (`operand0`, `operand1`).

A VRF operand is read per slice by an indexer, which puts three rules on it.

First, its `Chip` / `Cluster` / `Slice` are the stream's own: at slice `s` the op reads what slice

`s` holds, so an operand under a different partition is a compile error instead of a silent read of another slice's data. `VrfTensor::reshape` restates one under the stream's partition when the two describe the same slices, e.g. an operand replicated under an anonymous `m!` [256] feeding a stream partitioned by a named axis.

Second, one access (the `Packet` : 8 lanes in 8-way, 4 in 4-way) is one indexer step, and the indexer offers just two read options. **Broadcast** feeds every lane from one address; **contiguous** reads `Packet` consecutive `Element` cells. So the packet axes must be the operand's innermost, contiguous ones, or absent from it entirely; a packet that walks an outer axis of the operand, or mixes broadcast and live lanes inside one access, has no encoding and is rejected.

Third, the two directions of an axis mismatch are not the same. An axis the operand has and the stream does not is refused, with the stream leaves operand axes unmatched, because the cells it indexes would never be read. An axis the stream has and the operand does not is a broadcast, which is how one operand feeds every row of the stream.

The same op method picks up different sources by the argument type:

```
.vector_fxp(FxpBinaryOp::AddFxp, 100)           // operand from constant
.vector_fxp(FxpBinaryOp::MulInt, &vrf)          // operand from VRF tensor
.vector_clip(ClipBinaryOpI32::Max, Stash)        // operand from stash (set
earlier)
```

The `stash` source comes from `vector_stash()`, which snapshots the running tensor so a later binary or ternary op can read it back as the `stash` operand. The typical use is a residual or skip-connection like `max(f(x), x)`, where the original `x` must survive across intermediate stages. Although `stash` is presented as a separate operand source, it is backed by the VRF: part of the current running tensor is stored in the VRF and read back from there. Whenever a Tensor Unit invocation uses `stash`, the compiler conservatively reserves 1,024 B of VRF capacity per slice for it, regardless of the actual amount of data stashed. If that invocation also reads a pre-loaded VRF tensor as an RHS operand, size the operand with this reservation in mind: `stash` and the RHS share the 8 KiB VRF, so at most 7 KiB per slice remains for the RHS. Call `vector_stash()` at any `Stashable` stage (`Branch`, `Logic`, `Fxp`, `Narrow`, `Fp`, `FpDiv`, `Clip`); the snapshot stays live until the Tensor Unit invocation ends and feeds any later binary or ternary call that takes `stash`. The slot is single-use (a second `vector_stash()` is a compile-time error) and typed, so an `f32` `stash` only feeds `f32` ops: a conversion or reinterpret between the write and the read has to be undone first. The `stash` is also read-once: it feeds exactly one later op (reading it moves the slot past `occupied`, so a second `stash` read is a compile-time

error). A value that must be read more than once is not a stash: put it in a read-many VRF, passed as `&vrf`. The mapping follows the running tensor, so a stash taken before `Narrow` is still usable after `Widen`. Those seven stages are the seven chain steps that own a VRF write port, which is why `Widen` is not one of them: the operand register cannot be written from the widen's output at all. A value computed in the 4-way region therefore reaches the stash by widening first and stashing at a `clip` op, which puts both the write and the read on the 8-way side. Stash is unavailable in **Pair Mode**, and the `IntraFirst` transition to the **inter-slice reducer** drops it, so anything stashed before `vector_inter_slice_reduce()` is gone afterward.

An **argument mode** then picks which slots hold the stream versus the operands so the same op can compute, e.g., `stream + operand` or `operand - stream`. For example, `BinaryArgMode::Mode10` swaps the slots so `SubFxp` computes `operand - stream`:

```
.vector_fxp_with_mode(FxpBinaryOp::SubFxp, BinaryArgMode::Mode10, 7) //
computes 7 - stream
```

`BinaryArgMode` picks which two of a binary op's slots are stream vs operand (unary ops have no mode and always run as `op(stream)`):

BinaryArgMode	Slots	Computation
Mode00	stream / stream	<code>op(stream, stream)</code>
Mode01	stream / operand	<code>op(stream, operand)</code> (default)
Mode10	operand / stream	<code>op(operand, stream)</code>
Mode11	operand / operand	<code>op(operand, operand)</code>

`TernaryArgMode` does the same for ternary ops:

TernaryArgMode	Slots	Computation
Mode012	stream / operand0 / operand1	<code>op(stream, operand0, operand1)</code> (default)
Mode002	stream / stream / operand1	<code>op(stream, stream, operand1)</code>
Mode102	operand0 / stream / operand1	<code>op(operand0, stream, operand1)</code>
Mode112	operand0 / operand0 / operand1	<code>op(operand0, operand0, operand1)</code>
	stream / operand1 /	

Mode020	stream	op(stream, operand1, stream)
Mode021	stream / operand1 / operand0	op(stream, operand1, operand0)
Mode120	operand0 / operand1 / stream	op(operand0, operand1, stream)

Pair Mode

Pair mode runs the chain on a tensor whose elements split into two interleaved groups, so an op can relate the two groups (e.g., pair-wise add, asymmetric scale). Entry is `vector_intra_slice_unzip()` directly after `vector_init()`, applied to a collected tensor that carries a 2-way grouping axis. Starting the chain with `vector_intra_slice_unzip()` precludes the [Filter](#) stage downstream. Under the hood, `vector_intra_slice_unzip()` derives each element's `GroupId` from its position along the 2-way grouping axis, driving the branch unit's count augementer rather than any `TagMode`. The tag the chain was started with is untouched, which is why this is the supported route to a group split and `TagMode::AxisToggle` is not.

The flow has four steps:

1. `vector_intra_slice_unzip()` splits the input into two parallel streams (group 0 and group 1).
2. The chain runs through stages with both groups in lock-step (the **paired** phase).
3. A `_zip` op fuses the two streams back into one (the **merged** phase).
4. The merged stream continues to `vector_final()` like a normal chain.

Stages during the paired phase fall into two flavors:

- **Common stages** (`vector_fxp_to_fp`, `vector_narrow_split`, `vector_widen_concat`, `vector_fp_to_fxp`) act on both groups uniformly. `vector_narrow_trim` and `vector_widen_pad` are not available on pairs; use the `_split` / `_concat` variants instead.
- **Per-group ops** take one argument per group:
 - Binary and ternary (`vector_fxp`, `vector_fp_binary`, `vector_fp_ternary`, `vector_clip`, etc.) accept `()` on a side to skip it, or different operands on each side.
 - Unary (`vector_fp_unary`) is the exception: it takes flags (`op`, `group0_apply`,

`group1_apply)` and `false` skips that group.

Pair mode reinterprets `BinaryArgMode` depending on the op: per-group ops (`vector_fxp_with_mode`, `vector_fp_binary_with_mode`, etc.) apply the mode inside each group independently (`0` is that group's stream, `1` is that group's operand), while `_zip` ops (`vector_fxp_zip_with_mode`, etc.) take the two slots as the two grouped streams (`0` is Group 0's stream, `1` is Group 1's stream):

<code>_zip</code> <code>BinaryArgMode</code>	Slots	Computation
Mode00	group0 / group0	<code>op(group0, group0)</code>
Mode01	group0 / group1	<code>op(group0, group1)</code> (default)
Mode10	group1 / group0	<code>op(group1, group0)</code>
Mode11	group1 / group1	<code>op(group1, group1)</code>

Pair-mode constraints:

- `stash()` and `filter()` are unavailable throughout pair mode (both paired and merged phases).
- `vector_intra_slice_reduce()` is unavailable throughout pair mode as well, before and after the `_zip`: the reducer takes no group condition.
- Before `_zip` (the paired phase), the chain cannot transition to the `inter-slice reducer`, since `vector_inter_slice_reduce()` is not available on per-group tensors. After `_zip` (the merged phase), the result is `Commitable` again and can call `vector_inter_slice_reduce()` if the current stage supports the transition.
- ALU usage is shared across the two groups: an ALU used in either group counts as consumed for both.

Stages

Within a stage, each ALU runs at most once per Tensor Unit invocation. This matters mainly in `Logic`, `Fxp`, `Fp`, and `Clip`, where multiple operators share a stage-local ALU pool. For example, `tanh(sqrt(x))` cannot fit in a single Tensor Unit invocation because both `tanh` and `sqrt` consume the `FpFpu` ALU.

Tag

The Tag stage is the chain's entry point and assigns each 32-bit element in a flit a 4-bit `Tag` (0-15), which later stages use to apply conditional operations. Bit 3 (the MSB) is `GroupId`, used by Filter and pair mode to split elements into Group 0 / Group 1. Bits 0..2 are general-purpose flag bits filled by comparison results.

The `TagMode` selects how the 4 bits are computed for each element. Only `Zero` and `Comparison` are available today. `AxisToggle` is declared but not compilable, as the last column records; `ValidCount` and `Vrf` are withheld from the enum entirely until they run on both paths.

TagMode	How each tag bit is filled	Status
Zero	All four bits are 0. Every element has tag = 0.	Works
<code>Comparison([cmp0, cmp1, cmp2, cmp3])</code>	For each element x , bit $i = 1$ iff <code>cmp_i(x)</code> holds. Each <code>cmp_i</code> is a <code>Cmp<D></code> carrying the boundary it compares against, typed by the stream's scalar so the boundary is checked against it: <code>Equal</code> , <code>Less</code> , <code>Greater</code> , <code>LessUnsigned</code> , <code>GreaterUnsigned</code> , <code>True</code> , <code>False</code> . The <code>*Unsigned</code> pair compares raw bit patterns.	Works
<code>AxisToggle { axis }</code>	Bit 3 (<code>GroupId</code>) = <code>axis_index % 2</code> along <code>axis</code> . Bits 0..2 stay 0.	Not compilable. Runs on the host, but rejected with a kernel compile error when lowered. <code>vector_intra_slice_unzip()</code> is the supported way to get this grouping.

For example, with `i32` data and `Comparison([Cmp::Less(0), Cmp::Equal(5), Cmp::Greater(100), Cmp::True])`, an element `x = 7` yields bits `0/0/0/1` (LSB first), so its tag is `0b1000 = 8`.

Once tags are assigned, later binary and ternary ops can condition an operand on the tag, so different elements see different values. The rule is one sentence: with no condition write the operand bare, and with a condition name the hardware slot it drives, through `Branched.ve_elementwise_branched` in `furiosa-opt-examples` is a compiled kernel doing exactly that – a guarded immediate with a trailing unconditional `rf` read – and `ve_branch_bit_order` adds three immediates and a guarded stash read. Both are read here rather than restated, so an API change breaks them instead of leaving prose behind.

A `TagGuard` is `TagGuard::all()`, `TagGuard::group(id)` for one group, or `matches/not_matches` of a `[BitReq; 4]` pattern written LSB first, one `BitReq` per tag bit: `one` where the bit must be 1, `zero` where it must be 0, `ignore` where it does not matter. The requirement is on the bit's value, whatever the tag mode filled it from.

A pass has four operand slots: three immediate registers and the register-file port, which reads either a VRF register or the stash. A conditional operand says which *kind* of slot it drives — `Branched::imm` for an immediate, `Branched::rf` for the port — and gets back a builder offering only what may still follow. `imm` may be called three times and `rf` once, last; a fourth `imm`, or an `imm` after `rf`, does not compile.

The kernel does not pick *which* immediate register: `imm` takes the next one, and which physical register a value ends up in is command generation's choice. What is preserved is **call order** — the slots reach the hardware in the order they were filled, and that order is match priority, so an element takes the first slot whose guard it satisfies and an unconditional slot is the pass's `else` with nothing after it.

Two guards are rejected outright, because a slot carrying one is filled and inert while a pass has only four: `TagGuard::not_matches([Ignore, Ignore, Ignore, Ignore])`, which no execution id satisfies, and any slot placed after an unconditional one.

`TagGuard::matches([Ignore, Ignore, Ignore, Ignore])` is accepted and stored as `TagGuard::all()`, the same predicate spelled shorter.

Helpers that build a layout and hand it back are ordinary functions, so a kernel library can wrap whatever combinations it uses often. Two things such a helper cannot do, both of which the `operand` module documents in full: it cannot introduce a *new* operand type (an operand carries a claim about what it does to the stash, so the traits that say so are closed), and it cannot fill a slot conditionally, because how many slots are spoken for is

part of the builder's type and the two arms of an `if` would disagree. Vary the payload or the guard instead of the slot count, or branch around the whole op.

Logic Cluster

The Logic Cluster performs bitwise operations on `i32` or `f32` (bit-level). It runs 8-way.

The stage exposes five ALU classes (`LogicAnd`, `LogicOr`, `LogicXor`, `LogicLshift`, `LogicRshift`), each runnable once per Tensor Unit invocation. Operators sharing the same class cannot fuse into one invocation.

`i32` operations:

Op	ALU	Note
<code>BitAnd</code>	<code>LogicAnd</code>	bitwise and
<code>BitOr</code>	<code>LogicOr</code>	bitwise or
<code>BitXor</code>	<code>LogicXor</code>	bitwise xor
<code>LeftShift</code>	<code>LogicLshift</code>	logical left shift
<code>LogicRightShift</code>	<code>LogicRshift</code>	logical right shift
<code>ArithRightShift</code>	<code>LogicRshift</code>	arithmetic right shift

`f32` operations:

Op	ALU	Note
<code>BitAnd</code>	<code>LogicAnd</code>	bitwise and on fp bit patterns
<code>BitOr</code>	<code>LogicOr</code>	bitwise or on fp bit patterns
<code>BitXor</code>	<code>LogicXor</code>	bitwise xor on fp bit patterns

Fxp Cluster

The Fxp Cluster performs integer and fixed-point arithmetic on `i32`. It runs 8-way.

The stage exposes four ALU classes (`FxpAdd`, `FxpLshift`, `FxpMul`, `FxpRshift`), each runnable once per Tensor Unit invocation. Operators sharing the same class cannot fuse into one invocation.

Op	ALU	Note
AddFxp	FxpAdd	wrapping add
AddFxpSat	FxpAdd	saturating add
SubFxp	FxpAdd	wrapping subtract
SubFxpSat	FxpAdd	saturating subtract
LeftShift	FxpLshift	logical left shift
LeftShiftSat	FxpLshift	saturating left shift
MulFxp	FxpMul	fixed-point multiply
MulInt	FxpMul	integer multiply
LogicRightShift	FxpRshift	logical right shift
ArithRightShift	FxpRshift	arithmetic right shift
ArithRightShiftRound	FxpRshift	arithmetic right shift with rounding

The single-ALU rule rejects, for example, two ops that both target `FxpAdd` :

```
// PANICS: "FxpAdd is already in use"
input
  .vector_init()
  .vector_intra_slice_tag(TagMode::Zero)
  .vector_fxp(FxpBinaryOp::AddFxp, 10) // uses FxpAdd
  .vector_fxp(FxpBinaryOp::MulInt, 2)  // uses FxpMul ✓
  .vector_fxp(FxpBinaryOp::SubFxp, 5)  // uses FxpAdd again ✗
  .vector_final()
```

FxpToFp Conversion

The FxpToFp Conversion stage converts `i32` to `f32`. The `int_width` parameter specifies the integer bit width for the conversion; `int_width = 31` is the standard `i32` $\leftrightarrow$ `f32` conversion.

Method	Effect
<code>vector_fxp_to_fp(int_width)</code>	convert <code>i32</code> stream to <code>f32</code>

Narrow

The `Narrow` stage switches 8-way to 4-way. An 8-way packet carries 8 active elements

(`Packet = m![... # 8]`) and a 4-way packet carries 4 (`Packet = m![... # 4]`).

Narrowing halves throughput on the float and reduce path, so the same logical tensor shape takes twice as many packets or Tensor Unit invocations.

Method	Use When	Effect
<code>vector_narrow_split()</code>	both halves contain real data	split one 8-way flit into a front-4 and back-4 packet, updating <code>Time</code> and <code>Packet</code>
<code>vector_narrow_trim()</code>	back 4 elements are already padding or irrelevant	keep only the front 4 elements

Shape semantics:

```
axes![A = 512, B = 2, S = 64];
```

```
fn vector_narrow_split_semantics<'l, const T: Tu>(
  input: VectorBranchTensor<'l, T, i32, m![1], m![B], m![S / 4 # 256], m![S
% 4], m![A % 8], Fresh, { stage::VeOrder::IntraFirst }>,
) -> VectorNarrowTensor<'l, T, i32, m![1], m![B], m![S / 4 # 256], m![S % 4,
A / 4 % 2], m![A % 4], Fresh, { stage::VeOrder::IntraFirst }>
{
  input.vector_narrow_split::<m![S % 4, A / 4 % 2], m![A % 4]>()
  // shape semantics: [T], [P] -> [T, P / 2], [P % 4]
}
```

```
fn vector_narrow_trim_semantics<'l, const T: Tu>(
  input: VectorBranchTensor<'l, T, f32, m![1], m![B], m![A / 2], m![1], m!
[A % 2 # 8], Fresh, { stage::VeOrder::IntraFirst }>,
) -> VectorNarrowTensor<'l, T, f32, m![1], m![B], m![A / 2], m![1], m![A % 2
# 4], Fresh, { stage::VeOrder::IntraFirst }>
{
  input.vector_narrow_trim::<m![A % 2 # 4]>()
  // shape semantics: [T], [P] -> [T], [P = 4]
}
```

Float Cluster

The Float Cluster provides unary, binary, and ternary floating-point operations on `f32`. It runs 4-way, so the input must already have passed through `Narrow`.

It exposes five independent ALUs (`FpFma`, `FpFpu`, `FpExp`, `FpMul0`, `FpMul1`), each

runnable once per Tensor Unit invocation. This stage is where ALU planning matters most.

Unary ops:

Op	ALU	Note
Exp	FpExp	exponential
NegExp	FpExp	negative exponential
Sqrt	FpFpu	square root
Tanh	FpFpu	hyperbolic tangent
Sigmoid	FpFpu	sigmoid
Erf	FpFpu	error function
Log	FpFpu	natural logarithm
Sin	FpFpu	sine
Cos	FpFpu	cosine

Binary ops:

Op	ALU	Note
AddF	FpFma	floating-point add
SubF	FpFma	floating-point subtract
MulF(FpMulAlu::Mul0)	FpMul0	multiply
MulF(FpMulAlu::Mul1)	FpMul1	multiply
MulF(FpMulAlu::Fma)	FpFma	multiply
DivF	FpFpu	division inside Fp stage

Ternary ops:

Op	ALU	Note
FmaF	FpFma	fused multiply-add

For example, to compute $\exp(\sqrt{((x + 1) * 2) * 3})$:

- $x1 = x + 1$ via `FpFma ( FpBinaryOp::AddF )`
- $x2 = x1 * 2$ via `FpMul0 ( FpBinaryOp::MulF(FpMulAlu::Mul0) )`
- $x3 = x2 * 3$ via `FpMul1 ( FpBinaryOp::MulF(FpMulAlu::Mul1) )`
- $x4 = \sqrt{x3}$ via `FpFpu ( FpUnaryOp::Sqrt )`

- `x5 = exp(x4)` via `FpExp ( FpUnaryOp::Exp )`

IntraSliceReduce

The `IntraSliceReduce` stage reduces axes within a single slice. It runs 4-way. The stage uses a dedicated accumulator-tree ALU, so no user-selectable ALU is exposed.

Data Type	Supported Ops
i32	AddSat , Max , Min
f32	Add , Max , Min

See [Intra-Slice Reduce](#) for details.

FpDiv

The `FpDiv` stage performs floating-point division. It runs 4-way. The stage uses a dedicated floating-point divider, so no user-selectable ALU is exposed.

Op	Note
<code>FpDivBinaryOp::DivF</code>	dedicated floating-point division

Widen

The `Widen` stage transitions from 4-way back to 8-way. Later stages (`FpToFxp` , `Clip` , `Filter` , `Output`) then see 8-element packets again.

Method	Use When	Effect
<code>vector_widen_concat()</code>	reversing a prior <code>vector_narrow_split()</code>	merge two 4-way packets back into one 8-way flit
<code>vector_widen_pad()</code>	reversing a prior <code>vector_narrow_trim()</code>	pad a 4-way packet back to 8 elements with invalid fillers

Shape semantics:


```
axes![A = 512, B = 2, S = 64, R = 8];
```

```
fn vector_widen_concat_semantics<'l, const T: Tu>(
    input: VectorIntraSliceReduceTensor<'l, T, i32, m![1], m![B], m![S / 4 #
256], m![A / 4 % 2], m![A % 4], Fresh, { stage::VeOrder::IntraFirst }>,
) -> VectorWidenTensor<'l, T, i32, m![1], m![B], m![S / 4 # 256], m![1], m![A
% 8], Fresh, { stage::VeOrder::IntraFirst }>
{
    input.vector_widen_concat::<m![1], m![A % 8]>()
    // shape semantics: [T, P / 2], [P % 4] -> [T], [P]
}

fn vector_widen_pad_semantics<'l, const T: Tu>(
    input: VectorFpTensor<'l, T, f32, m![1], m![B], m![A / 2], m![1], m![A %
2 # 4], Fresh, { stage::VeOrder::IntraFirst }>,
) -> VectorWidenTensor<'l, T, f32, m![1], m![B], m![A / 2], m![1], m![A % 2 #
8], Fresh, { stage::VeOrder::IntraFirst }>
{
    input.vector_widen_pad::<m![A % 2 # 8]>()
    // shape semantics: [T], [P] -> [T], [P # 8]
}
```

FpToFxp Conversion

The FpToFxp Conversion stage converts `f32` back to `i32`. The `int_width` parameter specifies the integer bit width.

Method	Effect
<code>vector_fp_to_fxp(int_width)</code>	convert <code>f32</code> stream back to <code>i32</code>

Clip Cluster

The Clip Cluster performs clamping and comparison operations. It runs 8-way.

The stage exposes three ALU classes (`ClipAdd`, `ClipMax`, `ClipMin`), each runnable once per Tensor Unit invocation.

`i32` operations:

Op	ALU	Note
Min	ClipMin	minimum

Max	ClipMax	maximum
AbsMin	ClipMin	absolute minimum
AbsMax	ClipMax	absolute maximum
AddFxp	ClipAdd	wrapping add
AddFxpSat	ClipAdd	saturating add

f32 operations:

Op	ALU	Note
Min	ClipMin	minimum
Max	ClipMax	maximum
AbsMin	ClipMin	absolute minimum
AbsMax	ClipMax	absolute maximum
Add	ClipAdd	floating-point add

Filter

The Filter stage applies an execution mask derived from a `TagGuard` (typically matching on the `GroupId` MSB of each element's `Tag`) to filter output flits. Available 8-way and `Standalone` context only. The source `impl` lives on `VectorTensor` for any stage with `CanTransitionTo<Filter>`, which covers every intra-slice stage and `InterSliceReduce`.

Output

The Output stage exits the Vector Engine pipeline. The result can continue to the [Cast Engine](#), [Transpose Engine](#), or [Commit Engine](#).

Reinterpret

`vector_reinterpret::<D2>()` rereads the stream as the other 32-bit scalar with the bits untouched, so `1.0f32` becomes `0x3f80_0000` and not `1`. Every element is 32 bits and every cluster takes its operand type from the op it runs, so a reinterpret emits no instruction, claims no ALU, and holds both the current stage and the way. It can therefore

appear anywhere in the chain, any number of times, including between two ops of the same cluster.

One possible use for a reinterpret is applying a bitwise operation to floating-point values, as in [Bitwise abs on f32](#). For the value-preserving conversions use [FxpToFp](#) / [FpToFxp](#) instead.

Method	Effect
<code>vector_reinterpret::<i>i32</i>()</code>	read the <code>f32</code> stream's bits as <code>i32</code>
<code>vector_reinterpret::<i>f32</i>()</code>	read the <code>i32</code> stream's bits as <code>f32</code>

A reinterpret also decides what a following `vector_stash()` writes, since the stash takes the scalar the stream carries at the write. So a value reaches a reader of the other scalar by reinterpreting first and stashing after; a read at the other scalar does not compile.

Examples

`i32` Pipeline

A minimal `i32` chain that adds a constant after branching. The `Fxp` stage runs 8-way, so no narrow or widen is needed.

```
axes![A = 2048, B = 2];

fn add_constant<'l, const T: Tu>(
    input: CollectTensor<'l, T, i32, m![1], m![B], m![A / 8], m![1], m![A %
8]>,
) -> VectorFinalTensor<'l, T, i32, m![1], m![B], m![A / 8], m![1], m![A % 8]>
{
    input
        .vector_init()
        .vector_intra_slice_tag(TagMode::Zero)
        .vector_fxp(FxpBinaryOp::AddFxp, 100)
        .vector_final()
}
```

`f32` Pipeline

`vector_narrow_trim()` is the **Narrow** step that converts the tensor from 8-way to 4-way before the float operation. `vector_widen_pad()` is the **Widen** step that converts back to 8-way afterward.

```
axes![A = 512, B = 2];
```

```
fn sigmoid<'l, const T: Tu>(
  input: CollectTensor<'l, T, f32, m![1], m![B], m![A / 2], m![1], m![A % 2
# 8]>,
) -> VectorFinalTensor<'l, T, f32, m![1], m![B], m![A / 2], m![1], m![A % 2 #
8]> {
  input
    .vector_init()
    .vector_intra_slice_tag(TagMode::Zero)
    .vector_narrow_trim::<m![A % 2 # 4]>() // Narrow: Way8 -> Way4
    .vector_fp_unary(FpUnaryOp::Sigmoid)
    .vector_widen_pad::<m![A % 2 # 8]>() // Widen: Way4 -> Way8
    .vector_final()
}
```

Bitwise Abs on f32

Clearing the sign bit is $|x|$, reached with no float ALU: the mask is an ordinary `i32` literal because the stream is read as `i32` for the one op that needs it. Both reinterprets are free, so this costs exactly one `LogicAnd`.

```
axes![A = 512, B = 2];
```

```
fn abs<'l, const T: Tu>(
  input: CollectTensor<'l, T, f32, m![1], m![B], m![A / 2], m![1], m![A % 2
# 8]>,
) -> VectorFinalTensor<'l, T, f32, m![1], m![B], m![A / 2], m![1], m![A % 2 #
8]> {
  input
    .vector_init()
    .vector_intra_slice_tag(TagMode::Zero)
    .vector_reinterpret::<i32>() // read the bits as i32
    .vector_logic(LogicBinaryOpI32::BitAnd, 0x7fff_ffff) // clear the
sign bit
    .vector_reinterpret::<f32>() // and back
    .vector_final()
}
```

Single-Stream Argument Mode

`BinaryArgMode::Mode10` swaps the stream and operand positions, so `SubFxp` computes `operand - stream` (here, `7 - x`) rather than the default `stream - operand`.

```
axes![A = 2048, B = 2];

fn bias_minus_x<'l, const T: Tu>(
    input: CollectTensor<'l, T, i32, m![1], m![B], m![A / 8], m![1], m![A %
8]>,
) -> VectorFinalTensor<'l, T, i32, m![1], m![B], m![A / 8], m![1], m![A % 8]>
{
    input
        .vector_init()
        .vector_intra_slice_tag(TagMode::Zero)
        .vector_fxp_with_mode(FxpBinaryOp::SubFxp, BinaryArgMode::Mode10, 7)
// compute 7 - x
        .vector_final()
}
```

VRF Operand

Pre-loaded VRF data as an operand:

```
axes![A = 2048, B = 2, N = 256];

fn vrf_add<'l, const T: Tu>(
    input: CollectTensor<'l, T, i32, m![1], m![B], m![A / 8], m![N], m![A %
8]>,
    vrf: &VrfTensor<i32, m![1], m![B], m![A / 8], m![A % 8]>,
) -> VectorFinalTensor<'l, T, i32, m![1], m![B], m![A / 8], m![N], m![A % 8]>
{
    input
        .vector_init()
        .vector_intra_slice_tag(TagMode::Zero)
        .vector_fxp(FxpBinaryOp::AddFxp, vrf)
        .vector_final()
}
```

Stash on the Fp-Only Path

Stash at an early stage, then use it later in a Clip operation. This implements `max(2 * x,`

```

x) :

axes![A = 512, B = 2];

fn residual_max<'l, const T: Tu>(
    input: CollectTensor<'l, T, f32, m![1], m![B], m![A / 2], m![1], m![A % 2
# 8]>,
) -> VectorFinalTensor<'l, T, f32, m![1], m![B], m![A / 2], m![1], m![A % 2 #
8]> {
    input
        .vector_init() // enter VE
        .vector_intra_slice_tag(TagMode::Zero) // start the intra-slice path
        .vector_stash() // save
original x
        .vector_narrow_trim::<m![A % 2 # 4]>() // narrow to
Way4
        .vector_fp_binary(FpBinaryOp::MulF(FpMulAlu::Mul0), 2.0f32) //
compute 2 * x
        .vector_widen_pad::<m![A % 2 # 8]>() // widen back
to Way8
        .vector_clip(ClipBinaryOpF32::Max, Stash) // max(2 * x,
x)
        .vector_final()
}

```

Stash on the Fxp-Only Path

Stash at an early stage, then use it later in a Clip operation. This implements `max(x + bias, x)`:

```

axes![A = 2048, B = 2];

fn stash_at_fxp<'l, const T: Tu>(
    input: CollectTensor<'l, T, i32, m![1], m![B], m![A / 8], m![1], m![A %
8]>,
) -> VectorFinalTensor<'l, T, i32, m![1], m![B], m![A / 8], m![1], m![A % 8]>
{
    input
        .vector_init() // enter VE
        .vector_intra_slice_tag(TagMode::Zero) // start the intra-slice path
        .vector_stash() // save original
x
        .vector_fxp(FxpBinaryOp::AddFxp, 100) // compute x +
bias
        .vector_clip(ClipBinaryOpI32::Max, Stash) // compute max(x
+ bias, x)
        .vector_final()
}

```

Stash Across Narrow and Widen

Stash before narrowing, consume after widening. This computes $\max(\text{sigmoid}(x), x)$:

```

axes![A = 512, B = 2];

fn stash_across_narrow_widen<'l, const T: Tu>(
    input: CollectTensor<'l, T, f32, m![1], m![B], m![A / 2], m![1], m![A % 2
# 8]>,
) -> VectorFinalTensor<'l, T, f32, m![1], m![B], m![A / 2], m![1], m![A % 2 #
8]> {
    input
        .vector_init() // enter VE
        .vector_intra_slice_tag(TagMode::Zero) // start the intra-slice path
        .vector_stash() // save x (Way8)
        .vector_narrow_trim::<m![A % 2 # 4]>() // narrow to
Way4
        .vector_fp_unary(FpUnaryOp::Sigmoid) // compute
sigmoid(x) in Way4
        .vector_widen_pad::<m![A % 2 # 8]>() // widen back to
Way8
        .vector_clip(ClipBinaryOpF32::Max, Stash) // compute
max(sigmoid(x), x)
        .vector_final()
}

```

Pair Add

Zip two interleaved groups with integer add:

```
axes![A = 2048, B = 2, I = 2];

fn pair_add<'l, const T: Tu>(
    input: CollectTensor<'l, T, i32, m![1], m![B], m![A / 8], m![I], m![A %
8]>,
) -> VectorFinalTensor<'l, T, i32, m![1], m![B], m![A / 8], m![1], m![A % 8]>
{
    input
        .vector_init()
        .vector_intra_slice_unzip::<I, m![1 # 2], m![1]>()
        .vector_clip_zip(ClipBinaryOpI32::AddFxp)
        .vector_final()
}
```

Pair Per-Side Preprocessing

Asymmetric preprocessing scales only group 0 before zip:

```
axes![A = 2048, B = 2, I = 2];

fn pair_preprocess_one_side<'l, const T: Tu>(
    input: CollectTensor<'l, T, i32, m![1], m![B], m![A / 8], m![I], m![A %
8]>,
) -> VectorFinalTensor<'l, T, i32, m![1], m![B], m![A / 8], m![1], m![A % 8]>
{
    input
        .vector_init()
        .vector_intra_slice_unzip::<I, m![1 # 2], m![1]>()
        .vector_fxp(FxpBinaryOp::MulInt, 10, ()) // group 0 only
        .vector_clip_zip(ClipBinaryOpI32::AddFxp)
        .vector_final()
}
```

Pair Float Pipeline with Zip

Both groups traverse the float path (narrow -> fp -> zip -> widen):


```

axes![A = 512, B = 2, I = 2];

fn pair_fp_mul_zip<'l, const T: Tu>(
    input: CollectTensor<'l, T, f32, m![1], m![B], m![A / 2], m![I], m![A % 2
# 8]>,
) -> VectorFinalTensor<'l, T, f32, m![1], m![B], m![A / 2], m![1], m![A % 2 #
8]> {
    input
        .vector_init()
        .vector_intra_slice_unzip::<I, m![1 # 2], m![1]>()
        .vector_narrow_split::<m![1 # 2], m![A % 2 # 4]>()          // both
groups: Way8 -> Way4
        .vector_fp_zip(FpBinaryOp::MulF(FpMulAlu::Mul0))          // group0 * group1
(Way4)
        .vector_widen_concat::<m![1], m![A % 2 # 8]>()              // Way4 ->
Way8
        .vector_final()
}

```

Pair Per-Group Preprocessing

Apply a different operation to each group before zipping:

```

axes![A = 512, B = 2, I = 2];

fn pair_asymmetric_preprocess<'l, const T: Tu>(
    input: CollectTensor<'l, T, f32, m![1], m![B], m![A / 2], m![I], m![A % 2
# 8]>,
) -> VectorFinalTensor<'l, T, f32, m![1], m![B], m![A / 2], m![1], m![A % 2 #
8]> {
    input
        .vector_init()
        .vector_intra_slice_unzip::<I, m![1 # 2], m![1]>()
        .vector_narrow_split::<m![1 # 2], m![A % 2 # 4]>()
        .vector_fp_unary(FpUnaryOp::Exp, true, false)              // group 0:
exp(x), group 1: skip
        .vector_fp_zip(FpBinaryOp::MulF(FpMulAlu::Mul0))          // exp(group0) *
group1
        .vector_widen_concat::<m![1], m![A % 2 # 8]>()
        .vector_final()
}

```

Pair Zip Argument Mode

`BinaryArgMode::Mode10` swaps the two grouped streams when zipping:

```
axes![A = 512, B = 2, I = 2];

fn pair_sub_reverse<'l, const T: Tu>(
    input: CollectTensor<'l, T, f32, m![1], m![B], m![A / 2], m![I], m![A % 2
# 8]>,
) -> VectorFinalTensor<'l, T, f32, m![1], m![B], m![A / 2], m![1], m![A % 2 #
8]> {
    input
        .vector_init()
        .vector_intra_slice_unzip::<I, m![1 # 2], m![1]>()
        .vector_narrow_split::<m![1 # 2], m![A % 2 # 4]>()
        .vector_fp_zip_with_mode(FpBinaryOp::SubF, BinaryArgMode::Mode10) //
compute group1 - group0
        .vector_widen_concat::<m![1], m![A % 2 # 8]>()
        .vector_final()
}
```

Performance

Throughput is full 8-way (8 elements per cycle) on the Logic, Fxp, and Clip clusters. The Float cluster runs at 4-way, and the Narrow/Widen wrapping around the float path halves effective throughput in practice.

Latency adds one cycle per ALU used. Operations spanning multiple ALUs accumulate their latencies. For example, `exp(sqrt(x))` adds 2 cycles (FpFpu for sqrt plus FpExp for exp).

Intra-Slice Reduce

The `IntraSliceReduce` stage in the [Intra-Slice Chain](#) reduces dimensions that live in the `Time` and `Packet` of each slice (`Chip`, `Cluster`, and `Slice` pass through unchanged). The [Inter-Slice Reducer](#) covers the complementary case of reducing across the 256 slices of a cluster.

Examples

The reduce call's key parameters are below.

- **REDUCE_LABEL**: The axis to reduce. Reduction eliminates every factor in `Time` and `Packet` carrying this axis, so they must not appear in the output shape (`OutTime`, `OutPacket`). For example, if `R` is split as `R / 4` in `Time` and `R % 4` in `Packet`, specifying `REDUCE_LABEL = R` eliminates both.
- **op**: The reduce operation. `IntraSliceReduceOpI32` provides `AddSat`, `Max`, `Min`; `IntraSliceReduceOpF32` provides `Add`, `Max`, `Min`.
- **OutTime**, **OutPacket**: The output `Time` and `Packet` shape after reduction. These match the input `Time` and `Packet` with every `REDUCE_LABEL` factor removed.

The call is unavailable in [Pair Mode](#). The examples below exercise each parameter combination.

Reduction in Time

`R` exists only in `Time`, so the stage accumulates across time steps. It computes $output[a] = \sum_{r \in R} input[a, r]$ with saturating addition.

```

axes![A = 512, R = 16];

// R in Time → temporal accumulation. Packet is non-reduce.
fn reduce_time<'l, const T: Tu>(
    input: VectorBranchTensor<'l, T, i32, m![1], m![1 # 2], m![A / 2], m![R],
    m![A % 2 # 8], Fresh, { stage::VeOrder::IntraFirst }>,
) -> VectorIntraSliceReduceTensor<'l, T, i32, m![1], m![1 # 2], m![A / 2], m!
[1], m![A % 2 # 4], Fresh, { stage::VeOrder::IntraFirst }>
{
    input
        .vector_narrow_trim::<m![A % 2 # 4]>() // 8-way → 4-way
        // R eliminated from Time
        .vector_intra_slice_reduce::<R, m![1], m![A % 2 # 4]>(
            IntraSliceReduceOpI32::AddSat,
        )
}

```

Reduction in Packet

R exists only in `Packet`, so the hardware runs a 4-way tree reduce within each flit and skips temporal accumulation. It computes $output[a] = \sum_{r \in R} input[a, r]$.

```

axes![A = 512, R = 4];

// R in Packet → tree reduce within flit.
fn reduce_packet<'l, const T: Tu>(
    input: VectorBranchTensor<'l, T, f32, m![1], m![1 # 2], m![A / 2], m![A %
2], m![R # 8], Fresh, { stage::VeOrder::IntraFirst }>,
) -> VectorIntraSliceReduceTensor<'l, T, f32, m![1], m![1 # 2], m![A / 2], m!
[A % 2], m![1 # 4], Fresh, { stage::VeOrder::IntraFirst }>
{
    input
        .vector_narrow_trim::<m![R]>() // 8-way → 4-way
        // R eliminated from Packet
        .vector_intra_slice_reduce::<R, m![A % 2], m![1 # 4]>(
            IntraSliceReduceOpF32::Add,
        )
}

```

Reduction in Both

R splits across `Packet` and `Time`. $R \% 4$ in `Packet` tree-reduces within each flit, and $R / 4$ in `Time` accumulates across time steps. It computes

$$\text{output}[a] = \max_{r \in R} \text{input}[a, r].$$

```
axes![A = 256, R = 16];

// R % 4 in Packet → spatial tree reduce
// R / 4 in Time → temporal accumulation
fn reduce_time_packet<'l, const T: Tu>(
    input: VectorBranchTensor<'l, T, f32, m![1], m![1 # 2], m![A], m![R / 4],
    m![R % 4 # 8], Fresh, { stage::VeOrder::IntraFirst }>,
) -> VectorIntraSliceReduceTensor<'l, T, f32, m![1], m![1 # 2], m![A], m![1],
m![1 # 4], Fresh, { stage::VeOrder::IntraFirst }>
{
    input
        .vector_narrow_trim::<m![R % 4]>() // 8-way → 4-way
        // R eliminated from both Time and Packet
        .vector_intra_slice_reduce::<R, m![1], m![1 # 4]>(
            IntraSliceReduceOpF32::Max,
        )
}
```

Per-Slice Reduction

R has portions in `slice`, `time`, and `packet`, but the intra-slice reducer only collapses the `time` and `packet` portions, so the `slice` portion of R stays in the output. Here `R = 13` is padded to 32 to fit the layout (`R # 32`), and R is then split across 4 slices (8 R-positions per slice). Only positions 0-12 hold real elements, so the slice straddling that boundary (positions 8-15: 5 real followed by 3 pad) is a boundary slice, and slices past it are fully padding. The [VCG](#) drives the per-slice reduction count so each slice reduces only its real elements (see [Valid Count Generator](#) for the exact mapping).

```

axes![R = 13];

// R split across all three: Slice (groups of 8), Time (pairs within group),
// Packet (4 elements).
fn reduce_slice_time_packet<'l, const T: Tu>(
    input: VectorBranchTensor<'l, T, i32, m![1], m![1 # 2], m![R # 32 / 8 #
256], m![R # 32 / 4 % 2], m![R # 32 % 4 # 8], Fresh, {
stage::VeOrder::IntraFirst }>,
) -> VectorIntraSliceReduceTensor<'l, T, i32, m![1], m![1 # 2], m![R # 32 / 8
# 256], m![1], m![1 # 4], Fresh, { stage::VeOrder::IntraFirst }>
{
    input
        .vector_narrow_trim::<m![R # 32 % 4]>()          // 8-way → 4-way
        // R eliminated from Time and Packet (accumulated within each slice)
        .vector_intra_slice_reduce::<R, m![1], m![1 # 4]>(
            IntraSliceReduceOpI32::Min,
        )
}

```

Architecture

The stage runs separate machinery for the `Time` and `Packet` axes.

Reduction in Time

The stage applies the [temporal accumulator](#) model with slot capacity 8 (so `InnerTime::SIZE ≤ 8`).

For `reduce_time` above, `Time = m![R]` and `OutTime = m![1]`, so `R` is the outermost reduce dimension and `InnerTime = m![1]` (`InnerTime::SIZE = 1`). A single slot accumulates all `R` values into the output.

If `InnerTime::SIZE` exceeds 8, the API rejects the call. For example:

```

fn invalid_too_many_slots<'l, const T: Tu>(
    input: VectorBranchTensor<'l, T, i32, m![1], m![1 # 2], m![A / 3 # 256],
    m![R, A % 3, B % 4], m![B / 4 # 8], Fresh, { stage::VeOrder::IntraFirst }>,
) -> VectorIntraSliceReduceTensor<'l, T, i32, m![1], m![1 # 2], m![A / 3 #
256], m![A % 3, B % 4], m![B / 4], Fresh, { stage::VeOrder::IntraFirst }>
{
    input
        .vector_narrow_trim::<m![B / 4]>()
        // Time      = m![R, A % 3, B % 4]
        // OutTime   = m![A % 3, B % 4]
        // InnerTime = m![A % 3, B % 4], InnerTime::SIZE = 3 × 4 = 12 > 8
        .vector_intra_slice_reduce::<R, m![A % 3, B % 4], m![B / 4]>(
            IntraSliceReduceOpI32::AddSat,
        )
    // Rejected: 12 accumulator slots required, but only 8 are available.
}

```

Reduction in Packet

The 4 Packet elements per flit pick one of two paths:

- `OutPacket = Packet`: elements pass through unreduced. 4 outputs per cycle, each accumulated independently over `Time`.
- `OutPacket = m![1 # 4]`: elements collapse to a single value via a 2-level tree `op(op(a, b), op(c, d))`. 1 output per cycle plus 3 padding positions.

When a reduce axis is padded to fit hardware dimensions, the padded positions contain arbitrary data that the reduction must exclude. Two strategies handle padding exclusion.

- **VCG (Valid Count Generator)**: Preferred when its axis placement is supported. The compiler configures the VCG automatically from the mapping, and the VCG tags each flit with a `valid_count` so pad elements are excluded automatically. Not all axis placements across `Slice`, `Time`, and `Packet` are supported. See [Valid Count Generator](#) for details.
- **Identity-element padding**: Fill pad positions with the identity element of the reduce operation before data reaches the [Intra-Slice Chain](#). The book does not document a usable public masking operation; use a verified identity-producing path before reduction.

Operation	Identity Element
AddSat / Add	0 / 0.0

Max	<code>i32::MIN / f32::NEG_INFINITY</code>
Min	<code>i32::MAX / f32::INFINITY</code>

This strategy applies only when no non-invertible transformation precedes the reduce operation. For example, with $\exp(x) + \exp(y) + \dots$ (sum of exponentials), no value p satisfies $\exp(p) = 0$ (the additive identity). Identity padding does not apply.

Performance

Throughput stays at one flit per cycle since the tree reduce is fully pipelined within the Intra-Slice Chain, adding no extra per-flit cost.

Latency adds a first-output delay of n flit cycles where n is the number of time steps in the reduce axis, because the stage must accumulate all input flits for a reduction group before emitting the result. In a multi-engine pipeline, this accumulation delay stalls downstream engines waiting for the first flit.

Valid Count Generator

The [Intra-Slice Reduce](#) stage reduces the axis identified by `REDUCE_LABEL` (e.g., `R`) across its `Time` and `Packet` factors, leaving any `slice` factor in the output. That axis often needs padding to fit hardware dimensions, and the extra padded positions contain arbitrary data that the reduction must exclude.

The Valid Count Generator (VCG) solves this. The user places `R` as sub-expressions across `slice`, `Time`, and/or `Packet` in the mapping. The compiler then configures the VCG to tag each 8-element flit with a `valid_size` count of how many elements are real data. Each sub-expression in `Time` or `slice` maps to a sequencer counter assigned to a time filter. Each sub-expression in `Packet` drives the packet clipper.

Throughout this page, capitalized `slice`, `Time`, `Packet` refer to mapping dimensions; lowercase `slice` and `time step` refer to runtime instances.

The mapping must express `R` in a specific form for the VCG to work. `R` is padded to a hardware-aligned size, written `R # PADDED_SIZE` when discussed in general. Concrete examples use the actual padded value (e.g., `R # 16`, `R # 48`). Each sub-expression is then a factor of `R # PADDED_SIZE` of the form `R # PADDED_SIZE / n % m` (stride `n`, modulo `m`). See [Stride and Modulo](#) for `/ n` and `% m` semantics. Each sub-expression is assigned to one hardware dimension. One possible distribution is `R = 43` padded to `R # 48`, split across all three dimensions:

```

Slice:  R # 48 / 8      (stride 8, 6 positions)
Time:   R # 48 / 2 % 4  (stride 2, 4 positions)
Packet: R # 48 % 2      (stride 1, 2 positions)

```

Architecture

The VCG assigns a `valid_size(s, t) ∈ {0, 1, ..., 8}` to each flit, where `s` is the slice id (an integer encoding the flit's position across all `slice` sub-expressions) and `t` is the time step. The first `valid_size` elements of the flit are real data, the rest are padding.

```

struct VcgConfig {
    time_filters: [TimeFilterConfig; 3], // for R's sub-expressions in Time
and/or Slice
    packet_clipper: PacketClipperConfig, // for R's sub-expressions in
Packet
}

impl VcgConfig {
    fn valid_size(&self, s: u64, t: u64) -> u32 {
        if self.time_filters.iter().all(|tf| tf.valid(s, t)) {
            self.packet_clipper.valid_size(t)
        } else {
            0
        }
    }
}

```

When all timers report valid for flit (s, t) , the packet clipper decides how many elements in that flit are real data. When any timer reports invalid, all elements in the flit are excluded regardless of what the packet clipper would say. The two components, `TimeFilterConfig` and `PacketClipperConfig`, are explained in the sections below.

Time Filter

For each slice s , a time filter determines whether each time step t carries valid R data. The subsections below build up `fn valid()` step by step, starting from the simplest case and adding complexity. When R has no sub-expressions in `Time` or `Slice`, the time filter is disabled by setting `slice_mask = 0` and `slice_thres = 1`. Then $s \& 0 = 0 < 1$ for every s , hitting the `Less` arm so `fn valid()` always returns `true`. The choice of `slice_thres = 1` is conventional; any positive value works since 0 is always less than it.

```

struct TimeFilterConfig {
    // How R's index is reconstructed from t.
    sequencer: Sequencer,

    // Slice classification (see `R in Slice and Time`, `R in Time and
    Slice`).
    slice_mask: u32,
    slice_thres: u32,
    time_thres: u32,
    mode:      TimeFilterMode, // SliceMajor | TimeMajor
}

impl TimeFilterConfig {
    /// Returns true if flit (s, t) carries valid R data.
    fn valid(&self, s: u64, t: u64) -> bool {
        let idx = self.sequencer.index(t);
        match ((s & self.slice_mask).cmp(&self.slice_thres), self.mode) {
            (Less, _) => true,
            (Greater, SliceMajor) => false,
            _ => idx < self.time_thres as u64,
        }
    }
}

```

R as Time

In the simplest case, `R` occupies all of `Time` with no other axes. Each time step `t` corresponds directly to one `R` index, and is valid when `t < R::SIZE`.

```

// The compiler emits roughly:
TimeFilterConfig {
    sequencer: [R # PADDED_SIZE -> size PADDED_SIZE : stride 1], // idx =
t
    slice_mask: 0, // no slice partitioning
    slice_thres: 0, // 0 cmp 0 = Equal -> falls to `idx <
time_thres`
    time_thres: R::SIZE, // valid when idx < R::SIZE
    mode:      SliceMajor, // arbitrary; only the Equal arm is hit
}

```

```
axes![A = 8, R = 12, X = 128];
```

```
fn reduce_time_only<'l, const T: Tu>(
    input: VectorBranchTensor<'l, T, i32, m![1], m![1 # 2], m![X, A / 4], m!
[R # 16], m![A % 4 # 8], Fresh, { stage::VeOrder::IntraFirst }>,
) -> VectorIntraSliceReduceTensor<'l, T, i32, m![1], m![1 # 2], m![X, A / 4],
m![1], m![A % 4], Fresh, { stage::VeOrder::IntraFirst }>
{
    input
    .vector_narrow_trim::<m![A % 4]>()
    // Slice      = m![X, A / 4]
    // Time       = m![R # 16]      (steps < R::SIZE valid)
    // Packet     = m![A % 4]
    // OutTime    = m![1]           (R eliminated from Time)
    // OutPacket  = m![A % 4]
    .vector_intra_slice_reduce::<R, m![1], m![A % 4]>(
        IntraSliceReduceOpI32::AddSat,
    )
}
```

In the example above, the sequencer iterates $R \# 16$ once with `size 16 : stride 1`, so $idx = t$ for every time step. With `time_thres = R::SIZE = 12`, the first 12 time steps ($t = 0..11$) are valid and the remaining 4 ($t = 12..15$) are filtered out. The intra-slice reduce then folds exactly the 12 real R elements per slice.

R in Time

The VCG supports `Time` mappings where R shares space with other axes and appears as multiple sub-expressions, in any order. The time filter uses a [Sequencer](#) to decompose t into per-sub-expression counters; summing `value × stride` over the R -assigned counters gives idx , which encodes R 's index for that time step.

The following example uses $R = 10$ padded to $R \# 12$, where A sits between R 's two sub-expressions in `Time`.

```

axes![A = 3, B = 4, R = 10, X = 64];

// R = 10, padded to R # 12, split as (size 3, stride 4) × (size 4, stride
1).
// time filter sums (R # 12 / 4 value) * 4 + (R # 12 % 4 value) * 1 to
recover R index regardless of A.
fn reduce_time_reordered<'l, const T: Tu>(
    input: VectorBranchTensor<'l, T, i32, m![1], m![1 # 2], m![X # 256], m![R
# 12 / 4, A, R # 12 % 4], m![B # 8], Fresh, { stage::VeOrder::IntraFirst }>,
) -> VectorIntraSliceReduceTensor<'l, T, i32, m![1], m![1 # 2], m![X # 256],
m![A], m![B], Fresh, { stage::VeOrder::IntraFirst }>
{
    input
        .vector_narrow_trim::<m![B]>()
        // Slice      = m![X # 256]
        // Time       = m![R # 12 / 4, A, R # 12 % 4]
        // Packet     = m![B]
        // OutTime    = m![A]      (R eliminated; A survives)
        // OutPacket  = m![B]
        .vector_intra_slice_reduce::<R, m![A], m![B]>(
            IntraSliceReduceOpI32::AddSat,
        )
}

```

The compiler configures the time filter for this placement as follows:

```

TimeFilterConfig {
    sequencer: [R # 12 / 4 -> size 3 : stride 4, // assigned to time
filter
                A      -> size 3 : stride 0, // not assigned (A's
OutTime)
                R # 12 % 4 -> size 4 : stride 1], // assigned to time
filter
    slice_mask: 0, // no slice partitioning (R is only in Time)
    slice_thres: 0, // every slice falls to the `idx < time_thres`
arm
    time_thres: R::SIZE, // 10
    mode: SliceMajor, // arbitrary; only the Equal arm is hit
}

```

The `A` entry has stride 0, so it never contributes to `idx`, which means `A`'s position in `Time` has no effect on validity. The remaining two entries reconstruct `R` as $(R \# 12 / 4 \text{ value}) \times 4 + (R \# 12 \% 4 \text{ value})$.

For example:

- At $t = 13$, the sequencer state is $(R \# 12 / 4, A, R \# 12 \% 4) = (1, 0, 1)$,

giving $\text{idx} = 1 \times 4 + 0 + 1 \times 1 = 5$. $\text{idx} < 10$, so valid.

- At $t = 27$, the sequencer state is $(2, 0, 3)$, giving $\text{idx} = 2 \times 4 + 0 + 3 \times 1 = 11$. $\text{idx} \geq 10$, so invalid (it is one of the two padded R positions).

R in Slice and Time

`sliceMajor` mode allows multiple R sub-expressions in both `slice` and `Time`, with `slice` sub-expressions more major (larger stride) than `Time` sub-expressions. Within `slice`, sub-expressions must appear in descending stride order (major before minor), and each must have a power-of-2 size and a power-of-2 stride so that its bits occupy a contiguous run of `slice_mask`. Within `Time`, sub-expressions may appear in any order.

```
axes![R = 11, X = 64];
```

```
fn reduce_slice_time_slicemajor<'l, const T: Tu>(
    input: VectorBranchTensor<'l, T, i32, m![1], m![1 # 2], m![R # 16 / 8, X,
R # 16 / 4 % 2], m![R # 16 % 2, R # 16 / 2 % 2], m![1 # 8], Fresh, {
stage::VeOrder::IntraFirst }>,
) -> VectorIntraSliceReduceTensor<'l, T, i32, m![1], m![1 # 2], m![R # 16 /
8, X, R # 16 / 4 % 2], m![1], m![1 # 4], Fresh, { stage::VeOrder::IntraFirst
}>
{
    input
        .vector_narrow_trim::<m![1 # 4]>()
        // Slice      = m![R # 16 / 8, X, R # 16 / 4 % 2]    (major R sub-
exprs, descending R-stride order required)
        // Time      = m![R # 16 % 2, R # 16 / 2 % 2]      (minor R sub-
exprs, any order OK)
        // Packet    = m![1 # 4]
        // OutTime    = m![1]                                (R eliminated)
        // OutPacket  = m![1 # 4]
        .vector_intra_slice_reduce::<R, m![1], m![1 # 4]>(
            IntraSliceReduceOpI32::Min,
        )
}
```

The example places $R = 11$ (padded to $R \# 16$) across `slice` and `Time` with the following sub-expressions.

Dimension	Sub-expression	Stride
slice	$R \# 16 / 8$	8
slice	$R \# 16 / 4 \% 2$	4
Time	$R \# 16 \% 2$	1

Time	R # 16 / 2 % 2	2
------	----------------	---

The example divides into 4 slice groups (one per R contribution from slice: 0, 4, 8, or 12) and 4 time iterations per slice. The layout has three regimes: 2 slices are fully valid, 1 slice is partial, and 1 slice is fully invalid.

R contribution from Slice	R values across iterations	Valid time steps
0	0, 2, 1, 3	4 (all < R::SIZE)
4	4, 6, 5, 7	4 (all < R::SIZE)
8	8, 10, 9, 11	3 (R = 11 invalid)
12	12, 14, 13, 15	0 (all ≥ R::SIZE)

The compiler emits the time filter config below.

```
TimeFilterConfig {
  sequencer: [R # 16 % 2    -> size 2 : stride 1,
              R # 16 / 2 % 2 -> size 2 : stride 2],
  slice_mask: 0b1000001, // bits 0 (R # 16 / 4 % 2) and 6 (R # 16 / 8)
  carry the R contribution
  slice_thres: 64,        // masked id encoding the boundary R
  contribution (= 8)
  time_thres: 3,          // = R::SIZE - boundary = 11 - 8
  mode:      SliceMajor,
}
```

Each field encodes one part of the validity decision:

- `sequencer` records each `Time` sub-expression with its R stride, so the reconstructed `idx` equals the R contribution from `Time` at each time step `t`. For this example `idx` takes values in $\{0, 1, 2, 3\}$ as `t` ranges over $[0, 4)$.
- `slice_mask` extracts the bits of the slice id that carry the R contribution from `slice`. For this example bit 0 carries $R \# 16 / 4 \% 2$ and bit 6 carries $R \# 16 / 8$, so `slice_mask` = 0b1000001.
- `slice_thres` is the bit pattern within `slice_mask` that encodes the partial slice's R contribution. From the table above, the partial slice has R contribution 8, encoded by setting bit 6 ($R \# 16 / 8 = 1$) and clearing bit 0 ($R \# 16 / 4 \% 2 = 0$). So `slice_thres` = 64.
- `time_thres` is `R::SIZE` minus the partial slice's R contribution. For this example `time_thres` = $11 - 8 = 3$.

`fn valid` returns the correct validity for every flit. To verify, decompose the `R` index as $r = r_slice + idx$, where `r_slice` is the `R` contribution from `slice` (encoded by `s & slice_mask`) and `idx` is the contribution from `Time`, and consider the three cases of the slice comparison.

- When $(s \& slice_mask) < slice_thres$, every time step is valid: `r_slice` is at least one slice spacing below the partial slice's contribution, so $r = r_slice + idx < R::SIZE$ for every `idx`.
- When $(s \& slice_mask) = slice_thres$, a time step is valid iff $idx < time_thres$. This is the partial slice, and by definition of `time_thres` its `r_slice = R::SIZE - time_thres`. So $r = r_slice + idx < R::SIZE$ exactly when $idx < time_thres$.
- When $(s \& slice_mask) > slice_thres$, no time step is valid: `r_slice` is at least one slice spacing above the partial slice's contribution, so $r_slice \geq R::SIZE$.

R in Time and Slice

`TimeMajor` mode is the dual of `SliceMajor`: the major/minor roles are flipped, so `Time` sub-expressions carry the larger strides and `slice` sub-expressions carry the smaller ones. The within-`slice` and within-`Time` ordering rules and the power-of-2 size/stride requirement on `slice` sub-expressions carry over unchanged from `SliceMajor`.

`TimeMajor` adds one extra constraint on top of these inherited rules. Recall that `PADDED_SIZE` decomposes as $slice_span \times time_span$, where `slice_span` and `time_span` are the products of sizes of `R`'s sub-expressions in `slice` and `Time` respectively. `TimeMajor` requires $PADDED_SIZE - R::SIZE \leq slice_span$, meaning at most `slice_span` `R` positions may be over-padded. This constraint is essential, and placements that violate it are not supported by the VCG (see [R in Time and Slice](#)).


```
axes![R = 13, X = 64];
```

```
fn reduce_time_slice_timemajor<'l, const T: Tu>(
  input: VectorBranchTensor<'l, T, i32, m![1], m![1 # 2], m![R # 16 / 2 %
2, X, R # 16 % 2], m![R # 16 / 4 % 2, R # 16 / 8], m![1 # 8], Fresh, {
stage::VeOrder::IntraFirst }>,
) -> VectorIntraSliceReduceTensor<'l, T, i32, m![1], m![1 # 2], m![R # 16 / 2
% 2, X, R # 16 % 2], m![1], m![1 # 4], Fresh, { stage::VeOrder::IntraFirst }>
{
  input
    .vector_narrow_trim::<m![1 # 4]>()
    // Slice      = m![R # 16 / 2 % 2, X, R # 16 % 2]
    // Time       = m![R # 16 / 4 % 2, R # 16 / 8]
    // Packet     = m![1 # 4]
    // OutTime    = m![1]           (R eliminated)
    // OutPacket  = m![1 # 4]
    .vector_intra_slice_reduce::<R, m![1], m![1 # 4]>(
      IntraSliceReduceOpI32::AddSat,
    )
}
```

The example places $R = 13$ (padded to $R \# 16$) across `slice` and `Time` with the following sub-expressions.

Dimension	Sub-expression	Stride
Time	$R \# 16 / 4 \% 2$	4
Time	$R \# 16 / 8$	8
Slice	$R \# 16 / 2 \% 2$	2
Slice	$R \# 16 \% 2$	1

These sub-expressions give `slice_span = 4`, `time_span = 4`, and over-padding `PADDED_SIZE - R::SIZE = 3`, which satisfies the constraint $3 \leq 4$. Each of the 4 slices, distinguished by its R contribution from `slice` (0, 1, 2, or 3), sweeps through 4 R values across the 4 time iterations. The layout has two regimes: 1 slice is fully valid, and 3 are partial (each losing one iteration).

R contribution from Slice	R values across iterations	Valid time steps
0	0, 8, 4, 12	4 (all < $R::SIZE$)
1	1, 9, 5, 13	3 ($R = 13$ invalid)
2	2, 10, 6, 14	3 ($R = 14$

		invalid)
3	3, 11, 7, 15	3 (R = 15 invalid)

The compiler emits the time filter config below.

```
TimeFilterConfig {
    sequencer: [R # 16 / 4 % 2 -> size 2 : stride 1,    // 1 = 4 /
slice_span
                R # 16 / 8    -> size 2 : stride 2],    // 2 = 8 /
slice_span
    slice_mask: 0b1000001, // bits 0 (R # 16 % 2) and 6 (R # 16 / 2 % 2)
carry the R contribution
    slice_thres: 1,        // masked id encoding r_slice = 1
    time_thres: 3,        // = time_span - 1 (partial slices drop the
over-padded last iteration)
    mode:      TimeMajor,
}
```

The config differs from `SliceMajor` in three fields (`slice_mask` follows the same pattern):

- `sequencer`: every `Time` sub-expression stride is divided by `slice_span` first. For this example strides 4 and 8 become 1 and 2.
- `slice_thres`: encodes the partial slice's $r_slice = slice_span - (PADDED_SIZE - R::SIZE)$. For this example the target value is $4 - 3 = 1$, encoded with bit 0 set ($R \# 16 \% 2 = 1$) and bit 6 clear ($R \# 16 / 2 \% 2 = 0$), giving `slice_thres = 1`.
- `time_thres`: always `time_span - 1`. For this example `time_thres = 3`.

`fn valid` returns the correct validity for every flit. To verify, decompose the `R` index as $r = idx * slice_span + r_slice$, where `r_slice` is the slice's `R` contribution decoded from `s & slice_mask`, and consider the two cases of the slice comparison.

- When $(s \& slice_mask) < slice_thres$, every time step is valid: $r_slice < slice_span - (PADDED_SIZE - R::SIZE)$, so $r \leq (time_span - 1) * slice_span + r_slice < R::SIZE$ for every `idx`.
- When $(s \& slice_mask) \geq slice_thres$, a time step is valid iff $idx < time_thres = time_span - 1$. The slice's $r_slice \geq slice_span - (PADDED_SIZE - R::SIZE)$. For $idx \leq time_span - 2$, $r \leq (time_span - 2) * slice_span + (slice_span - 1) < (time_span - 1) * slice_span \leq R::SIZE$. For $idx = time_span - 1$, $r \geq (time_span - 1) * slice_span + slice_span - (PADDED_SIZE - R::SIZE) = R::SIZE$.

Unlike `SliceMajor`, `TimeMajor` has no “all invalid” regime. The padding constraint $\text{PADDED_SIZE} - R::\text{SIZE} \leq \text{slice_span}$ caps over-padding tightly enough that no slice becomes fully invalid.

Packet Clipper

For each flit, the packet clipper computes `valid_size(t)`, which depends only on `t` (not on the slice `s`), so all slices receive the same count at the same time step. This slice-independence constrains which placements the VCG can express. The subsections below build up `fn valid_size()` step by step. When `R` has no sub-expression in `Packet`, the packet clipper is disabled by setting `axis_size = packet_span = 8` with an empty sequencer, making `fn valid_size()` always return `8` (the full flit).

```
struct PacketClipperConfig {
    sequencer: Sequencer,
    axis_size: u32, // R::SIZE
    packet_span: u32, // R positions per flit
}

impl PacketClipperConfig {
    /// Returns the valid element count for the flit at time step t.
    fn valid_size(&self, t: u64) -> u32 {
        let idx = self.sequencer.index(t);
        (self.axis_size - idx).clamp(0, self.packet_span)
    }
}
```

The packet clipper requires `Packet = m![R # PADDED_SIZE % packet_span # 8]`. Any other axis sharing `Packet` with `R`, or `R` being split into multiple sub-expressions within `Packet`, breaks the contiguous-prefix property that `fn valid_size()` relies on (see [Inexpressible Patterns](#)).

R as Packet

In the simplest case, `R` fits in a single flit ($R::\text{SIZE} \leq 8$), so every flit has the same `valid_size = R::SIZE` regardless of time step or slice.

```

axes![A = 8, R = 3, X = 64];

fn reduce_packet_only<'l, const T: Tu>(
    input: VectorBranchTensor<'l, T, f32, m![1], m![1 # 2], m![X, A / 2], m!
[1], m![R # 8], Fresh, { stage::VeOrder::IntraFirst }>,
) -> VectorIntraSliceReduceTensor<'l, T, f32, m![1], m![1 # 2], m![X, A / 2],
m![1], m![1 # 4], Fresh, { stage::VeOrder::IntraFirst }>
{
    input
        .vector_narrow_trim::<m![R # 4]>()
        // Slice      = m![X, A / 2]
        // Time       = m![1]
        // Packet      = m![R # 4]
        // OutTime     = m![1]
        // OutPacket    = m![1 # 4] (R eliminated from Packet)
        .vector_intra_slice_reduce::<R, m![1], m![1 # 4]>(
            IntraSliceReduceOpF32::Add,
        )
}

```

The example above places $R = 3$ (padded to $R \# 8$) entirely in `Packet` with a single sub-expression. The compiler configures the packet clipper as follows:

```

PacketClipperConfig {
    sequencer: [], // empty: a single flit holds all of R
    axis_size: 3,  // R::SIZE
    packet_span: 8, // R fits in 8 flit positions
}

```

Every flit has `valid_size = clamp(3 - 0, 0, 8) = 3` (constant across time steps and slices).

R in Time and Packet

The VCG supports R with sub-expressions in both `Time` and `Packet`.

```

axes![A = 8, R = 10, X = 64];

fn reduce_time_packet<'l, const T: Tu>(
  input: VectorBranchTensor<'l, T, f32, m![1], m![1 # 2], m![X, A / 2], m!
[R # 16 / 4], m![R # 16 % 4 # 8], Fresh, { stage::VeOrder::IntraFirst }>,
) -> VectorIntraSliceReduceTensor<'l, T, f32, m![1], m![1 # 2], m![X, A / 2],
m![1], m![1 # 4], Fresh, { stage::VeOrder::IntraFirst }>
{
  input
    .vector_narrow_trim::<m![R # 16 % 4]>()
    // Slice      = m![X, A / 2]
    // Time       = m![R # 16 / 4]
    // Packet     = m![R # 16 % 4 # 8]
    // OutTime    = m![1]          (R eliminated)
    // OutPacket  = m![1 # 4]
    .vector_intra_slice_reduce::<R, m![1], m![1 # 4]>(<
      IntraSliceReduceOpF32::Add,
    )
}

```

The example above places $R = 10$ (padded to $R \# 16$) across Time and Packet:

Dimension	Sub-expression	Stride
Time	$R \# 16 / 4$	4
Packet	$R \# 16 \% 4 \# 8$	1

The compiler configures the packet clipper as follows:

```

PacketClipperConfig {
  sequencer: [R # 16 / 4 -> size 4 : stride 4],
  axis_size: 10, // R::SIZE
  packet_span: 4, // 4 R positions per flit (the trailing 4 flit
positions are always padding)
}

```

With this config, the sequencer reconstructs $\text{idx}(t)$, the Time contribution to the R index (see [R in Time](#)). After Time covers $\text{idx}(t)$ of the $\text{axis_size} = 10$ elements, $\text{axis_size} - \text{idx}(t)$ remain. The packet clipper fits as many of these into the flit as it can, capped at $\text{packet_span} = 4$. So $\text{fn valid_size}(t) = \text{clamp}(10 - \text{idx}(t), 0, 4)$, with the trailing 4 flit positions always padding and the last flit carrying a partial count when $R::\text{SIZE}$ is not a multiple of packet_span :


```
valid_size: 8, 8, 3
            ^   ^
            full 19 - 16 = 3 (partial)
```

Since there are no time filters, every slice gets this exact same pattern:

```
All slices, all flits:
flit 0: [8,8,3] (valid_size=8)
flit 1: [8,8,3] (valid_size=8)
flit 2: [8,0,0] (valid_size=3)
```

Step 2: Add C=5 (packet clipper + time filter 0)

Now enable the c -axis timer (time filter 0). $C=5$ is split into c_0 (slice, size 4) $\times$ c_i (time, size 2). The c time filter config: `slice_mask=0b0011` (extracts c_0 from `slice_id`), `slice_thres=2`, `time_thres=1`, `SliceMajor`.

Each slice now runs 6 flits: c_i size 2 $\times$ w_0 size 3. The c time filter classifies slices by their c_0 value:

c_0	Group	Effect
0	below (< 2)	all 6 flits pass packet clipper's pattern
1	below (< 2)	same
2	boundary ($= 2$)	valid for $c_i=0$, invalid for $c_i=1$
3	above (> 2)	all 6 flits have <code>valid_size = 0</code>

Result per slice (6 flits = c_i size 2 $\times$ w_0 size 3):

```
Co=0: [8,8,3, 8,8,3] (both Ci steps valid)
Co=1: [8,8,3, 8,8,3] (same)
Co=2: [8,8,3, 0,0,0] (Ci=0 valid, Ci=1 invalid)
Co=3: [0,0,0, 0,0,0] (all invalid)
```

Notice the timer's effect. Some slices go entirely to zero, and the boundary slice loses its second half. But within the valid flits, the `[8,8,3]` pattern from packet clipper is unchanged.

Step 3: Add H=5 (packet clipper + time filter 0 + time filter 1)

Now enable the H -axis timer (time filter 1). $H=5$ is split into H_0 (slice, size 4) $\times$ H_i (time, size 2). The H time filter config: `slice_mask=0b1100` (extracts H_0 from `slice_id`), `slice_thres=0b1000`, `time_thres=1`, `SliceMajor`.

The slice id encodes both slice factors as `slice_id =  $H_0 * 4 + C_0$` , giving 16 slices. Each slice now runs 12 flits: H_i size 2 $\times$ C_i size 2 $\times$ W_0 size 3.

Component	Axis	Tracks	Config
packet clipper	$W=19$	per-flit R count	<code>axis_size=19</code> , <code>packet_span=8</code> , <code>sequencer = [$W_0 \rightarrow$ size 3 : stride 8]</code>
time filter 0	$C=5$	per-slice C_0 validity	<code>slice_mask=0b0011</code> , <code>slice_thres=2</code> , <code>time_thres=1</code> , <code>SliceMajor</code>
time filter 1	$H=5$	per-slice H_0 validity	<code>slice_mask=0b1100</code> , <code>slice_thres=0b1000</code> , <code>time_thres=1</code> , <code>SliceMajor</code>

The H time filter classifies slices by H_0 , same logic as C time filter by C_0 :

H_0	Group	Effect
0	below	open
1	below	open
2	boundary	open for $H_i=0$, closed for $H_i=1$
3	above	closed

The final `valid_size` is the packet clipper's count when both time filters return `true`, or 0 if either returns `false`.

The complete heatmap below has 16 slices (columns, grouped by H_0) by 12 flits (rows, grouped by (H_i, C_i)). Right-side annotations ($H:$, $C:$) label which timers are active for each row: `v` = valid, `>` = boundary, `x` = invalid. Scan these annotations first to predict which row $\times$ column blocks should be all-zero (any timer `x`) versus carry data, then read the cells to confirm the `[8, 8, 3]` packet-dim sawtooth.

		Ho=0				Ho=1				Ho=2				Ho=3						
		Co:	0	1	2	3	0	1	2	3	0	1	2	3	0	1	2	3		
H time filter:		v	v	v	v		v	v	v	v	>	>	>	>	x	x	x	x		
C time filter:		v	v	>	x		v	v	>	x	v	v	>	x	v	v	>	x		

t= 0	Hi=0,Ci=0	W	8	8	8	0	8	8	8	0	8	8	8	0	0	0	0	0	H:v	C:v
t= 1			8	8	8	0	8	8	8	0	8	8	8	0	0	0	0	0		
t= 2			3	3	3	0	3	3	3	0	3	3	3	0	0	0	0	0		
t= 3	Hi=0,Ci=1	W	8	8	0	0	8	8	0	0	8	8	0	0	0	0	0	0	H:v	C:>
t= 4			8	8	0	0	8	8	0	0	8	8	0	0	0	0	0	0		
t= 5			3	3	0	0	3	3	0	0	3	3	0	0	0	0	0	0		
t= 6	Hi=1,Ci=0	W	8	8	8	0	8	8	8	0	0	0	0	0	0	0	0	0	H:>	C:v
t= 7			8	8	8	0	8	8	8	0	0	0	0	0	0	0	0	0		
t= 8			3	3	3	0	3	3	3	0	0	0	0	0	0	0	0	0		
t= 9	Hi=1,Ci=1	W	8	8	0	0	8	8	0	0	0	0	0	0	0	0	0	0	H:>	C:>
t=10			8	8	0	0	8	8	0	0	0	0	0	0	0	0	0	0		
t=11			3	3	0	0	3	3	0	0	0	0	0	0	0	0	0	0		

Legend: `v` = below (all valid), `>` = boundary (partial), `x` = above (all invalid)

- Ho=3 columns (rightmost 4): all 0 (H time filter x, always closed).
- Co=3 columns (every 4th): all 0 (C time filter x).
- Co=2 columns (H:v C:>): C time filter is boundary, so only rows with Ci=0 pass. Compare Co=1 vs Co=2.
- Ho=2 columns (H:> C:v): H time filter is boundary, so only rows with Hi=0 pass. Compare Ho=1 vs Ho=2.
- Ho=2 × Co=2 (both >): only (Hi=0, Ci=0) rows pass, the intersection of both boundaries.
- Within valid cells, the [8, 8, 3] sawtooth from packet clipper always appears, the same regardless of slice.

Inexpressible Patterns

The following placements cannot be expressed by the VCG. Each subsection shows the placement and explains why.

R in Slice, Out of Order

When `R` has multiple sub-expressions in `slice`, each outer sub-expression must have a larger stride than the inner ones. Reversing this order produces non-monotonic per-slice `R` index ranges that a single `slice_thres` cannot capture.

```
axes![R = 13, X = 32];
```

```
// NOT supported: inner sub-expression (/ 2 % 4, stride 2) placed outside
major (/ 8, stride 8) in Slice.
```

```
// Produces non-monotonic slice validity (S6 valid after S5 partial); VCG
cannot express this.
```

```
fn reduce_wrong_ordering<'l, const T: Tu>(
    input: VectorBranchTensor<'l, T, i32, m![1], m![1 # 2], m![X, R # 16 / 2
% 4, R # 16 / 8], m![R # 16 % 2], m![1 # 8], Fresh, {
stage::VeOrder::IntraFirst }>,
) -> VectorIntraSliceReduceTensor<'l, T, i32, m![1], m![1 # 2], m![X, R # 16
/ 2 % 4, R # 16 / 8], m![1], m![1 # 4], Fresh, { stage::VeOrder::IntraFirst
}>
{
    input
        .vector_narrow_trim::<m![1 # 4]>()
        // Slice      = m![X, R # 16 / 2 % 4, R # 16 / 8]
        // Time       = m![R # 16 % 2]
        // Packet      = m![1 # 8]
        // OutTime     = m![1]                (R eliminated)
        // OutPacket   = m![1 # 4]
        .vector_intra_slice_reduce::<R, m![1], m![1 # 4]>(
            IntraSliceReduceOpI32::Min,
        )
}
```

R in Slice and Time, Interleaved

When `R`'s sub-expressions are distributed such that a `Slice` factor falls between two `Time` factors, different slices end up needing different numbers of valid time steps. A single `slice_thres` cannot express this per-slice variation.

```

axes![R = 13, X = 64];

// NOT supported: Time-Slice-Time interleave.
// Different slices need different valid step counts (e.g., S2: 3/4, S3:
// 2/4); single threshold cannot express this.
fn reduce_wrong_interleave<'l, const T: Tu>(
    input: VectorBranchTensor<'l, T, i32, m![1], m![1 # 2], m![X, R # 16 / 2
    % 4], m![R # 16 / 8, R # 16 % 2], m![1 # 8], Fresh, {
    stage::VeOrder::IntraFirst }>,
) -> VectorIntraSliceReduceTensor<'l, T, i32, m![1], m![1 # 2], m![X, R # 16
/ 2 % 4], m![1], m![1 # 4], Fresh, { stage::VeOrder::IntraFirst }>
{
    input
        .vector_narrow_trim::<m![1 # 4]>()
        // Slice      = m![X, R # 16 / 2 % 4]
        // Time       = m![R # 16 / 8, R # 16 % 2]
        // Packet     = m![1 # 8]
        // OutTime    = m![1]                (R eliminated)
        // OutPacket  = m![1 # 4]
        .vector_intra_slice_reduce::<R, m![1], m![1 # 4]>(
            IntraSliceReduceOpI32::Min,
        )
}

```

R in Slice and Time, Over-padded

TimeMajor mode requires $\text{PADDED_SIZE} - R::\text{SIZE} \leq \text{slice_span}$. At most slice_span R positions are over-padded.

In the example below, $R = 14$, $\text{Slice} = m![X, R \# 20 \% 4]$ ($\text{slice_span} = 4$), and $\text{Time} = m![A, R \# 20 / 4]$ with $A = 3$ (so $\text{time_span} = 5$ from $R \# 20 / 4$, while $\text{Time}::\text{SIZE} = A \times \text{time_span} = 15$). The constraint is on time_span , not $\text{Time}::\text{SIZE}$: non- R axes in Time like A cycle without changing R 's index, so they do not affect the constraint. The correct padding is $R \# 16$ (since $16 - 14 = 2 \leq \text{slice_span} = 4$), giving $\text{time_span} = 4$. Using $R \# 20$ over-pads R , making $\text{time_span} = 5$ and adding an extra Time iteration that contains no real data. For slices below slice_thres ($\text{slice_contribution} = 0$), the sequencer-reconstructed idx reaches $4 \times 4 = 16$ when $R \# 20 / 4 = 4$, giving R index $16 \geq R::\text{SIZE}$: padding that shouldn't be reachable. The CPU engine catches exactly this: reducing over the over-padded layout panics ($\text{out} \times \text{residue}$ must factor the operand) rather than running with a stale, unreachable R index.

```
axes![A = 3, R = 14, X = 64];
```

```
fn reduce_time_major_wrong<'l, const T: Tu>(
    input: VectorBranchTensor<'l, T, i32, m![1], m![1 # 2], m![X, R # 20 %
4], m![A, R # 20 / 4], m![1 # 8], Fresh, { stage::VeOrder::IntraFirst }>,
) -> VectorIntraSliceReduceTensor<'l, T, i32, m![1], m![1 # 2], m![X, R # 20
% 4], m![A], m![1 # 4], Fresh, { stage::VeOrder::IntraFirst }>
{
    input
        .vector_narrow_trim::<m![1 # 4]>()
        // Slice      = m![X, R # 20 % 4]
        // Time       = m![A, R # 20 / 4]    (A is non-R; time_span = 5 from
R # 20 / 4)
        // Packet     = m![1 # 8]
        // OutTime     = m![A]                (R eliminated; A survives)
        // OutPacket   = m![1 # 4]
        // NOT supported: R # 20 over-pads (20 - 14 = 6 > slice_span = 4).
        // time_span = 5 > 4. Below-group slices include time steps where R #
20 / 4 = 4 (R = 16 padding).
        .vector_intra_slice_reduce::<R, m![A], m![1 # 4]>(
            IntraSliceReduceOpI32::AddSat,
        )
}
```

R in Packet, Complex

The packet clipper requires `Packet = m![R # PADDED_SIZE % packet_span # 8]`. Other forms break the contiguous-prefix property that `fn valid_size()` relies on.

The first example places `R`'s major part in `Packet` (form `R # 24 / 8` instead of `R # 24 % 8`), so the prefix mixes positions from different `R`-strides rather than holding `R`'s next contiguous run.

```
axes![A = 8, R = 19, X = 64];
```

```
fn reduce_wrong_packet_outer<'l, const T: Tu>(
    input: VectorBranchTensor<'l, T, f32, m![1], m![1 # 2], m![X, A / 2], m!
[R # 24 % 8], m![R # 24 / 8 # 8], Fresh, { stage::VeOrder::IntraFirst }>,
) -> VectorIntraSliceReduceTensor<'l, T, f32, m![1], m![1 # 2], m![X, A / 2],
m![1], m![1 # 4], Fresh, { stage::VeOrder::IntraFirst }>
{
    input
        .vector_narrow_trim::<m![R # 24 / 8 # 4]>()
        // Slice      = m![X, A / 2]
        // Time       = m![R # 24 % 8]
        // Packet      = m![R # 24 / 8 # 8]    (NOT supported: major R in
Packet)
        // OutTime    = m![1]                (R eliminated)
        // OutPacket   = m![1 # 4]
        .vector_intra_slice_reduce::<R, m![1], m![1 # 4]>(<
            IntraSliceReduceOpF32::Add,
        )
}
```

The second example has `R` sharing `Packet` with another axis `A`, so `A`'s elements occupy positions that the prefix-based count treats as padding.

```
axes![A = 4, R = 19, X = 256];
```

```
fn reduce_wrong_mixed_packet<'l, const T: Tu>(
    input: VectorBranchTensor<'l, T, f32, m![1], m![1 # 2], m![X], m![R # 24
/ 2], m![R # 24 % 2, A], Fresh, { stage::VeOrder::IntraFirst }>,
) -> VectorIntraSliceReduceTensor<'l, T, f32, m![1], m![1 # 2], m![X], m![1],
m![A], Fresh, { stage::VeOrder::IntraFirst }>
{
    input
        .vector_narrow_split::<m![R # 24], m![A]>()
        // at Vector Init:
        // Slice      = m![X]
        // Time       = m![R # 24 / 2]
        // Packet      = m![R # 24 % 2, A]    (NOT supported: A shares Packet
with R)
        // OutTime    = m![1]                (R eliminated; A silently excluded by
prefix valid_size)
        // OutPacket   = m![A]
        .vector_intra_slice_reduce::<R, m![1], m![A]>(<
            IntraSliceReduceOpF32::Add,
        )
}
```

R in Slice and Packet

`R` splits between `Slice` and `Packet`. To see why this is inexpressible, consider `R = 2045` (padded to `R # 2048`) split as `Slice = m![R # 2048 / 8]` (256 slices) and `Packet = m![R # 2048 % 8]` (8-element flits). At the only time step `t = 0`, `fn valid_size` computes `clamp(2045 - 0, 0, 8) = 8` for every slice. But slice 255's flit holds `R` indices 2040–2047, of which only five are real data:

At `t = 0`, `slice = 255`:

flit position	0	1	2	3	4	5	6
R index	2040	2041	2042	2043	2044	2045	2046
valid?	yes	yes	yes	yes	yes	[pad]	[pad]

Slices 0–254 legitimately need `valid_size = 8`, but slice 255 needs `valid_size = 5`. `fn valid_size(t)` can only return one value for a given `t`, so no single configuration works.

The degenerate sub-case where `R::SIZE % packet_span = 0` (every packet is full or empty) reduces to `Slice` only and is supported.

```
axes![R = 2045];
```

```
// NOT supported: R = 2045 split across Slice (/ 8, 256 slices) and Packet (% 8).
```

```
// Slices 0-254 need valid_size = 8; slice 255 needs valid_size = 5. fn valid_size(t) cannot vary by slice.
```

```
fn reduce_wrong_slice_packet<'l, const T: Tu>(
    input: VectorBranchTensor<'l, T, i32, m![1], m![1 # 2], m![R # 2048 / 8],
    m![1], m![R # 2048 % 8], Fresh, { stage::VeOrder::IntraFirst }>,
) -> VectorIntraSliceReduceTensor<'l, T, i32, m![1], m![1 # 2], m![R # 2048 / 8], m![1], m![1 # 4], Fresh, { stage::VeOrder::IntraFirst }>
{
    input
        .vector_narrow_trim::<m![R # 2048 % 4]>()
        // Slice      = m![R # 2048 / 8]
        // Time       = m![1]
        // Packet      = m![R # 2048 % 8]
        // OutTime     = m![1]           (R eliminated)
        // OutPacket   = m![1 # 4]
        .vector_intra_slice_reduce::<R, m![1], m![1 # 4]>(
            IntraSliceReduceOpI32::AddSat,
        )
}
```

Constraints

Component	Capacity
Packet clippers	1 instance
Time filters	3 instances
Sequencer entries per time filter / packet clipper	8 (see Sequencer)

Each padded axis that needs validity tracking occupies one time filter or the packet clipper. At most 4 axes can be tracked in one invocation (1 packet clipper + 3 time filters). Unpadded axes need no slot (`slice_mask=0`, `slice_thres=1` disables the time filter, making it always return `true`).

Intra-slice reduce takes a single `REDUCE_LABEL` , so “multi-axis” means one reduce axis `R` plus extra padded non-reduce axes, not multiple simultaneous reductions.

Downstream 4-Way Operations

The VCG produces `valid_size` per 8-way flit before any narrowing. A downstream `Narrow` stage splits each 8-way flit into 4-way halves, and the way the narrow is applied determines how each `valid_size` is split between the halves.

Operation	Input	Output	Valid Count Transformation
<code>vector_narrow_split</code>	8-way flit (<code>valid_size = v</code>)	two 4-way flits	low: <code>min(v, 4)</code> , high: <code>max(v - 4, 0)</code>
<code>vector_narrow_trim</code>	8-way flit (<code>valid_size = v</code>)	one 4-way flit	<code>min(v, 4)</code>
<code>vector_widen_concat</code>	two 4-way flits (<code>v_low, v_high</code>)	8-way flit	<code>v_low + v_high</code>
<code>vector_widen_pad</code>	4-way flit	8-way flit	unchanged

`vector_narrow_split` and `vector_widen_concat` preserve the prefix property. For `vector_narrow_trim` , the mapping must statically guarantee `v <= 4` . If the upper 4

elements could be valid, trimming them would lose data.

Inter-Slice Reducer

The Inter-Slice Reducer reduces a tensor across the 256 slices in a cluster. It preserves `Chip`, `Cluster`, and `Packet`, and rewrites `slice` and `Time` into `OutSlice` and `OutTime`. The output tensor is always `way8` regardless of the input mode.

Interface

The reducer can be entered right after `vector_init()` (the `InterFirst` path, shown below) or from a [compatible intra-slice stage](#) (the `IntraFirst` path, with the same `vector_inter_slice_reduce()` method called on the intra-slice tensor). The signatures shown below are the `VectorInitTensor` variants. The same methods also exist on intra-slice tensors at the stages that support the transition, so the call site looks identical.

The inter-slice reducer provides separate APIs for `i32` and `f32`.

i32 Operations

```
impl<'l, const T: Tu, Chip: M, Cluster: M, Slice: M, Time: M, Packet: M>
  VectorInitTensor<'l, T, i32, Chip, Cluster, Slice, Time, Packet>
{
  /// Performs inter-slice reduce for i32 as the first VE operation.
  #[primitive(VectorInitTensor::vector_inter_slice_reduce)]
  pub fn vector_inter_slice_reduce<OutSlice: M, OutTime: M>(
    self,
    op: InterSliceReduceOpI32,
  ) -> VectorInterSliceReduceTensor<'l, T, i32, Chip, Cluster, OutSlice,
    OutTime, Packet, { VeOrder::InterFirst }>
  {
    let reduced = self.inner.reduce(op.reduce_fn(), op.identity(), true);
    create_inter_slice_reduce_tensor(self.ctx, reduced)
  }
}
```

InterSliceReduceOpI32 operations:

Operation	Description
Add	Wrapping addition

AddSat	Saturating addition
Max	Maximum value
Min	Minimum value

f32 Operations

```
impl<'l, const T: Tu, Chip: M, Cluster: M, Slice: M, Time: M, Packet: M>
  VectorInitTensor<'l, T, f32, Chip, Cluster, Slice, Time, Packet>
{
  /// Performs inter-slice reduce for f32 as the first VE operation.
  #[primitive(VectorInitTensor::vector_inter_slice_reduce)]
  pub fn vector_inter_slice_reduce<OutSlice: M, OutTime: M>(
    self,
    op: InterSliceReduceOpF32,
  ) -> VectorInterSliceReduceTensor<'l, T, f32, Chip, Cluster, OutSlice,
    OutTime, Packet, { VeOrder::InterFirst }>
  {
    let reduced = self.inner.reduce(op.reduce_fn(), op.identity(), true);
    create_inter_slice_reduce_tensor(self.ctx, reduced)
  }
}
```

InterSliceReduceOpF32 operations:

Operation	Description
Add	Floating-point addition
Max	Maximum value
Min	Minimum value
Mul	Floating-point multiplication

Constraints

The supported `Slice` $\rightarrow$ `OutSlice` and `Time` $\rightarrow$ `OutTime` shapes follow four rules:

1. **Reduce from innermost.** The reduced portion of `slice` must be the innermost factors, contiguous, with stride 1 through the reduction ratio `r`.
2. **Replacement on the reduced axis.** Each reduced factor's slot in `outSlice` is filled by one of: a dummy (`1 # n`), a broadcast over a fresh dimension, or a promotion

from `Time` .

3. **Replacement kinds mix freely.** Dummy, broadcast, and promotion slots can appear together in `OutSlice` in any order.
4. **Promotion from `Time` to `OutSlice` reorders.** The `Time` $\rightarrow$ `OutTime` portion preserves the relative order of surviving factors, but the `Time` $\rightarrow$ `OutSlice` promotion path does not preserve order: a promoted factor's position in `OutSlice` is independent of its position in `Time` .

Examples

The math in the examples below uses einsum notation. A dimension that appears on the input side but not on the output side is reduced (summed), and a dimension that appears on the output side but not on the input side is broadcast.

Dummy Replacement

This pass sums input across `R` and places the result in a dummy slot. The einsum form is `AR -> A` .

```
axes![A = 512, B = 2, R = 4];

// When R is reduced and no other dimension fills its slot, the output keeps
// the slot as a 1 # n dummy.
// One position holds the reduced value, and the remaining n - 1 are padding.
// `# n` denotes dimension multiplicity (see the Mapping Expressions doc).
fn inter_slice_add<'l, const T: Tu>(
    input: CollectTensor<'l, T, i32, m![1], m![B], m![A / 8, R], m![1], m![A
% 8]>,
) -> VectorFinalTensor<'l, T, i32, m![1], m![B], m![A / 8, 1 # 4], m![1], m!
[A % 8]> {
    input
        .vector_init()
        // sum across R, the freed R-slot becomes the 1 # 4 dummy in OutSlice
        .vector_inter_slice_reduce::

```

```

Slice = [A / 8, R]  ->  [A / 8, 1 # 4]
Time  = [1]         ->  [1]

```

Broadcast Into a New Slice Dimension

This pass reduces `R` and broadcasts the result over a fresh dimension `x`. The einsum form is `PRW -> PWX`, with the fresh `x` on the output side broadcasting.

```

axes![B = 2, P = 8, R = 4, W = 64, X = 4];

// A fresh non-reduce dimension X takes the slot that R leaves behind.
// The reduced value broadcasts across every position of X.
fn broadcast_into_x<'l, const T: Tu>(
    input: CollectTensor<'l, T, f32, m![1], m![B], m![W, R], m![1], m![P]>,
) -> VectorFinalTensor<'l, T, f32, m![1], m![B], m![W, X], m![1], m![P]> {
    input
        .vector_init()
        // sum across R, broadcast result over X (fresh OutSlice dimension)
        .vector_inter_slice_reduce::<m![W, X], m![1]>
(InterSliceReduceOpF32::Add)
        .vector_final()
}

Slice = [W, R]  ->  [W, X]
Time  = [1]     ->  [1]

```

Promotion from Time into OutSlice

This pass reduces `R` and promotes the `Time` dimension `v` into `OutSlice`. The einsum form is `PRSUUVW -> PSUUVW`.

```

axes![B = 2, P = 8, R = 4, S = 2, U = 2, V = 4, W = 64];

// A dimension from Time (here V) is promoted into OutSlice to fill R's slot.
// The promoted dimension does not need to be outermost in Time.
fn axis_promotion<'l, const T: Tu>(
    input: CollectTensor<'l, T, f32, m![1], m![B], m![W, R], m![S, V, U], m!
[P]>,
) -> VectorFinalTensor<'l, T, f32, m![1], m![B], m![W, V], m![S, U], m![P]> {
    input
        .vector_init()
        // sum across R, V moves from Time to OutSlice
        .vector_inter_slice_reduce::<m![W, V], m![S, U]>
(InterSliceReduceOpF32::Add)
        .vector_final()
}

Slice = [W, R]      -> [W, V]
Time   = [S, V, U]  -> [S, U]

```

For examples that combine the reducer with the intra-slice chain in either order, see the [Vector Engine Examples](#).

Performance

The reduction ratio r (the number of slices in one reduction group) is the main tuning knob. Inter-slice reduce latency is $O(r)$ cycles, approximately one ring traversal of the reduction group. Total time equals the input streaming time plus that ring-sized tail.

In practice, upstream work (contraction producing partial sums or intra-slice work before `vector_inter_slice_reduce()`) often dominates and hides this ring tail, so the reducer isn't the bottleneck. The reducer becomes visible at large r (longer tails) and on small tensors that can't amortize the fixed tail across many packets.

Cast Engine

The Cast Engine narrows `f32 / i32` pipeline results to lower-precision types (e.g., `bf16`) before the [Commit Engine](#) writes them to DM, reducing storage cost.

Interface

`CollectTensor`, `ContractTensor`, and `VectorFinalTensor` all expose `.cast()` with the same semantics.

```
//
// The Cast Engine accepts only `VeScalar` inputs (hardware constraint), so
// the
// bound lives on the impl rather than on a wider trait.
impl<'l, const T: Tu, P: CanApplyCast, D: VeScalar, Chip: M, Cluster: M,
Slice: M, Time: M, Packet: M, B: Backend>
    TuTensor<'l, T, P, D, Chip, Cluster, Slice, Time, Packet, B>
{
    /// Casts each element to type `OutD` and pads the output packet back to
    one
    /// 32-byte flit.
    #[primitive(TuTensor::cast)]
    pub fn cast<OutD: Scalar, OutPacket: M>(self) -> CastTensor<'l, T, OutD,
Chip, Cluster, Slice, Time, OutPacket, B>
    where
        D: CastEngineCast<OutD>,
    {
        verify_cast::<D, OutD, Packet, OutPacket>();
        CastTensor::new(self.ctx, self.inner.map(|v|
v.cast()).transpose(false))
    }
}
```

`.cast::<OutD, OutPacket>()` converts each element to type `outD` and pads the output back to one 32-byte flit. The kernel writer chooses `outD` (the target type) and `OutPacket` (the output element layout). The compiler derives the rest.

Although the Cast Engine is not a [mathematical tensor move](#), it preserves the tensor's shape and only changes the element type. All dimensions pass through unchanged, except for the `Packet` layout, which repads so the output still fits one 32-byte flit.

The example below casts an 8-element `i32` packet ($8 \times 4 = 32$ bytes) to `i8`. After the

cast, the 8 elements occupy 8 bytes, so `A # 32` pads the output back to 32 bytes:

```
axes![B = 4, A = 8];

fn cast_i32_to_i8<'l, const T: Tu>(
    input: CollectTensor<'l, T, i32, m![1], m![1 # 2], m![1 # 256], m![B], m!
[A]>,
) -> CastTensor<'l, T, i8, m![1], m![1 # 2], m![1 # 256], m![B], m![A # 32]>
{
    input.cast()
}
```

The input data may not fill 32 bytes. The example below casts an `i32` input where 4 data elements are padded to 8 (`A # 8`, 32 bytes) into an `i8` output where the same 4 elements are padded to 32 (`A # 32`, also 32 bytes):

```
axes![A = 4];

fn cast_padded<'l, const T: Tu>(
    input: CollectTensor<'l, T, i32, m![1], m![1 # 2], m![1 # 256], m![1], m!
[A # 8]>,
) -> CastTensor<'l, T, i8, m![1], m![1 # 2], m![1 # 256], m![1], m![A # 32]>
{
    input.cast()
}
```

Supported Casts

Each input is a 32-byte flit. The supported source types are `f32` and `i32`, each with specific target types:

Input Type (<code>D</code>)	Supported Output Types (<code>OutD</code>)
<code>i32</code>	<code>i4</code> , <code>i8</code> , <code>i16</code>
<code>f32</code>	<code>f8e5m2</code> , <code>f8e4m3</code> , <code>bf16</code>

Performance

The Cast Engine is never the pipeline bottleneck: it processes one flit per cycle regardless

of how much of the flit carries valid data. The downstream [Commit Engine](#) aggregates under-utilized flits into dense DM writes, so no DM bandwidth is wasted.

Transpose Engine

The Transpose Engine swaps the `Time` and `Packet` dimensions, while leaving the `chip`, `cluster`, and `slice` dimensions unchanged.

Interface

`CollectTensor` and `VectorFinalTensor` both expose `.transpose()`. The `VectorFinalTensor` entry point feeds the Transpose Engine directly from the Vector Engine output.

```
// `D: MaterializableScalar` (see its doc) excludes i5/i9 stagings from
transpose.
impl<
    'l,
    const T: Tu,
    P: CanApplyTranspose,
    D: MaterializableScalar,
    Chip: M,
    Cluster: M,
    Slice: M,
    Time: M,
    Packet: M,
    B: Backend,
> TuTensor<'l, T, P, D, Chip, Cluster, Slice, Time, Packet, B>
{
    /// Performs the transpose operation.
    #[primitive(TuTensor::transpose)]
    pub fn transpose<OutTime: M, OutPacket: M>(
        self,
    ) -> TransposeTensor<'l, T, D, Chip, Cluster, Slice, OutTime, OutPacket,
B> {
        verify_transpose::<D, Time, Packet, OutTime, OutPacket>();
        TransposeTensor::new(self.ctx, self.inner.transpose(false))
    }
}
```

The kernel writer chooses `OutTime` and `OutPacket` (the output dimension layouts), and the compiler verifies the result against the hardware constraints listed under [Parameters](#).

The example below transposes an 8×16 `i8` matrix whose 16-wide rows are each gathered from two input packets (`D = 2`). It is reused as the running example

throughout the rest of this page.

```
axes![P = 256, B = 2, C = 8, D = 2, E = 8];

fn basic_transpose<'l, const T: Tu>(
    input: CollectTensor<'l, T, i8, m![1], m![1 # 2], m![P], m![B, C, D], m!
[E # 32]>,
) -> TransposeTensor<'l, T, i8, m![1], m![1 # 2], m![P], m![B, D, E], m![C #
32]> {
    input.transpose()
}
```

Architecture

The five transpose stages below are illustrated using the running example from [Interface](#).

Parameters

`valid_size` is the number of valid elements the Transpose Engine reads per cycle from its 32-byte input bus, and every input flit arrives in `bit-width × valid_size` form, with any remaining bytes of the 32-byte flit treated as padding and discarded by the Unpack stage. For 8-bit and 16-bit elements, the engine can select doubled input packing and read 16 valid elements from each flit instead of the base 8. Doubled packing is not supported for 4-bit or 32-bit elements. Data reaches the Transpose Engine via `CollectTensor::transpose()` (after `Fetch` → `[Switch` →] `Collect` → `[Cast` →] `Transpose`) or `VectorFinalTensor::transpose()` (directly from the [Vector Engine](#)). The [Contraction Engine](#) emits `32b × 8` only, while the [Vector Engine](#) and [Fetch Engine](#) emit any combination from the table below.

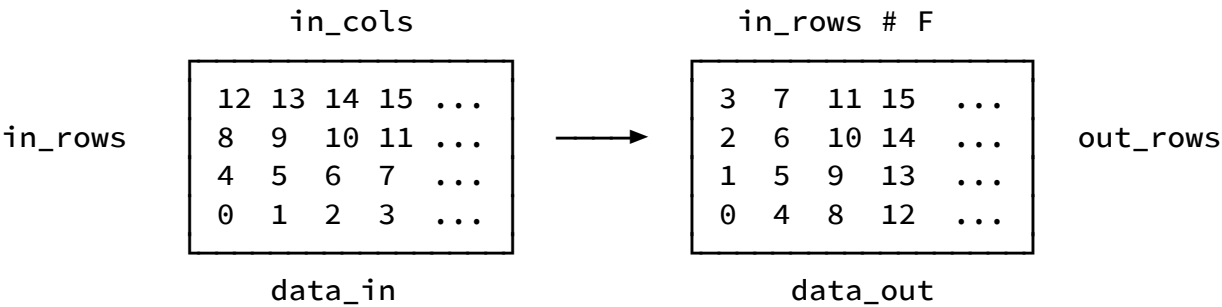
`in_cols`, `in_rows`, and `out_rows` are fixed by the kernel writer's `OutTime` and `OutPacket` choices. All four are constrained by the element size:

Element size	Single <code>valid_size</code>	Doubled <code>valid_size</code>	Max <code>in_rows</code>	Valid <code>in_cols</code>
4-bit	16	unsupported	16	16, 32
8-bit	8	16	8	8, 16, 32
16-bit	8	16	4	8, 16, 32

32-bit	8	unsupported	2	8, 16, 32
--------	---	-------------	---	-----------

For the running example (`i8`, so `valid_size = 8`), the compiler derives:

Parameter	Value	Notes
<code>in_cols</code>	16	<code>D = 2 packets gathered × valid_size = 8</code>
<code>in_rows</code>	8	<code>C::SIZE</code>
<code>out_rows</code>	16	<code>D•E (= in_cols , fully utilized)</code>



Unpack

Each 32-byte input packet carries `valid_size` valid elements; the Unpack stage discards the rest of the flit as padding. The compiler selects doubled packing when an 8-bit or 16-bit packet has live elements beyond the base 8-element prefix. The selected `valid_size` must still cover the complete live prefix; any remaining lanes must be padding.

In the running example: `[C, D, E # 32] → [C, D, E]` .

For example, an `i8` packet `[B=16 # 32]` uses doubled packing and contributes all 16 live elements in one cycle. The equivalent single-packed layout would contribute 8 elements from each of two consecutive packets. An `i4` packet cannot request the doubled form: its only supported `valid_size` is 16.

Gather

One row of the input matrix is `in_cols = packets_per_col × valid_size` elements wide, assembled from `packets_per_col` consecutive packets — the innermost time steps. The Gather stage concatenates those packets into a single row, and `in_rows` further time steps stack into the `[in_rows × in_cols]` input matrix. (Unpack and Gather

both happen as the engine reads its input — they are not separate buffered passes.)

In the running example, the innermost $D = 2$ packets each contribute `valid_size = 8`, forming `in_cols = 16`-wide rows, and the $C = 8$ time steps above them stack into the $[8 \times 16]$ input matrix.

Transpose

The matrix is transposed: $[in_rows \times in_cols] \rightarrow [in_cols \times in_rows]$.

In the running example: $[C, D, E] \rightarrow [D, E, C]$.

Trim

When some input packets carry fewer valid elements than `valid_size`, the transposed matrix has padded rows. The Trim stage drops those rows, producing $[out_rows \times in_rows]$ where $out_rows \leq in_cols$.

In the running example: $[D, E, C] \rightarrow [D, E, C]$ (the input is fully utilized, so no rows are trimmed). See the [Small Matrix](#) example for a case where Trim actually discards rows.

Align

The transposed rows are `in_rows` elements wide, but DM packets must be 32 bytes. The Align stage pads each row to a 32-byte flit, producing shape $[out_rows \times (in_rows \# F)]$ where F is chosen so that $D[F]$ is 32 bytes.

In the running example: $[D, E, C] \rightarrow [D, E, C \# 32]$.

Latency

Note

Read [Performance](#) first for the formulas.

For the running example, $in_cols = 16 \leq 16$ selects double buffering. With $in_flits =$

16, `out_rows = 16`, and `n = 2`, the total latency is $16 + 1 \times \max(16, 16) + 16 = 48$ cycles.

Examples

Small Matrix

This example demonstrates the Trim stage discarding padded rows when `out_rows < in_cols`:

```
axes![P = 256, A = 4, B = 2];

fn small_transpose<'l, const T: Tu>(
    input: CollectTensor<'l, T, i8, m![1], m![1 # 2], m![P], m![A], m![B #
32]>,
) -> TransposeTensor<'l, T, i8, m![1], m![1 # 2], m![P], m![B], m![A # 32]> {
    input.transpose()
}
```

Parameters:

Parameter	Value	Notes
<code>in_cols</code>	8	<code>B::SIZE = 2</code> , padded to 8
<code>in_rows</code>	4	<code>A::SIZE</code>
<code>out_rows</code>	2	<code>B::SIZE</code>

Stages:

- **Unpack:** $[A, B \# 32] \rightarrow [A, B \# 8]$.
- **Gather:** $[A, B \# 8] \rightarrow [A, B \# 8]$ (`packets_per_col = 1`, so each packet is already a full `in_cols = 8` row).
- **Transpose:** $[A, B \# 8] \rightarrow [B \# 8, A]$.
- **Trim:** $[B \# 8, A] \rightarrow [B, A]$ (6 padded rows trimmed).
- **Align:** $[B, A] \rightarrow [B, A \# 32]$.

Latency: `in_cols = 8 ≤ 16` selects double buffering. With `in_flits = 4`, `out_rows = 2`, and `n = 1`, the total is $4 + 0 \times \max(4, 2) + 2 = 6$ cycles.

Large Column

This example forces single buffering by making `in_cols > 16`, which prevents input and output from overlapping and so increases total cycles:

```
axes![P = 256, B = 2, C = 8, D = 4, E = 8];

fn large_col_transpose<'l, const T: Tu>(
    input: CollectTensor<'l, T, i8, m![1], m![1 # 2], m![P], m![B, C, D], m!
[E # 32]>,
) -> TransposeTensor<'l, T, i8, m![1], m![1 # 2], m![P], m![B, D, E], m![C #
32]> {
    input.transpose()
}
```

Parameters:

Parameter	Value	Notes
<code>in_cols</code>	32	$D::SIZE \times E::SIZE$
<code>in_rows</code>	8	$C::SIZE$
<code>out_rows</code>	32	$D::SIZE \times E::SIZE$

Stages:

- **Unpack:** $[C, D, E \# 32] \rightarrow [C, D, E]$.
- **Gather:** the innermost $D = 4$ packets form each $in_cols = 32$ row, and the $c = 8$ time steps stack into the $[8 \times 32]$ input matrix.
- **Transpose:** $[C, D, E] \rightarrow [D, E, C]$.
- **Trim:** $[D, E, C] \rightarrow [D, E, C]$ (no rows trimmed).
- **Align:** $[D, E, C] \rightarrow [D, E, C \# 32]$.

Latency: $in_cols = 32 > 16$ selects single buffering. With $in_flits = 32$, $out_rows = 32$, and $n = 2$ (B), the total is $2 \times (32 + 32) = 128$ cycles.

16-bit Data Type

This example uses `bf16`, where the wider element halves max `in_rows` (4 instead of 8) and shrinks the 32-byte output flit to 16 elements (instead of 32 for `i8`):

```
axes![P = 256, C = 8, D = 4, E = 8];

fn bf16_transpose<'l, const T: Tu>(
    input: CollectTensor<'l, T, bf16, m![1], m![1 # 2], m![P], m![C, D], m![E
# 16]>,
) -> TransposeTensor<'l, T, bf16, m![1], m![1 # 2], m![P], m![C, E], m![D #
16]> {
    input.transpose()
}
```

Parameters:

Parameter	Value	Notes
in_cols	8	E::SIZE
in_rows	4	D::SIZE
out_rows	8	E::SIZE

Stages:

- **Unpack:** $[D, E \# 16] \rightarrow [D, E]$.
- **Gather:** $[D, E] \rightarrow [D, E]$ (packets_per_col = 1; each packet is already a full in_cols = 8 row).
- **Transpose:** $[D, E] \rightarrow [E, D]$.
- **Trim:** $[E, D] \rightarrow [E, D]$ (no rows trimmed).
- **Align:** $[E, D] \rightarrow [E, D \# 16]$.

Latency: in_cols = 8 $\leq$ 16 selects double buffering. With in_flits = 4, out_rows = 8, and n = 8 (C), the total is $4 + 7 \times \max(4, 8) + 8 = 68$ cycles.

4-bit Data Type

This example uses i4, where valid_size = 16 doubles the per-cycle element count and max in_rows rises to 16 (16×4 bits = 8 bytes), while the 32-byte flit grows to 64 elements:

```
axes![P = 256, B = 4, C = 16, E = 16];

fn i4_transpose<'l, const T: Tu>(
    input: CollectTensor<'l, T, i4, m![1], m![1 # 2], m![P], m![B, C], m![E #
64]>,
) -> TransposeTensor<'l, T, i4, m![1], m![1 # 2], m![P], m![B, E], m![C #
64]> {
    input.transpose()
}
```

Parameters:

Parameter	Value	Notes
in_cols	16	E::SIZE
in_rows	16	C::SIZE
out_rows	16	E::SIZE

Stages:

- **Unpack:** $[C, E \# 64] \rightarrow [C, E]$.
- **Gather:** $[C, E] \rightarrow [C, E]$ (packets_per_col = 1; each packet is already a full in_cols = 16 row).
- **Transpose:** $[C, E] \rightarrow [E, C]$.
- **Trim:** $[E, C] \rightarrow [E, C]$ (no rows trimmed).
- **Align:** $[E, C] \rightarrow [E, C \# 64]$.

Latency: in_cols = 16 ≤ 16 selects double buffering. With in_flits = 16, out_rows = 16, and n = 4 (B), the total is $16 + 3 \times \max(16, 16) + 16 = 80$ cycles.

32-bit Data Type

This example uses f32, one of the two 32-bit formats the [Contraction Engine](#) emits (32b × 8, either f32 or i32). The wider element drops max in_rows to 2 (2 × 4 bytes = 8 bytes) and shrinks the 32-byte flit to 8 elements:


```
axes![P = 256, B = 4, D = 2, E = 8];
```

```
fn f32_transpose<'l, const T: Tu>(  
    input: CollectTensor<'l, T, f32, m![1], m![1 # 2], m![P], m![B, D], m![E  
# 8]>,  
) -> TransposeTensor<'l, T, f32, m![1], m![1 # 2], m![P], m![B, E], m![D #  
8]> {  
    input.transpose()  
}
```

Parameters:

Parameter	Value	Notes
in_cols	8	E::SIZE
in_rows	2	D::SIZE
out_rows	8	E::SIZE

Stages:

- **Unpack:** $[D, E \# 8] \rightarrow [D, E]$.
- **Gather:** $[D, E] \rightarrow [D, E]$ (packets_per_col = 1; each packet is already a full in_cols = 8 row).
- **Transpose:** $[D, E] \rightarrow [E, D]$.
- **Trim:** $[E, D] \rightarrow [E, D]$ (no rows trimmed).
- **Align:** $[E, D] \rightarrow [E, D \# 8]$.

Latency: in_cols = 8 $\leq$ 16 selects double buffering. With in_flits = 2, out_rows = 8, and n = 4 (B), the total is $2 + 3 \times \max(2, 8) + 8 = 34$ cycles.

Performance

A burst runs $n = \text{OutTime}::\text{SIZE} / \text{out_rows}$ transpose iterations. Each iteration moves in_flits = in_rows $\times$ (in_cols / valid_size) input flits and out_rows output flits.

Buffering Modes

The Transpose Engine has two internal buffers, each holding 16 columns. The compiler picks double buffering when in_cols $\leq$ 16 and single buffering otherwise: double

buffering overlaps input and output to reduce total cycles, while single buffering serializes the two phases.

Single Buffering Latency

The total burst latency is $n \times (\text{in_flits} + \text{out_rows})$. Both buffers are used together, so input and output add together each iteration.

Double Buffering Latency

The total burst latency is $\text{in_flits} + (n - 1) \times \max(\text{in_flits}, \text{out_rows}) + \text{out_rows}$, which breaks into three phases:

- **Input-only phase** (in_flits cycles): the first buffer fills.
- **Overlap phase** ($(n - 1) \times \max(\text{in_flits}, \text{out_rows})$ cycles): one buffer receives input while the other produces output simultaneously, so the slower side gates each iteration.
- **Output-only phase** (out_rows cycles): the last buffer drains.

Commit Adapter

The Commit Adapter applies element-wise transformations to the packet stream before the [Commit Engine](#) writes it to DM. It mirrors the [Fetch Adapter](#) on the output side of the Tensor Unit.

The adapter's stages chain as dedicated `.commit_xxx(...)` methods on the upstream tensor, and the chain always ends in `.commit(...)` for the actual DM write. [Trimming](#) is the mandatory first stage: `.commit()` / `.commit_view()` are reachable only after `.commit_trim(...)`, so every commit is trimmed first (it is how flit padding is dropped). The other public stage is optional type casting, and main and sub contexts then diverge. Generate Mode is a separate sub-context path used internally by `memset` for immediate fills; it does not chain off a `TuTensor`.

- Main pipeline: [Trimming](#) → [Type Casting](#) (optionally fusing ReLU) → `.commit()`.
- Sub bypass: Generate Mode writes a constant directly for `memset`, without consuming an upstream Tensor Unit stream.

Operation	Main	Sub
Trimming	✓	✓
Type Casting (optional fused ReLU)	✓	✗
Generate Mode (internal <code>memset</code> path)	✗	✓

Trimming

Stream packets in the Tensor Unit pipeline are always 32-byte *flits* (see [Collect Engine](#)), but a flit may carry fewer valid elements than its capacity, with trailing elements filled by padding. Writing the full flit verbatim would clobber DM bytes beyond the valid region with the flit's padding values.

Trimming solves this by writing only the leading `valid_size` elements of each flit to DM, discarding the trailing padding. The compiler derives `valid_size` from the output tensor mapping. Users do not set it directly. `D[valid_size]` must be 8, 16, 24, or 32 bytes (where 32 means no trim). Trimming adds nearly zero latency.

Trimming is the mandatory first stage of the Commit Adapter, even though not every

`commit` has padding to drop: when `valid_size` is already 32 bytes the flit is fully valid and the trim is a no-op. It is mandatory because `.commit()` is reachable only after `.commit_trim(...)`, so it anchors the chain and runs ahead of [Type Casting](#) (main).

```
// `D: MaterializableScalar` here (trim is the commit path's mandatory first
stage) keeps i5/i9 uncommittable.
impl<
  'l,
  const T: Tu,
  P: CanApplyCommitTrim,
  D: MaterializableScalar,
  Chip: M,
  Cluster: M,
  Slice: M,
  Time: M,
  Packet: M,
  B: Backend,
> TuTensor<'l, T, P, D, Chip, Cluster, Slice, Time, Packet, B>
{
  /// Runs the Commit Adapter's trimming stage.
  ///
  /// Drops the trailing padding from each flit so DM stores only valid
  /// elements. `OutPacket` is the post-trim layout the kernel
  /// promises; the compiler derives the trim count from the input and
  /// output mappings.
  #[primitive(TuTensor::commit_trim)]
  pub fn commit_trim<OutPacket: M>(self) -> CommitTrimTensor<'l, T, D,
Chip, Cluster, Slice, Time, OutPacket, B> {
    verify_commit_trim::<D, Packet, OutPacket>();
    // `transpose(false)` is type-system filler; real trim lowering lands
with the backend wiring.
    CommitTrimTensor::new(self.ctx, self.inner.transpose(false))
  }
}
```

`.commit_trim::<OutPacket>()` declares the post-trim packet, and the chained `.commit(...)` then performs the DM write on the trimmed stream. The two are fully separate.

```

axes![M = 4, K = 2, W = 8, N = 16, J = 64];

fn commit_trim_i8_padding<'l, const T: Tu>(
    input: CastTensor<'l, T, i8, m![1], m![1], m![1], m![M, K], m![W # 32]>,
) -> CommitTrimTensor<'l, T, i8, m![1], m![1], m![1], m![M, K], m![W]> {
    // 8 valid i8 out of 32 padded; OutPacket drops the `# 32` padding.
    input.commit_trim::<m![W]>()
}

fn commit_trim_f32_non_padding<'l, const T: Tu>(
    input: ContractTensor<'l, T, f32, m![1], m![1], m![1], m![M, K], m![W]>,
) -> CommitTrimTensor<'l, T, f32, m![1], m![1], m![1], m![M, K], m![W = 4]> {
    // 4 valid f32 out of 8; OutPacket resizes `W` to 4.
    input.commit_trim::<m![W = 4]>()
}

fn commit_trim_bf16_with_transpose<'l, const T: Tu>(
    input: CastTensor<'l, T, bf16, m![1], m![1], m![1], m![M, K], m![N]>,
) -> CommitTrimTensor<'l, T, bf16, m![1], m![1], m![1], m![M, K], m![N = 8]> {
    // 8 valid bf16 out of 16; OutPacket resizes `N` to 8.
    input.commit_trim::<m![N = 8]>()
}

fn commit_trim_i4_no_trim<'l, const T: Tu>(
    input: CastTensor<'l, T, i4, m![1], m![1], m![1], m![M, K], m![J]>,
) -> CommitTrimTensor<'l, T, i4, m![1], m![1], m![1], m![M, K], m![J]> {
    // No trimming; `OutPacket == Packet`.
    input.commit_trim::<m![J]>()
}

```

Type Casting

Type casting converts `f32` data to `bf16` format on the commit path, optionally fusing a ReLU activation into the same pass. The [Cast Engine](#) handles most type conversions in the Tensor Unit pipeline. Commit Adapter type casting exists for one specific case, running main-context contraction in parallel with sub-context Vector Engine work. The Cast Engine sits on top of the Vector Engine and so occupies it during a conversion. If the main-context performed its `f32` $\rightarrow$ `bf16` conversion through the Cast Engine, the Vector Engine would be busy and the sub-context could not run in parallel. Routing the conversion through the Commit Adapter instead leaves the Vector Engine free for the sub-context. Sub-context itself does not support type casting (consistent with the support matrix above).

`commit_cast` and `commit_cast_relu` are separate methods because they are separate hardware conversions, not one conversion with a mode: ReLU has no standalone stage to select at run time, and exists only fused with the narrowing cast.

```

impl<
  'l,
  const T: Tu,
  P: CanApplyCommitCast,
  D: MaterializableScalar,
  Chip: M,
  Cluster: M,
  Slice: M,
  Time: M,
  Packet: M,
  B: Backend,
> TuTensor<'l, T, P, D, Chip, Cluster, Slice, Time, Packet, B>
{
  /// Runs the Commit Adapter's type-casting stage.
  ///
  /// Folds the `f32` → `bf16` cast into the commit path, leaving the
  /// [Cast Engine](crate::engine::cast) free for sub-context Vector Engine
  /// work. See [`CommitCast`](crate::prelude::CommitCast) for why this is
the
  /// only conversion, and [`commit_cast_relu`](Self::commit_cast_relu) for
the
  /// variant that fuses a ReLU.
  #[primitive(TuTensor::commit_cast)]
  pub fn commit_cast<OutD: Scalar>(self) -> CommitCastTensor<'l, T, OutD,
Chip, Cluster, Slice, Time, Packet, B>
  where
    D: CommitCast<OutD>,
    {
      verify_commit_cast::<Packet>();
      CommitCastTensor::new(self.ctx, self.inner.map(|v| v.cast()))
    }

  /// The same cast with a ReLU fused in, clamping negative values to zero.
  ///
  /// A separate method rather than an argument because the two are
separate
  /// hardware conversions (`CommitF32ToBf16` and `CommitF32ToBf16Relu`),
and
  /// ReLU has no standalone stage to select at run time.
  #[primitive(TuTensor::commit_cast_relu)]
  pub fn commit_cast_relu<OutD: Scalar>(self) -> CommitCastTensor<'l, T,
OutD, Chip, Cluster, Slice, Time, Packet, B>
  where
    D: CommitCast<OutD>,
    {
      verify_commit_cast::<Packet>();
      CommitCastTensor::new(self.ctx, self.inner.map(|v| v.cast_relu()))
    }
}

```

```
axes![N = 4, C = 3, H = 4, W = 8];
```

```
fn commit_cast_example<'l, const T: Tu>(
    input: ContractTensor<'l, T, f32, m![1], m![1], m![1], m![N, C, H], m![W]>,
) -> CommitCastTensor<'l, T, bf16, m![1], m![1], m![1], m![N, C, H], m![W]> {
    // Cast f32 to bf16 (values preserved), no activation. A real main
    // commit runs `.commit_trim()` first, then `.commit(...)` after.
    // W = 8 f32 elements (32 bytes) stays 8 bf16 elements (16 bytes).
    input.commit_cast::<bf16>()
}

fn commit_cast_relu_example<'l, const T: Tu>(
    input: ContractTensor<'l, T, f32, m![1], m![1], m![1], m![N, C, H], m![W]>,
) -> CommitCastTensor<'l, T, bf16, m![1], m![1], m![1], m![N, C, H], m![W]> {
    // f32 -> bf16 with a fused ReLU: negative values clamped to zero.
    // e.g. [-5.0, -0.1, 0.0, 3.7] -> [0.0, 0.0, 0.0, 3.7]
    input.commit_cast_relu::<bf16>()
}
```


End-to-End Cases

End-to-End Cases connect the mapping, movement, computation, and scheduling contracts into composed workloads. Choose the nearest case below, then verify its referenced API before adapting mappings.

Legend: **Verified and tested** identifies a reader-ready pattern with current source and tests. **Design guide** identifies explanatory material whose API details must be verified before implementation.

Choose a Starting Point

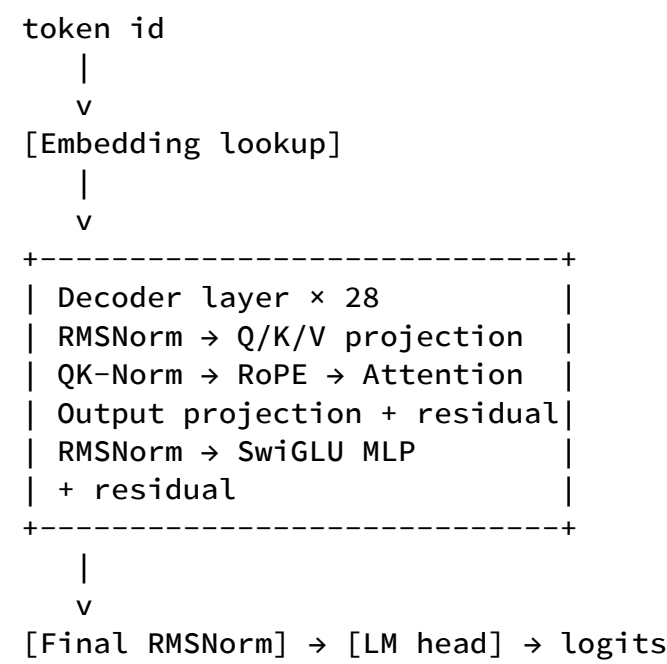
Start implementation from a [Quick Start](#) pattern whenever it can express the required API behavior. The remaining pages explain design choices; verify their code against the current API before using it.

Need	Start with	Technical focus and status
Qwen3 decoder step	Case Study: Transformer	Model mental model, baseline kernel map, decode-only boundaries, portable oracle, and schedule data from the current transformer example.

Case Study: Transformer

This Transformer page uses Qwen3-0.6B as a concrete tutorial case, explaining its model mental model and following one decoder token from a host-prepared embedding to padded vocabulary logits. Each boundary is checked with a portable host oracle and a reproducible schedule artifact. Preserve those host-prepared boundaries when adapting it.

The model is a decoder-only transformer with 28 layers. Each layer transforms one running hidden vector for one token at a time.



The model uses hidden size 1024, 16 query heads, 8 key/value heads, head dimension 128, and intermediate size 3072. Grouped-query attention shares each key/value head across two query heads. RoPE uses theta 1,000,000, and the embedding and output weights are tied by the model design.

Component	Shape or count
Decoder layers	28
Vocabulary	151,936 tokens
Embedding and tied LM head	151,936 × 1,024
Attention projections per layer	6,291,456 parameters
SwiGLU MLP per layer	9,437,184 parameters

Total model

approximately 596 million parameters

The RMSNorm, RoPE, attention, KV-cache, SwiGLU, residual, and output-projection decisions are summarized in this case study and implemented in the local transformer example.

The example is **decode-only and single-token**. It does not implement prefill, sampling, or device-side token lookup. The executable source is the [transformer example](#). This case study explains its decisions and points to the source and tests instead of copying the implementation.

Runnable decoder flow

The local `furiosa-opt-examples/src/transformer/mod.rs` exposes these device launches:

- `embedding`
- `projection`
- `attention_forward_first`
- `attention_forward`
- `decoder`
- `final_layer`

This local launch map is the canonical execution surface for this case study. The vISA and API contracts remain in the reference chapters.

Workload semantics

The workload is one decoder step:

host-prepared embedding → 28 decoder layers → final RMSNorm and LM head → logits [Wp]

The current example describes Qwen3-0.6B with 28 layers and hidden size 1024. Each decoder layer has these boundaries:

1. `projection`: input RMSNorm, Q/K/V projections, Q/K RMSNorm, RoPE, and KV-

cache writes.

2. `attention_forward_first` or `attention_forward`: one online-softmax attention chunk.
3. `decoder`: output projection, residual, post-attention RMSNorm, MLP, and residual.

`final_layer` applies the final RMSNorm and computes logits into the padded vocabulary shape. The exact axis sizes and tensor types are defined in [axes.rs](#).

Decision trace

The case study follows the same order as the rest of the book:

1. **Workload semantics:** one token and a growing KV cache define the data movement.
2. **Hard constraints:** every kernel has a typed public boundary. The vocabulary output uses `wp`, while the model vocabulary is `w`.
3. **Kernel boundaries:** the Rust module exposes `embedding`, `projection`, `attention_forward_first`, `attention_forward`, `decoder`, and `final_layer` as separate device entry points.
4. **Mapping and movement:** the source moves host-prepared tensors through HBM and DM and uses the representations required by each engine. See [Mapping Tensors](#) and [Moving Tensors](#) for those contracts.
5. **Engine choices:** projection and LM-head contractions use the Contraction Engine. RMSNorm and elementwise stages use the Vector Engine. Transfers use the DMA and Fetch/Commit paths. See [Computing Tensors](#) for the contracts.
6. **Checks:** host-oracle tests establish values, and a dumped schedule records the static execution plan. See [Scheduling and Tuning](#).

The case study therefore demonstrates how choices compose. It does not redefine any individual engine API.

Host-prepared embedding

The `embedding` entry point is a host-prepared HBM copy. It accepts an `HbmTensor<bf16, ..., m![H]>` and copies its view into the output HBM tensor with `to_hbm_view`. It is not a device lookup from token IDs and must not be described as one.

The current source implements this step as `ops::embedding`. The smoke test is `test_embedding`.

Decoder step

`projection` receives one hidden vector, the projection and RMSNorm weights, the position-dependent `cos` and `sin` vectors, and mutable K/V caches. It writes Q to HBM and updates the cache. The first attention chunk uses `attention_forward_first`. Later chunks use `attention_forward` to continue the online reduction state.

`decoder` consumes the attention output and its reduction state, writes the residual stream back to HBM, and applies the output projection, residual path, RMSNorm, MLP, and final residual path. The function signatures define the required inputs and outputs. The implementation details are in `furiosa-opt-examples/src/transformer/mod.rs`.

The boundary test `test_decoder_step_wires_kernel_boundaries` passes host-prepared input through projection, one attention chunk, and decoder with zero weights. Its scalar-oracle tests separately cover projection, chunked attention, and decoder arithmetic.

Final layer and padded logits

`final_layer` takes the residual stream, applies RMSNorm, reshapes the model vocabulary weight view to the padded output representation, and writes logits to `m! [Wp]`. The source accepts a separate `lm_head_weight`. This example makes no tied-embedding claim.

The deterministic test `test_final_layer_matches_scalar_reference` compares the valid `w` logits with a host RMSNorm-and-dot-product oracle. Padding is part of the tensor's required shape and storage behavior. Interpretation of padded entries belongs to the caller.

Verify and inspect

Run the transformer test target with the repository's normal command:

```
cargo furiosa-opt test --test transformer_tests
```

The tests use seeded small fixtures and compare each supported boundary against host equations. They do not require model files or a sampling loop. Compile a selected device entry point with `--dump-schedule` to produce schedule data. Inspect the result with the [Schedule Viewer](#) and follow the [Scheduling and Tuning](#) comparison protocol.

Keep changes disciplined: establish the oracle first, record a baseline schedule, change one mapping, movement, engine, or shape choice, and compare the same artifact. Do not infer throughput from the oracle or from source order.

Scheduling and Tuning

Scheduling and tuning turn a correct kernel into a measured plan. Inspect the schedule, identify the limiting dependency or resource, change one supported choice, and compare under the same conditions.

Scheduling turns a compiled kernel into a testable hypothesis about context overlap, memory dependencies, and resource conflicts. The chapter follows one loop:

1. [Schedule](#): understand contexts, dependencies, and operation order.
2. [Diagnosis](#): inspect a dumped schedule and state a bottleneck hypothesis.
3. [Tuning](#): test one change, compare the same metric, and keep or revert it.

The chapter does not re-explain mapping, tensor movement, or compute-engine contracts. It links to those chapters when a scheduling decision depends on them.

For the four validation meanings and their caveats, see [Quick Start: Kernel Validation](#). Use [Diagnosis](#) and [Tuning](#) to interpret fixed schedule candidates and make decisions.

Schedule

The scheduler uses contexts, source order, and memory locations to organize operations. It preserves dependencies and resource constraints needed for sequentially equivalent results.

Execution contexts

The public execution contexts are:

- **Main** (`ctx.main`): drives the main Tensor Unit pipeline.
- **Sub** (`ctx.sub`): drives the subset of the Tensor Unit pipeline used by operations such as register-file preloads.
- **DMA** (`ctx.tdma`): drives tensor movement between HBM, DM, and other memory tiers.

Operations in one context are ordered. In the MNIST schedule, operations in different contexts overlap when they do not share a scheduling resource or violate a memory dependency. For example, a DMA load overlaps main-context computation until the consumer needs the loaded value.

Resource and bank contention

- **Scheduling model:** The scheduler uses context occupancy information: if operation A occupies a context (e.g., main-context), the next operation B using that context waits until A completes. Understanding which contexts operations occupy enables predicting parallel execution.
- **Compiler scheduling behavior:** When Tensor Unit operations would violate the 64-access limit, the compiler schedules them as if they occupy DMA, preventing concurrent DMA operations. This sacrifices the TCP architecture's inherent main/sub/DMA context parallelism where data preparation and computation occur in parallel, but avoids catastrophic hardware resets. Treat this as a hard constraint: never use patterns with 64+ consecutive same-bank accesses.
- **The 64-access limit details:**
 - The limit is cumulative: total accesses from all engines to the same bank must

stay below 64, since even interleaved accesses across commands accumulate toward this total. For example, main: 30, sub: 20, DMA: 1 totals 51 (safe), but main: 30, sub: 35, DMA: 1 totals 66 (triggers starvation).

- The compiler keeps each individual command below 64 consecutive same-bank accesses, but cannot prevent the total from reaching 64 when multiple commands run concurrently.
- In practice, sub-context rarely accesses the same bank consecutively (`StoTrf` , `StoVrf` operations typically use sequential addresses and tiling prevents same-bank access).
- Sub-context operations that would exceed the limit are also not scheduled concurrently with DMA.
- **Main/sub-context contention:** Main-context can starve sub-context, but this is less severe:
 - Unlike DMA starvation, sub-context starvation does not cause NoC timeout or hardware reset and only increases processing time.
 - Collision probability is lower: DMA Engine occupies 16 banks at once, while sub fetch/commit engines occupy only one bank.
 - Starvation does not occur between fetch and commit engines within the same context due to pipeline back-pressure.
 - **Performance impact example:** If main-context exec command continuously accesses a specific bank while sub-context stos command is scheduled, sub-context processing is delayed. Worst case: total time = main-context time + sub-context time. Ideal case: main and sub access different banks, achieving total time = max(main-context time, sub-context time).

Operation order and dependencies

The scheduler may order independent operations to expose overlap. A dependency prevents reordering when the changed order could alter a value. Do not infer a performance improvement from written source order. Inspect the emitted schedule and compare its makespan under the [tuning protocol](#).

Tensor Unit operations cannot consume HBM tensors directly. Data must move through the memory tiers described in [Quick Start](#). Memory locations are shared by contexts, so the scheduler must preserve these hazards:

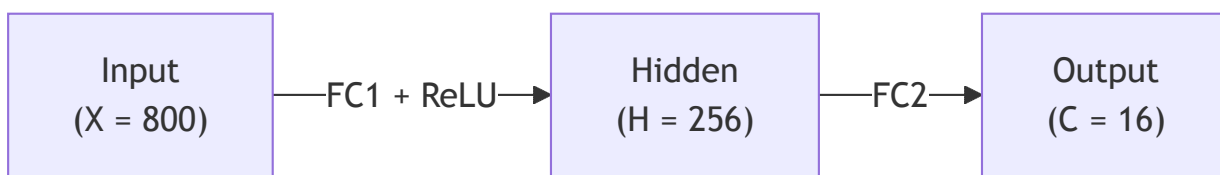
- **Read after write:** a consumer waits for the producer's write.
- **Write after read:** a later writer waits until earlier readers finish.

- **Write after write:** overlapping writes retain their program order.

These hazards explain why independent contexts sometimes wait even when their operation types differ.

Different contexts still serialize when they require the same scheduling resource. For example, a main-context Vector Engine operation and a sub-context Vector Engine operation cannot occupy that resource simultaneously. A diagnosis must distinguish this resource wait from a data dependency before choosing a tuning lever.

The [MNIST example](#) provides a schedule example for a two-layer MLP; its first layer applies ReLU. Its schedule images show context overlap and waits:



The device function calls the two layers in source order. Each layer prepares its operands in the memory tier required by its compute path:

```
axes![X = 800, H = 256, C = 16];
```

```
type Chip = m![1];
```

```
type Cluster = m![1 # 2];
```

```
#[device(chip = 1)]
```

```
pub fn forward(
```

```
    ctx: &mut Context,
```

```
    input: &HbmTensor<bf16, Chip, m![X]>,
```

```
    fc1_weight: &HbmTensor<bf16, Chip, m![H, X]>,
```

```
    fc1_bias: &HbmTensor<bf16, Chip, m![H]>,
```

```
    fc2_weight: &HbmTensor<bf16, Chip, m![C, H]>,
```

```
    fc2_bias: &HbmTensor<bf16, Chip, m![C]>,
```

```
) -> HbmTensor<bf16, Chip, m![C]> {
```

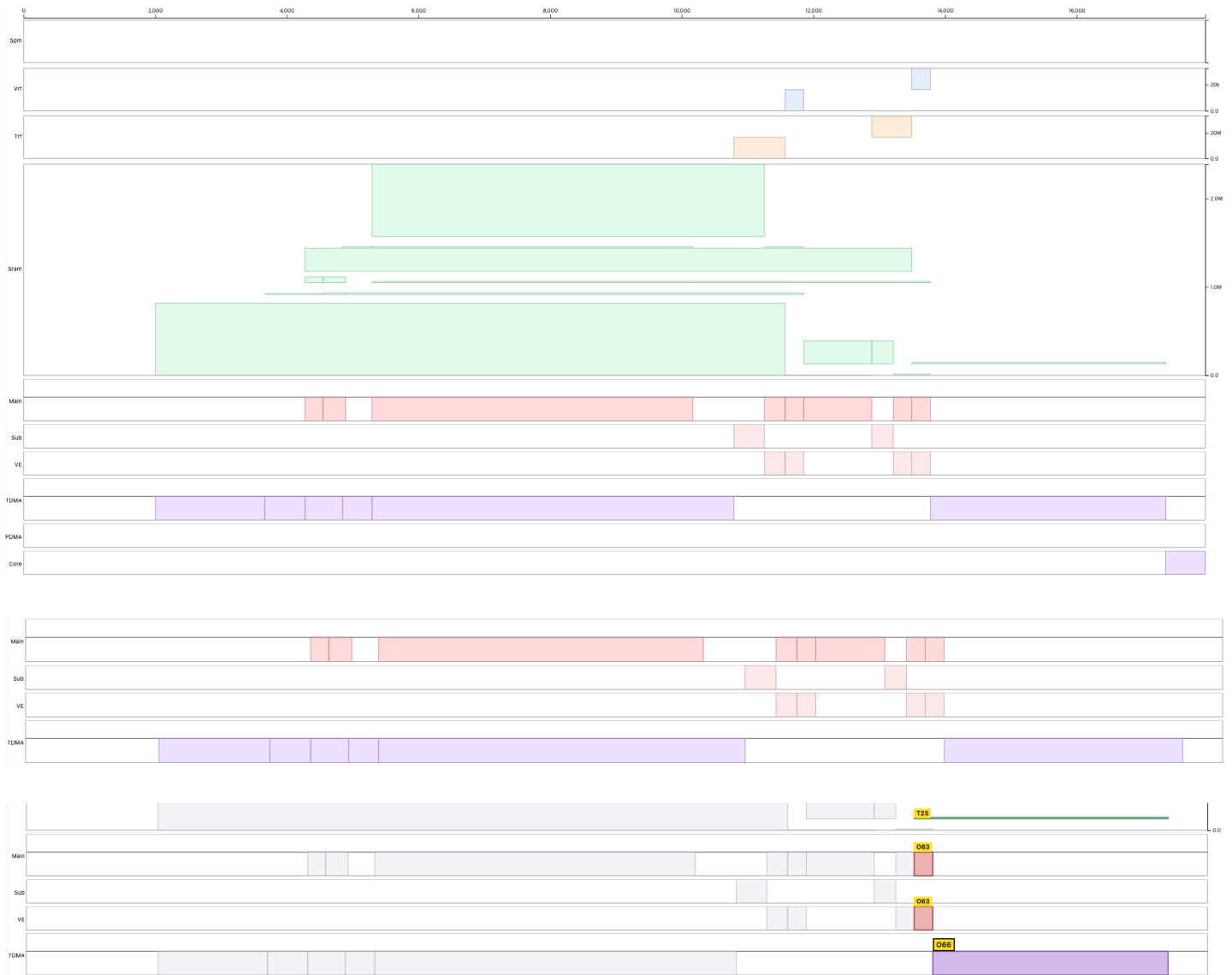
```
    let hidden = fc1_relu(ctx, input, fc1_weight, fc1_bias);
```

```
    fc2(ctx, hidden, fc2_weight, fc2_bias)
```

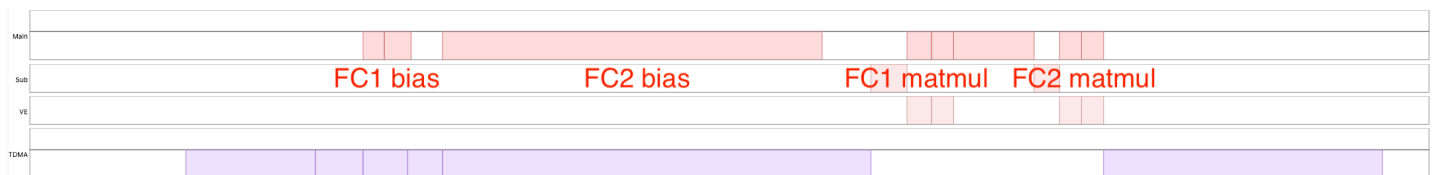
```
}
```

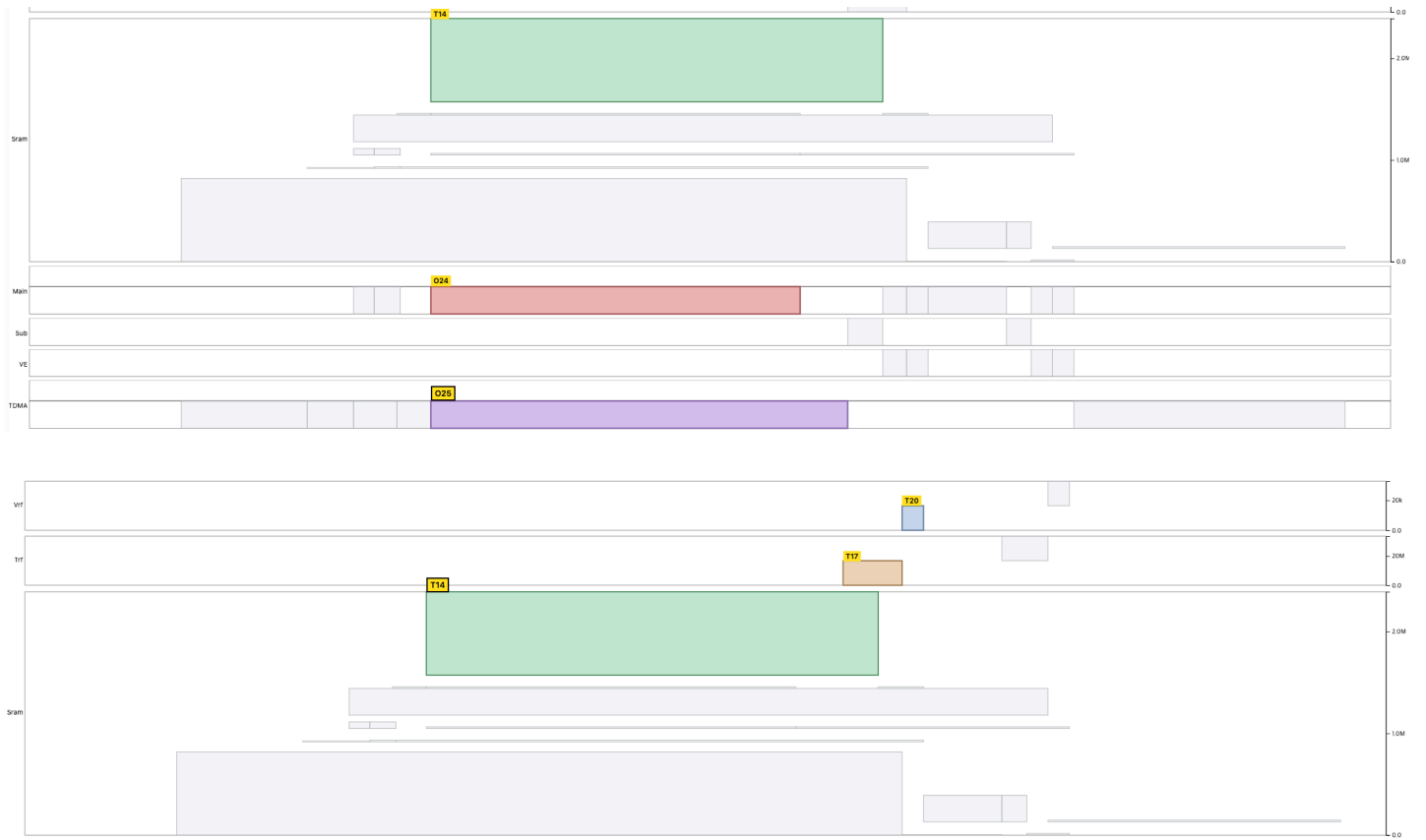
Each fully connected layer computes a matrix-vector product and adds a bias. The first layer also applies ReLU in the same pass.

The complete schedule is shown here. The following images zoom into its context and dependency behavior.



In the MNIST schedule, bias preparation can precede a matrix-vector operation while a long input fetch is in flight.





Diagnosis

Diagnosis converts a schedule view into one testable bottleneck hypothesis. Separate data dependencies from shared-resource waits. A schedule view does not establish a speedup by inspection alone.

Inspect a dumped schedule

Generate and open the JSON with the [Schedule Viewer tool reference](#). The viewer exposes the scheduled timeline, operation context, lifetime, connected tensors, and operator description. Select a node to trace its inputs and outputs. Zoom the cycle range to isolate the suspected region.

Read the timeline

The final scheduled cycle defines the makespan, and the intervals that determine it provide the diagnosis:

1. Identify the interval that reaches the final cycle.
2. Check whether its predecessor is a memory dependency, a shared resource, or an unscheduled gap.
3. Compare the relevant context lanes to see whether another operation could overlap.
4. State one candidate change and the reason it should alter that interval.

The schedule shows static compiler behavior. It does not replace a value oracle, and it does not prove device throughput.

Keep the comparison valid

Record the source revision, release profile, device-function filter, shapes, element types, mappings, and schedule file for each candidate. Change only the selected kernel choice between candidates. If any of those inputs changes, treat the result as a new baseline

rather than a tuning comparison.

Tuning

Tuning is a controlled experiment: preserve correctness, record a baseline, change one supported lever, and compare the same schedule metric before deciding whether to keep the change.

Control the experiment

Follow this order for every candidate:

1. Run the type-checking path to validate mapping and shape constraints.
2. Run a CPU test with a host oracle, including the relevant boundary cases.
3. Compile the baseline and record its schedule makespan.
4. Change one lever while keeping release, shapes, data types, and mappings fixed.
5. Run the same checks and dump the candidate schedule.
6. Keep the change only if correctness still passes and the measured metric improves; otherwise restore the baseline.

Schedule makespan is the primary static metric in this chapter. Do not report a throughput or cycle improvement without a reproducible schedule comparison or separately documented device evidence.

Choose one lever

Choose a lever only after diagnosis names the dependency or resource it targets:

- change the execution-engine path when the current resource is the bottleneck;
- change a tile or split shape when the schedule exposes avoidable serial work or an ill-fitting reduction;
- change a mapping, padding, or transfer boundary when the limiting interval is movement or an address dependency.

The [Moving Tensors](#), [Computing Tensors](#), and [Mapping Tensors](#) chapters define these choices. This page defines how to test them, not their contracts.

Double buffering

The scheduler splits the TRF into halves on its own: a tensor that fits in half the file is placed in one, so another operation can hold the other. A kernel does not name the region. Both halves share banks, so a candidate must still be checked for resource contention in the schedule.

See [Register Files: Double Buffering](#) for capacities and address modes. Use the schedule viewer to verify whether the chosen pattern actually overlaps operations; do not assume a fixed cycle reduction.

Decide and record

For each experiment, record the hypothesis, the changed lever, the baseline and candidate makespans, the correctness commands, and the decision. This record makes a later Qwen3 capstone reproducible without repeating the engine contracts here.

Tools

These tools support installation and the development process for kernel work.

- [Kernel Optimizer](#): The cargo subcommand that drives the toolchain: backend selection, automatic kernel builds, and direct kernel compilation.
- [Language Server](#): Installation and configuration of `furiosa-rust-analyzer-proxy` for IDE integration.
- [Schedule Viewer](#): An interactive viewer for inspecting schedule JSON files.

Kernel Optimizer

The Kernel Optimizer provides build, test, run, and direct-compile workflows for Furiosa Optimizer kernels. The `cargo-furiosa-opt` command is the entry point for these workflows.

`cargo-furiosa-opt` is a thin wrapper around `cargo` for the Furiosa NPU compiler toolchain. `cargo furiosa-opt` is the NPU build; a plain `cargo` invocation builds and runs on the CPU. Every `cargo` argument passes through verbatim, and the wrapper compiles the kernels your build needs.

For installation, see [Installation and First Program](#).

First Commands

Run a binary on the CPU, then check mappings, compare values with an oracle, and export schedule data.

```
cargo run --release --bin gemm
cargo furiosa-opt compile
cargo test --release --bin gemm
cargo furiosa-opt compile gemm_kernel --dump-schedule schedule.json
```

Quick Start introduces these commands in context. This section gives the complete `cargo-furiosa-opt` command reference. Use [Kernel Validation](#) to interpret compile, CPU, NPU, and schedule results.

Usage

```
cargo furiosa-opt <command> [args]
cargo furiosa-opt compile [FILTER]... [options]
```

The first form is the cargo passthrough: `<command>` is any cargo subcommand (`build`, `test`, `run`, ...) and `[args]` are forwarded to cargo unchanged, compiled for the NPU. The second form compiles kernels directly. See [Direct compilation](#).

A plain `cargo build / cargo test / cargo run` is the CPU build: kernels run on the host, and no NPU hardware or SDK is required.

What the wrapper adds

Compared to plain cargo, `cargo furiosa-opt` does two things around the cargo invocation:

- **Targets the NPU.** Kernels dispatch to real hardware. It requires a physical NPU and the Furiosa SDK.
- **Builds the kernels you need automatically.** It compiles necessary `#[device]` functions before the cargo build so the resulting binary can load them at runtime. See [Automatic kernel builds](#).

See [Quick Start: Kernel Optimizer](#) for a task-oriented overview.

Automatic kernel builds

`cargo furiosa-opt` runs a kernel pre-compilation step before handing off to cargo. The pre-step runs only when it matters. It is skipped unless **both** of the following hold:

- The cargo subcommand builds or executes code: `build`, `check`, `test`, `run`, `bench`, or `doc`.
- The invocation is not a `-h / --help` query.

When it runs, the pre-step compiles only the kernels the build actually needs:

- It reads cargo's unit graph, honoring your `-p / --package` and workspace selection, to find kernel packages. A crate is a kernel package if its `Cargo.toml` declares `[package.metadata.furiosa-opt]`.
- When the command resolves to specific runnable targets, such as a test, example, or binary, the compiler scans each target and compiles only the `#[device]` functions reachable from it.
- Otherwise, it falls back to compiling every kernel in the selected packages.

Compilation is cached per kernel so unchanged kernels are not recompiled on the next run. Artifacts are written under the output directory (see [FURIOSA_OPT_OUT_DIR](#)).

Direct compilation: `cargo furiosa-opt compile`

`cargo furiosa-opt compile` compiles `#[device]` functions directly, without the cargo passthrough.

```
# Compile every #[device] function in every kernel package.
cargo furiosa-opt compile
```

```
# Compile only the functions matching a filter, in one package.
cargo furiosa-opt compile transpose_simple -p my_kernels
```

```
# Compile a single function and dump its schedule for the Schedule Viewer.
cargo furiosa-opt compile transpose::transpose_simple \
  --dump-schedule schedule.json
```

Compilation pipeline and dump options

The dump flags below each capture a different point of the kernel compilation pipeline, which runs in three stages:

1. **Stage 1 — MIR → vISA.**
2. **Stage 2 — vISA → LIR.**
3. **Stage 3 — LIR → EDF (the kernel binary).**

Each public dump flag targets a stage and emits an artifact for a specific consumer. All dump flags are single-kernel options.

- `--dump-visa` : Stage 1 vISA text for compiler/backend inspection.
- `--dump-ir` : Stage 2 LIR as a bincode binary for compiler tooling.
- `--dump-dfg` : Stage 1 vISA data-flow graph serialized as compiler IR binary data to the requested file for graph inspection.
- `--dump-graph` : Stage 2 LIR graph as JSON for the IR graph viewer.
- `--dump-summary` : Stage 3 LIR-to-EDF compilation summary written to the requested directory.
- `--dump-schedule` : Stage 3 scheduling after LIR is lowered to `ResourceLir`, emitted as JSON for the [Schedule Viewer](#).

[FILTER]...

Specifies the set of `#[device]` functions to compile. Filters are matched as a substring against `#[device]` function names as a full path (`abc::def::foo`). A function is compiled if it matches any filter. When omitted, all device functions are compiled.

`-p, --package name`

Restrict compilation to the named kernel package. The option may be repeated to select multiple kernel packages. When omitted, all kernel packages are compiled.

`--message-format format`

Diagnostic format forwarded to the compiler (e.g. `json`), so tools can machine-parse kernel-compile failure diagnostics.

`--dump-visa file`

Dump the intermediate vISA to a file. Should only be used when compiling a single kernel.

`--dump-ir file`

Dump the intermediate LIR (Low-Level IR) as a bincode binary to a file. Should only be used when compiling a single kernel.

`--dump-schedule file`

Lower LIR to `ResourceLir` and dump the resulting schedule as JSON for the [Schedule Viewer](#). Should only be used when compiling a single kernel.

`--dump-dfg file`

Dump the Stage 1 vISA data-flow graph to a file. Should only be used when compiling a single kernel.

`--dump-graph file`

Dump the Stage 2 LIR graph as JSON for the IR graph viewer. Should only be used when compiling a single kernel.

`--dump-summary dir`

Dump the Stage 3 LIR-to-EDF compilation summary to a directory for diagnostic tooling. Should only be used when compiling a single kernel.

Environment variables

FURIOSA_OPT_OUT_DIR

Kernel output directory. Defaults to `<workspace target>/furiosa-opt/kernel`.

Language Server

The language-server proxy provides IDE navigation, diagnostics, and readable mapping-expression types. Editor rendering does not change type checking.

This tool reference details the installation and configuration of `furiosa-rust-analyzer-proxy`, a proxy for `rust-analyzer` that provides IDE support for mapping expressions. Quick Start introduces this tool alongside the other development tools; this section gives the installation and configuration steps. The proxy runs `rust-analyzer` underneath, forwards normal Rust language-server traffic to it, and rewrites editor-facing results so mapping types are displayed in `m![...]` notation.

Installation

1. Ensure `rust-analyzer` is installed and available in your `PATH`. The proxy launches this upstream `rust-analyzer` process to provide standard Rust IDE features.
2. Download the `furiosa-rust-analyzer-proxy` binary from the [GitHub releases](#) and make it executable:

```
curl -L -o furiosa-rust-analyzer-proxy \
  https://github.com/furiosa-ai/furiosa-
  opt/releases/latest/download/furiosa-rust-analyzer-proxy-x86_64-unknown-
  linux-gnu
chmod +x furiosa-rust-analyzer-proxy
```

3. Configure your IDE to use the downloaded binary instead of the default language server. For example, in VSCode, update your `settings.json`:

```
{
  "rust-analyzer.server.path": "/path/to/furiosa-rust-analyzer-proxy",
  "rust-analyzer.inlayHints.maxLength": null // recommended to reduce
  '_' truncation
}
```

Environment variables

You can configure the language server using environment variables. For example, in VSCode, update your `settings.json`:

```
{
  "rust-analyzer.server.extraEnv": {
    "ENV_NAME": "env_value"
  }
}
```

- `FURIOSA_RUST_ANALYZER_PROXY_UPSTREAM`: Custom path to the upstream `rust-analyzer` binary that the proxy launches. Defaults to `rust-analyzer` in `PATH`.

Features

The proxy delegates standard Rust IDE features to `rust-analyzer` and rewrites mapping expressions in editor-facing results.

Call Hierarchy

Provides incoming and outgoing call hierarchy views. Function details shown in hierarchy entries are converted into mapping expressions.

```

  ✓  binary_add_kernel fn binary_add_kernel(ctx: &mut Context, lhs: HbmTensorView<'_, i8, Chip, m![A]>, rhs: Hb...
  | >  __tcp_binary_add_2048 fn __tcp_binary_add_2048(ctx: &mut Context, lhs: &HbmTensor<i8, m![1], m![A]>, ...
```

Code Actions

Provides quick fixes, refactors, and other editor actions. Action title and text edits are converted into mapping expressions.


```
pub fn new_hbm(address: u64) → HbmTensor<i8, m![1], m![B, A]> {
    ...let output: HbmTensor<i8, m![1], m![B, A]> = unsafe { HbmTensor::<i8, m![1], m![B, A]>::from_addr(address) };
    ...output
}
```

Code Completions

Provides completion items for names, methods, functions, types, and snippets. Completion labels, detail text, and text edits are converted into mapping expressions.

ve_ Not Committed Yet

- __tcp_ve_elementwise_fp_binary_with_mo... fn(&mut Context, &HbmTensor<f32, m![1], m![A], Simu...
- __tcp_ve_elementwise_fp_div_with_mo... fn(&mut Context, &HbmTensor<f32, m![1], m![A], Simulat...
- __tcp_ve_elementwise_fp_ternary_with_m... fn(&mut Context, &HbmTensor<f32, m![1], m![A], Sim...
- __tcp_ve_elementwise_fp_unary... fn(&mut Context, &HbmTensor<f32, m![1], m![A], Simulation>) ...
- __tcp_ve_elementwise_full_pipelin... fn(&mut Context, &HbmTensor<i32, m![1], m![A], Simulatio...
- __tcp_ve_elementwise_fxp_chain... fn(&mut Context, &HbmTensor<i32, m![1], m![A], Simulation>) ...
- __tcp_ve_elementwise_fxp_const... fn(&mut Context, &HbmTensor<i32, m![1], m![A], Simulation>) ...
- __tcp_ve_elementwise_fxp_with_mod... fn(&mut Context, &HbmTensor<i32, m![1], m![A], Simulatio...
- __tcp_ve_elementwise_logic... fn(&mut Context, &HbmTensor<i32, m![1], m![A], Simulation>) → H...
- __tcp_ve_elementwise_logic_with_mo... fn(&mut Context, &HbmTensor<i32, m![1], m![A], Simulatio...
- __tcp_ve_elementwise_multi... fn(&mut Context, &HbmTensor<i32, m![1], m![A, B], Simulation>, ...
- __tcp_ve_elementwise_stash_f32... fn(&mut Context, &HbmTensor<f32, m![1], m![A], Simulation>) ...

Diagnostics

Provides diagnostics from `rust-analyzer` and `rustc`. Diagnostic messages and related information are converted into mapping expressions.

output

mismatched types
 expected struct `furiosa\_visa\_std::prelude::HbmTensor<\_, \_, m![A, B]>`
 found struct `furiosa\_visa\_std::prelude::HbmTensor<\_, \_, m![B, A]>`
 the full name for the type has been written to '/root/FuriosaAI/npu-tools/target/debug/deps/furiosa_visa_examples-469df6fd9add2d08.long-type-16261697815141885790.txt'
 consider using `--verbose` to print the full type name to the console [rustc\(Click for full compiler diagnostic\)](#)
 tile.rs(7, 89): expected `HbmTensor<i8, m![1], m![A, B]>` because of return type

Hover

Shows additional information when hovering over a symbol. Hover contents such as inferred types, function signatures, and documentation are converted into mapping expressions.

```

/// Add two
#[device(ch
pub fn bina
... ctx: &m
... lhs: &H
... rhs: &H
) → HbmTen
... // Allo
... // Chip
... type Re
... let mut
... // Perf
... binary_
... // Writ
... result.to_hbm(&mut ctx.tdma, 0x3000)
}

```

furiosa_visa_std::tensor::memory::DmTensor

impl<D, Chip, Cluster, Slice, Element, B> DmTensor<D, Chip, Cluster, Slice, Element, B>

pub fn to_hbm<Element2>(&self, _dma: &mut DmaContext<<expr>>, address: Address) → HbmTensor<D, Chip, Element2, B>

where

Element2: M,

// Bounds from impl:

D: Scalar,

Chip: M,

Cluster: M,

Slice: M,

Element: M,

B: Backend,

D = i32, Chip = m![1], Cluster = m![1 # 2], Slice = m![A / 8], Element = m![A % 8], B = Simulation, Element2 = m![A]

Converts to HBM tensor.

Inlay Hints

Shows additional information inline with the source code. Inlay hints are converted into mapping expressions.

💡 Tip

For the most accurate conversion, set `rust-analyzer.inlayHints.maxLength` to `null` (unlimited length). This reduces how often long inlay hints are truncated into `'_'`.

```

// Generate random input tensors and allocate output tensor.
let mut rng: SmallRng = SmallRng::seed_from_u64(state: 42);
let lhs: HostTensor<i8, m![A]> = HostTensor::<i8, m![A]>::rand(&mut rng);
let rhs: HostTensor<i8, m![A]> = HostTensor::<i8, m![A]>::rand(&mut rng);
let lhs_hbm: HbmTensor<i8, m![1], m![A]> = lhs.to_hbm(_dma: &mut ctx.pdma, address: 0 << 28).await;
let rhs_hbm: HbmTensor<i8, m![1], m![A]> = rhs.to_hbm(_dma: &mut ctx.pdma, address: 1 << 28).await;

```

Signature Help

Shows function signatures and the active parameter while writing a call. Signature labels, parameter labels, and documentation are rewritten to use mapping notation, including offset-based parameter labels returned by LSP clients.

```

>vice(chip = 1)]
fn binary_add_2048(
  ctx: &mut Context,
  lhs: &HbmTensor<i8, m![1], m![A]>,
  rhs: &HbmTensor<i8, m![1], m![A]>,
  > HbmTensor<i32, m![1], m![A]> {
  // Allocate DM tensor for output with layout:
  // Chip: m![1], Cluster: m![1#2], Slice: [A/8], Element: [A%8]
  type ResultDmTensor = DmTensor<i32, Chip, Cluster, m![A / 8], m![A % 8]>;
  let mut result: DmTensor<i32, m![1], m![1 # 2], m![A / 8]... = unsafe { ResultDmTensor::from_addr(0) };

  // Perform binary addition in DM

```

Caveats

The language server may incorrectly interpret user-defined types as mapping expressions if they share names with internal mapping components. For instance, if you define a custom `Symbol<T>` struct, the language server might mistakenly display it as `m![T]` in your IDE. This is purely a UI display issue and does not affect the other LSP behaviors.

Schedule Viewer

The Schedule Viewer supports schedule compilation and makespan comparison between fixed candidates. The viewer shows the static execution plan; it does not validate values or throughput.

The Schedule Viewer loads a schedule JSON file generated by `cargo furiosa-opt` and displays it as an interactive execution timeline. Quick Start introduces the viewer as part of the development tools; this section explains how to install it and inspect a schedule. Use it to inspect which operations run in parallel, which context or resource each operation occupies, and which operations block pipeline progress.

Review a Schedule

Use schedule makespan, the final scheduled cycle, to compare fixed kernel workloads. Keep release, shapes, data types, and mapping unchanged between candidates. Use the timeline to explain differences in overlap, dependencies, and resource occupancy.

Getting Started

Install and Run the Viewer

Install `furiosa-schedule-viewer` binary from crates.io:

```
cargo install furiosa-schedule-viewer
```

Run the viewer to start the local web UI:

```
furiosa-schedule-viewer
```

By default the server binds to `127.0.0.1:9254` and opens the page in your default browser. Use `--host` and `--port` to change the address that the server listens on:

```
furiosa-schedule-viewer --host 127.0.0.1 --port 9254
```

Generate a Schedule JSON File

`--dump-schedule` writes the compiled schedule to a JSON file. It is available when compiling a single kernel with `cargo furiosa-opt compile`:

```
cargo furiosa-opt compile <device-function> \  
  --dump-schedule <path-to-json-file>
```

Provide the function name as the positional `filter`. If the function name is ambiguous, use the full Rust path, such as `kernel::gemm_kernel::gemm_kernel`. The command still emits the normal kernel artifacts. The JSON file is the input for the Schedule Viewer.

Usage

Click the drop zone or drag a schedule JSON file onto the page to visualize it.

Inspect Tensors and Operators

Click any node to inspect it. The left sidebar shows details such as its name, lifetime, context, and connected nodes.

When you hover over or select a node, related nodes are highlighted. For example, selecting an operator highlights its input and output tensors. Selecting a tensor highlights operators connected to that tensor.

Because tensors and operators are based on actual hardware instructions, they may differ from the tensors and operators defined in vISA. Whenever possible, the Schedule Viewer shows the vISA tensor name and shape, and the operator description.

Zoom In on the Schedule

You can zoom in on the schedule to inspect a specific region.

To adjust only the cycle range, drag across the timeline at the top or use the **Cycle Range** inputs at the top right. To adjust the memory-address range, click **Enable Brush**, then drag across the schedule view. The brush sets both the cycle range and the memory-address range.

To restore the full schedule view, click the timeline background or click **Reset** at the top right.